

Daehee Kim
Sejun Song
Baek-Young Choi

Data Deduplication for Data Optimization for Storage and Network Systems

Data Deduplication for Data Optimization for Storage and Network Systems

Daehee Kim • Sejun Song • Baek-Young Choi

Data Deduplication for Data Optimization for Storage and Network Systems

Daehee Kim
Department of Computing
and New Media Technologies
University of Wisconsin-Stevens Point
Stevens Point, Wisconsin, USA

Sejun Song
Department of Computer Science
and Electrical Engineering
University of Missouri-Kansas City
Kansas City, Missouri, USA

Baek-Young Choi
Department of Computer Science
and Electrical Engineering
University of Missouri-Kansas City
Kansas City, Missouri, USA

ISBN 978-3-319-42278-7 ISBN 978-3-319-42280-0 (eBook)
DOI 10.1007/978-3-319-42280-0

Library of Congress Control Number: 2016949407

© Springer International Publishing Switzerland 2017

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made.

Printed on acid-free paper

This Springer imprint is published by Springer Nature
The registered company is Springer International Publishing AG Switzerland

Contents

Part I Traditional Deduplication Techniques and Solutions

1	Introduction	3
1.1	Data Explosion	3
1.2	Redundancies	4
1.3	Existing Deduplication Solutions to Remove Redundancies	5
1.4	Issues Related to Existing Solutions	7
1.5	Deduplication Framework	7
1.6	Redundant Array of Inexpensive Disks	8
1.7	Direct-Attached Storage	9
1.8	Storage Area Network	10
1.9	Network-Attached Storage	12
1.10	Comparison of DAS, NAS and SAN	13
1.11	Storage Virtualization	13
1.12	In-Memory Storage	15
1.13	Object-Oriented Storage	16
1.14	Standards and Efforts to Develop Data Storage Systems	16
1.15	Summary and Organization	20
	References	21
2	Existing Deduplication Techniques	23
2.1	Deduplication Techniques Classification	23
2.2	Common Modules	25
2.2.1	Chunk Index Cache	25
2.2.2	Bloom Filter	30
2.3	Deduplication Techniques by Granularity	34
2.3.1	File-Level Deduplication	34
2.3.2	Fixed-Size Block Deduplication	38
2.3.3	Variable-Sized Block Deduplication	44
2.3.4	Hybrid Deduplication	54
2.3.5	Object-Level Deduplication	55
2.3.6	Comparison of Deduplications by Granularity	55

2.4	Deduplication Techniques by Place	56
2.4.1	Server-Based Deduplication	56
2.4.2	Client-Based Deduplication	57
2.4.3	End-to-End Redundancy Elimination	58
2.4.4	Network-Wide Redundancy Elimination	60
2.5	Deduplication Techniques by Time	71
2.5.1	Inline Deduplication	71
2.5.2	Offline Deduplication	73
2.6	Summary	74
	References	75

Part II Storage Data Deduplication

3	HEDS: Hybrid Email Deduplication System	79
3.1	Large Redundancies in Emails	79
3.2	Hybrid System Design	80
3.3	EDMilter	80
3.4	Metadata Server	82
3.5	Bloom Filter	82
3.6	Chunk Index Cache	82
3.7	Storage Server	83
3.8	EDA	83
3.9	Evaluation	85
3.9.1	Metrics	85
3.9.2	Data Sets	86
3.9.3	Deduplication Performance	89
3.9.4	Memory Overhead	92
3.9.5	CPU Overhead	94
3.10	Summary	94
	References	94
4	SAFE: Structure-Aware File and Email Deduplication for Cloud-Based Storage Systems	97
4.1	Large Redundancies in Cloud Storage Systems	97
4.2	SAFE Modules	98
4.3	Email Parser	99
4.4	File Parser	100
4.5	Object-Level Deduplication and Store Manager	103
4.6	SAFE in Dropbox	104
4.7	Evaluation	106
4.7.1	Metrics	107
4.7.2	Data Sets	107
4.7.3	Storage Data Reduction Performance	109
4.7.4	Data Traffic Reduction Performance	109
4.7.5	CPU Overhead	110
4.7.6	Memory Overhead	113

4.8	Summary	113
	References	115

Part III Network Deduplication

5	SoftDance: Software-Defined Deduplication as a Network and Storage Service	119
5.1	Large Redundancies in Network	119
5.2	Software-Defined Network	121
5.3	Control and Data Flow	121
5.4	Encoding Algorithms in Middlebox (SDMB)	124
5.5	Index Distribution Algorithms	125
	5.5.1 SoftDANCE-Full (SD-Full)	125
	5.5.2 SoftDance-Uniform (SD-Uniform)	126
	5.5.3 SoftDANCE-Merge (SD-Merge)	127
	5.5.4 SoftDANCE-Optimize (SD-opt)	128
5.6	Implementation	130
	5.6.1 Floodlight, REST, JSON	130
	5.6.2 CPLEX Optimizer: Installation	130
	5.6.3 CPLEX Optimizer: Run Simple CPLEX Using Interactive Optimizer	135
	5.6.4 CPLEX Optimizer: Run Simple CPLEX Using Java Application (with CPLEX API)	137
5.7	Setup	139
	5.7.1 Experiment	139
	5.7.2 Emulation	140
5.8	Evaluation	140
	5.8.1 Metrics	140
	5.8.2 Data Sets	142
	5.8.3 Storage Space and Network Bandwidth Saving	142
	5.8.4 CPU and Memory Overhead	143
	5.8.5 Performance and Overhead per Topology	145
	5.8.6 SoftDance vs. Combined Existing Deduplication Techniques	147
5.9	Summary	150
	References	151

Part IV Future Directions

6	Mobile De-Duplication	155
6.1	Large Redundancies in Mobile Devices	155
6.2	Approaches and Observations	156
6.3	JPEG and MPEG4	156
6.4	Evaluation	156
	6.4.1 Setup	157

6.4.2	Throughput and Running Time per File Type	158
6.4.3	Throughput and Running Time per File Size	161
6.5	Summary	161
	References	164
7	Conclusions	165
 Part V Appendixes		
	Appendices	169
A	Index Creation with SHA1	171
A.1	sha1Wrapper.h	171
A.2	sha1Wrapper.cc	172
A.3	sha1.h	173
A.4	sha1.cc	177
B	Index Table Implementation using Unordered Map	193
B.1	cacheInterface.h	193
B.2	cache.h	195
B.3	cache.cc	198
C	Bloom Filter Implementation	201
C.1	bf.h	201
C.2	bf.c	202
D	Rabin Fingerprinting Implementation	209
D.1	rabinpoly.h	209
D.2	rabinpoly.cc	211
D.3	rabinpoly_main.cc	216
E	Chunking Core Implementation	219
E.1	chunk.h	219
E.2	chunk_main.cc	221
E.3	chunk_sub.cc	223
E.4	common.h	226
E.5	util.cc	227
F	Chunking Wrapper Implementation	231
F.1	chunkInterface.h	231
F.2	chunkWrapper.h	233
F.3	chunkWrapper.cc	233
F.4	chunkWrapperTest	237

G Sample Programs Using libnetfilter_queue Library 239

 G.1 ndedup.h..... 239

 G.2 ndedup.cc 243

 G.3 ndedup_main.cc..... 255

 References 260

Glossary 261

Acronyms

ACK	Acknowledgement
AES_NI	AES New Instruction
AES	Advanced Encryption Standard
AFS	Andrew File System
CAS	Content address storage
CDB	Command descriptor block
CDMI	Cloud Data Management Interface
CDN	Content Delivery Network
CDNI	Content delivery network interconnection
CIFS	Common Internet File System
CRC	Cyclic redundancy check
CSP	Content service provider
DAS	Direct-attached storage
dCDN	downstream CDN
DCN	Data centre network
DCT	Discrete cosine transformation
DDFS	Data Domain File System
DES	Data Encryption Standard
DHT	Distributed hash table
EDA	Email deduplication algorithm
EMC	EMC Corporation
FC	Fibre channel
FIPS	Federal Information Processing Standard
FUSE	File System in UserSpace
HEDS	Hybrid email deduplication system
ICN	Information-centric networking
IDC	International Data Corporation
IDE	Integrated development environment
iFCP	Internet Fibre Channel Protocol
I-frame	Intra frame
IP	Internet Protocol

iSCSI	Internet Small Computer System Interface Architecture
ISP	Internet service provider
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
LAN	Local area network
LBFS	Low-bandwidth file system
LP	Linear programming
LRU	Least Recently Used
MAC	Medium Access Control
MD5	Message Digested Algorithm
MIME	Multipurpose Internet Mail Extensions
MPEG	Moving Picture Experts Group
MTA	Mail Transfer Agent
MTTF	Mean time to failure
MTTR	Mean time to repair
NAS	Network-attached storage
NFS	Network File System
ONC RPC	Open Networking Computing Remote Procedure Call
PATA	Parallel ATA
PDF	Portable Document Format
P-frame	Predicted frame
RAID	Redundant Array of Inexpensive Disks
RE	Redundancy elimination
REST	Representational State Transfer
RPC	Remote Procedure Call
SAFE	Structure-Aware File and Email Deduplication for Cloud-based Storage Systems
SAN	Storage area network
SATA	Serial ATA
SCSI	Small Computer Interface Architecture
SDDC	Software-defined data centre
SDMB	SoftDance Middlebox
SDN	Software-defined network
SDS	Software-defined storage
SHA1	Secure Hash Algorithm 1
SHA2	Secure Hash Algorithm 2
SIS	Single Instance Store
SLED	Single large expensive magnetic disks
SMI	Storage management interface
SMTP	Simple Mail Transfer Protocol
SoftDance	Software-defined deduplication as a network and storage service
SSHD	Solid-state hybrid drive
SSL	Secure Socket Layer
TCP	Transmission Control Protocol
TOS	Type Of Service

TTL	Time to live
Ubuntu LTS	Ubuntu Long-Term Support
uCDN	upstream CDN
WAN	Wide area network
XDR	External Data Representation Standard
XML	Extensible Markup Language

Part I

Traditional Deduplication Techniques and Solutions

In this part, we present an overview of data deduplication. In Chap. 1, we show the importance of data deduplication by pointing out data explosion and large amounts of redundancies. We describe design and issues in connection with current solutions, including storage data deduplication, redundancy elimination, and information-centric networking. We introduce a deduplication framework that optimizes data from clients to servers through networks. The framework consists of three components based on the level of deduplication: the client component removes local redundancies that occur in a client, the network component removes redundant transfers coming from different clients using redundancy elimination (RE) devices, and the server component eliminates redundancies coming from different networks. We also present the evolution of data storage systems. Data storage systems evolved from storage devices attached to a single computer (direct-attached storage) into storage devices attached to computer networks (storage area network and network-attached storage). We discuss the different kinds of storage being developed and how they differ from one another. We explain the concepts redundant array of inexpensive disks (RAID), direct-attached storage (DAS), storage area network (SAN), and network-attached storage (NAS). A storage virtualization technique known as software-defined storage is discussed.

In Chap. 2, we classify various deduplication techniques and existing solutions that have been proposed and used. Brief implementation codes are given for each technique. This chapter explains how deduplication techniques have been developed with different designs considering the characteristics of datasets, system capacity, and deduplication time based on performance and overhead. Based on methods related to granularity, file-level deduplication, fixed- and variable-size block deduplication, hybrid deduplication, and object-level deduplication are explained. Based on the deduplication location, server-based deduplication, client-based deduplication, and RE (end-to-end and network-wide) are explained. Based on deduplication time, inline deduplication and offline deduplication are introduced.

Chapter 1

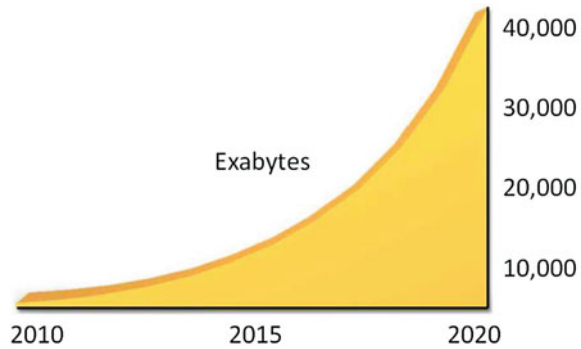
Introduction

Abstract In this chapter, we show why data deduplication is important by stressing data explosion and large amounts of redundancies. We elaborate on current solutions (including storage data deduplication, redundancy elimination, information-centric networking) for data deduplication and the limitations of current solutions. We introduce a deduplication framework that optimizes data from clients to servers through networks. The framework consists of three components based on the level of deduplication. The client component removes local redundancies that occur in a client, the network component removes redundant transfers coming from different clients using redundancy elimination (RE) devices, and the server component eliminates redundancies coming from different networks. Then we show the evolution of data storage. Data storage has evolved from storage devices attached to a single computer (direct-attached storage) into storage devices attached to computer networks (storage area network and network-attached storage). We discuss the different kinds of storage devices and how they differ from one another. A redundant array of inexpensive disks (RAID), which improves storage access performance, is explained, and direct-attached storage (DAS), where storage is incorporated into a computer, is illustrated. We elaborate on storage area networks (SANs) and network-attached storage (NAS), where data from computers are transferred to storage devices through a dedicated network (SAN) or a general local area network used for sending and receiving application data (NAS). SAN and NAS consolidate and efficiently provide storage without wasting storage space compared to a DAS device. We describe a storage virtualization technique known as software-defined storage.

1.1 Data Explosion

We live in an era of data explosion. Based on the International Data Corporation's (IDC's) Digital Universe Study [6], as shown in Fig. 1.1, data volume will increase by 50 times by the end of 2020 over its 2010 level; this amounts to 40 zetabytes (40 million petabytes – more than 5200 gigabytes for every person). This huge increase in data volume will have a critical impact on the overhead costs of

Fig. 1.1 Data explosion:
IDC's Digital Universe
Study [6]



computation, storage and networks. Also, large portions of the data will contain massive redundancies created by users, applications, systems and communication models.

Interestingly, massive portions of this enormous amount of data will be derived from redundancies in storage devices and networks. One study [9] showed that there is a redundancy of 70 % in data sets collected from file systems of almost 1000 computers in an enterprise. Another study [17] found that 30 % of incoming traffic and 60 % of outgoing traffic are redundant based on packet traces on a corporate research environment with 3000 users and Web servers.

1.2 Redundancies

Redundancies are produced in clients, servers and networks in various manners as shown in Fig. 1.2. Redundancies increase on the client side. A user copies a file with a different file name and creates similar files with small updates. These redundancies further increase when users copy redundant files back and forth among people within an organization. Another type of redundancy is generated by applications. For example, currently there it is popular to take pictures of moving objects, in what is called the *burst shooting mode*. In this mode, 30 pictures can be taken within 1 s and good pictures can be saved or bad pictures removed. However, this type of application produces large redundancies among similar pictures. Another type of redundancy occurs in similar frames in video files. A video file consists of many frames. In scenes where actors keep talking with the same background, large portions of the background become redundant.

Redundancies also occur on the network side. When a user first requests a file, a unique transfer occurs produces no redundant transfers in a network. However, when a user requests the same file again, a redundant transfer occurs. Redundancies are also generated by data dissemination, such as video streaming. For example, when different clients receive a streaming file from YouTube, redundant packets must travel through multiple Internet service providers (ISPs).

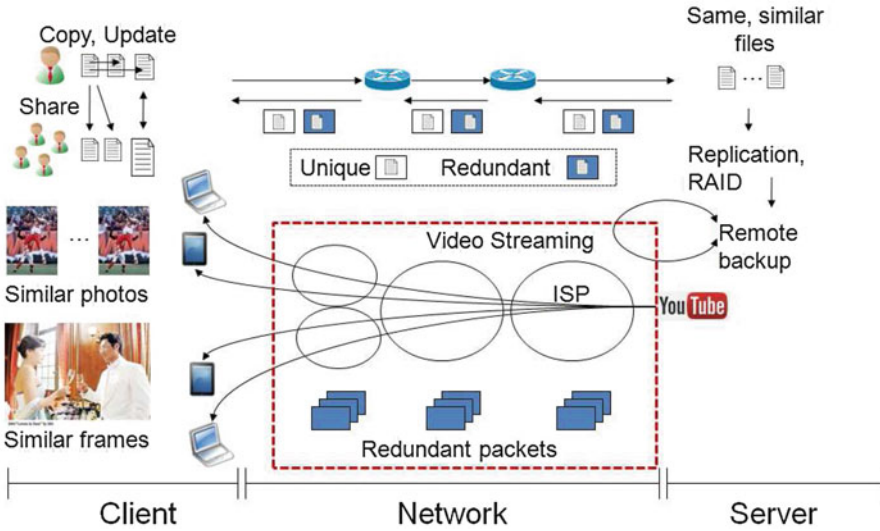


Fig. 1.2 Redundancies

On the server side, redundancies are greatly expanded when people in the same organization upload the same (or similar) files. The redundancies are accelerated by replication, a RAID and remote backup for reliability.

Then one of the problems arising from these redundancies from the client and server sides is that storage consumption increases. On the network side, network bandwidth consumption increases. For clients, latency increases because users keep downloading the same files from distant source servers each time. We find that redundancies significantly impact storage devices and networks. The next question is what solutions exist for removing (or reducing) these redundancies.

1.3 Existing Deduplication Solutions to Remove Redundancies

As shown in Fig. 1.3, there are three types of approaches to removing redundancies from storage devices and networks. The first approach is called storage data deduplication, whose aim is to save storage space. In this approach, only a unique file or chunk is saved, but redundant data are replaced by indexes. Likewise, an image is decomposed into multiple chunks, and redundant chunks are replaced by indexes. A video file consists of I-frames that contain the image itself and P-frames that contain the delta information between images in an I-frame. In a video file where the backgrounds are the same, I-frames have large redundancies that are replaced by indexes. Servers deduplicate redundancies coming from clients by using storage data deduplication.

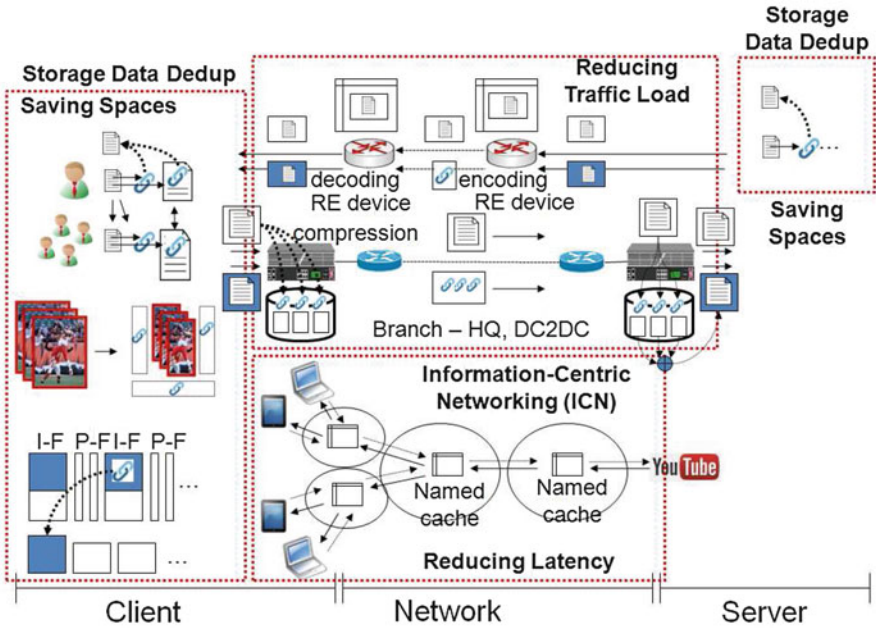


Fig. 1.3 Existing solutions to remove redundancies

The second approach to removing redundancies is called redundancy elimination (RE). With this approach the aim is to reduce traffic loads in networks. The typical example is the wide area network (WAN) optimizer that removes redundant network transfers between branches (or a branch) to a headquarter and one data centre to another. The WAN optimizer works as follows. Suppose a user sends a file to a remote server. Before the file moves through the network, the WAN optimizer splits the file into chunks and saves the chunks and corresponding indexes. The file is compressed and delivered to the WAN optimizer on the other side, where the file is again split into chunks that are saved along with the indexes. The next time the same file passes through the network, the WAN optimizer replaces it with small indexes. On the other side, the WAN optimizer reassembles the file with previously saved chunks based on indexes in a packet.

Another example is network-wide RE, which involves the use of a router (or switch) called a RE device. In this approach, for a unique transfer, the RE device saves the unique packets. When transfers become redundant, the RE device replaces the redundant payload within a packet with an index (called encoding) and reconstructs the encoded packet (called decoding).

The third approach to removing redundancies is called information-centric networking (ICN), which aims to reduce latency. In ICN, any router can cache data packets that are passing by. Thus, when a client requests data, any router with the proper cache can send the requested data.

1.4 Issues Related to Existing Solutions

Problems exist within these current solutions. First, storage data deduplication carries considerable computational and memory overhead in clients and servers. Many studies have focused on the trade-off between space savings and overhead based on granularity. The use of small-scale granularity, like 4 KB, makes it possible to find more redundancies than large-scale granularity, such as a file, but it requires a long processing time and high index overhead. Second, RE entails resource-intensive operations, such as fingerprinting, encoding and decoding at routers. Additionally, a representative RE study proposed a control module that involves a traffic matrix, routing policies and resource configurations, but few details are given, and some of those details are based on assumptions. Thus, we need to have an efficient way to adapt RE devices to dynamic changes. Third, ICN uses name-based forwarding tables that grow much faster than IP forwarding tables. Thus, long table-lookup times and scalability issues arise.

1.5 Deduplication Framework

To resolve (or reduce) issues of the existing solutions, an approach suggested in this book is to develop a deduplication framework that optimizes data from clients to servers throughout networks. The framework consists of three components that have different levels of redundancy removal (Fig. 1.4).

The client component removes local redundancies from a client and is basically comprised of functions to decompose and reconstruct files. These components

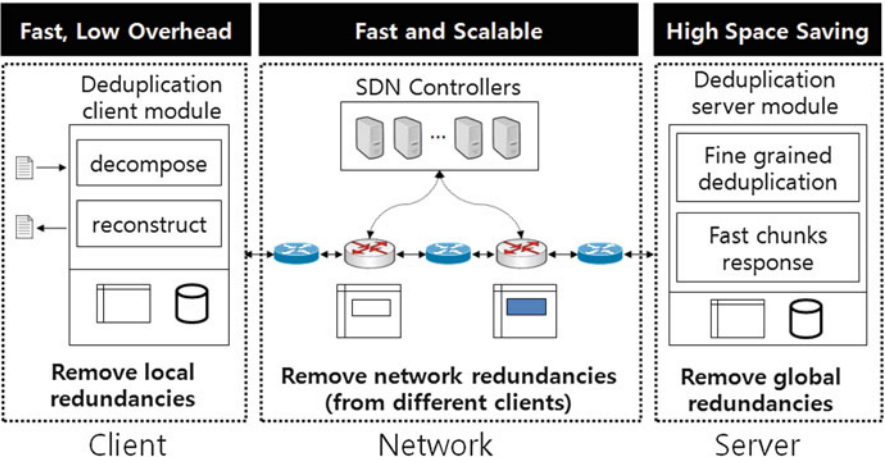


Fig. 1.4 Deduplication framework

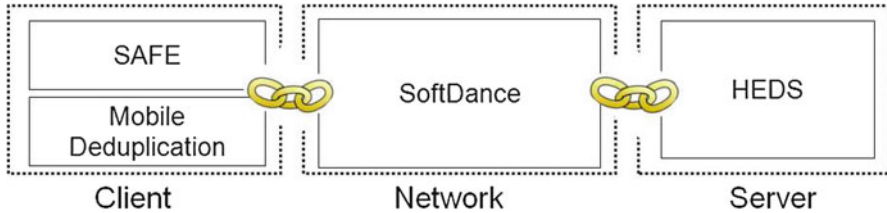


Fig. 1.5 Components developed for deduplication framework

should be fast and have low overhead considering the low capacity of most clients. The network component removes redundant transfers from different clients. In this component, the RE devices intercept data packets and eliminate redundant data. RE devices are dynamically controlled by software-defined network (SDN) controllers. This component should be fast when analysing large numbers of packets and be scalable to a large number of RE devices. Finally, the server component removes redundancies from different networks. This component should provide high space savings. Thus, fine-grained deduplication and fast responses are fundamental functions.

This book discusses practical implementations of the components of a deduplication framework (Fig. 1.5). For the server component, a Hybrid Email Deduplication System (HEDS) is presented. The HEDS achieves a balanced trade-off between space savings and overhead for email systems. For the client component, Structure-Aware File and Email Deduplication for Cloud-based Storage Systems (SAFE) is shown. The SAFE is fast and provides high storage space savings through structure-based granularity. For the network component, Software-Defined Deduplication as a Network and Storage Service (SoftDance) is presented. SoftDance is an in-network deduplication approach that chains storage data deduplication and redundancy elimination functions using SDN and achieves both storage space and network bandwidth savings with low processing time and memory overhead. Mobile deduplication is a client component that removes redundancies of popular files like images and video files on mobile devices.

1.6 Redundant Array of Inexpensive Disks

The RAID was proposed to increase storage access performance using disk arrays. We show three types of RAID, RAID 0, RAID 1 and RAID 5, that are widely used to increase read and write performance or fault tolerance by redundancy. RAID 0 divides a file into blocks that are evenly striped into disks. Figure 1.6 illustrates how RAID 0 works. Suppose we have four blocks, 1, 2, 3, and 4. Logically the four blocks are identified as being in the same logical disk, but physically the blocks are separated (striped) into two physical disks. Blocks 1 and 3 are saved to the

Fig. 1.6 RAID 0 (striping)

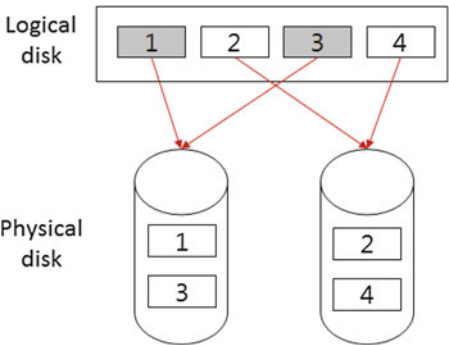
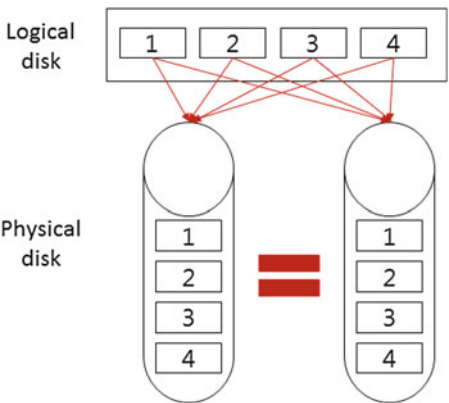


Fig. 1.7 RAID 1 (mirroring)



left disk, while blocks 2 and 4 are saved to the right disk. Because of independent parallel access to blocks on different disks, RAID 0 increases the read performance on the disks. RAID 0 could also make a large logical disk with small physical disks. However, the failure of a disk results in the loss of all data.

RAID 1 focuses on fault tolerance by mirroring blocks between disks (Fig. 1.7). The left and right blocks have the same blocks (blocks 1, 2, 3, and 4). Even if one disk fails, RAID 1 can recover the lost data using blocks on the other disk. RAID 1 increases read performance owing to parallel access but decreases write performance owing to the creation of duplicates. RAID 5 uses block-level striping with distributed parity. As shown in Fig. 1.8, each disk contains a parity representing blocks: for example, C_p is a parity for C_1 and C_2 . RAID 5 requires at least three disks. RAID 5 increases read and write performance and fault tolerance.

1.7 Direct-Attached Storage

The first data storage is called direct-attached storage (DAS), where a storage device, like a hard disk, is attached to a computer through a parallel or serial data cable (Fig. 1.9). A computer has slots where the cables for multiple hard disks can

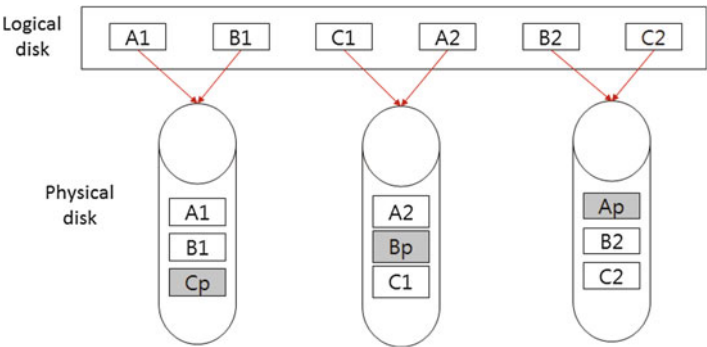
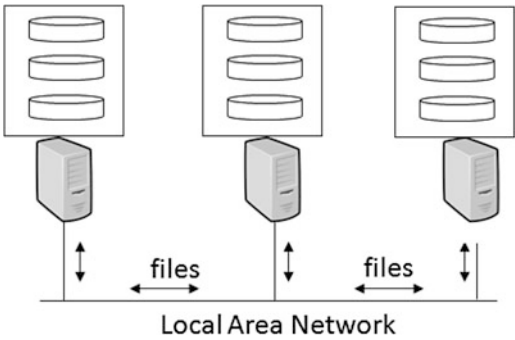


Fig. 1.8 RAID 5 (Block-level striping with distributed parity)

Fig. 1.9 Direct-attached storage (DAS)



be inserted. DAS is mainly used to run applications on a computer. The first DAS interface standard was called Parallel Advanced Technology Attachment (PATA) and is used for hard disk drives, optical disk drives and floppy disk drives. In PATA, data are transferred from/to a storage device through a 16-bit wide cable. Figure 1.10 shows a PATA data cable. The PATA cable supports various data rates, including 16, 33, 66, 100 and 133 MB/s.

PATA was replaced by Serial ATA (SATA) (Fig. 1.11), which has faster speeds – 150, 300, 600 and 1900 MB/s – than PATA. SATA uses a serial cable (Fig. 1.13). Figure 1.12 shows a power cable adapter for a SATA cable. Hard disks that support SATA provide a 7-pin data cable connector and a 15-pin power cable connector (Fig. 1.13).

1.8 Storage Area Network

A storage area network (SAN) allows multiple computers to share disk arrays through a dedicated network. While DAS is a one-to-one mapping between a computer and storage devices on a computer, a SAN is a many-to-many mapping

Fig. 1.10 Parallel Advanced Technology Attachment (PATA) data cable

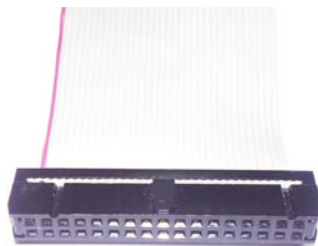


Fig. 1.11 SATA (Serial ATA) data cable



Fig. 1.12 Serial Advanced Technology Attachment (SATA) power cable



Fig. 1.13 SATA connectors: 7-pin data and 15-pin power connectors



between computers and storage devices located in a dedicated place. A computer normally refers to an application server that runs specific applications such as email, a file server or a Web server. Servers send (or save) data to storage through a dedicated network that is used to store data but not deliver application data. As shown in Fig. 1.14, client messages are transferred through a local area network (LAN), and disk input/output (I/O) messages are transferred through a SAN. The unit of data delivered through a SAN is a block rather than a file. Application servers

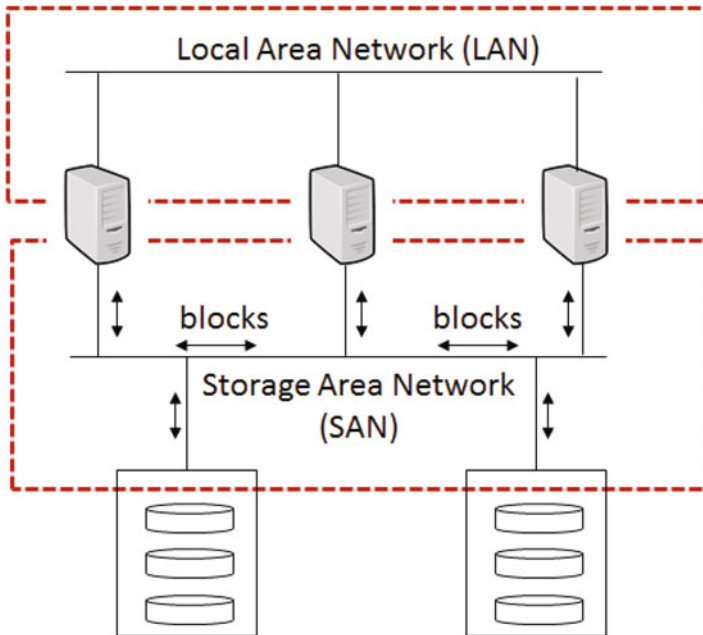


Fig. 1.14 Storage area network

send blocks (rather than files) to storage, and each storage device is shown to the application servers as if the storage were a hard disk drive like DAS.

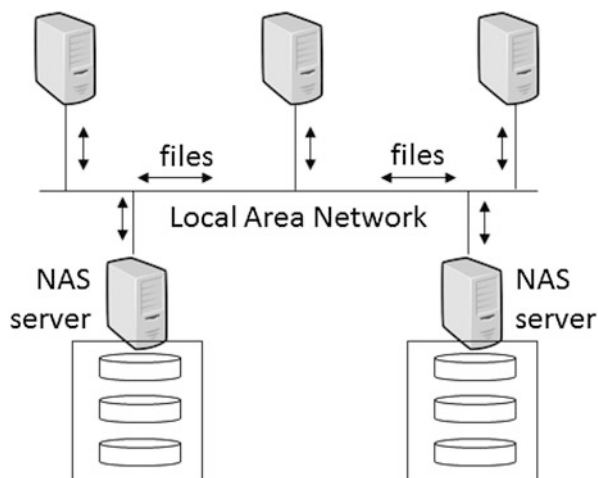
A SAN has two main attributes. One is availability, the other is scalability. Storage data should be recoverable after a failure without having to stop applications. Also, as the number of disks increases, performance should increase linearly (or more). SAN protocols include Fibre Channel (FC), Internet Small Computer System Interface (iSCSI), and ATA over Ethernet (AoE).

1.9 Network-Attached Storage

Network-attached storage (NAS) refers to a computer that serves as a remote file server. While a SAN delivers blocks through a dedicated network, NAS, with disk arrays, receives files through a LAN, through which application data flow. As shown in Fig. 1.15, application servers send files to NAS servers that subsequently save the received files to disk arrays. NAS uses file-based protocols such as Network File System (NFS), Common Internet File System (CIFS), and Andrew File System (AFS).

NAS is used in enterprise and home networks. In home networks, NAS is mainly used to save multimedia files or as a backup system of files. The NAS server supports

Fig. 1.15 Network-attached storage



a browser-based configuration and management based on an IP address. As more capacity is needed, NAS servers support clustering and provide extra capacity by collaborating with cloud storage providers.

1.10 Comparison of DAS, NAS and SAN

The three types of storage system DAS, NAS, and SAN have different characteristics (Table 1.1). Data storage in DAS is owned by individual computers, but in NAS and SAN it is shared by multiple computers. Data in DAS are transferred to data storage directly through I/O cables, but data using NAS and SAN should be transferred through a LAN for NAS and a fast storage area network for SAN. Data units to be transferred to storage are sectors on hard disks for DAS, files for NAS and blocks for SAN. DAS is limited in terms of the number of disks owing to the space on the computer and operators need to manage data storage independently on each computer. By contrast, SAN and NAS can have centralized management tools and can increase the size of data storage easily by just adding storage devices.

1.11 Storage Virtualization

Storage virtualization is the separation of logical storage and physical storage. A hard disk (physical storage) can be partitioned into multiple logical disks. The opposite case also applies: multiple physical hard disks can be combined into a logical disk. Storage virtualization hides physical storage from applications and presents a logical view of storage resources to the applications. Virtualized storage

Table 1.1 Comparison of DAS, NAS and SAN

	DAS	NAS	SAN
Shared(?)	Individual	Shared	Shared
Network	Not required	Local area network	Storage area network
Protocols	PATA, SATA	NFS, CIFS, AFS	Fibre Channel, iSCSI, AoE
Data unit	Sector	File	Block
Capacity	Low	Moderate/High	High
Complexity	Easy	Moderate	Difficult
Management	High	Moderate	Low

has a common name, where the physical storage can be complex with multiple networks. Storage virtualization has multiple benefits, as follows:

- **Fast provisioning:** available free storage space is found rapidly by storage virtualization. By contrast, without storage virtualization, operators should find the available storage that encompasses enough space for the requested applications.
- **Consolidation:** without storage virtualization, some spaces in individual storage can be wasted because the remaining spaces are insufficient for applications. However, storage virtualization combines the multiple remaining spaces that are created as a logical storage space. Thus, spaces are efficiently utilized.
- **Reduction of management costs:** the number of operators that assign storage space for requested applications is reduced.

Software-defined storage (SDS) [1] has emerged as a form of software-based storage virtualization. SDS separates storage hardware from software and controls physically disparate data storage devices that are made by different storage companies or that represent different storage types, such as a single disk or disk arrays. SDS is an important component of a software-defined data centre (SDDC) along with software-defined compute and software-defined networks (SDN).

Figure 1.16 shows the components of SDS that are recommended by Storage Networking Industry Association (SNIA) [1]. SDS aggregates storage resources into pools. Data services, including provisioning, data protection, data availability, data performance and data security, are applied to meet storage service requirements. These services are provided to storage administrators through a SDS application program interface (API). SDS is located in a virtualized data path between physical storage devices and application servers to handle files, blocks and objects. SDS interacts with physical storage devices including flash drives, hard disks or the disk arrays of hard disks through a storage management interface like SMI-S. Software developers and deployers access SDS through a data management interface like Cloud Data Management Interface (CDMI). In short, SDS enables software-based control over different types of disks.

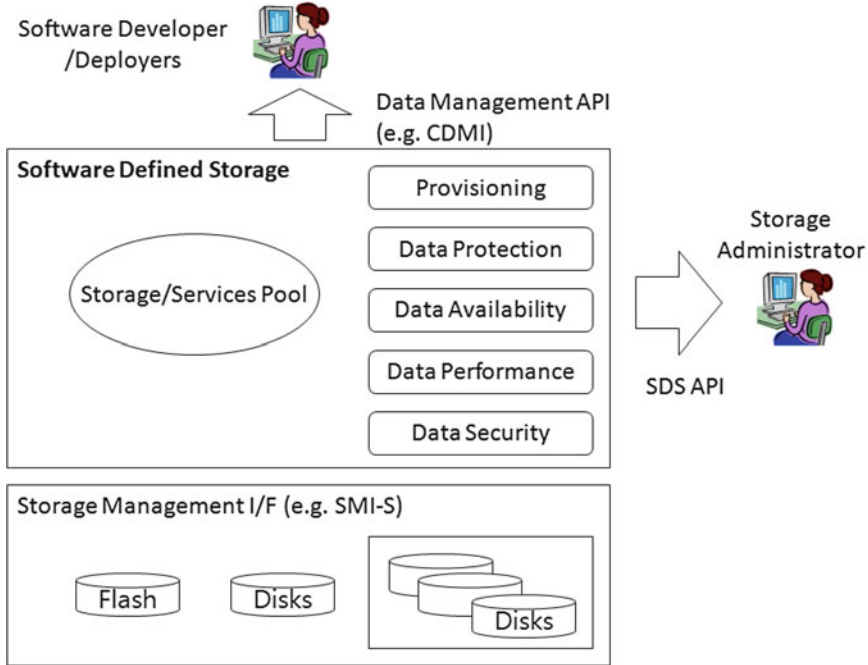


Fig. 1.16 Big picture of SDS [1]

1.12 In-Memory Storage

In-memory storage or in-memory database (IMDB) has been developed to cope with the fast saving and retrieving of data to/from databases. Traditionally a database resides on a hard disk, and access to the disk is constrained by the mechanical movement of the disk head. Using a solid-state disk (SSD) or memory rather than disk as a storage device will result in an increase in the speed of data write and read. The explosive growth of big data requires fast data processing in memory. Thus, IMDB is becoming popular for real-time big data analysis applications.

In-memory data grids (IMDGs) extend IMDBs in terms of scalability. IMDG is similar to IMDB in that it stores data in main memory, but it is different in that (1) data are distributed and stored in multiple servers, (2) data are usually object-oriented and non-relational, and (3) servers can be added and removed often in IMDGs. There are open source and commercial IMDG products, such as Hazelcast [4], Oracle Coherence [12], VMWare Gemfire [20] and IBM eXtreme Scale [5]. IMDG provides horizontal scalability using a distributed architecture and resolves the issue of reliability through a replication system. IMDG uses the concept of in-memory key value to store and retrieve data (or objects).

1.13 Object-Oriented Storage

Object-oriented storage saves data as objects, whereas block-based storage stores data as fixed-size blocks. Object storage abstracts lower layers of storage, and data are managed objects instead of files or blocks. Object storage provides addressing and identification of individual objects rather than file name and path. Object storage separates metadata and data, and applications access objects through an application program interface (API), for example, RESTful API. In object storage, administrators do not have to create and manage logical volumes to use disk capacity.

Lustre [8] is a parallel distributed file system using object storage. Lustre consists of compute nodes (Lustre clients), Lustre object storage servers (OSSs), Lustre object storage targets (OSTs), Lustre metadata servers (MDSs) and Lustre metadata targets (MDTs). A MDS manages metadata such as file names and directories. A MDT is a block device where metadata are stored. An OSS handles I/O requests for file data, and an OST is a block device where file data are stored. OpenStack Swift [11] is object-based cloud storage that is a distributed and consistent object/blob store. Swift creates and retrieves objects and metadata using the Object Storage RESTful API. This RESTful API makes it easier for clients to integrate Swift service into client applications. With the API, the resource path is defined based on a format such as `/v1/{account}/{container}/{object}`. Then the object can be retrieved at a URL like the following: `http://server/v1/{account}/{container}/{object}`.

1.14 Standards and Efforts to Develop Data Storage Systems

In this section, we discuss the efforts made and standards developed in the evolution of data storage. We start from a SATA and RAID. Then we explain a FC standard (FC encapsulation), iSCSI and Internet Fibre Channel Protocol (iFCP) for a SAN, and a NFS for NAS. We end by explaining the content deduplication standard and Cloud data management interface.

SATA [14, 15] is a popular storage interface. The fastest speed of SATA is currently 16 Gb/s, as described in the SATA revision 3.2 specification [15]. SATA replaced PATA and achieves a higher throughput and reduced cable width than PATA (33–133 MB/s). SATA revision 3.0 [14] (for 6 Gb/s speed) gives various benefits compared to PATA. SATA 6 Gb/s can operate at over 580 MB/s by increasing data transfer speeds from a cache on a hard disk, which does not incur rotational delay. SATA revision 3.2 [15] contains new features, including SATA express, new form factors, power management enhancement and enhancement of solid-state hybrid drives. SATA express enables SATA and PCIe interfaces to coexist. It contains the M.2 form factor used in tablets and notebooks and minimizes energy use. This SATA revision complies with specifications for a solid-state hybrid drive (SSHD).

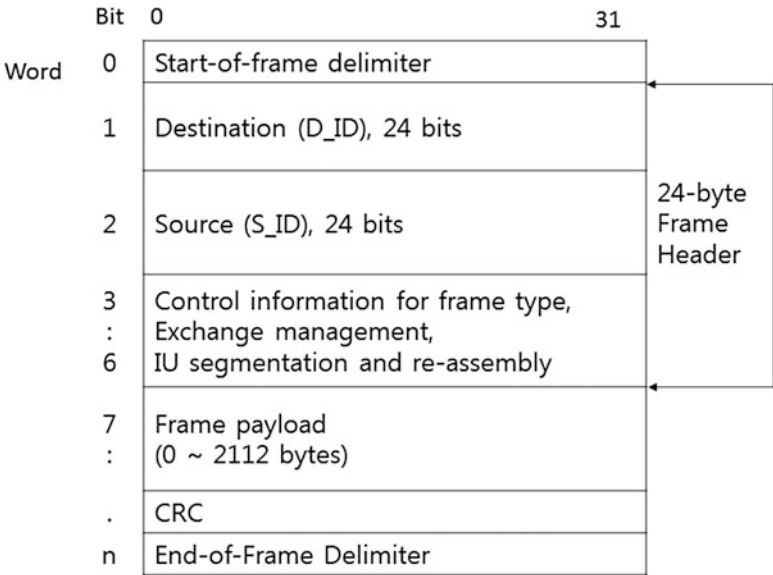


Fig. 1.17 Fibre Channel frame format

Patterson et al. [13] proposed a method, called RAID, to improve I/O performance by clustering inexpensive disks; this represents an alternative to single large expensive magnetic disks (SLEDs). Each disk in a RAID has a short mean time to failure (MTTF) compared to high-performance SLEDs. The paper focuses on the reliability and price performance of disk arrays, which shortens the mean time to repair (MTTR) due to disk failure by having redundant disks. When a disk fails, another disk replaces it. RAID 1 mirrors disks that duplicate all disks. RAID 2 uses hamming code to check and correct errors, where data are interleaved across disks and a sufficient number of check disks are used to identify errors. RAID 3 uses only one check disk. RAID 4 saves a data unit to a single sector, improving the performance of small transfers owing to parallelism. RAID 5 does not use separate check disks but distributes parity bits to all disks.

RFC 3643 [21] defines a common FC frame encapsulation format and usage of the format in data transfers on an IP network. Figure 1.17 illustrates the FC frame format. A frame consists of a 24-byte frame header, a frame payload that can be up to 2112 bytes, and cyclic redundancy check (CRC), along with a start-of-frame delimiter and end-of-frame delimiter. A FC has five layers: FC-0, FC-1, FC-2, FC-3 and FC-4. FC-0 defines the interface of the physical medium. FC-1 shows the encoding and decoding of data. FC-2 specifies the transfer of frames and sequences. FC-3 indicates common services, and FC-4 represents application protocols. The FC address is 24 bits and consists of a domain ID (7 bits), area ID (7 bits) and port ID (9 bits). A FC address is acquired when the channel device is loaded, and the domain ID ranges from 1 to 239.

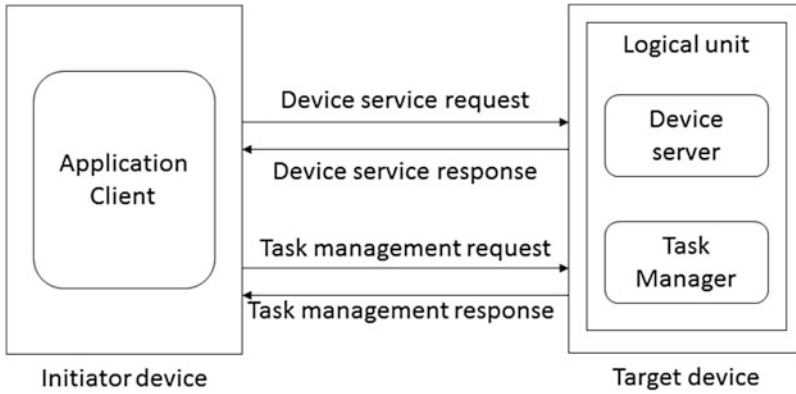


Fig. 1.18 Small Computer System Interface architecture: interaction between client and server: cited from [19]

The SCSI architecture [19] is an interface for saving data to I/O devices and is defined in ANSI INCITS 366-2003 and ISO/IEC 14776-412. As shown in Fig. 1.18, the application client within an initiator device (like a device driver) sends the SCSI commands to the device server in a logical unit located in a target device. The device server processes the SCSI commands and returns a response to the initiated client. The task manager receives and processes the task management requests, responding to the client as well. An application client sends requests by a remote procedure with input parameters, including command descriptor blocks (CDBs). CDBs are command parameters that define the operations to be performed by the device server.

The iSCSI architecture is defined in RFC 7143 [2], where the SCSI runs through the TCP connections on an IP network. This allows an application client in an initiator device to send commands and data to a device server on a remote target device on a LAN, WAN, or the Internet. iSCSI is a protocol of a SAN but runs on an IP network without the need for special cables like FC. The application client communicates with the device server through a session that consists of one or more TCP connections. A session has a session ID. Likewise, each connection in the session has a connection ID. Commands are numbered in a session and are ordered over multiple connections in the session.

RFC 4172 [10] defines the iFCP that allows FC devices to communicate through TCP connections on an IP network. That is, IP components replace the FC switching and routing infrastructure. Figure 1.19 shows how iFCP works on an IP network. In the figure, N_PORT is the end point for the FC traffic, the FC device is the FC device that is connected to the N_PORT, and the Fabric port is the interface within a FC network that is attached to the end point (N_PORT) for FC traffic. FC frames are encapsulated in a TCP segment by the iFCP layer and routed to a destination through the IP network. On receiving FC frames from the IP network, the iFCP layer de-encapsulates and delivers the frames to the appropriate end point for FC traffic, N_PORT.

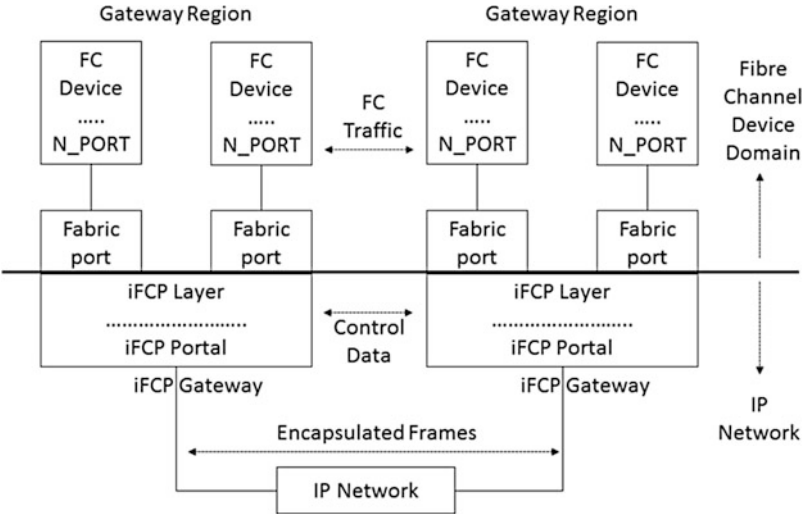


Fig. 1.19 Internet Fibre Channel Protocol (iFCP): cited from RFC 4172 [10]

The NFS that is defined in RFC 7530 [16] is a distributed file system, which is widely used in NAS. NFS is based on the Open Network Computing (ONC) Remote Procedure Call (RPC) (RFC 1831) [18]. The “Network File System (NFS) Version 4 External Data Representation Standard (XDR) Description” (RFC 7531) [3] defines XDR structures used by NFS version 4. NFS consists of a NFS server and NFS client: the NFS server runs a daemon on a remote server where a file is located and the NFS client accesses the file on the remote server using RPC. NFS provides the same operations on the remote files as those on the local files. When an application needs a remote file, the application opens a remote file to obtain access, reads data from the file, writes data to the file, seeks specified data in the file and closes the file when the application finishes. NFS is different from a file transfer service because the application does not retrieve and store the entire file but rather transfers small blocks of data at a time.

Jin et al. [7] report an effort on content deduplication for content delivery network interconnection (CDNi) optimization. CDN caches the duplicate contents multiple times, increasing storage size, and the duplicate contents are delivered through the CDN, decreasing available network bandwidth. This effort focuses on the elimination or reduction of duplicate contents in the content delivery network (CDN). A typical example of duplicate contents is data backup and recovery through a network. The main case of redundancy in a CDN is where a downstream CDN caches the same content copy multiple times from a content service provider (CSP) or upstream CDN (uCDN) owing to the different URLs for the same content. In short, using URLs is not enough to find identical contents and ultimately to remove duplicate content. The authors propose a feasible solution whereby content can be named using a content identifier and resource identifiers because the content can be

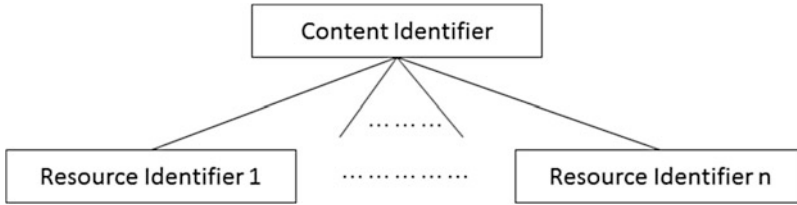


Fig. 1.20 Content delivery network interconnection (CDNi): content naming mechanism, cited from [7]

located in multiple locations. Figure 1.20 shows the relationship between a content identifier and resource identifiers. The content identifier should be globally unique.

1.15 Summary and Organization

In this chapter, we have presented a deduplication framework that consists of a client, server and network components. We also illustrated the evolution of data storage systems. Data storage has evolved from a single hard disk attached to a single computer by DAS. As the amount of data increases and large amounts of storage are required for multiple computers, storage is located in different places where data are shared from multiple computers (including application servers) by a SAN or NAS. To increase the read or write performance and fault tolerance, RAIDs are used with different levels of services, including striping, mirroring or striping with distributed parity. SDS, which is a critical component of a SDDC, consolidates and virtualizes disparate data storage devices using storage/service pools, data service, SDS API and data management API.

This book follows the order of components that we developed for the deduplication framework. We provide background information on how deduplication works and discuss existing deduplication studies in Chap. 2. After that, we elaborate on each component for the deduplication framework one by one. In Chaps. 3 and 4, we present a server component and a client component: Hybrid Email Deduplication System (HEDS) and Structure-Aware File and Email Deduplication for Cloud-based Storage Systems (SAFE) respectively. In Chap. 5, we elaborate on how deduplication can be used for networks and storage to reduce data volumes using Software-defined Deduplication as a Network and Storage Service, or SoftDance. We present our on-going project, mobile deduplication, in Chap. 6. Chapter 7 concludes the book.

References

1. Carson, M., Yoder, A., Schoeb, L., Deel, D., Pratt, C.: Software defined storage. <http://www.snia.org/sites/default/files/SNIA%20Software%20Defined%20Storage%20White%20Paper-%20v1.0k-DRAFT.pdf> (2014)
2. Chadalapaka, M., Satran, J., Meth, K., Black, D.: Internet Small Computer System Interface (iSCSI) Protocol (Consolidated). <http://www.rfc-editor.org/info/rfc7143> (2014)
3. Haynes, T., Noveck, D., Primary Data: Network File System (NFS) Version 4, External Data Representation Standard (XDR) Description. <https://tools.ietf.org/html/rfc7531> (2015)
4. Hazelcast.org: Hazelcast. <http://hazelcast.org/> (2016)
5. IBM: eXtremeScale. <http://www-03.ibm.com/software/products/en/websphere-extreme-scale> (2016)
6. IDC: The digital universe in 2020. <https://www.emc.com/collateral/analyst-reports/idc-the-digital-universe-in-2020.pdf> (2012)
7. Jin, W., Li, M., Khasnabish, B.: Content De-duplication for CDNi Optimization. <https://tools.ietf.org/html/draft-jin-cdni-content-deduplication-optimization-04> (2013)
8. lustre.org: Lustre. <http://lustre.org/> (2016)
9. Meyer, D.T., Bolosky, W.J.: A study of practical deduplication. In: Proceeding of the USENIX Conference on File and Storage Technologies (FAST) (2011)
10. Monia, C., Mullendore, R., Travostino, F., Jeong, W., Edwards, M.: iFCP - A Protocol for Internet Fibre Channel Storage Networking. <http://www.rfc-editor.org/info/rfc4172> (2005)
11. openstack.org: OpenStack Swift. <http://www.openstack.org/software/releases/liberty/components/swift> (2016)
12. Oracle: Coherence. <http://www.oracle.com/technetwork/middleware/coherence/overview/index.html> (2016)
13. Patterson, D.A., Gibson, G., Katz, R.H.: A case for redundant arrays of inexpensive disks (raid). In: Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data, SIGMOD '88 (1988)
14. Serial_ATA: Fast Just Got Faster: SATA 6Gb/s. https://www.sata-io.org/system/files/member-downloads/SATA-6Gbs-Fast-Just-Got-Faster_2.pdf (2009)
15. Serial_ATA: SATA revision 3.2 specification. https://www.sata-io.org/sites/default/files/documents/SATA_v3%202020_PR_Final_BusinessWire_8.20.13.pdf (2013)
16. Shepler, S., Callaghan, B., Robinson, D., Thurlow, R., Beame, C., Eisler, M., Noveck, D.: Network File System (NFS) Version 4 Protocol. <http://www.rfc-editor.org/info/rfc7530> (2015)
17. Spring, N.T., Wetherall, D.: A protocol-independent technique for eliminating redundant network traffic. In: Proceedings of the ACM SIGCOMM 2000 conference on Data communication (2000)
18. Srinivasan, R.: RPC: Remote Procedure Call Protocol Specification Version 2. <https://tools.ietf.org/html/rfc1831> (1995)
19. T10, I.T.C.: SCSI Architecture Model-2 (SAM-2). ANSI INCITS 366-2003, ISO/IEC 14776-412 (2003)
20. VMWare: Gemfire. <https://www.vmware.com/support/pubs/vfabric-gemfire.html> (2016)
21. Weber, R., Rajagopal, M., Travostino, F., O'Donnell, M., Monia, C., Merhar, M.: Fibre Channel (FC) Frame Encapsulation. <http://www.rfc-editor.org/info/rfc3643> (2003)

Chapter 2

Existing Deduplication Techniques

Abstract Though various deduplication techniques have been proposed and used, no single best solution has been developed to handle all types of redundancies. Considering performance and overhead, each deduplication technique has been developed with different designs considering the characteristics of data sets, system capacity and deduplication time. For example, if the data sets to be handled have many duplicate files, deduplication can compare files themselves without looking at the file content for faster running time. However, if data sets have similar files rather than identical files, deduplication should look inside the files to check what parts of the contents are the same as previously saved data for better storage space savings. Also, deduplication should consider different designs of system capacity. High-capacity servers can handle considerable overhead for deduplication, but low-capacity clients should have lightweight deduplication designs for fast performance. Studies have been conducted to reduce redundancies at routers (or switches) within a network. This approach requires the fast processing of data packets at the routers, which is of crucial necessity for Internet service providers (ISPs). Meanwhile, if a system removes redundancies directly in a write path within a confined storage space, it is better to eliminate redundant data before storage. On the other hand, if a system has residual (or idle) time or enough space to store data temporarily, deduplication can be performed after the data are placed in temporary storage. In this chapter, we classify existing deduplication techniques based on granularity, place of deduplication and deduplication time. We start by explaining how to efficiently detect redundancy using chunk index caches and bloom filters. Then we describe how each deduplication technique works along with existing approaches and elaborate on commercially and academically existing deduplication solutions. All implementation codes are tested and run on Ubuntu 12.04 precise.

2.1 Deduplication Techniques Classification

Deduplication can be divided based on granularity (the unit of compared data), deduplication place, and deduplication time (Table 2.1). The main components of these three classification criteria are chunking, hashing and indexing. Chunking is a process that generates the unit of compared data, called a chunk. To compare

Table 2.1 Deduplication classification

Methods based on granularity	Place	Time
File-level deduplication	Server-based deduplication	Inline deduplication
Fixed-size block deduplication	Client-based deduplication	Offline deduplication
Variable-sized block deduplication	Redundancy elimination (end-to-end RE, network-wide RE)	

duplicate chunks, hash keys of chunks are computed and compared, and a hash key is saved as an index for future comparison with other chunks.

Deduplication is classified based on granularity. The unit of compared data can be at the file level or subfile level, which are further subdivided into fixed-size blocks, variable-sized chunks, packet payload or byte streams in a packet payload. The smaller the granularity used, the larger number of indexes created, but the more redundant data are detected and removed.

For place of deduplication, deduplication is divided into server-based and client-based deduplication for end-to-end systems. Server-based deduplication traditionally runs on high-capacity servers, whereas client-based deduplication runs on clients that normally have limited capacity. Deduplication can occur on the network side; this is known as *redundancy elimination* (RE). The main goal of RE techniques is to save bandwidth and reduce latency by reducing repeating transfers through the network links. RE is further subdivided into end-to-end RE, where deduplication runs at end points on a network, and network-wide RE (or in-network deduplication), where deduplication runs on network routers.

In terms of deduplication time, deduplication is divided into inline and offline deduplication. With inline deduplication, deduplication is performed before data are stored on disks, whereas offline deduplication involving performing deduplication after data are stored. Thus, inline deduplication does not require extra storage space but incurs latency overhead within a write path. Conversely, offline deduplication does not have latency overhead but requires extra storage space and more disk bandwidth because data saved in temporary storage are loaded for deduplication and deduplicated chunks are saved again to more permanent storage. Inline deduplication mainly focuses on latency-sensitive primary workloads, whereas offline deduplication concentrates on throughput-sensitive secondary workloads. Thus, inline deduplication studies tend to show trade-offs between storage space savings and fast running time.

First we explain chunk index caches and bloom filters that are used to identify redundant data based on indexes and small arrays, respectively. We then go into detail about classified deduplication techniques, discussing each one by one, in the order of granularity, place and time. Note that a deduplication technique can belong to multiple categories, such as a combination of variable-sized block deduplication, server-based deduplication and inline deduplication.

2.2 Common Modules

2.2.1 *Chunk Index Cache*

Deduplication aims to find as many redundancies as possible while maintaining processing time. To reduce processing time, one typical technique is to check indexes of data in memory before accessing disks. If the data indexes are the same, deduplication does not involve accessing the disks where the indexes are stored, which would reduce processing time. An index represents essential metadata that are used to compare data (or chunks). In this section, we show what can be indexed and how indexes are computed, stored and used for comparisons.

2.2.1.1 Fundamentals

To compare redundant data, deduplication involves the computation of data indexes. Thus, an index should be unique for all data with different content. To ensure the uniqueness of an index, one-way hash functions, such as message digest 5 (MD5), secure hash algorithm 1 (SHA-1), or secure hash algorithm 2 (SHA-2) are used. These hash functions should not create the same index for different data. In other words, an index is normally considered a hash key that represents data. Indexes should be saved to permanent storage devices like a hard disk, but to speed up the comparison of indexes, they are prefetched in memory. The indexes in memory should provide temporal locality to reduce the number of evictions of indexes from memory owing to filled memory as well as a decrease in the number of prefetches. In the same sense, to prefetch related indexes, the indexes should be grouped by spatial locality. That is, indexes of similar data are stored close to each other in storage.

An index table is a place where indexes are temporarily located for fast comparison. Such tables can be deployed using many different methods, but mainly they are built using hash tables, which allows comparisons to be made very quickly due to the time complexity of $O(1)$ with the overhead of hash table size. In the next section, we present a simple implementation of an index table using an `unordered_map` container.

2.2.1.2 Implementation: Hash Computation

We show an implementation of an index computation using an SHA-1 hash function. The whole code for this example is in Appendix A. The codes in the appendix are written in C++. The unit of data can be a file or a byte stream data (like chunk). Thus, we show codes to compute a SHA-1 hash key from a file and data. We use the FIPS-180-1-compliant SHA-1 implementation created by Paul Bakker. We developed a wrapper class with two functions, such as `getHashKeyOfFile(string`

filePath) and getHashKey(string data). Following are code snippets that use the two functions.

```
string hashKey;
hashKey = sha1Wrapper.getHashKey(data);
```

```
string hashKey;
hashKey = sha1Wrapper.getHashKeyOfFile(fileName);
```

We provide a main function to test the computation of a hash key and a Makefile to make compilation easy. In the main function, the first paragraph shows how to compute a hash key of a file, and the second paragraph shows how to calculate a hash key of a string block:

```
#ifdef SHA-IWRAPPER_TEST

int main() {

    Sha1Wrapper obj;
    string filePath = "hello.dat";
    string data = "hello_danny_how_are_you_??_";

    string hashKey;

    // get hash key of a file
    hashKey = obj.getHashKeyOfFile(filePath);
    cout << "hashkey_of_" << filePath << "_:_" << hashKey << endl;
    cout << endl;

    // get hash key of data
    cout << data << endl;
    hashKey = obj.getHashKey(data);
    cout << "hashkey_of_data_:_" << hashKey << endl;

    return 0;
}

#endif

## make
all:
    g++ -DSHA1WRAPPER_TEST -o SHA-1 sha1.cc sha1Wrapper.cc

clean:
    rm -f *.o SHA-1
```

We compile and build an executable file to test SHA-1 as follows:

```
root@server:~/lib/SHA-1# make
g++ -DSHA1WRAPPER_TEST -o SHA-1 sha1.cc sha1Wrapper.cc
root@server:~/lib/sha1# ls -l
-rw-r--r-- 1 root root 12 Jul 20 20:28 hello.dat
-rw-r--r-- 1 root root 98 Jul 20 20:28 Makefile
-rwxr-xr-x 1 root root 37383 Jul 20 20:33 SHA-1
```

```

-rw-r--r-- 1 root root 20297 Jul 20 20:28 sha1.cc
-rw-r--r-- 1 root root 4606 Jul 20 20:28 sha1.h
-rw-r--r-- 1 root root 1187 Jul 20 20:28 sha1Wrapper.cc
-rw-r--r-- 1 root root 522 Jul 20 20:28 sha1Wrapper.h

```

Following are the results of running a SHA-1 executable file. We retrieve an index string with 40 characters created from 20 bytes; 1 byte is denoted by two hexadecimals. Thus, the size of the index amounts to 40 bytes (40 characters). The first hash key that starts with 49a32... is computed from a file (here, hello.dat). The second hash key starting with e69927 is computed from a string “hello danny how are you??”:

```

root@server:~/lib/sha1# SHA-1
hashkey of hello.dat : 49a32112d754917ca799d684895c5bbc4e25828b

hello danny how are you ??
hashkey of data : e69927c529b145fa729ae2664c07929853f59994

```

2.2.1.3 Implementation: Index Table

We show an implementation of an index table using an unordered_map. The implementation codes are in Appendix B. We compile and build a cache executable file. To compile using an unordered_map, we need to add ‘-std=c++0x’ at compilation:

```

root@server:~/lib/cache# make
g++ -DCACHE_TEST -o cache cache.cc -std=c++0x

root@server:~/lib/cache# ls -l
total 72
-rwxr-xr-x 1 root root 54227 Jul 20 21:34 cache
-rw-r--r-- 1 root root 2235 Jul 20 20:28 cache.cc
-rw-r--r-- 1 root root 4079 Jul 20 20:28 cache.h
-rw-r--r-- 1 root root 1278 Jul 20 20:28 cacheInterface.h
-rw-r--r-- 1 root root 91 Jul 20 21:34 Makefile

root@server:~/lib/cache# cat Makefile
## make
all:
    g++ -DCACHE_TEST -o cache cache.cc -std=c++0x

clean:
    rm -f *.o cache

```

What follows shows how to test the implementation codes of an index table. First, an index table is created with a pair consisting of a key and a value. ‘cache.empty()’ is used to check whether the index table is empty. To save an index to the table, we use the set() method, for example, ‘cache.set(<key>, <value>)’. To obtain an index from the table, we use ‘cache.get(<key>)’. ‘cache.size()’ retrieves the number of indexes. To check whether an index with a key exists, the ‘cache.exist(<key>)’ function is used:

```

UMapCache<string , string> cache;

string key = "1";
string value = "Danny";
string key2 = "2";
string value2 = "Kim";
string key3 = "3";

// check if cache is empty
cout << "****_current_cache_****" << endl;
if (cache.empty())
    cout << "empty" << endl;
else
    cout << "filled" << endl;
cout << endl;

// save an entry
cout << "****_save_entries_****" << endl;
cout << "<" << key << "," << value << ">" << endl;
cout << "<" << key2 << "," << value2 << ">" << endl;
cache.set(key, value);
cache.set(key2, value2);
cout << endl;

// check if cache is empty
cout << "****_current_cache_****" << endl;
if (cache.empty())
    cout << "empty" << endl;
else
    cout << "filled" << endl;
cout << endl;

// get an entry
cout << "****_get_an_entry_****" << endl;
cout << "key=_ " << key << "_ ";
cout << cache.get(key) << endl;
cout << endl;

// get number of entries
cout << "****_get_number_of_entries_****" << endl;
cout << "size:_ " << cache.size() << endl;
cout << endl;

// check if an entry with key exists
cout << "****_existence_of_a_key_****" << endl;
string tmp = key2;
if (cache.exist(tmp))
    cout << tmp << "_exists" << endl;
else
    cout << tmp << "_doesn't_exist" << endl;
cout << endl;

```

To show all entries, 'cache.showAll()' is used. We determine the size of the index table using various functions, such as 'sizeOfAllEntries()', 'sizeOfAll-

EntriesDouble()', 'sizeOfKeys()', 'sizeOfKeysDouble()' and 'sizeOfValues()'. That is, 'cache.sizeOfAllEntriesDouble()' shows the size of the index table, including all pairs; 'cache.sizeOfKeys()' and 'cache.sizeOfValues()' return the size of keys or values in the index table respectively; 'cache.sizeOfKeysDouble()' and 'cache.sizeOfValuesDouble()' return the size of double data type; and 'cache.removeAll()' removes all indexes in the index table.

```
// show all entries
cout << "****_show_all_entries_****" << endl;
cache.showAll();
cout << endl;

// show size of all entries in bytes
cout << "size_of_all_entries_(bytes):_" << cache.sizeOfAllEntries() << endl;
cout << "size_of_all_entries(double_value)_(bytes):_" << cache.sizeOfAllEntriesDouble() << endl;

// show size of keys of all entries in bytes
cout << "size_of_keys_(bytes):_" << cache.sizeOfKeys() << endl;
cout << "size_of_keys(double_value)(bytes):_" << cache.sizeOfKeysDouble() << endl;

// show size of values of all entries in bytes
cout << "size_of_values_(bytes):_" << cache.sizeOfValues() << endl;
cout << endl;

// remove all entries
cout << "****_remove_all_entries_****" << endl;
cache.removeAll();
cout << "size_of_all_entries:_" << cache.sizeOfAllEntries() << endl;
cout << endl;
```

The following code shows the results of running a 'cache' executable file. In the following result, for the first time the index table is empty, and then two entries are saved. The keys are '1' and '2', and the values are 'Danny' and 'Kim'. There are two entries. Keys occupy 2 bytes of two characters, and values have 8 bytes of eight characters.

```
root@server:~/lib/cache# cache
**** current cache ****
empty

**** save entries ****
<1,Danny>
<2,Kim>

**** current cache ****
filled
```

```

**** get an entry ****
key = 1 Danny

**** get number of entries ****
size : 2

**** existence of a key ****
2 exists

**** show all entries ****
1, Danny
2, Kim

size of all entries (bytes) : 10
size of all entries(double value) (bytes) : 10
size of keys (bytes) : 2
size of keys(double value)(bytes) : 2
size of values (bytes) : 8

**** remove all entries ****
size of all entries : 0

```

2.2.2 Bloom Filter

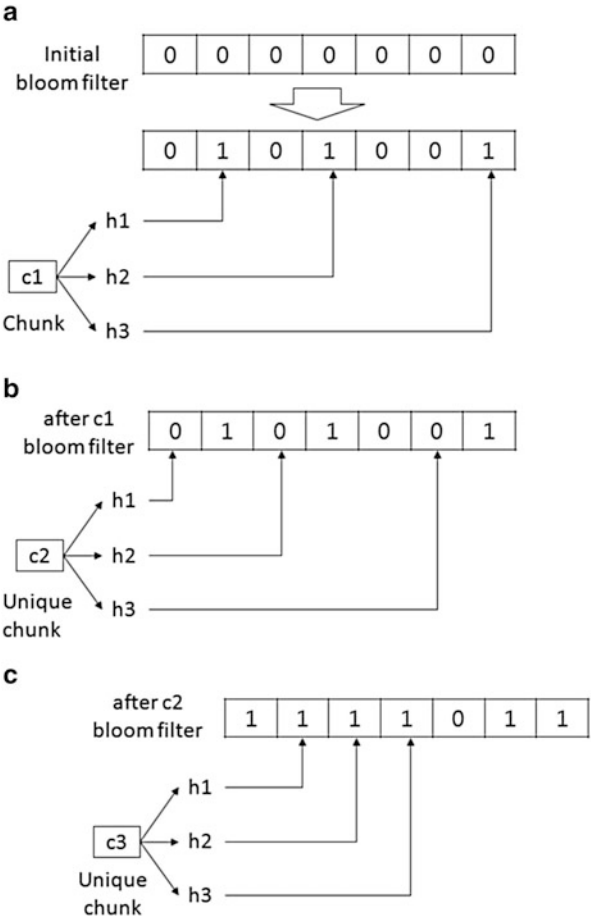
To prevent an index table occupying memory as the number of indexes grows in the index table, a small summary vector, called a Bloom filter, is used to quickly check whether data are unique using small sized metadata. In this section, we see how Bloom filter codes are implemented.

2.2.2.1 Fundamentals

A Bloom filter is used to see whether duplicate chunks of data exist in storage. The Bloom filter is a bit array of m bits initially set to 0. Given a set U , each element u ($u \in U$) of the set is hashed using k hash functions $h_1, \dots, h_k$. Each hash function $h_i(u)$ returns an array index in the bit array that ranges from 0 to $m-1$. Subsequently, a bit of the index is set to 1. This Bloom filter is used to check whether an element was already saved to a set. When an element attempts to be added to a set, if one of the bits corresponding to the return values of hash functions $h_1, \dots, h_k$ is 0, then the element is considered a new one in the set. If bits corresponding to return values of hash functions are all 1, the element is considered to exist in the set.

Let us explain how the Bloom filter works in an example as shown in Fig. 2.1. The Bloom filter initially has all 0 bits. When a chunk $c1$ is saved, the array indexes of the Bloom filter are computed using three different hash functions ($h1, h2, h3$). Here, $h1, h2$ and $h3$ functions return the second, fourth and seventh indexes respectively.

Fig. 2.1 How the Bloom filter works. **(a)** Bloom filter after *c1* chunk is saved. **(b)** Bloom filter when *c2*, a unique chunk, is compared. **(c)** Bloom filter when *c3*, a unique chunk, is compared (false positive). A unique chunk is found to be redundant



Subsequently, the indexes of the Bloom filter are set to 1. Suppose the same chunk *c1* is saved again. The chunk is found to be redundant because all three indexes by hash functions are already set to 1. As shown in Fig. 2.1b, when a unique chunk (*c2*) is saved, indexes by three hash functions are computed again. Now, the elements of the three indexes are all 0. Thus, a chunk *c2* is determined to be unique. However, in Fig. 2.1c, the Bloom filter can have a false positive, that is, the Bloom filter says that a chunk is redundant, but the chunk is actually unique. The array indexes for *c3* are the second, third and fourth indexes, which were set by other chunks. In this case, we will lose a unique chunk without saving it. Thus, the Bloom filter guarantees that a chunk is unique with one 0 index, but it does not guarantee that a chunk is redundant with all three 1 indexes. Thus, in this case, the chunk index cache should be checked after the Bloom filter is used.

2.2.2.2 Implementation

We show the implementation of a Bloom filter using four hash functions. The size of a Bloom filter bit array is calculated based on the SHA-1 hash key. The first step in implementing a Bloom filter is to determine the size of the Bloom filter bit array. Considering a 2 % false positive, we calculate the size of the Bloom filter bit array as shown in [50]. m is the number of bits in a Bloom filter array, n is the number of bits of a fingerprint (which means a hash key in this case), and k is the number of hash functions. To achieve a 2 % false positive, the smallest size of the Bloom filter is $m = 8 * n$ bits ($m/n = 8$), and the number of hash functions is four. Thus, we compute the size of the Bloom filter bit array (m) to 1280 bits as follows. We choose 1283 rather than 1280 for the size of the Bloom filter bit array because the prime number shows a good uniform distribution, reducing the primary cluster as shown in Weiss' book [47] when the mod() function is used for the hash function:

```
n = 160 bits (SHA-1 hash key)
m = 8 * 160 = 1280 bits
k = 4 (four hash functions)
```

The codes are compiled and built by typing 'make', and bf is the executable file used to test the Bloom filter codes:

```
root@server:~/bf# make
gcc -DBF_TEST -DDEBUG -o bf bf.c

root@server:~/bf# ls -l
-rwxr-xr-x 1 root root 12861 Jul 21 08:59 bf
-rw-r--r-- 1 root root 5627 Jul 21 00:39 bf.c
-rw-r--r-- 1 root root 1585 Jul 21 00:38 bf.h
-rw-r--r-- 1 root root 51 Jul 21 08:59 Makefile
```

Following are the results of the testing program of the Bloom filter codes. First, we assign two fingerprints (which are considered to be hash keys). The Bloom filter is initialized with a bit array with all 0s in each bit. We use 11 bits for the Bloom filter; readers can extend the size if needed. When data with the first hash key (fingerprint1) are saved, deduplication checks the Bloom filter. The bit indexes that four hash functions compute are 2, 4, 7 and 8. Please note that the bit index starts at 0. The data are found to be unique because 0 is found among the bit values. Then the values of the bit indexes (2, 4, 7, 8) are changed to 1, and the Bloom filter has 00101001100 bit arrays:

```
root@server:~/bf# bf
#####
# Test : Input Data          ##
#####
[bloom filter] fingerprint1   : 4543863031426141731
[bloom filter] fingerprint2   : 4543863041425141743

#####
[bloom filter] initialize
#####
```

```

>>> Bloom Filter (11 bits)
0 0 0 0 0 0 0 0 0 0 0

#####
[bloom filter] insert      : 4543863031426141731
#####
hash1() : fringerpt = 4543863031426141731
hash1() : temp      = 454386303142614
hash1() : bf_index  = 7
hash2() : fringerpt = 4543863031426141731
hash2() : temp      = 45438630314261
hash2() : bf_index  = 8
hash3() : fringerpt = 4543863031426141731
hash3() : temp      = 4543863031426
hash3() : bf_index  = 4
hash4() : fringerpt = 4543863031426141731
hash4() : temp      = 454386303142
hash4() : bf_index  = 2

>>> Bloom Filter (11 bits)
0 0 1 0 1 0 0 1 1 0 0

#####
[bloom filter] lookup : 4543863031426141731
#####
hash1() : fringerpt = 4543863031426141731
hash1() : temp      = 454386303142614
hash1() : bf_index  = 7
hash2() : fringerpt = 4543863031426141731
hash2() : temp      = 45438630314261
hash2() : bf_index  = 8
hash3() : fringerpt = 4543863031426141731
hash3() : temp      = 4543863031426
hash3() : bf_index  = 4
hash4() : fringerpt = 4543863031426141731
hash4() : temp      = 454386303142
hash4() : bf_index  = 2
[bloom filter] exist      : 4543863031426141731

#####
[bloom filter] lookup : 4543863041425141743
#####
hash1() : fringerpt = 4543863041425141743
hash1() : temp      = 454386304142514
hash1() : bf_index  = 7
hash2() : fringerpt = 4543863041425141743
hash2() : temp      = 45438630414251
hash2() : bf_index  = 8
hash3() : fringerpt = 4543863041425141743
hash3() : temp      = 4543863041425
hash3() : bf_index  = 4
hash4() : fringerpt = 4543863041425141743
hash4() : temp      = 454386304142
hash4() : bf_index  = 1

```

```
[bloom filter] doesn't_exist:_4543863041425141743
root@server:~/bf#
```

When data with the same hash key (fingerprint1) are saved, four hash functions compute the bit indexes, including 2, 4, 7 and 8, which were set to 1 already. Thus, the Bloom filter finds the current data to exist and to be redundant. Now, when the new data with a different hash key (fingerprint2) are saved, four hash functions again calculate the 1, 4, 7 and 8 bit indexes. Though the bit values of the 4, 7, 8 indexes are found to be 1, the bit values of the 1 bit index (second bit) is still 0. This means the current data are unique. Therefore, the Bloom filter says the current data do not exist in the previously saved data. Then the bit value of the 1 bit index (second bit) is changed to 1.

2.3 Deduplication Techniques by Granularity

2.3.1 File-Level Deduplication

File-level deduplication uses file-level granularity, which is the most coarse-grained granularity. File-level deduplication compares entire files based on a hash value of a file, like SHA-1 [34], to avoid saving the same files. In this section, we demonstrate how file-level deduplication works and its implementation.

2.3.1.1 Fundamentals

We begin by explaining how file-level deduplication works. As shown in Fig. 2.2, suppose we have two identical files. When we save the first file, deduplication computes an index that is a hash value using a one-way hash function. If the index is not found in the index table, the file is unique. In this case, the index and the file are saved to the index table and storage respectively. For the second file, the index of the file is found in the index table, so the corresponding file is not saved.

File-level deduplication has been used to remove redundancies of identical files in storage, email systems and cloud-based storage systems. For storage, EMC Corporation's (EMC's) Centera [17] uses file-level deduplication to reduce redundancies in storage. For email systems, Microsoft Exchange 2003 [29] and 2007 [30] use file-level deduplication, called the Single Instance storage (SIS) [5]. An email with multiple recipients is copied to multiple mailboxes, resulting in multiple copies of the email. In this case, SIS saves only one copy of an email in the recipient's mailbox and saves only the pointers of the email in other recipients' mailboxes without storing the email redundantly in the individual recipients' mailboxes. Many cloud-based storage services such as JustCloud [22] and Mozy [32] also use file-level deduplication. One study [28] on corporate users' file systems showed that

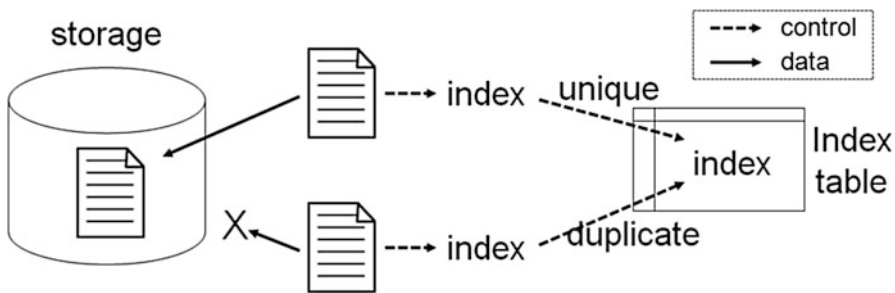


Fig. 2.2 File-level deduplication

simple file-level deduplication can achieve three-quarters of the space savings of aggressive, expensive block deduplications (to be discussed in the next two sections) at a lower cost in terms of performance and complexity.

2.3.1.2 Implementation

The first step of file-level deduplication is to compute an index (hash key) of a file, and the hash key and data of the file are fed into a file-level deduplication function, called the `dedupFile()`. The SHA-1 hash key is used as an index computed by the `getHashKey()` with the data of the file:

```
FileOper fileOper;
ShalWrapper shalWrapper;
string data, hashKey;

data = fileOper.getData(filePath); // path of the file to be
    saved
hashKey = shalWrapper.getHashKey(data);

dedupFile(hashKey, data);
```

`getData()` in `FileOper` class read a file and load the content to a string typed variable. `getData()` is implemented using `ifstream()` as follows:

```
string
FileOper::getData(string filePath) {

    string result, line;

    ifstream file((char*)filePath.c_str());
    if (file.is_open()) {
        while (file.good()) {
            getline(file, line);
            if (!file.eof())
                result += line + "\n";
            else
                result += line;
        }
    }
```

```

    }
} else {
    cout << "getData()_:_" << filePath << "_open_error_" << endl;
    return "";
}
file.close();

return result;
}

```

The dedupFile() function begins by comparing the hash key in arguments with pre-existing indexes in a hash table, checking whether data corresponding to the hash key are unique or duplicates. In code, we use a flag variable, called ‘isUnique’, that has ‘false’ initially. File-level deduplication checks the Bloom filter with an index (hash key). If the Bloom filter returns ‘true’, then we further check the chunk index cache because there may be false positives. If the Bloom filter returns ‘false’, the current data are determined to be unique with 100 % certainty, and ‘isUnique’ is changed to ‘true’:

```

void
SDedup::dedupFile(string fileHashKey , string data) {

    boolean isUnique = false;

    //-----
    // check bloom filter
    //-----
    if (existInBloomFilter(fileHashKey)) {
        // due to false positive , we check chunk index subsequently.

        //-----
        // check chunk index cache
        //-----
        // duplicate data
        if (!isDuplicateInCache(fileHashKey)) {
            isUnique = true;
        }
    } else {
        isUnique = true;
    }

    if (isUnique) {
        saveInCache(fileHashKey);

        // save to storage
        sm.setBufferedData(fileHashKey , data);
    }
}

```

existInBloomFilter() is a wrapper function of bf_lookup(<bloom filter array>, <index>) of the Bloom filter implementation; that is, the Boolean result of bf_lookup() is forwarded to existInBloomFilter(). isDuplicateInCache() is a wrapper function of ‘cache.exist(key)’ in the index table implementation as follows:

```

bool
SDedup::isDuplicateInCache(string key) {
    if (cache.exist(key))
        return true;
    else
        return false;
}

void
SDedup::saveInCache(string key) {
    cache.set(key, "");
}

```

If the current data are determined to be unique by the Bloom filter or chunk index cache, the index is saved to the chunk index cache using the `saveInCache()` function. `'sm.setBufferedData(fileHashKey, data)'` buffers data contents, compresses the data, and saves them to storage. `'sm'` is an object of the `'StoreManager'` class.

2.3.1.3 Existing Solutions

File-level deduplication is used for Microsoft Exchange 2003 and 2007 based on a SIS [5]. SIS stores file contents to a 'SIS Common Store'. In SIS, a user file is managed by a SIS link that is a reference to a file called the 'Common Store File'. Whenever SIS detects duplicate files, SIS links are created automatically and file contents are saved to the common store. SIS consists of a file system filter library that implements links and a user-level service detecting duplicate files that are replaced by links. SIS can find duplicate files but not large redundancies within similar files. We address this issue by developing the Hybrid Email Deduplication System (HEDS) [23].

File-level deduplication is used for popular cloud storage systems, such as JustCloud [22] and Mozy [32], to reduce latency in a client. Cloud storage system client applications run file-level deduplication that computes an index (hash key) of each file and checks whether the index exists in a server. If the server has the index, the client does not send the duplicate file. Running the file-level deduplication in the client before sending data to a server allows cloud storage systems to consume less storage space and bandwidth. One study [20] measured the performance of several cloud storage systems including Mozy.

One study [28] evaluated the trade-off in space savings between file-level deduplication and block-based (fixed-size and variable-size) deduplications, claiming that file-level deduplication provided a simpler complexity and reduced more file fragmentation than block-based deduplications. The study collected file system contents from almost 1000 desktop computers in a corporation and measured file redundancies and space savings. The authors showed that File-level deduplication saves less space than block-level deduplication. Figure 2.3 shows the evaluation setup of the study. A file system scanner computes the indexes of blocks or chunks by running fixed-size and variable-sized block deduplication with the minimum and

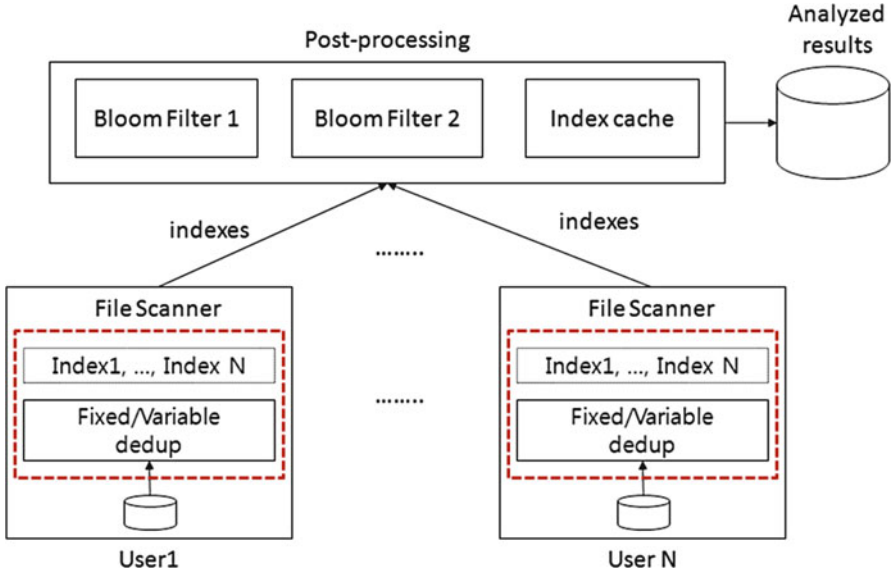


Fig. 2.3 A study of practical deduplication: evaluation set-up

maximum chunk sizes, 4 KB and 128 KB respectively. The expected chunk size ranges from 8 to 64 KB. The computed indexes are collected by a post-processing module that checks the redundancies of indexes using two Bloom filters. The size of each Bloom filter is 2 GB. The analysed results are saved to a database. The computed total size of the files is 40 TB, and the number of files is 200 million. File duplicates are found in post-processing by identifying files where all chunks matched. This study also mentions that a semantic knowledge of file structures would be useful to reduce redundancies with less overhead, and our Structure-Aware File and Email Deduplication for Cloud-based Storage Systems (SAFE) approach exploits the semantic information of file structures, as shown in Chap. 4.

2.3.2 Fixed-Size Block Deduplication

File-level deduplication can find redundancies of identical files but not redundancies within similar files. To find redundancies in similar files, fixed-size block deduplication has been proposed and uses fixed-size blocks for the granularity. In this section, we show how fixed-size block deduplication works and the implementation codes.

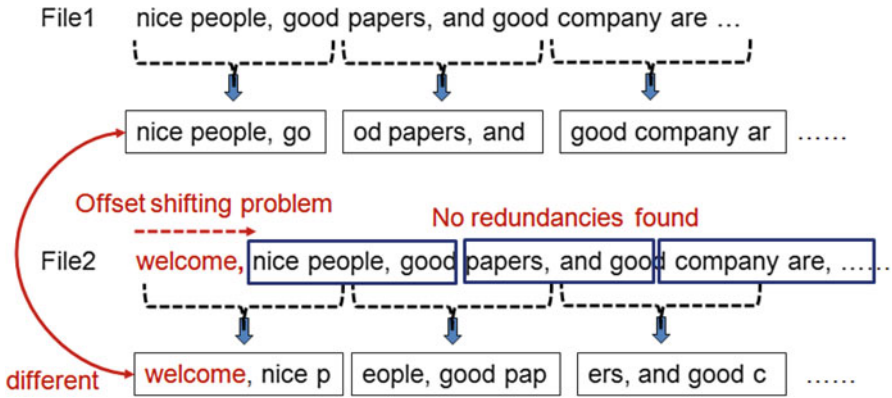


Fig. 2.4 Fixed-size block deduplication

2.3.2.1 Fundamentals

Fixed-size block-level deduplication separates a file into the same sized blocks and finds redundant blocks by comparing the indexes of the blocks. It runs fast because it only relies on offsets in a file to separate a file into blocks. However, fixed-size block deduplication has an issue when it comes to finding matching contents in similar files when the content at the beginning of the files is changed. For example, as shown in Fig. 2.4, suppose deduplication uses a 15 byte fixed-size block as granularity. When we save an original file *File1*, deduplication splits the file into 15 byte fixed-size blocks. Likewise, when we save an updated file *File2*, in which we add the small text ‘welcome’ at the beginning of the original file, deduplication again splits the file into fixed-size blocks. However, blocks split from the updated second file are totally different from blocks split from the original first file. This is because the contents are shifted in the file; this is called the *offset-shifting problem*.

Fixed-size block deduplication has been used for archival storage systems like Venti [39]. Venti uses fixed-size blocks as the granularity level and compares SHA-1 hash keys of blocks with previously saved hash keys following an on-disk index hierarchy. A popular cloud storage system, Dropbox [12], uses very large fixed-size (4 MB) block deduplication. Dropbox reduces network redundant traffic and redundant savings in the server by communicating with indexes between clients and servers before sending the data. Detailed information on how Dropbox works is explained in Chap. 4.

2.3.2.2 Implementation

Fixed-size block deduplication requires three arguments, including an index (a hash key) of a file, the data (content of a file) and a block size based on which data are

split into blocks. To retrieve an index, we can use the `getHashKey()` function in the `sha1Wrapper` class (Sect. 2.2.1.2):

```
hashKey = sha1Wrapper.getHashKey(data);
dedupBlock(hashKey, data, blkSize);
```

The following is an example of a fixed-size block deduplication function, `dedupBlock()`. `dedupBlock()` has three parameters corresponding to the called method, `dedupBlock()`. The first local variable `block` is a reference variable to point to the array of blocks whose type is the string. We also need a variable, ‘`numOfBlocks`’, that shows the number of blocks in a file. The `hashKey` variable indicates the index of each block. In this code, the Bloom filter is not shown, but the checking of redundancy by the Bloom filter can be located before checking using the `isDuplicateInCache()` function with ‘`chunk index cache`’.

```
void
SDedup::dedupBlock(string fileHashKey, string data, int blkSize){

    string *blocks;
    int numOfBlocks = 0;
    string hashKey;
    int i;

    //-----
    // check duplicate file
    //-----
    // *** A duplicate file does not need to be de-duplicated
        to blocks
    if (isDuplicateInCache(fileHashKey)) {
        return;
    }
    else {
        saveInCache(fileHashKey);
    }

    //-----
    // check duplicate blocks
    //-----
    // set block size
    chunkWrapper.setAvgChunkSize(blkSize);

    // get blocks from a data
    blocks = chunkWrapper.getBlocks(data, numOfBlocks);

    for (i=0; i < numOfBlocks; i++) {

        // get hash key of a block
        hashKey = sha1Wrapper.getHashKey(blocks[i]);

        if (!isDuplicateInCache(hashKey)) {
            // save an index of a block
            saveInCache(hashKey);
```

```

        // save to storage
        sm.setBufferedData(hashKey, blocks[i]);
    }

}

// clear memory
delete[] blocks;
}

```

In pure fixed-size block deduplication, a file is directly split into blocks without checking whether the file itself exists, causing redundant processing overhead and memory overhead. Thus, the `dedupBlock()` function first checks whether there is a duplicate file with an index of the current file. If a file is redundant, the file is not separated into blocks. An index of the file is saved to the index table using the `saveInCache()` function, and the `dedupBlock()` function ends.

If the current file is not a duplicate, there could be similar files with the same blocks. First, the program sets the block size using `chunkWrapper.setAvgChunkSize(blkSize)`. The `chunkWrapper` object maintains all environment variables related to chunking. We go into detail on the `chunkWrapper` class in the next section. The `getBlock()` function in `chunkWrapper()` splits the data (file) into blocks of string and returns the split blocks and the number of blocks (`numOfBlocks`). Then, for each block, we check that the block is a duplicate based on the index (or hash key) of the block on the index table. If the block is unique [that is, if `isDuplicateInCache(hashKey)` returns ‘false’], the index of the block is saved to the index table, and each block is filled into buffer so that buffered data are stored when the buffer is full or when it reaches the time threshold. After deduplication is done, the memory allocated for blocks is deleted (the memory was allocated as the type of dynamic array).

```

string *
ChunkWrapper::getBlocks(string data, int & numOfChunks) {

    string *chunks;
    int chunkIndex = 0;
    size_t beginOffset = 0, endOffset = 0;

    int chunkSize = getAvgChunkSize();

    // get number of fixed chunks
    numOfChunks = (data.length() / chunkSize) + 1;

    // get fixed chunks
    chunks = new string[numOfChunks];
    endOffset = beginOffset + (chunkSize - 1);
    while (endOffset < data.length()) {

        chunks[chunkIndex++] = data.substr(beginOffset, chunkSize);

        beginOffset = endOffset + 1;
        endOffset = beginOffset + (chunkSize - 1);
    }
}

```

```

}

// get fixed last chunk
chunks[chunkIndex] =
    data.substr(beginOffset, data.length() - beginOffset);

return chunks;
}

```

The preceding code shows the `getBlocks()` function implementation. Please note that chunk and block terms are used interchangeably. In `getBlocks()`, the number of blocks is computed by dividing the size of the data by the block size (`chunkSize`). We maintain `beginOffset` and `endOffset` for each block, and each block is split from the data and ultimately contained in the string element of the chunk array using a `substr()` function. After all blocks are contained in a chunk string array, the reference variable of the chunks are returned.

2.3.2.3 Existing Solutions

Venti [39] is a fixed-size block deduplication system and uses a write-once policy, preventing data being inconsistent or causing malicious data loss. The main idea is that a file is divided into several blocks, and the index (hash key) of each block is created by a SHA-1 hash function. If the index of the block is the same as a previously saved index, the block is not saved. The index is arranged into a hash tree for reconstructing a file that contains the block. To improve performance, Venti uses three techniques: caching, striping and write buffering. The block and index are cached. Venti shows the possibility of using a hash key to differentiate each block in a file. Most deduplication applications that have been published split a file into several blocks (or chunks) and save each block based on the index (hash key) of each block.

Figure 2.5 shows how files are saved into the tree structure of Venti. A data block is pointed to by an index (hash key) of the block, and the indexes are packed into a pointer block with pointers. As shown in Fig. 2.5a, Venti creates a hash key of a pointer block P_0 that is a root pointer block of *file1*. Venti creates new pointer blocks P_1 and P_2 that subsequently point to D_0 , D_1 , D_2 and D_3 . Thus, data blocks of *file1* are retrieved following on the tree structure of pointer blocks starting from P_0 . Figure 2.5b demonstrates how the tree structure is changed when a similar file (*file2*) is saved. Suppose *file2* has two identical data blocks (D_0 and D_1), like *file1*, but two unique data blocks (D_4 and D_5). Venti does not change the pointer blocks but instead creates new pointer blocks (P_3 and P_4) for *file2*. *File2* can be retrieved using pointer blocks P_3 , P_1 , and P_4 .

Dropbox [12] uses fixed-size block deduplication with a 4 MB fixed block as its granularity. One study [11] discovered internal mechanisms of Dropbox by measuring and analysing packet traces between clients and Dropbox servers. Dropbox is accessed by Web UI (<http://www.dropbox.com>) or a Dropbox client.

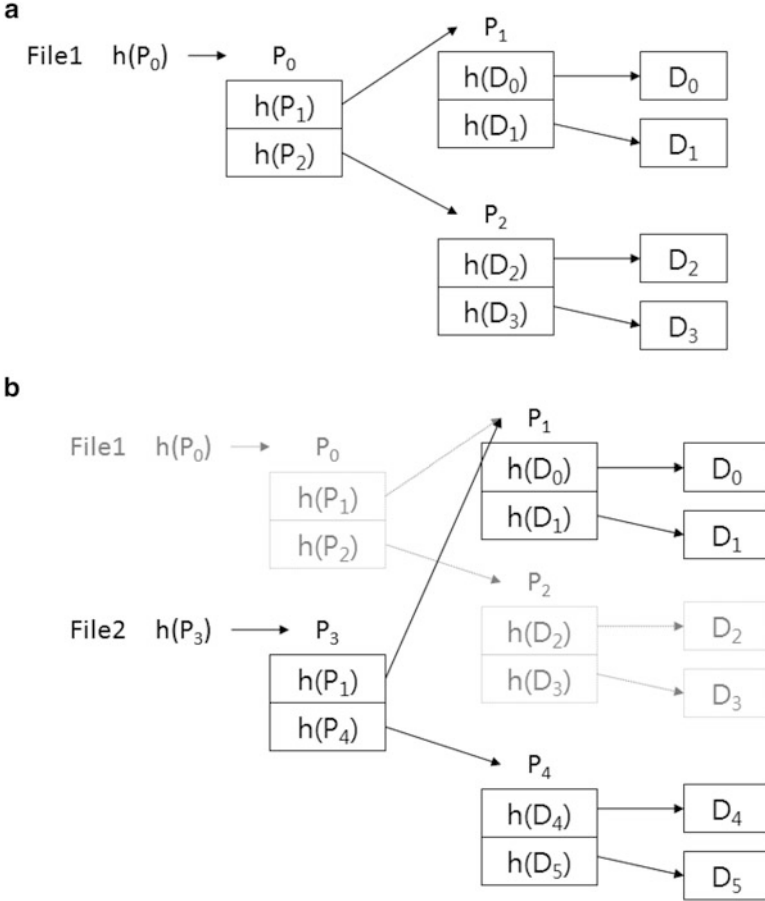


Fig. 2.5 Venti tree structure of data blocks [39]. (a) Tree structure of an original file (File1). *File1* consists of four data blocks, including D_0 , D_1 , D_2 , and D_3 . (b) Tree structure of a similar file (File2). *File2* consists of four data blocks, including D_0 , D_1 , D_4 , and D_5

We leverage SAFE into a Dropbox client to deduplicate structured files on the client side. Dropbox consists of two type of servers, one a control server and the other a storage server. Control servers hold metadata of files such as the hash value of individual blocks and mapping between a file and its blocks. Storage servers contain unique blocks in Amazon S3 [2]. Dropbox client synchronizes its own data and indexes with Dropbox servers.

Figure 2.6 shows how Dropbox works. Circles with numbers represent the order in which a file is saved. *File-A* is a file and *Blk-X* is a block that is separated from a file. $h(Blk-X)$ denotes the hash value of a block. Thick $h(Blk-X)$ and $Blk-X$ are considered hash values and blocks that already existed before a file was saved. The user device is a mobile phone, tablet, laptop or desktop. Dropbox goes through

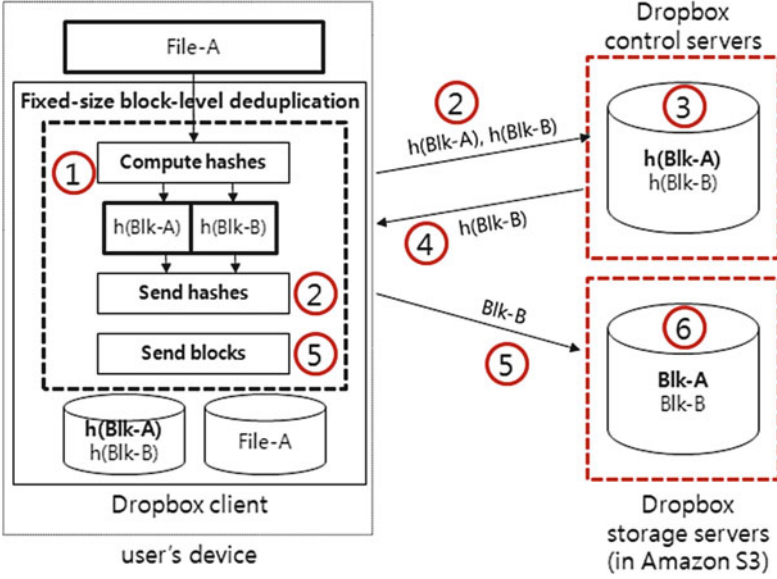


Fig. 2.6 Dropbox internal mechanism

the following steps to save a file. (1) As soon as a user saves *File-A* to a shared folder in a Dropbox client, the fixed-size block deduplication of Dropbox splits the file into blocks based on 4 MB granularity and computes hashes of the objects. If a file is larger than 4 MB, then the file is the same as an object and a hash value of the file is computed. Dropbox uses SHA256 [35] to compute a hash value. (2–4) The Dropbox client sends all computed hash values of a file to a control server that returns only unique hash values after checking previously saved hash values. In this example, the hash key of *Blk-B* is returned to a client because the hash key of *Blk-A* is found to be a duplicate. (5–6) The Dropbox client sends the blocks of returned indexes to the storage server. Ultimately, storage servers have unique blocks across all Dropbox clients. Note that storage saving occurs in a server (thanks to not saving *Blk-A* again), and the incurred network load is reduced because only *Blk-B* is sent.

2.3.3 Variable-Sized Block Deduplication

Variable-sized block deduplication resolves the offset-shifting issue in fixed-size block deduplication, finding more redundant data in similar files. variable-sized block deduplication relies not on a fixed-size offset but on content-based chunking. In this section, we show how variable-sized block deduplication works and present the implementation codes.

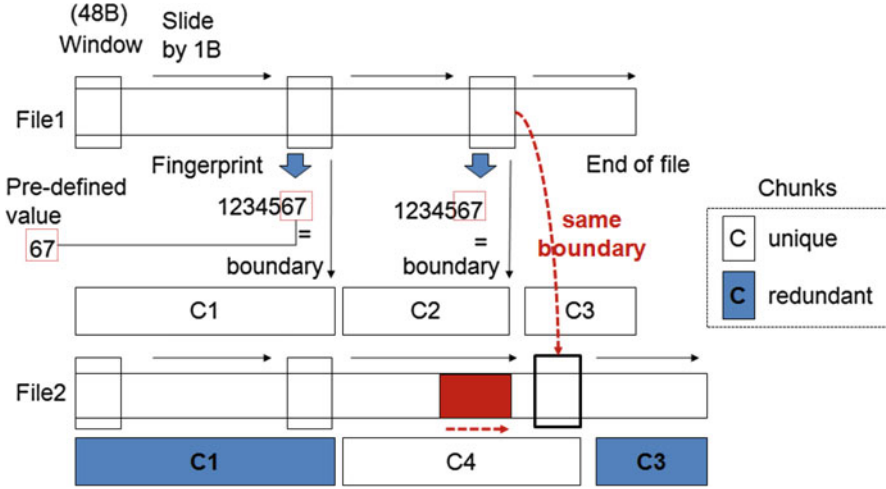


Fig. 2.7 Variable-sized block deduplication

2.3.3.1 Fundamentals

Variable-sized block deduplication has been proposed to solve the *offset-shifting problem* of fixed-size block deduplication. Variable-sized block deduplication relies on contents rather than a fixed offset. Figure 2.7 illustrates how variable-sized block deduplication works. Suppose we have two files. *File1* is an original file and *File2* is an updated file in which we add brief texts in the middle of the file. When we save *File1*, deduplication slides a small window from the beginning of the file. While the window is sliding byte by byte, a fingerprint [40] of each window is computed and the two lowest digits are compared with a pre-defined value. If they are the same, the window is set to a chunk boundary. Then the contents ranging from the previous chunk boundary to the current chunk boundary is treated as a chunk. The window keeps sliding and finding chunk boundaries in the same manner. As a result, three unique chunks (*C1*, *C2*, *C3*) and the corresponding indexes are saved. When we save the updated second file, deduplication again slides a window and finds chunks. *C4* is found to be unique, and *C1* and *C3* are found to be redundant. Here, we see that chunk boundaries are maintained, though the contents are shifted in a file. Thus, content-based variable-sized block deduplication can find more redundancies than offset-based fixed-size block deduplication.

Since variable-sized block deduplication provides fine-granularity chunking techniques to achieve high storage space savings, it has been used for backup [10, 13, 19, 26, 48, 50] or file systems [6, 42]. However, to speed up the processing time by reducing the number of disk accesses, this approach, like the DDfs [50], exploits efficient caching schemes, like the Bloom filter and the chunk index cache, and locality-based disk layout.

2.3.3.2 Implementation: dedupChunk()

We show the dedupChunk() function where variable-sized deduplication is used. Like fixed-size block deduplication, the Bloom filter is not shown, but it can run before an index is checked in the index table using the isDuplicateInCache() function. The dedupChunk() function is almost the same as the dedupBlock() function, except that dedupChunk() uses chunkWrapper.getChunks() rather than chunkWrapper.getBlocks(). The getChunks() function is explained in more detail in Sect. 2.3.3.5. Overall, the unique chunk is passed to a buffer where the buffered data are saved in storage.

```

void
SDedup::dedupChunk(string fileHashKey, string data, int avgChunk
    Size,
                    int minChunkSize, int maxChunkSize) {

    string *chunks;
    int numOfChunks = 0;
    int i;
    string hashKey;
    string filePath;

    //-----
    // check duplicate file
    //-----
    // *** A duplicate file does not need to be de-duplicated to
    blocks
    if (isDuplicateInCache(fileHashKey)) {
        return;
    }
    else {
        saveInCache(fileHashKey);
    }

    //-----
    // check duplicate chunks
    //-----

    // get chunks from a data
    chunks = chunkWrapper.getChunks(data, numOfChunks,
        avgChunkSize, minChunkSize, maxChunkSize);

    for (i=0; i < numOfChunks; i++) {
        // get hash key of a block
        hashKey = sha1Wrapper.getHashKey(chunks[i]);

        if (!isDuplicateInCache(hashKey)) {
            saveInCache(hashKey);
            sm.setBufferedData(hashKey, chunks[i]);
        }
    }
    // clear memory
    delete [] chunks;
}

```

2.3.3.3 Implementation: Rabin Fingerprint

The Rabin fingerprint [40] is used to find chunk boundaries, resulting in the identification of a chunk. The Rabin fingerprint is a 64 bit key. When we compute fingerprints in data (byte stream using sliding windows), a fingerprint of each window can be computed quickly based on the previous fingerprints using the following equation. Detailed information can be found in [43]. The full implementation codes are in Appendix D:

$$\text{RF}(t_{i+1} \dots t_{\beta+i}) = (\text{RF}(t_i \dots t_{\beta+i-1}) - t_i \times p^\beta) + t_{\beta+i} \bmod M. \quad (2.1)$$

We can compile and build a test program to compute the Rabin fingerprint by typing ‘make’. The results from running the executable file, rabin, show a fingerprint (with long integer type) for the ‘hello tom danny’ string.

```
root@server:~/rabin# make
g++ -o rabin rabinpoly.cc rabinpoly_main.cc

root@server:~/rabin# ls -l
total 40
-rw-r--r-- 1 root root 83 Jul 24 16:47 Makefile
-rwxr-xr-x 1 root root 13503 Jul 24 16:47 rabin
-rw-r--r-- 1 root root 9662 Jul 24 16:46 rabinpoly.cc
-rw-r--r-- 1 root root 2756 Jul 24 16:46 rabinpoly.h
-rw-r--r-- 1 root root 1399 Jul 24 16:47 rabinpoly_main.cc

root@server:~/rabin# rabin
rabinpoly_main : input data
hello tom danny
rabinpoly_main : fingerpt = 379718595532164463
```

2.3.3.4 Implementation: Chunking Core

Chunking is the first of three steps in deduplication (the other steps are hashing and indexing). The snippet codes for chunking are found in Appendix E. The core function in chunking is process_chunk in chunk_sub.cc. The process_chunk() function slides a small window byte by byte on the data, finds the chunk boundaries, and saves the beginning and ending indexes for all chunks to the ‘begin_indexes’ and ‘end_indexes’ integer arrays respectively. That is, the goal of the process_chunk() function is to identify the boundaries of the chunks. Based on the boundaries, the chunking Wrapper class, as shown in Appendix F, splits the data into chunks.

The following code is the function call to process_chunk(), where ‘buf’ and ‘size’ are the data and the size of the data to be separated, num_of_breakpoints is the number of break points, BOUNDARY_SIZE, defined in rabinpoly.h, is 48 bytes and the size of the sliding windows avg_chunk_size, min_chunk_size and max_chunk_size are average, minimum and maximum chunk size respectively.

`num_of_chunks` is literally the number of chunks. `begin_indexes` and `end_indexes` are integer arrays, where the beginning and ending indexes of all chunks are to be held after the breakpoints are found.

```
process_chunk(buf, size, &num_of_breakpoints, BOUNDARY_SIZE,
              avg_chunk_size, min_chunk_size, max_chunk_size,
              &num_of_chunks, begin_indexes, end_indexes);
```

The entire code of `process_chunk()` is in `chunk_sub.cc` of Appendix E.3. We explain how `process_chunk()` works using snippet codes. The first *for* loop slides a window by one byte based on `cur_pos`. At STEP1, `b_region` means each window, and at STEP2, `fingerprint` is computed for each `b_region` using the `fingerprint()` function in `rabinpoly.cc`. Readers should note that they can further optimize the running time not by copying window to the `b_region` char array but by using the offset of window in data. At STEP3, low-order bits are calculated as the remainder of `fingerprint` divided by `avg_chunk_size`. At STEP5, the low-order bits of `fingerprint` are compared with `BREAKMARK_VALUE`, which is defined at `0x78` in `chunk.h`. If they are the same, then the current window is determined to be a chunk boundary. The beginning (`chunk_b_pos`) and ending (`chunk_e_pos`) indexes of the chunk are saved to `begin_indexes` and `end_indexes` integer arrays by the `set_breakpoint()` function. Note that if the chunk size is less than the predetermined `min_chunk_size` ('if (`cur_chunk_size` < `min_chunk_size`)'), `process_chunk()` continues to slide a window to find the next chunk boundaries. Also, as at STEP4, if the window keeps sliding without finding chunk boundaries and the size of a chunk is supposed to be larger than `max_chunk_size`, `process_chunk()` forcibly sets a chunk boundary (break mark) and creates a chunk.

```
:
for (cur_pos=0; cur_pos < data_size; cur_pos += ONE_BYTE)
{
    :
    //-----
    // STEP1. get boundary region
    //-----
    :
    // allocate boundary region
    b_region = (unsigned char *)malloc(sizeof(unsigned char)*
        b_size);
    memset(b_region, '\0', b_size);

    // get boundary region
    strncpy(b_region, data + cur_pos, b_size);
    b_region[b_size] = '\0';
    :
    //-----
    // STEP2 compute rabin fingerprint
    //-----
    fingerprint = fingerprint(b_region, strlen(b_region), FINGER
        PRINT_PT);

    //-----
```

```

// STEP3. compare to breakpoint value
//          to extract chunk
//
//          fingerprint % K(avg_chunk_size) == BREAKMARK_VALUE
//_____
low_order_bits = fingerprint % avg_chunk_size;

//_____
// STEP4. chunk size is larger than maximum chunk size
//_____
cur_chunk_size = get_chunk_size(chunk_b_pos, cur_pos,
b_size);
if (cur_chunk_size >= max_chunk_size)
{
    //get chunk_b_pos and chunk_e_pos
    set_breakpoint(num_of_breakpoints, &chunk_b_pos,
&chunk_e_pos, &cur_pos,
(char*)"MAX_CHUNK_SIZE", b_size, data_size,
num_of_chunks, begin_indexes, end_indexes);

    // set position for the next chunk
    chunk_b_pos = chunk_e_pos + 1;
    cur_pos      = chunk_b_pos;
}

//_____
// STEP5 chunk size is less than minimum chunk size or
//          is in between minimum chunk size and maximum chunk
//          size
//
//          (f(A) mod K == x)
//          (chunk size is less than minimum or
//          is in range of minimum and maximum chunk size)
//
//          : f(A) → fingerprint
//          : K    → expected average chunk size
//          : x    → BREAKMARK_VALUE
//_____
if (low_order_bits == BREAKMARK_VALUE)
{
    if (cur_chunk_size < min_chunk_size)
    {
        // do not set breakpoint
        ;
    }
    else if ( (cur_chunk_size >= min_chunk_size)
&& (cur_chunk_size < max_chunk_size) )
    {

        // get chunk_b_pos and chunk_e_pos
        set_breakpoint(num_of_breakpoints, &chunk_b_pos,
&chunk_e_pos, &cur_pos,
(char*)"BREAKMARK", b_size, data_size,
num_of_chunks, begin_indexes, end_indexes);
    }
}

```

```

        // set position for the next chunk
        chunk_b_pos = chunk_e_pos + 1;
        cur_pos      = chunk_b_pos;

    }
}
}

```

To compile and build a ‘chunk’ executable, we type ‘make’. Following are the results of running the ‘chunk’ executable. ‘body’ is data that are split into chunks. The chunking core computes 14 boundaries from the body data. The beginning and ending indexes of each chunk are shown in the results. For example, the second chunk has the 3493rd byte as the beginning index and 12,427th byte as the ending index.

```

root@server:~/lib/chunk/chunk_lib# make
g++ -o chunk chunk_main.cc chunk_sub.cc rabinpoly.cc util.cc

root@server:~/lib/chunk/chunk_lib# ls -l
total 356
-rw-r--r-- 1 root root 106342 Jul 20 20:29 body
-rwxr-xr-x 1 root root 24070 Jul 25 13:49 chunk
-rw-r--r-- 1 root root 5417 Jul 20 20:29 chunk.h
-rw-r--r-- 1 root root 4592 Jul 20 20:29 chunk_main.cc
-rw-r--r-- 1 root root 30079 Jul 20 20:29 chunk_sub.cc
-rw-r--r-- 1 root root 409 Jul 20 20:29 common.h
-rw-r--r-- 1 root root 209 Jul 20 20:29 Makefile
-rwxr-xr-x 1 root root 13503 Jul 24 17:36 rabin
-rw-r--r-- 1 root root 9662 Jul 20 20:29 rabinpoly.cc
-rw-r--r-- 1 root root 2756 Jul 20 20:29 rabinpoly.h
-rw-r--r-- 1 root root 1399 Jul 24 17:36 rabinpoly_main.cc
-rwxr-xr-x 1 root root 8845 Jul 24 17:30 util
-rw-r--r-- 1 root root 3088 Jul 20 20:29 util.cc

root@server:~/lib/chunk/chunk_lib# chunk body 8192 2048 65535
### set_breakpoint [1] : size(3493) : 0 ~ 3492, BREAKMARK
### set_breakpoint [2] : size(8935) : 3493 ~ 12427, BREAKMARK
### set_breakpoint [3] : size(23575) : 12428 ~ 36002, BREAKMARK
### set_breakpoint [4] : size(5917) : 36003 ~ 41919, BREAKMARK
### set_breakpoint [5] : size(9126) : 41920 ~ 51045, BREAKMARK
### set_breakpoint [6] : size(3076) : 51046 ~ 54121, BREAKMARK
### set_breakpoint [7] : size(4246) : 54122 ~ 58367, BREAKMARK
### set_breakpoint [8] : size(8408) : 58368 ~ 66775, BREAKMARK
### set_breakpoint [9] : size(18804) : 66776 ~ 85579, BREAKMARK
### set_breakpoint [10] : size(2109) : 85580 ~ 87688, BREAKMARK
### set_breakpoint [11] : size(11416) : 87689 ~ 99104, BREAKMARK
### set_breakpoint [12] : size(2180) : 99105 ~ 101284, BREAKMARK
### set_breakpoint [13] : size(4326) : 101285 ~ 105610, BREAKMARK
### set_breakpoint [13] : size(731) : 105611 ~ 106341, LAST_CHUNK

```

2.3.3.5 Implementation: Chunking Wrapper

The chunkWrapper class defines functions [getChunks()] to separate data into variable-sized chunks based on beginning and ending indexes computed by the chunking core class. The chunkWrapper class also defines functions to obtain fixed-size blocks [getBlocks()]. The chunkWrapper class requires other libraries, including the chunking core class (Sect. 2.3.3.4), Rabin fingerprint class (Sect. 2.3.3.3) and SHA-1 hashing (Sect. 2.2.1.2). The chunkWrapper class also requires a file operation class based on the C++ Boost library. The file operation class is not shown in this book owing to the large code size.

```
root@server:~/lib/chunk# make
g++ -DCHUNK_WRAPPER_TEST -o chunk chunkWrapper.cc
chunkWrapperTest .cc
fileOper/fileOper.cc chunk_lib/chunk_sub.cc chunk_lib/rabinpoly
.cc
chunk_lib/util.cc -I. -Iboost_1_58_0 -L/usr/local/lib
-lboost_filesystem -IfileOper -LfileOper -Ichunk_lib -Lchunk_lib
-Isha1 -Lsha1 sha1/sha1.cc sha1/sha1Wrapper.cc

root@server:~/lib/chunk# ls -l
drwx----- 9 501 staff 4096 Jul 25 15:37 boost_1_58_0
-rw-r--r-- 1 root root 83581760 Apr 16 03:58 boost_1_58_0.tar.gz
-rwxr-xr-x 1 root root 198084 Jul 25 16:01 chunk
-rw-r--r-- 1 root root 1355 Jul 20 20:28 chunkInterface.h
drwxr-xr-x 2 root root 4096 Jul 25 13:49 chunk_lib
-rw-r--r-- 1 root root 4162 Jul 20 20:28 chunkWrapper.cc
-rw-r--r-- 1 root root 915 Jul 20 20:28 chunkWrapper.h
-rw-r--r-- 1 root root 2536 Jul 20 20:28 chunkWrapperTest.cc
-rw-r--r-- 1 root root 18666 Jul 20 20:28 document.xml
.changed
drwxr-xr-x 2 root root 4096 Jul 25 16:01 fileOper
-rw-r--r-- 1 root root 934 Jul 25 16:01 Makefile
-rw-r--r-- 1 root root 1509 Jul 20 20:28 readme
drwxr-xr-x 2 root root 4096 Jul 25 16:01 SHA-1

root@server:~/lib/chunk# ls -l chunk_lib
-rw-r--r-- 1 root root 106342 Jul 20 20:29 body
-rwxr-xr-x 1 root root 24070 Jul 25 13:49 chunk
-rw-r--r-- 1 root root 5417 Jul 20 20:29 chunk.h
-rw-r--r-- 1 root root 4592 Jul 20 20:29 chunk_main.cc
-rw-r--r-- 1 root root 30079 Jul 20 20:29 chunk_sub.cc
-rw-r--r-- 1 root root 409 Jul 20 20:29 common.h
-rw-r--r-- 1 root root 209 Jul 20 20:29 Makefile
-rwxr-xr-x 1 root root 13503 Jul 24 17:36 rabin
-rw-r--r-- 1 root root 9662 Jul 20 20:29 rabinpoly.cc
-rw-r--r-- 1 root root 2756 Jul 20 20:29 rabinpoly.h
-rw-r--r-- 1 root root 1399 Jul 24 17:36 rabinpoly_main.cc
-rwxr-xr-x 1 root root 8845 Jul 24 17:30 util
-rw-r--r-- 1 root root 3088 Jul 20 20:29 util.cc

root@server:~/lib/chunk# ls -l SHA-1
```

```

-rw-r--r-- 1 root root 98 Jul 25 16:01 Makefile
-rwxr-xr-x 1 root root 37383 Jul 25 16:01 SHA-1
-rw-r--r-- 1 root root 20297 Jul 25 16:01 sha1.cc
-rw-r--r-- 1 root root 4606 Jul 25 16:01 sha1.h
-rw-r--r-- 1 root root 1908 Jul 25 16:01 sha1_test.cc
-rw-r--r-- 1 root root 1187 Jul 25 16:01 sha1Wrapper.cc
-rw-r--r-- 1 root root 522 Jul 25 16:01 sha1Wrapper.h

root@server:~/lib/chunk# ls -l fileOper
-rw-r--r-- 1 root root 52812 Jul 25 16:01 fileOper.cc
-rw-r--r-- 1 root root 22577 Jul 25 16:01 fileOper.h

```

We show three results obtained from running the program. The first variable-sized chunking extracts 14 chunks based on 8 KB (8192 bytes) average chunk size. Lines show the chunk boundaries and the chunk sizes after showing the boundaries. The second variable-sized chunking extracts 45 chunks from the same data. This is because an average chunk size (2 KB) that is smaller than the first chunking is used. The smaller the average chunk size used, the more chunks are created. The third result shows blocks extracted by `getBlocks()`, which means fixed-size blocks.

```

root@server:~/lib/chunk# chunk

***** variable sized chunking *****
average chunk size = 8192
minimum chunk size = 2048
maximum chunk size = 65535
### set_breakpoint [1] : size(3493) : 0 ~ 3492, BREAKMARK
### set_breakpoint [2] : size(8935) : 3493 ~ 12427, BREAKMARK
### set_breakpoint [3] : size(23575) : 12428 ~ 36002, BREAKMARK
### set_breakpoint [4] : size(5917) : 36003 ~ 41919, BREAKMARK
:
### set_breakpoint [12] : size(2180) : 99105 ~ 101284, BREAKMARK
### set_breakpoint [13] : size(4326) : 101285 ~ 105610, BREAKMARK
### set_breakpoint [13] : size(731) : 105611 ~ 106341, LAST_CHUNK
chunk [0] 3493
chunk [1] 8935
chunk [2] 23575
chunk [3] 5917
chunk [4] 9126
:
chunk [11] 2180
chunk [12] 4326
chunk [13] 731

***** variable sized chunking (with parameters) *****
average chunk size = 2048
minimum chunk size = 512
maximum chunk size = 65535
### set_breakpoint [1] : size(1329) : 0 ~ 1328, BREAKMARK
### set_breakpoint [2] : size(2164) : 1329 ~ 3492, BREAKMARK
### set_breakpoint [3] : size(3284) : 3493 ~ 6776, BREAKMARK
### set_breakpoint [4] : size(2974) : 6777 ~ 9750, BREAKMARK
:

```

```

### set_breakpoint [41] : size(2180) : 99105 ~ 101284, BREAKMARK
### set_breakpoint [42] : size(609) : 101285 ~ 101893, BREAKMARK
### set_breakpoint [43] : size(1674) : 101894 ~ 103567, BREAKMARK
### set_breakpoint [44] : size(1623) : 103568 ~ 105190, BREAKMARK
### set_breakpoint [44] : size(1151) : 105191 ~ 106341,
  LAST_CHUNK
chunk [0] 1329
chunk [1] 2164
chunk [2] 3284
chunk [3] 2974
:
chunk [40] 2180
chunk [41] 609
chunk [42] 1674
chunk [43] 1623
chunk [44] 1151

***** fixed sized chunking *****
chunk [0] 8192
chunk [1] 8192
chunk [2] 8192
:
chunk [8] 8192
chunk [9] 8192
chunk [10] 8192
chunk [11] 8192
chunk [12] 8038

```

2.3.3.6 Existing Solutions

Variable-sized block deduplication involves expensive chunking and indexing for finding large redundancies, requiring an efficient in-memory cache and on-disk layout on high-capacity servers. DDFS [50] exploits three techniques to relieve a disk bottleneck, reducing processing time. A summary vector, which is a compact in-memory data structure, is used to find new data. Stream-informed segment layout, on-disk layout, is used to improve spatial locality for both data and indexes. The idea of a stream-informed segment layout is that a segment tends to reappear in similar sequences with other segments. This spatial locality is called *segment duplicate locality*. Locality-preserved caching uses segment duplicate locality to acquire a high hit ratio in the memory cache. The study removes 99 % of disk accesses and achieves 100 MB/s and 210 MB/s for single-stream throughput and multi-stream throughput respectively.

Sparse indexing [26] uses sampling and a sparse index to reduce the number of indexes, decreasing RAM requirements. Sparse indexing chooses small portions of chunks in the byte stream as a sample and avoids full chunk indexes, unlike DDFS. This approach employs *chunk locality*, the tendency of chunks in backup data streams to reoccur together. Figure 2.8 shows the deduplication process of sparse indexing. In sparse indexing, a segment is the unit of storage and retrieval and

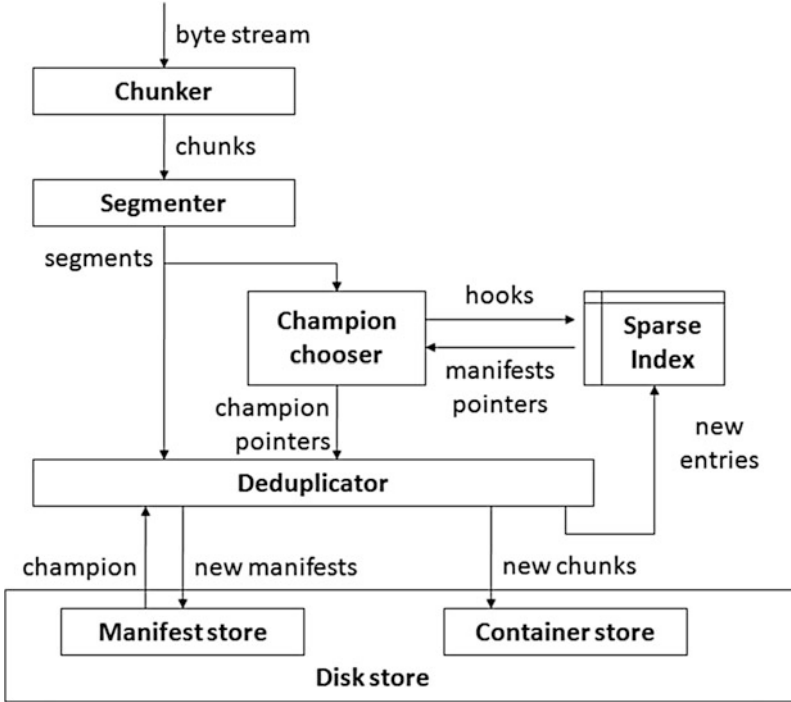


Fig. 2.8 Sparse indexing: deduplication process [26]

a sequence of chunks. A byte stream is split into chunks by Chunker using variable-sized chunking, and a sequence of chunks becomes a segment by Segmenter. Two segments are similar if they share a number of chunks. The Champion chooser chooses sampled segments, called champion, from a sparse index (in-memory index). Deduplicator compares chunks in incoming segments with chunks in champions (selected segments). Unique segments are saved to the sparse index for future comparison, and new chunks are saved to the Container store.

2.3.4 Hybrid Deduplication

Hybrid approaches have been proposed by adaptively using variable-sized block-level deduplication and file-level deduplication, based on either a fixed policy or dynamically changed file information [23, 31]. Min et al. [31] employed context-aware chunking using a file-level deduplication for multimedia content, compressed files or encrypted content and uses variable-sized block-level deduplication for

text files. Our approach, HEDS [23], first separates the message body and individual attachments and performs a variable-sized block-level deduplication if the object size exceeds a predefined threshold. Otherwise, a file-level deduplication is used.

2.3.5 Object-Level Deduplication

Fixed-size block deduplication and variable-sized block deduplication can be used for all types of files because they rely on the physical byte-string format of a file. However, for specific file formats, they may be inefficient owing to expensive chunking. Thus, object-level deduplication that splits a file based on the semantic (or logical) format of a file has been proposed. A few *structure-aware data deduplication* techniques [24, 25, 27, 49] have been proposed to simplify the chunking mechanism by using objects. Our approach, SAFE [24], splits structure files, including compressed files, document files (docx, pptx and pdf) and emails, based on files' structured formats. ADMAD [27] separates a file into variable-sized semantic segments, called meaningful chunks (MCs), based on the metadata of each file. Although the idea of ADMAD decomposing a file into objects according to the object structure is similar to that of SAFE, ADMAD is limited to a specific file format. For example, ADMAD does not deal with document file types such as docx, pptx and pdf. In addition, ADMAD does not handle emails with multiple attachments. Similar concepts involving the deduplication of structured objects are presented in [25] and [49].

2.3.6 Comparison of Deduplications by Granularity

Overall, as shown in Fig. 2.9, the deduplication ratio indicates how many redundancies are removed, and variable-sized block deduplication is much better than others. In terms of processing time, variable-sized block deduplication is the worst owing to expensive chunking. In terms of index overhead, fixed-size and variable-sized block deduplication is much worse than file-level deduplication, and the index overhead of fixed-size and variable-sized block deduplication changes depending on the block or chunk size. Thus, variable-sized block deduplication is good for the deduplication of updated files or server-based deduplication because high-capacity servers can handle excessive processing time and index overhead. On the other hand, file-level deduplication is good for the deduplication of copied files or client-based deduplication given low-capacity clients.

	Good for client-based Or copied files		Good for server-based Or updated files	
 Deduplication ratio	File-level	< Fixed size << better	Variable size	
 Processing time	File-level	< Fixed size <<<< worse	Variable size	
 Index overhead	File-level	<< Fixed size worse	Variable size	

Fig. 2.9 Comparisons of deduplications

2.4 Deduplication Techniques by Place

2.4.1 Server-Based Deduplication

Server-based deduplication has emerged as a disk-based substitute for tape storage and backs up large amounts of data at fast speeds using high-performance and dedicated backup systems. There are many commercial products [18, 36, 45] that can be used for this type of deduplication.

In this approach, clients send backup data to servers where data are deduplicated. Clients have lightweight backup through which data are sent to servers, avoiding large CPU computation and memory overhead of sources for backup purposes. Figure 2.10 shows how server-based deduplication works. A file is transferred to a server through a client application. On the server, the file is separated into chunks typically using variable-sized block deduplication. Indexes of chunks are computed and compared with indexes previously saved using a Bloom filter or a chunk index cache. Suppose a chunk *c1* is redundant and a chunk *c2* is unique in this example. Then, chunk *c2* and its corresponding index are saved to storage.

Server-based deduplication finds significant redundancies but incurs excessive redundant data traffic because duplicate data are delivered to servers to be deduplicated. What is worse, servers have large CPU computation and memory overhead for chunking and indexing of all backup data. To handle backup quickly with this overhead within a limited backup window, efficient in-memory and on-disk layout is required, such as in DDFS [50].

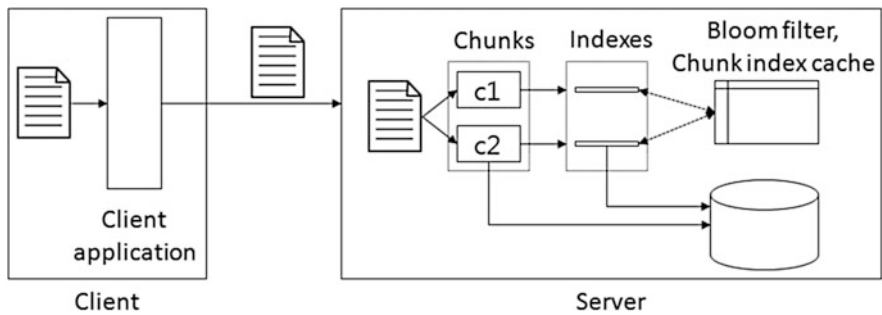


Fig. 2.10 Server-based deduplication

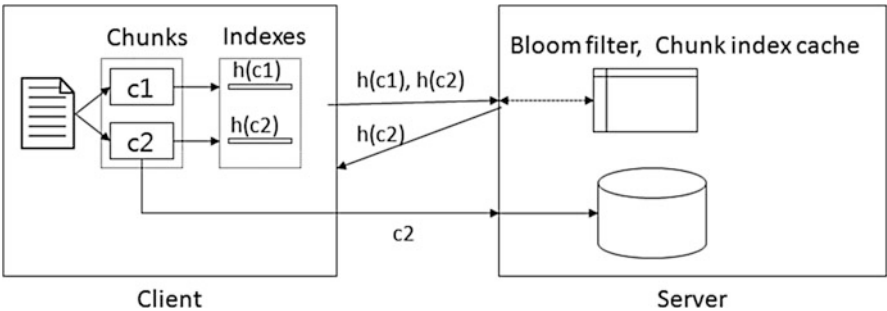


Fig. 2.11 Client-based deduplication: $c1$ and $c2$ are chunks. $h(c1)$ and $h(c2)$ are hash keys (indexes) of chunks

2.4.2 Client-Based Deduplication

In *client-based deduplication*, clients can keep indexes of deduplicated data or have a backup agent to check indexes that exist on servers. In either case, clients check the uniqueness of data in local indexes or in remote indexes through a backup agent. Only unique data are then delivered to servers. Client-based deduplication [16, 46] removes excessive redundant network traffic by performing deduplication at the client before data are transmitted. However, clients incur CPU computation and memory overhead for backup.

Pure client-based deduplication, where a client removes redundant data before sending data to a server, does not collaborate with a server (or servers); redundant data among clients are transferred to a server, which increases data traffic on the network. Thus, client-based deduplication typically communicates with a server, and Fig. 2.11 illustrates how client-based deduplication works with the help of a server. The client splits a file into chunks ($c1$ and $c2$) and computes indexes ($h(c1)$, $h(c2)$). Then the client sends all the indexes of the file to a server, which then returns indexes ($h(c2)$) of unique chunks that have not yet been saved. In this way, the client then can send only unique chunks ($c2$).



Fig. 2.12 End-to-end RE: $c1$ and $c2$ are chunks; $h(c1)$ and $h(c2)$ are hash keys (indexes) of chunks

A low-bandwidth network file system (LBFS) [33] improves space savings by adding a communication protocol that sends indexes to a server before sending a real data chunk. However, it introduces latency to run the protocol. Overall, a client-based deduplication system has difficulties associated with the limited capacity of clients to carry out an expensive deduplication process.

2.4.3 End-to-End Redundancy Elimination

End-to-end RE, like WAN optimizers [7, 8, 41], removes redundant network traffic at two end points (e.g. branch to headquarter and data centre to data centre). Figure 2.12 illustrates how end-to-end RE works. An end-to-end RE device, like a WAN optimizer, is located just before an ingress router (sending side) and just after an egress router (receiving side). Suppose clients send the same files ($f1$ and $f2$) to a server. When a unique file $f1$ is transferred, the file is split into chunks (here $c1$ and $c2$) and corresponding indexes ($h(c1)$ and $h(c2)$) are saved to the cache; subsequently, chunks and indexes are saved to disk (shown here). The file is compressed and delivered to the server side, where chunks and indexes of the received file are saved to the cache.

Now, when another client sends the same file ($f2$), chunks of $f2$ are split and indexes of the chunks are compared with previously saved indexes. The file $f2$ is found to be a duplicate because the same indexes $h(c1)$ and $h(c2)$ are found in the cache. Thus, the contents of the file are replaced (or encoded) by small indexes $h(c1)$ and $h(c2)$, which reduces packet size. When an encoded packet arrives at the server side, a file $f2$ is reassembled with chunks $c1$ and $c2$ based on indexes in the packet. The reassembled file is directed to a specific server.

A LBFS [33] reduces latency and network bandwidth through the collaboration of the client and server. That is, LBFS avoids sending data over the network when the same data can already be found in the server's file system or the client's cache. To reduce the bandwidth requirement, LBFS exploits cross-file similarities. As shown in Fig. 2.13a, a LBFS consists of a LBFS client and server, and both sides maintain chunk indexes in a chunk database.

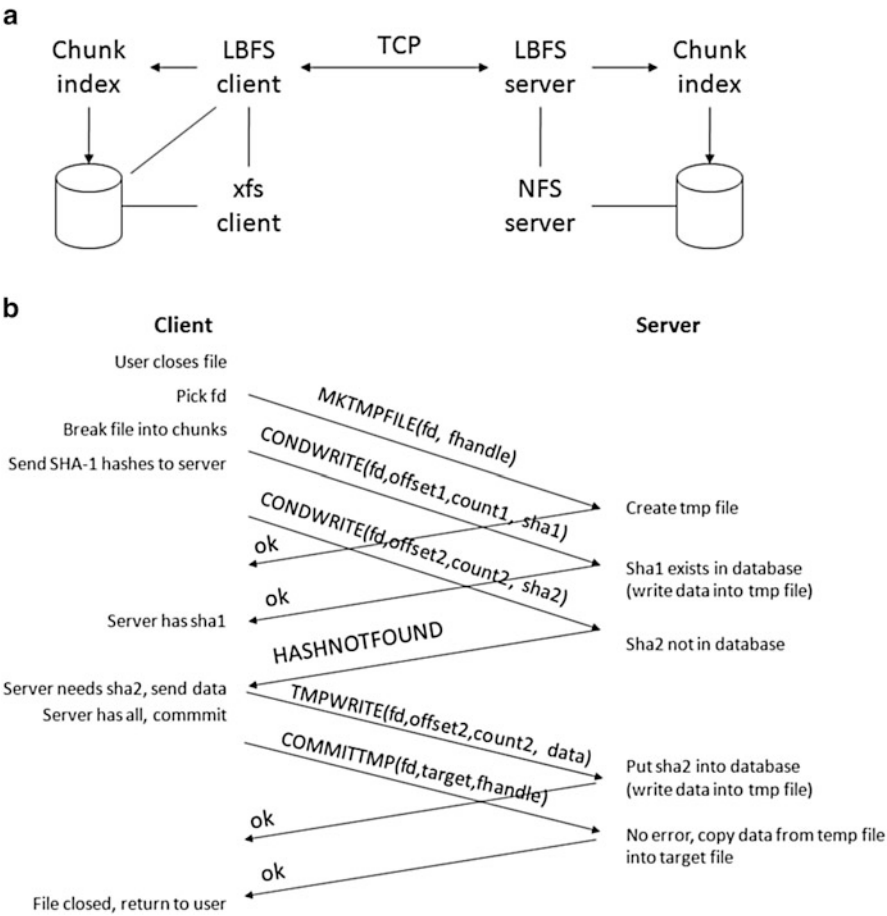


Fig. 2.13 Low-bandwidth file system (LBFS) [33]. (a) LBFS implementation. (b) LBFS: write a file

Figure 2.13b shows how a LBFS works when a file is written to a server from a client. When a user closes a file, a client chooses a file descriptor and calls MKTMPFILE RPC; subsequently, a server creates a temporary file. A client splits a file into chunks (chunk1 and chunk2), computes the hash keys of the chunks and calls CONDWRITE RPCs with hash keys. Suppose the server has SHA-1 (hash key for chunk1) but not SHA-2 (hash key for chunk2). The server returns HASHNOTFOUND for the SHA-2 request; that is, the server does not have chunk2. The client sends only chunk2 to the server, and the server creates a file with chunk1 (previously saved chunk) and chunk2 (chunk received by TMPWRITE RPC). A LBFS can be considered client-based deduplication because the client splits the file

into chunks and saves the indexes. Also, LBFS can be considered end-to-end RE because the client and the server hold the same chunks and indexes, and only unique chunks are transferred through the network, with both sides (client and server) maintaining chunks for unique and redundant files.

2.4.4 *Network-Wide Redundancy Elimination*

Network-wide RE runs deduplication at routers on the network rather than running it at a host (either a client or a server). The unit of deduplication becomes a payload of the packet or byte string on the payload, which is generally smaller than a file (or block or chunk) used for storage data deduplication. This section shows how network-wide RE works and implementation code. For implementation, the first step in the deduplication of a payload of a packet or a byte stream of the payload is to capture a packet on the fly in a router (or at network end points in a router). We show how to intercept a packet using a user space library called `libnetfilter_queue`. We also show how to intercept a packet using a kernel-level module, `libnetfilter`, to achieve better performance.

2.4.4.1 Fundamentals

Network-wide RE [3, 4, 43] eliminates repeating network traffic across network elements such as routers and switches. Network-wide RE computes indexes [40] for the incoming packet payload and eliminates redundant packets by comparing indexes with the packets saved previously. Redundant payload is encoded by small shims and decoded before exiting the network. However, this approach suffers from high processing time owing to sliding fingerprinting on routers and high memory overhead for saving packets and indexes.

The goal of network-wide RE is to remove redundancies of packet payloads, and the granularity is byte strings in a payload. Figure 2.14 shows how network-wide RE works. In network-wide RE, there are special routers (or switches) called RE devices. When an RE device receives a packet, it slides a small window on the payload and computes the fingerprints of all windows. Then some fingerprints are compared with fingerprints in the local cache. If they are the same, the indicated byte regions are expanded to the left and to the right while being compared with a packet in the local cache. The expanded byte region is replaced by a small shim header with a fingerprint and byte offsets. These processes are encoding processes. The encoded payload is reconstructed by a RE device on a path, called decoding. Decoded packets are delivered to a server.

As we see here, network-work RE saves bandwidth in links between an encoder and a decoder. However, as shown in Fig. 2.15, sliding fingerprinting requires excessive processing time, and packets that are saved in cache increase memory requirements. More importantly, redundancies removed in the network are restored

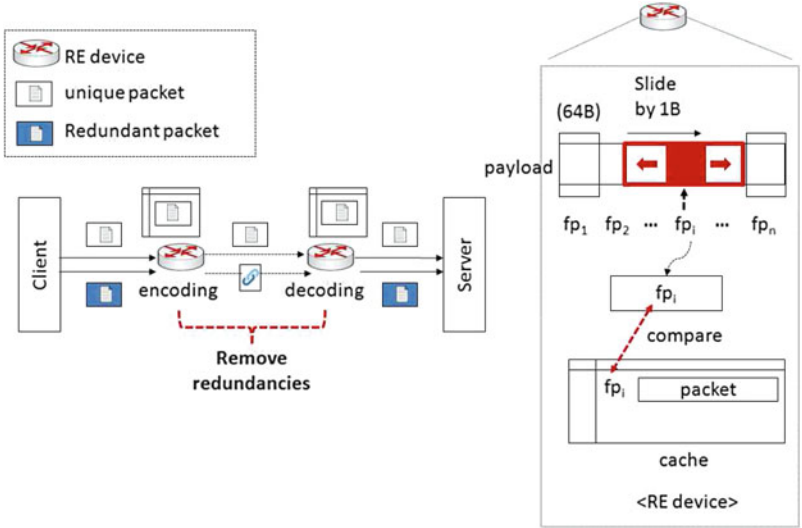


Fig. 2.14 Network-wide redundancy elimination

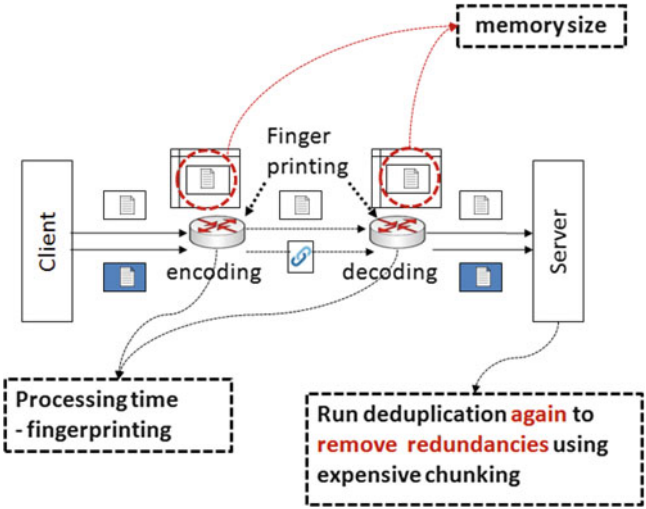


Fig. 2.15 Network-wide redundancy elimination: issue

in a decoder before reaching the server. Thus, the server should run deduplication again to remove redundancies using expensive chunking. That is, there are redundant deduplication operations in the network as well as on the server. We address this issue by developing SoftDance in Chap. 5.

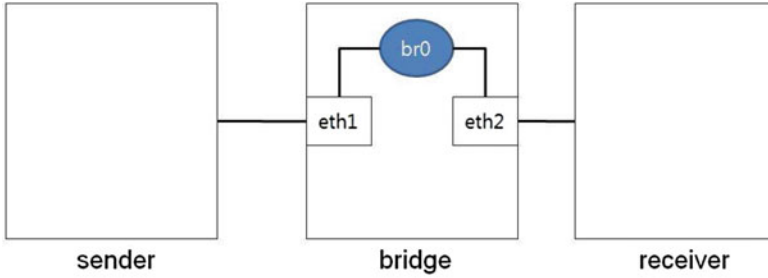


Fig. 2.16 Example deployment – Linux Bridge

2.4.4.2 Implementation: Linux Bridge

To deduplicate payloads in packets, a packet should be captured on a router, and the redundant payload (or byte strings in the payload) is (are) replaced by small indexes. The current router itself does not support these processes, so we need to use different types of router, like software-defined switches or middlebox based on a generic server. We show that a middlebox acts as a router and performs deduplication. In this example of implementation, we use a Linux system as a middlebox. To capture a packet, we set up a Linux bridge in a Linux system, capture an incoming packet and check whether the payload is redundant based on the index of the payload of packets that have been passed. This section shows how to set up a Linux bridge.

Figure 2.16 shows a basic Linux bridge that forwards a packet in Layer 2. A bridge is not a router. A sender and a receiver are in the same network. We show how to deploy a Linux bridge. The deployed environment is one in which there are three Linux computers. One has two network interface cards (NICs) for a bridge and one NIC for Internet access on which ‘brctl’ is installed. The two other computers have just one NIC. This example deployment is based on Ubuntu 12.04 LTS. The IP address of the bridge is 192.168.2.4, and those of the other two computers (a sender and a receiver) are 192.168.2.11 and 192.168.2.5 respectively. In this example deployment, we set an IP address on a bridge. However, if we do not have to access the bridge, it is okay not to set the IP address on the bridge. The first step is to install ‘brctl’ on a computer to be used as a Linux bridge as follows.

```

root@bridge:~# apt-get update
:
root@bridge:~# apt-get install bridge-utils
:
root@bridge:~# brctl show
bridge name      bridge id        STP enabled      interfaces

```

The next step is to create and configure a bridge, called br0, on the Linux computer, and connect two ports to the new bridge. That is, two ports communicate through the bridge. One port acts as an incoming or outgoing port interchangeably. After tying two ports to a bridge (br0), the bridge is set up with an IP address.

We enable promiscuous mode for two ports (eth1 and eth2). By typing 'brctl show', we find that a bridge (br0) is bound to two ports such as eth1 and eth2.

```
root@bridge:~# brctl addbr br0
root@bridge:~# brctl show
bridge name      bridge id          STP enabled      interfaces
br0              8000.000000000000  no
// ← we create a bridge (br0)

root@bridge:~# brctl addif br0 eth1
root@bridge:~# brctl addif br0 eth2
root@bridge:~# ifconfig br0 192.168.2.4 netmask 255.255.255.0 up
root@bridge:~# ifconfig eth1 0 promisc up
root@bridge:~# ifconfig eth2 0 promisc up

root@bridge:~# brctl show
bridge name      bridge id          STP enabled      interfaces
br0              8000.001018076b3d  no               eth1
                                                         eth2
```

The next step is to set a Linux parameter to forward traffic as follows. We change ip_forward under the '/proc/sys/net/ipv4/' directory from 0 to 1.

```
root@bridge:~# cat /proc/sys/net/ipv4/ip_forward
0
root@bridge:~# echo 1 > /proc/sys/net/ipv4/ip_forward
root@bridge:~# cat /proc/sys/net/ipv4/ip_forward
1
```

What follows is the result showing that all connections work properly. First, it shows a connection from a bridge to both end systems (a sender and a receiver). Second, we check whether a bridge is working properly by pinging from a sender to a receiver, and vice versa.

```
// Connection test to both ends
- bridge → receiver
root@bridge:~# ping 192.168.2.5
PING 192.168.2.5 (192.168.2.5) 56(84) bytes of data:
64 bytes from 192.168.2.5: icmp_req=1 ttl=64 time=0.678 ms
64 bytes from 192.168.2.5: icmp_req=2 ttl=64 time=0.369 ms
— 192.168.2.5 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 0.369/0.523/0.678/0.156 ms

- bridge → sender
root@bridge:~# ping 192.168.2.11
PING 192.168.2.11 (192.168.2.11) 56(84) bytes of data:
64 bytes from 192.168.2.11: icmp_req=1 ttl=64 time=254 ms
64 bytes from 192.168.2.11: icmp_req=2 ttl=64 time=1.46 ms
— 192.168.2.11 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.467/127.972/254.478/126.506 ms

// check if a bridge works
```

```

- sender -> receiver
[root@sender ~]# ping 192.168.2.5
PING 192.168.2.5 (192.168.2.5) 56(84) bytes of data.
64 bytes from 192.168.2.5: icmp_seq=1 ttl=64 time=83.9 ms
64 bytes from 192.168.2.5: icmp_seq=2 ttl=64 time=2.17 ms
64 bytes from 192.168.2.5: icmp_seq=3 ttl=64 time=1.57 ms
:

- receiver -> sender
root@receiver:~# ping 192.168.2.11
PING 192.168.2.11 (192.168.2.11) 56(84) bytes of data.
64 bytes from 192.168.2.11: icmp_req=1 ttl=64 time=126 ms
64 bytes from 192.168.2.11: icmp_req=2 ttl=64 time=2.32 ms
64 bytes from 192.168.2.11: icmp_req=3 ttl=64 time=1.47 ms

```

2.4.4.3 Implementation: Packet Flow in Netfilter

After we deploy a Linux bridge, the next question is how and where to capture a packet. For this purpose, we need to understand packet flow in Netfilter within the Linux network stack. As shown in Fig. 2.17, an incoming packet flows through Netfilter modules in the Linux operating system. Each rectangle means a process for a packet. For example, a rectangle with ‘filter’ (upper part of the rectangle) and ‘forward’ (lower part of the rectangle) means that a packet is processed in a forward chain based on the iptable rule (‘filter’). Thus, to capture (intercept) a forwarding packet, we add an iptable rule of the forward chain as follows:

```

iptables -I FORWARD -p tcp -j NFQUEUE --queue-num 0

```

-I selects a chain among INPUT, FORWARD, and OUTPUT chains. -p indicates a protocol by which a packet is delivered. -j means a target at which a packet is intercepted; in this case, a packet is forwarded to NFQUEUE where packets are buffered before being forwarded to a packet processing application. --queue-num means the number of the NFQUEUE. The valid queue numbers are 0 to 65536, and NFQUEUE 0 indicates the first NFQUEUE.

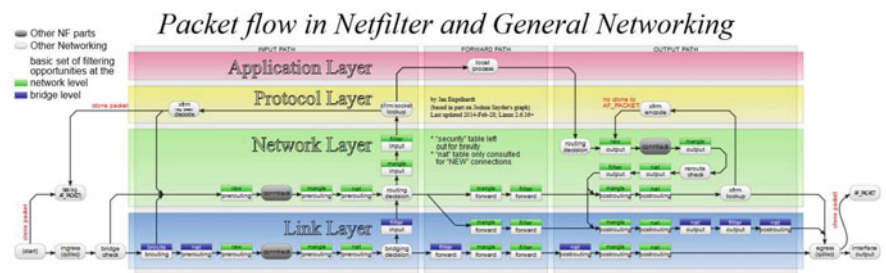


Fig. 2.17 Packet flow in Netfilter [37]

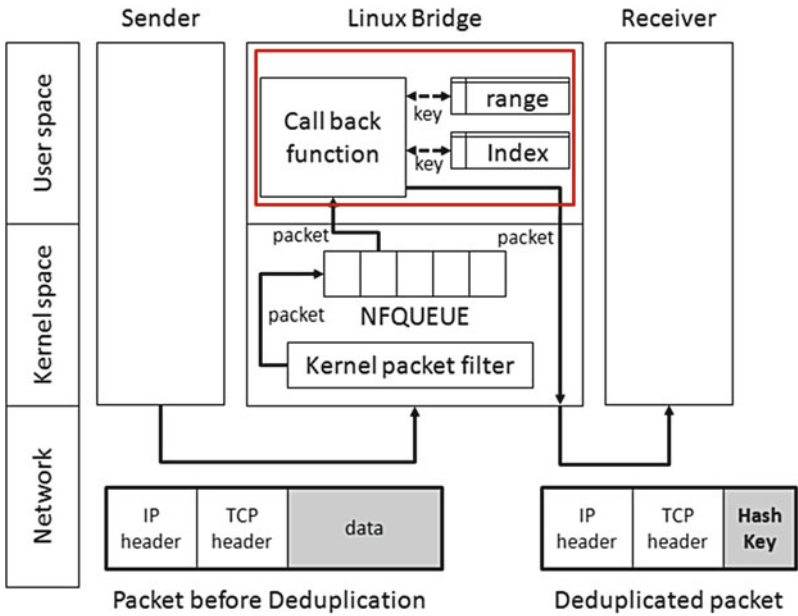


Fig. 2.18 NFQUEUE in Linux bridge

Figure 2.18 illustrates a Linux bridge intercepting an incoming packet. An incoming packet is intercepted from Netfilter (forward chain) and forwarded to NFQUEUE. The packet in NFQUEUE is sent to a call-back function (explained in the next section, `libnetfilter_queue`) which changes the duplicate payload of a packet to a small index (or changes duplicate byte strings in the payload to small indexes). Then, a deduplicated packet is sent out through an outgoing interface to a receiver.

2.4.4.4 Implementation: Packet Capture: `libnetfilter_queue`

`libnetfilter_queue` is a user space library providing an API to packets that have been queued to a kernel packet filter. We can implement a network deduplication program using the `libnetfilter_queue` library. In this section, we show how to install the `libnetfilter_queue` library and then how to run a simple network deduplication program based on the library. This installation is done with Ubuntu 12.04.

First, we need to install prerequisite packages such as `nfnetlink` and `libmnl` for `libnetfilter_queue`. To install an `nfnetlink`, ‘`apt-get install`’ is used with the package name, `libnfnetlink-dev`. For the `libmnl` package, `libmnl` higher than 1.0.3 is needed for `libnetfilter_queue`. (In default, Ubuntu 12.04 has a lower version of `libmnl` than 1.0.3.) Thus, we download `libmnl_1.0.3` source codes and install by typing ‘`./configure`’, ‘`make`’ and ‘`make install`’ in that order. To extract the bzip2 file, ‘`tar -xjvf <file name>`’ is used.

```

root@server:~# apt-get install libnfnctlink-dev
:
Unpacking libnfnctlink-dev
Setting up libnfnctlink-dev (1.0.0-1) ...
root@server:~#

root@server:~# wget https://launchpad.net/ubuntu/+archive/
primary/+files/libmnl_1.0.3.orig.tar.bz2
:
Length: 337375 (329K) [application/octet-stream]
Saving to: 'libmnl_1.0.3.orig.tar.bz2'

100%[=====>]

'libmnl_1.0.3.orig.tar.bz2' saved [337375/337375]

root@server:~# ls
:
libmnl_1.0.3.orig.tar.bz2
:

root@server:~# tar -xjvf libmnl_1.0.3.orig.tar.bz2
libmnl-1.0.3/
:
libmnl-1.0.3/examples/rtnl/rtnl-route-dump.c

root@server:~# ls
:
libmnl-1.0.3

root@server:~# cd libmnl-1.0.3
root@server:~/libmnl-1.0.3# ./configure
root@server:~/libmnl-1.0.3# make
root@server:~/libmnl-1.0.3# make install

```

We install `libnetfilter_queue` library. We download the source code of the library using the `'wget'` command. The latest version is 1.0.2. To extract the downloaded compressed file with `bzip2`, `'tar -xjvf <file name>'` is used. We type `'./configure'`, `'make'` and `'make install'` to compile and build `libnetfilter_queue` library. The final step is to set `'LD_LIBRARY_PATH'` to the `'/usr/local/lib'` directory where shared library files are located.

```

root@server:~# wget http://www.netfilter.org/projects/
libnetfilter_queue/files/libnetfilter_queue-1.0.2.tar.bz2

root@server:~# ls
libnetfilter_queue-1.0.2.tar.bz2 ...

root@server:~# tar -xjvf libnetfilter_queue*.bz2
libnetfilter_queue-1.0.2/
:
root@server:~# ls
libnetfilter_queue-1.0.2 ....

```

```

root@server:~# cd libnetfilter_queue-1.0.2
root@server:~/libnetfilter_queue-1.0.2# ./configure
root@server:~/libnetfilter_queue-1.0.2# make
root@server:~/libnetfilter_queue-1.0.2# make install

// set up LD_LIBRARY_PATH
root@server:~/nfqueue# export LD_LIBRARY_PATH=/usr/local/lib

root@server:~/nfqueue# env | grep LD_LIBRARY_PATH
LD_LIBRARY_PATH=/usr/local/lib

```

2.4.4.5 Implementation: Network dedup Sample Program Using libnetfilter_queue

Network deduplication sample programs using the libnetfilter_queue library are in Appendix G. The sample program works as a call back function. In this section, we show how to compile and build an executable file for the sample program. Then we demonstrate testing of this program to intercept and process a packet. The process is to change the lowercase letters of a payload in a packet to uppercase letters. To compile and build, we type ‘make’. Then an executable file, nd, is created.

```

root@server:~/nfqueue# make
g++ -DNDEDUP_TEST -o nd ndedup_main.cc ndedup.cc -lnetfilter_queue
-lnfnetlink

```

Prepare Testing For testing, we add a rule in iptable using the ‘INPUT’ chain as follows. ‘-dport 50000’ means that a packet only destined to the 50000 port is captured. In this example, we use the ‘INPUT’ chain for a receiver, but we can change the ‘INPUT’ chain to a ‘FORWARD’ chain for a forwarder. In either case, packets are intercepted and forwarded to the sample program through NFQUEUE. To check whether a rule has been added, we type ‘iptables -L -n’. We see there is a rule starting from ‘NFQUEUE tcp ...’. Then we run the sample program by typing the executable file ‘nd’.

```

iptables -I INPUT -p tcp -j NFQUEUE --dport 50000 --queue-num 0
--queue-bypass

```

```

root@server:~/nfqueue# iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source      destination
NFQUEUE    tcp  --  0.0.0.0/0   0.0.0.0/0   tcp dpt:50000 NFQUEUE
num 0 bypass

```

```

Chain FORWARD (policy ACCEPT)
target     prot opt source      destination

```

```

Chain OUTPUT (policy ACCEPT)
target     prot opt source      destination

```

```

root@server:~/nfqueue# nd
opening library handle
unbinding existing nf_queue handler for AF_INET (if any)
binding nfnetlink_queue as nf_queue handler for AF_INET
binding this socket to queue '0'
setting copy_packet mode
// <— network dedup module is waiting for packets.

```

Initial Connection As an example, we open two terminals. One terminal is used for a sender, and the other terminal is used for a receiver. Please note that we need three terminals: one for a sender, another for a receiver and the last one for a network deduplication sample program. We run a TCP server with port 50000 using 'nc -l 50000' at a receiver. We also run a TCP client connecting to a receiver in a localhost.

```

// receiver
root@server:~# nc -l 50000
// waiting to receive a message

// sender
root@server:~# nc localhost 50000
// waiting to send a message

```

The following shows standard output that is printed by the network deduplication sample program when a sender and a receiver are connected through an initial TCP connection. The output contains an IP header and a TCP header. The IP header information includes the IP version, header length, total packet length, IP identification number, TTL, protocol (TCP), checksum, source IP address and destination IP address. The TCP header information contains the port numbers of the sender and receiver, sequence number, acknowledgement of sequence number, TCP header length, flags (urgent, ack, push, rst, syn, fin), window size and checksum. As we see, when a sender is connected to a receiver, two packets are transferred to each other: one is a SYN packet from a sender to a receiver, and the other is an acknowledge (ACK) packet from a receiver to a sender.

```

// output by network deduplication sample program
pkt received
**** IP header ****
version      : 4
header length : 20 (byte)
total length  : 60 (byte)
id           : 14648
ttl          : 64
protocol     : 6
checksum     : 0x382
source       : 127.0.0.1
destination  : 127.0.0.1
**** TCP header ****
sport        : 55492
dport        : 50000
seq          : 2993819844
ack seq      : 0
header length : 40 (byte)

```

```

flag (urgent) : 0
flag (ack)    : 0
flag (push)   : 0
flag (rst)    : 0
flag (syn)    : 1
flag (fin)    : 0
window size   : 43690 (byte)
checksum      : 0x1301
entering callback

pkt received
**** IP header ****
version       : 4
header length : 20 (byte)
total length  : 52 (byte)
id            : 14649
ttl           : 64
protocol      : 6
checksum      : 0x389
source        : 127.0.0.1
destination   : 127.0.0.1
**** TCP header ****
sport         : 55492
dport         : 50000
seq           : 2993819845
ack seq       : 1922593124
header length : 32 (byte)
flag (urgent) : 0
flag (ack)    : 1
flag (push)   : 0
flag (rst)    : 0
flag (syn)    : 0
flag (fin)    : 0
window size   : 342 (byte)
checksum      : 0xce56
entering callback

```

Packet Payload Change by Sample Program After a connection between sender and receiver is established, we send a message with ‘hello deduplication’ from a sender to a receiver. The following results show that a sender sends the ‘hello deduplication’ message (which has all lowercase letters) to a receiver. Then a receiver receives a ‘HELLO DEDUPLICATION’ message (with all uppercase letters) that was changed by the network deduplication sample program in a Linux bridge.

```

// sender side
root@server:~# nc localhost 50000
hello deduplication

// receiver side
root@server:~# nc -l 50000
HELLO DEDUPLICATION

```

```
// output by network deduplication sample program
pkt received
**** IP header ****
version      : 4
header length : 20 (byte)
total length  : 72 (byte)
id           : 14650
ttl          : 64
protocol     : 6
checksum     : 0x374
source       : 127.0.0.1
destination  : 127.0.0.1
**** TCP header ****
sport        : 55492
dport        : 50000
seq          : 2993819845
ack seq      : 1922593124
header length : 32 (byte)
flag (urgent) : 0
flag (ack)    : 1
flag (push)   : 1
flag (rst)    : 0
flag (syn)    : 0
flag (fin)    : 0
window size   : 342 (byte)
checksum      : 0x681d
**** Block ****
begin offset  : 52
end   offset  : 71
size         : 20
**** nf_payload(IP header + TCP header + TCP data) ****
>>> before modification
ip   checksum = 0x374
tcp  checksum = 0x681d
45000048393A4000400603747F0000017F000001D8C4C350B27210C
57298716480180156681D00000101080A08E6708E08E629FC68656C
6C6F2064656475706C696361746966F6E0A
HELLO DEDUPLICATION
>>> after modification
ip   checksum = 0x374
tcp  checksum = 0xa91e
45000048393A4000400603747F0000017F000001D8C4C350B27210C
57298716480180156A91E00000101080A08E6708E08E629FC48454C
4C4F2044454455504C49434154494F4E0A
entering callback
```

The output of the sample program shows the information of a payload (shown as a block) including ‘begin offset’, ‘end offset’ and ‘size’. ‘begin offset’ and ‘end offset’ are literally the beginning and ending offsets of a packet. ‘size’ is the size of the payload (‘hello deduplication’ including a newline character), which is 20 bytes. After the ‘**** Block ****’ section, it shows some useful information, including the change of checksum after modification of the payload. IP checksum (0x374) is not changed because the size of the payload is not changed, which means no

change is made to the IP header. However, TCP checksum (0x681d $\rightarrow$ 0xa91e) is changed owing to the change in payload. The output also shows hexa codes of a packet before modification and after modification.

2.4.4.6 Existing Solution

One study [3] proposes the network-wide deployment of RE technology. Authors assume that the routers have the ability to detect and encode redundant content from network packets on the fly by comparing packet contents that were stored in a cache previously. In this approach, unique packets and the corresponding fingerprints of bytes in the packet payload are saved to a packet store and fingerprint store. When a packet arrives at a router, a small window slides on the payload in a packet, and fingerprints are computed for all windows. From among all the fingerprints representative ones are selected randomly. If the same fingerprints are found in the cache, the matched region from the pointed byte regions on a payload are expanded both to the left and to the right while comparing the two packets (incoming packet and packet in cache). The expanded region is replaced by a small shim header.

Figure 2.19 illustrates how many redundant packets are removed. Figure 2.19a shows the traditional shortest path routing where 18 packets are transferred from a sender to two destinations, D_1 and D_2 . Using RE on the routers, packet P_1 on each link is removed, as shown in Fig. 2.19b, which is a 33 % reduction of the total packets. This study proposes redundancy-aware routes based on the redundancy profile (which explains how often content is replicated across different destinations) for intra- and inter-domain routing. Figure 2.19c supports the idea that redundancies are further reduced using redundancy-aware routing, which amounts to a 44 % reduction of the total packets.

2.5 Deduplication Techniques by Time

2.5.1 Inline Deduplication

Inline deduplication is a deduplication technique that removes redundancies before data are stored on disk. Inline deduplication can be applied to primary workloads like email, user directories, databases and secondary workloads like archives and backups. Figure 2.20 elaborates how inline deduplication works for primary workloads (latency sensitive) as well as secondary workloads (throughput sensitive). For primary workloads, as shown in Fig. 2.20a, deduplication runs on a direct write and read path. When a user or client writes data, deduplication intercepts the data and checks for redundancies. Only unique data and indexes are saved to storage along with cache. Applications using primary workloads are highly latency sensitive; thus, deduplication typically uses in-memory cache to reduce disk I/O

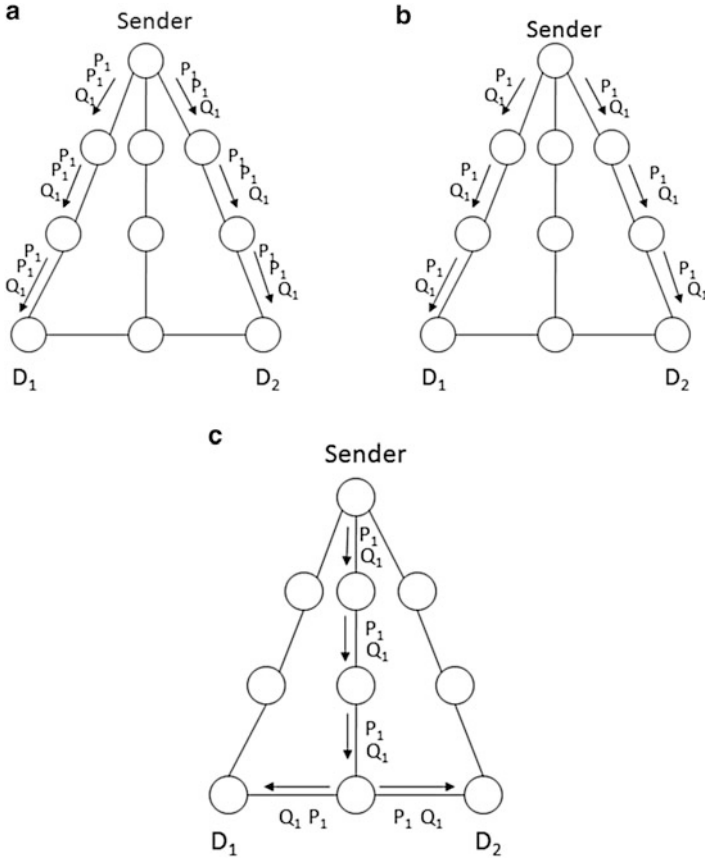


Fig. 2.19 Redundant traffic elimination with packet caches on routers. (a) No RE. (b) RE. (c) RE with redundancy-aware routing

requests. Figure 2.20b shows how deduplication works for secondary workloads. In these workloads, deduplication runs when data are archived or backed up on a backup server. The backup server does not maintain additional storage.

Inline deduplication has been proposed to remove redundancies for the primary workload [14, 44] and secondary workload [9, 26, 36, 50] without incurring extra space overhead or requiring more disk bandwidth. However, this approach requires latency overhead in a write path. iDedup [44] exploits temporal locality and spatial locality to maintain fast processing times in a write path. Content address storage (CAS) systems [17, 39] run inline deduplication because blocks are addressed by their fingerprints. A few file systems [6, 42] use inline deduplication for primary storage.

Inline deduplication [9, 13] runs deduplication before data are saved to disk storage. iDedup [44] has been proposed for inline deduplication for a primary workload.

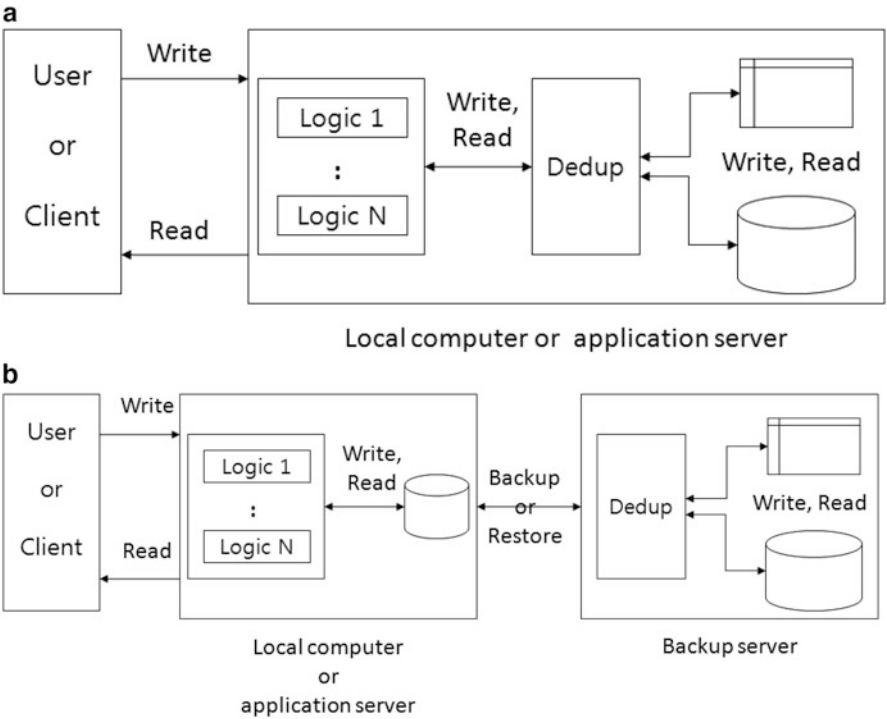


Fig. 2.20 Inline deduplication. (a) Inline deduplication for primary workloads. (b) Inline deduplication for secondary workloads

iDedup exploits spatial locality and temporal locality to achieve high performance (running time). For spatial locality, iDedup performs selective deduplication and mitigates the extra seek time for sequentially read files. For this purpose, iDedup examines blocks at write time and deduplicates full sequences of file blocks if and only if the sequences of blocks are (1) sequential in the file and (2) have duplicates that are sequential on disk. For temporal locality, iDedup maintains dedup-metadata as a *Least Recently Used (LRU)* cache by which iDedup avoids dedup-metadata I/O.

2.5.2 Offline Deduplication

Offline deduplication [1, 15, 21] runs deduplication after data are stored on disk; thus, it does not involve latency overhead in a write path but requires extra storage space. As shown in Fig. 2.21, data are saved to storage without deduplication. Offline deduplication runs out of a critical write and read path using already saved data, which does not hurt latency to write and read data. However, offline deduplication has several drawbacks: (1) extra disk space is needed to hold data

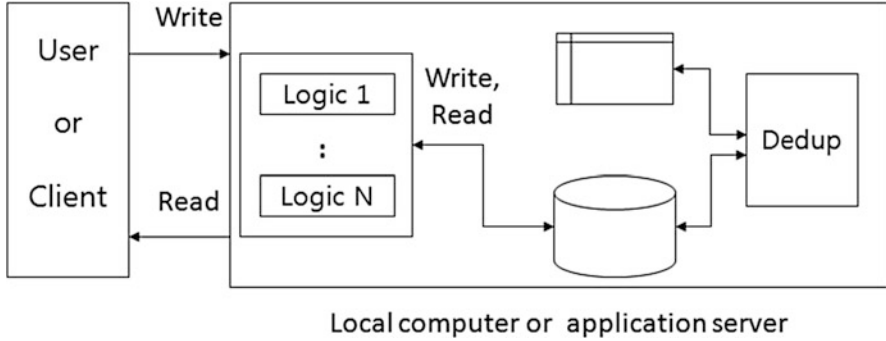


Fig. 2.21 Offline deduplication

temporarily before deduplication, (2) deduplication runs on system idle time, so deduplication can be very delayed if the system is running almost all the time, and (3) data on disk are loaded to memory for deduplication, so disk bandwidth is unnecessarily consumed.

ChunkStash [9] is a flash-assisted inline deduplication system where chunk metadata (with chunk index as key, and with chunk location and length as value) are saved to flash memory rather than disk. Considering that flash memory is 50 times faster than disk, ChunkStash reduces the penalty of index lookup misses in RAM, which increases inline deduplication throughput. ChunkStash also uses in-memory hash tables using the variant of cuckoo hashing [38], and compact key signatures rather than full keys are stored in the hash table, which reduces RAM size.

HYDRAsstor [13] is a grid of storage nodes. It works based on a distributed hash table (DHT) to save blocks to distributed storages. HYDRAsstor uses inline deduplication based on immutable and content-addressed and variable-sized blocks, data resilience by erasure coding, load balancing, and preservation of locality of data streams by prefetching. HYDRAsstor achieves scalability (by DHT), efficient utilization (by deduplication), fault tolerance (by data resiliency) and system performance (by load balancing, locality and prefetching).

2.6 Summary

In this chapter, we showed techniques for deduplication. We classified deduplication techniques based on various criteria, including granularity, deduplication place and deduplication time. We explained fundamental deduplication components, including chunk index cache and Bloom filters, along with implemented codes. Based on these criteria, we illustrated deduplication techniques such as file-level deduplication, fixed-size block deduplication, variable-sized block deduplication, server-based deduplication, client-based deduplication, end-to-end RE, network-wide RE, inline deduplication and offline deduplication.

References

1. Alvarez, C.: Netapp deduplication for FAS and v-series deployment and implementation guide (TR-3505). <http://www.netapp.com/us/media/tr-3505.pdf> (2011)
2. Amazon: Amazon simple storage service. <http://aws.amazon.com/s3/>
3. Anand, A., Gupta, A., Akella, A., Seshan, S., Shenker, S.: Packet caches on routers: the implications of universal redundant traffic elimination. In: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication (2008)
4. Anand, A., Sekar, V., Akella, A.: SmartRE: an architecture for coordinated network-wide redundancy elimination. In: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication (2009)
5. Bolosky, W., Corbin, S., Goebel, D., Douceur, J.: Single instance storage in Windows 2000. In: Proceedings of the 4th USENIX Windows Systems Symposium (2000)
6. Bonwick, J.: ZFS deduplication. https://blogs.oracle.com/bonwick/entry/zfs_dedup (2009)
7. Cisco: Wide area application services. <http://www.cisco.com/c/en/us/products/routers/wide-area-application-services/index.html>
8. Citrix: Cloudbridge. <http://www.citrix.com/products/cloudbridge/overview.html>
9. Debnath, B., Sengupta, S., Li, J.: ChunkStash: speeding up inline storage deduplication using flash memory. In: USENIX Annual Technical Conference (2010)
10. Dong, W., Douglass, F., Li, K., Patterson, R.H., Reddy, S., Shilane, P.: Tradeoffs in scalable data routing for deduplication clusters. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST) (2011)
11. Drago, I., Mellia, M., Munafo, M., Sperotto, A., Sadre, R., Pras, A.: Inside dropbox: understanding personal cloud storage services. In: Proceedings of the 2012 ACM Conference on Internet Measurement Conference (IMC), pp. 481–494 (2012)
12. Dropbox: <http://www.dropbox.com>
13. Dubnicki, C., Gryz, L., Heldt, L., Kaczmarczyk, M., Kilian, W., Strzelczak, P., Szczepkowski, J., Ungureanu, C., Welnicki, M.: HYDRASor: a scalable secondary storage. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST) (2009)
14. ElShimi, A., Kalach, R., Kumar, A., Oltean, A., Li, J., Sengupta, S.: Primary data deduplication-large scale study and system design. In: USENIX Annual Technical Conference (2012)
15. EMC: Achieving storage efficiency through EMC celerra data deduplication. <http://china.emc.com/collateral/hardware/white-papers/h6265-achieving-storage-efficiency-celerra-wp.pdf> (2009)
16. EMC: Avamar. <http://www.emc.com/backup-and-recovery/avamar/avamar.htm>
17. EMC: Centera: Content Addresses Storage System, Data Sheet. <http://www.emc.com/collateral/hardware/data-sheet/c931-emc-centera-cas-ds.pdf>
18. EMC: Networker. <http://www.emc.com/domains/legato/index.htm>
19. Guo, F., Efstathiopoulos, P.: Building a high-performance deduplication system. In: USENIX Annual Technical Conference (2011)
20. Hu, W., Yang, T., Matthews, J.N.: The good, the bad and the ugly of consumer cloud storage. ACM SIGOPS Oper. Syst. Rev. **44**(3), 110–115 (2010)
21. IBM: IBM white paper: IBM storage tank - a distributed storage system. <https://www.usenix.org/legacy/events/fast02/wips/pease.pdf> (2002)
22. JustCloud: <http://www.justcloud.com/>
23. Kim, D., Choi, B.Y.: HEDS: hybrid deduplication approach for email servers. In: 2012 Fourth International Conference on Ubiquitous and Future Networks (ICUFN) (2012)
24. Kim, D., Song, S., Choi, B.Y.: SAFE: structure-aware file and email deduplication for cloud-based storage systems. In: Proceedings of the 2nd IEEE International Conference on Cloud Networking (2013)
25. Li, J., He, L.W., Sengupta, S., Aiyer, A.: Multimodal object de-duplication. Microsoft Corporation (2009). Patent

26. Lillibridge, M., Eshghi, K., Bhagwat, D., Deolalikar, V., Trezise, G., Camble, P.: Sparse indexing: large scale, inline deduplication using sampling and locality. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST) (2009)
27. Liu, C., Lu, Y., Shi, C., Lu, G., Du, D., Wang, D.: ADMAD: Application-driven metadata aware de-duplication archival storage system. In: Fifth IEEE International Workshop on Storage Network Architecture and Parallel I/Os (SNAPI), pp. 29–35 (2008)
28. Meyer, D.T., Bolosky, W.J.: A study of practical deduplication. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST) (2011)
29. Microsoft: Exchange server 2003. <http://technet.microsoft.com/en-us/library/bb123872%28EXCHG.65%29.aspx>
30. Microsoft: Exchange server 2007. <http://www.microsoft.com/exchange/en-us/exchange-2007-overview.aspx>
31. Min, J., Yoon, D., Won, Y.: Efficient deduplication techniques for modern backup operation. *IEEE Trans. Comput.* **60**, 824–840 (2011)
32. Mozy: <http://mozy.com/>
33. Muthitacharoen, A., Chen, B., Mazières, D.: A low-bandwidth network file system. In: SOSP (2001)
34. National Institute of Standards and Technology (NIST): Secure Hash Standard 1 (SHA-1). <http://csrc.nist.gov/publications/fips/fips180-4/fips-180-4.pdf> (2015)
35. National Institute of Standards and Technology (NIST): Secure hash standard 256 (sha256). <http://csrc.nist.gov/groups/STM/cavp/documents/shs/sha256-384-512.pdf>
36. NEC: Hydrastor. <https://www.necam.com/hydrastor/>
37. Netfilter: Packet Flow. <https://upload.wikimedia.org/wikipedia/commons/3/37/Netfilter-packet-flow.svg>
38. Pagh, R., Rodler, F.F.: Cuckoo hashing. *J. Algorithms* **51**(2), 122–144 (2004). doi:10.1016/j.jalgor.2003.12.002. <http://dx.doi.org/10.1016/j.jalgor.2003.12.002>
39. Quinlan, S., Dorward, S.: Venti: a new approach to archival storage. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST) (2002)
40. Rabin, M.O.: Fingerprinting by random polynomials. Tech. Rep. Report TR-15-81, Harvard University (1981)
41. Riverbed: Steelhead for wan optimization. <http://www.riverbed.com/products/wan-optimization/>
42. Silverberg, S.: SDFS. <http://opendedup.org>
43. Spring, N.T., Wetherall, D.: A protocol-independent technique for eliminating redundant network traffic. In: Proceedings of the ACM SIGCOMM 2000 Conference on Data Communication (2000)
44. Srinivasan, K., Bisson, T., Goodson, G., Voruganti, K.: iDedup: latency-aware, inline data deduplication for primary storage. In: Proceedings of the Tenth USENIX Conference on File and Storage Technologies (FAST) (2012)
45. Symantec: Netbackup. <http://www.symantec.com/netbackup>
46. Symantec: Puredisk. <http://www.symantec.com/netbackup-puredisk>
47. Weiss, M.A.: Data Structures and Algorithm Analysis in C++, 3rd edn. Addison Wesley, Reading, MA (2005)
48. Xia, W., Jiang, H., Feng, D., Hua, Y.: SiLo: a similarity-locality based near-exact deduplication scheme with low RAM overhead and high throughput. In: USENIX Annual Technical Conference (2011)
49. Yan, F., Tan, Y.: A method of object-based de-duplication. *J. Netw.* **6**(12), 1705–1712 (2011)
50. Zhu, B., Li, K., Patterson, H.: Avoiding the disk bottleneck in the data domain deduplication file system. In: Proceedings of the USENIX Conference on File and Storage Technologies (FAST) (2008)

Part II

Storage Data Deduplication

In this part, we show our Hybrid Email Deduplication System (HEDS) and Structure-Aware File and Email Deduplication for Cloud-based Storage Systems (SAFE) approaches to storage data deduplication. In Chap. 3, HEDS is introduced as a server-side deduplication component. HEDS removes redundancies by trading off of file-level and block deduplication for email systems while achieving good storage space savings and low processing overhead. HEDS minimizes data storage size while minimizing the CPU and memory overhead. It performs hybrid data deduplication adaptively, at the granularity of a file level or chunk level, based on the size of emails and the existence of attachments. The implementation of HEDS and evaluation results with real email data sets are presented.

In Chap. 4, SAFE is introduced as a client-based deduplication approach. SAFE is fast and achieves the same space savings as variable-size block deduplication by using structure-based granularity rather than physical chunk granularity for cloud-based storage. SAFE removes redundant objects based on a structure-based granularity instead of using a physical chunk granularity. Unlike traditional deduplication that involves a trade-off between the deduplication ratio and processing overhead, SAFE realizes the benefits of both a high deduplication ratio and low processing overhead.

Chapter 3

HEDS: Hybrid Email Deduplication System

Abstract In this chapter, we show a server-side deduplication component, HEDS (Hybrid Email Deduplication System) for the proposed deduplication framework. HEDS removes redundancies by trading-off of file-level and block deduplication for email systems while achieving good storage space savings and low processing overhead.

3.1 Large Redundancies in Emails

Email is a commonly used method of communication today, and the volume of emails is greatly increasing and requires huge storage space on email servers. Email servers have large amounts of redundancies. For example, an email with multiple recipients is copied into multiple mailboxes, and email threads (where emails on the same topic are repeatedly sent and received with the same or similar attachments) increase redundant attachments. The redundancies in the emails are further increased as they are copied over multiple storage devices for reliability or performance.

The volume of email data can be reduced by properly removing the redundancies. Fixed-size and variable-size block deduplication can be used to find redundant contents in emails. However, fixed-size block deduplication cannot find redundancies between similar emails whose beginning contents are changed due to the offset shifting problem. Variable-size block deduplication efficiently finds redundancies in similar emails but increases processing time overhead because of the expensive chunking associated with sliding windows. File-level deduplication can be used to find redundancies in duplicate emails with multiple recipients quickly, but it cannot find redundancies inside emails and attachments, resulting in low space savings.

Few studies have been conducted on how to remove redundancies in email. Single-Instance Store (SIS) [2] uses file-level deduplication where an email is the unit of compared data. In this approach, only unique email is saved, and redundant emails are linked by pointers, which increases storage space savings by not saving the same emails. However, SIS does not exploit redundancies within email messages and attachments.

Considering the overhead as well as performance of deduplication, we developed the Hybrid Email Deduplication System (HEDS), which trades off file-level and variable-size block deduplication in terms of space savings and index overhead. HEDS separates attachments from emails, runs file-level deduplication on message bodies and separated attachments, and adaptively runs variable-size block deduplication only if data size exceeds a predetermined threshold. This threshold exists because small message bodies and attachments are generally unique, and using block deduplication for small data does not give any performance benefits considering the processing overhead. Evaluation of results using real email data sets shows that HEDS achieves a good deduplication ratio while keeping the CPU and memory overhead manageable. Therefore, it sheds light on inline deduplication for email servers. This chapter is organized as follows. We describe the design and implementation of HEDS in Sect. 3.2. The evaluation results are shown in Sect. 3.9. We conclude the chapter in Sect. 3.10.

3.2 Hybrid System Design

To explain the architecture of HEDS, we begin by presenting an overview of it and then go into detail on each module. HEDS is a server-based deduplication system that consists of six modules: EDMilter, metadata server, chunk index cache, Bloom filter, storage server, and an email deduplication algorithm (EDA). When an email arrives at the Mail Transfer Agent of a receiving Sendmail server, EDMilter intercepts the email and divides it into metadata and content. The content consists of the message body and attachments. EDMilter forwards the content to the EDA through a virtual file system and delivers the metadata to the metadata server. The metadata server holds metadata such as email ID, recipients, and sent date. The EDA deduplicates content that is intercepted by Libfuse through a virtual file system. The EDA plays a key role in the deduplication and communicates with all other modules. We implemented the EDA with filesystem on userspace (FUSE) [3]. The chunk index cache and Bloom filter speed up the processing time by reducing the number of disk accesses. In what follows we explain each module (Fig. 3.1).

3.3 EDMilter

As shown in Fig. 3.2, we have developed the Email Deduplication Milter, referred to as EDMilter, based on the Milter [7] API. Milter is an email filter that intercepts emails coming into the sendmail server. When a sendmail server receives an email, the Milter Library accepts an email from the Mail Transfer Agent (MTA) and passes the email to the EDMilter with a callback. The EDMilter extracts needed metadata from the SMTP header such as a mail id, senders, recipients, the number of

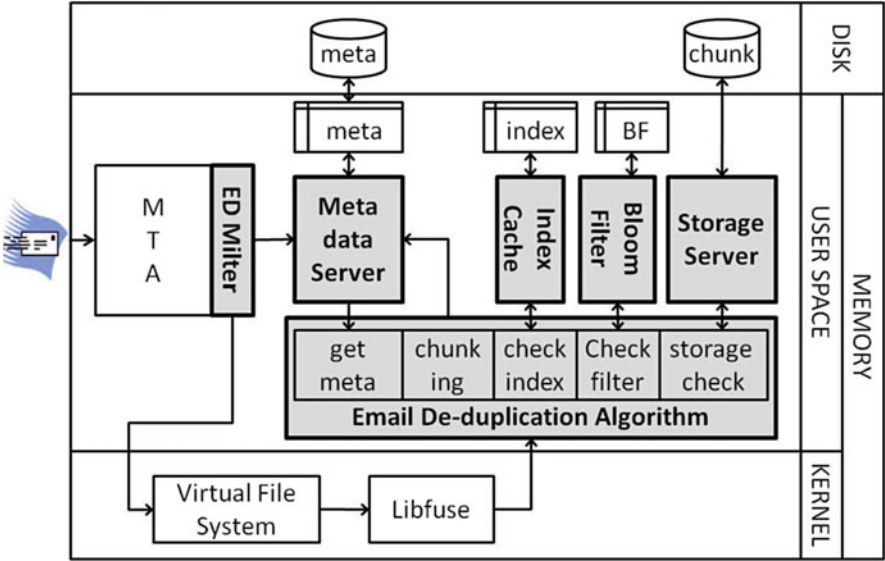


Fig. 3.1 Proposed hybrid email deduplication system (HEDS)

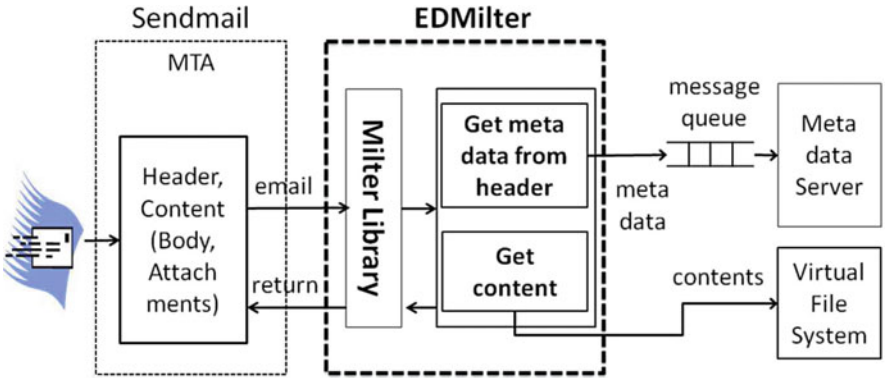


Fig. 3.2 EDMilter

recipients, and the size of email content that comprises of the body and attachments. At the same time, the EDMilter receives the content from the email and requests to save it to a directory that is a mounting point in a virtual file system. EDMilter also sends the email metadata to the metadata server through a message queue.

3.4 Metadata Server

The metadata server saves email metadata and chunk indexes for each email. Email metadata include an email ID comprised of 14 byte strings, recipients, the number of recipients and the size of the email contents. The chunk indexes are received from the EDA when the EDA splits an email content into chunks if the contents exceed the threshold size. Ultimately, the metadata server saves the metadata and chunk indexes to a metadata store. Meanwhile, to speed up reading and writing based on temporal locality, the metadata server maintains metadata and chunk indexes of the latest emails in a metadata cache. Each metadata in the cache has a timestamp to evict old metadata based on Least Recently Used (LRU) in case the cache grows over the cache size limit. The size of the metadata cache is configurable.

3.5 Bloom Filter

A Bloom filter is used to determine whether duplicate chunks of a current email exist in chunk storage. The Bloom filter is a bit array of m bits initially set to 0. Given a set U , each element u ($u \in U$) of the set is hashed using k hash functions $h_1, \dots, h_k$. Each hash function $h_i(u)$ returns an array index in the bit array that ranges from 0 to $m - 1$. Subsequently, a bit of the index is set to 1, in which case it remains 1. The Bloom filter is used to check whether an element was already saved to a set. When an element attempts to be added to the set, if one of the bits corresponding to the return values of hash functions $h_1, \dots, h_k$ is 0, the element is considered a new one in the set. If bits corresponding to return values of hash functions are all 1, the element is considered to exist in the set. However, the Bloom filter has a false positive where it says that an element exists even though it does not. In HEDS, the purpose for using the Bloom filter [1] is to speed up the writing of emails or chunks by reducing the number of disk accesses. Data Domain File System (DDFS) [10] shows how big the memory size of the Bloom filter should be for use with a certain false positive rate; for example, to achieve a 2 % false positive rate, the smallest size of the Bloom filter is $m = 8n$ bits ($m/n = 8$), and the number of hash functions can be four ($k = 4$), where m is the size of the Bloom filter in bits, and n is the key size. We use four hash functions, and the key size is 160 bits, which is the size of the SHA1 [8] hash key. Therefore, the smallest size of the Bloom filter for our case is $8 * 160$ bits = 1280 bits.

3.6 Chunk Index Cache

The chunk index cache is the next level of cache after the Bloom filter; it saves indexes for chunks that are saved as unique chunks. The chunk indexes are classified into three different categories based on the source of the chunk. The source can be

an entire email, an attachment or part of an email text. Chunk indexes of the latest emails are saved to the chunk index cache, and if chunks or a chunk of a current email exists in the chunk index cache, the chunk is not saved to the chunk store on a disk. Because of the limited size of the chunk index cache, old indexes are evicted to chunk storage if the size of cache grows over a certain threshold. Likewise, loaded indexes from the chunk store have new timestamps. The key of each entry in the chunk index cache is a SHA1 hash value of a chunk.

3.7 Storage Server

The storage server checks whether or not the chunks of the current email exist in chunk storage by accessing the disk, saving non-existent chunks to the chunk storage, or reading chunks from chunk storage. We used BerkeleyDB for chunk storage, and pairs of <chunk index, chunk> are saved to the chunk storage.

3.8 EDA

The EDA interacts with all other modules in HEDS. Algorithms 1–3 show how the EDA works on an email with or without attachments. As shown in Algorithm 1, the EDA separates email contents into a message body and attachments, which are further subdivided into individual attachments. As shown in Algorithm 2, the EDA checks the size of a divided attachment or message body. If the size of the data is over a configurable threshold, EDA splits the data into chunks using variable-size chunking based on Rabin fingerprinting [9]. Then, as shown in Algorithm 3, the EDA checks to determine whether each chunk or datum has already been saved based on the Bloom filter and then the chunk index cache. If the chunk index cache misses, the EDA checks indexes in the chunk store on disk. If the Bloom filter says

Algorithm 1 Email Deduplication Algorithm

Input: email content

```

1: if Not Exist(attachments) then                                ▷ email without attachments
2:   messageBody ← email content
3:   HybridDedupDecision(messageBody)
4: else                                                            ▷ email with attachments
5:   separate email content into message body, attachments
6:   HybridDedupDecision(messageBody)
7:   for all each attachment ∈ attachments do
8:     HybridDedupDecision(attachment)
9:   end for
10: end if

```

Algorithm 2 HybridDedupDecision**Input:** data, size_threshold

```

1: if size(data) > size_threshold then                                ▷ variable-size block deduplication
2:   chunks ← variableSizeChunking(data)
3:   for all each chunk ∈ chunks do
4:     checkIndexAndSaveData(chunk)
5:   end for
6: else                                                                ▷ file-level deduplication
7:   checkIndexAndSaveData(data)
8: end if

```

Algorithm 3 Check Index and Save Data**Input:** data**Output:** saved data

```

1: if ExistInBloomFilter(data) then
2:   index ← hash(data)
3:   if ExistInChunkIndexCache(index) then                                ▷ duplicate in cache
4:     return
5:   if ExistInChunkStore(index) then                                ▷ cache miss, duplicate in storage
6:     return
7:   end if
8: end if
9: end if                                                                ▷ unique data
10: index ← hash(chunk)
11: saveToStore(index, chunk)

```

‘no’ (this means the chunk or data are unique), the EDA saves a chunk (or data) and the corresponding index to the store. The reason to check the chunk index cache after the Bloom filter says ‘yes’ (meaning the chunk or data may be redundant) is because of false positives of the Bloom filter: the chunk or data may or may not be redundant. Also, the reason for relying on size is that small message bodies or attachments tend to be unique, and using variable-size block deduplication does not give any benefits considering overhead costs.

Figure 3.3 shows how variable-size block deduplication works in a case where the EDA adaptively runs block deduplication. As shown in Fig. 3.4, the EDA basically runs file-level deduplication if the size is under the threshold. In this case, the EDA does not separate the content into chunks but treats the content as a chunk, that is, no variable-size chunking is necessary, which reduces the index and processing overhead.

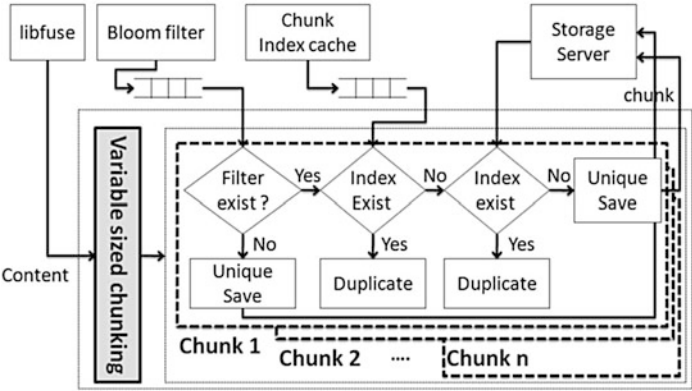


Fig. 3.3 Block deduplication at EDA

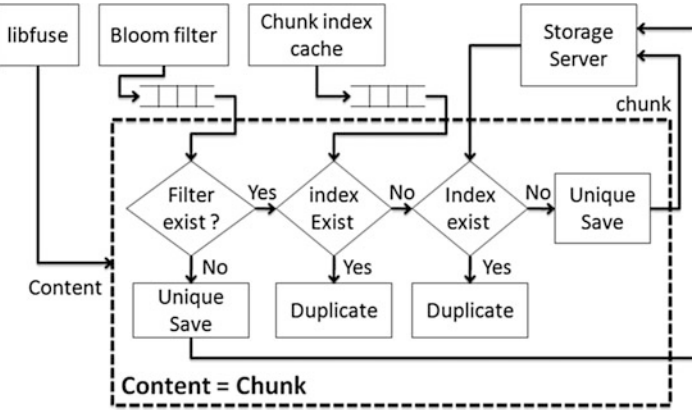


Fig. 3.4 File-level deduplication in EDA

3.9 Evaluation

In this section, we show the performance and overhead of HEDS compared to file-level and variable-size block deduplication. We explain the metrics, data set and measured results.

3.9.1 Metrics

We evaluate HEDS with respect to deduplication performance and the overhead costs of memory and CPU usage with the generated chunk indexes. We compare

Table 3.1 Data sets

	Company data set	Personal data set
Type	Corporate (Enron) email	Gmail
Attachment	Removed	Retained
Number of users	150	1
Number of emails	0.5 million	0.01 million
Size of data set	1.3 GB	1 GB
Duration	1998–2002	2007–2011

HEDS with file-level and variable-size block deduplications. As for the deduplication performance, we use a deduplication ratio that is computed as follows:

$$\text{Dedup ratio} = 1 - (\text{deduped size}/\text{original size}).$$

3.9.2 Data Sets

We experiment with two data sets, including a corporate email data set, called the Enron data set [4], and a single user gmail data set. Table 3.1 shows the summary information of the two data sets.

For the Enron data set experiment, we created 150 mail users for the receiving email server, according to the recipients shown in the data set. With the gmail data set, we created only one email user who receives all emails since the gmail data set belongs to one person. At a sender’s email server, emails were sent one by one sequentially, in the order of emails in the data sets. For all cases, we analyse the deduplication ratios and the overhead of the CPU and memory to process and store the chunk indexes.

We adjusted the threshold size, on the basis of which HEDS performs deduplication, adaptively, on either the file level or a block. To gain insight into the proper threshold size, we observed the distributions of the email sizes in the data sets.

Figures 3.5a and 3.6b display complete distributions, and Figs. 3.5b and 3.6b show distributions with zoomed-in ranges for a closer look. The mean and median email sizes for the Enron data set were about 1.9 and 0.7 KB respectively. Meanwhile, the mean and median email sizes for the gmail data set were around 28 and 5 KB respectively. Note that the Enron data set did not include attachments in the emails. Thus, we select the threshold sizes that are about or greater than the mean value of 1 and 2 KB for the Enron data set. Larger thresholds are chosen, including 51, 128, 16 and 4 KB for the gmail data set. As for the expected average chunk size, we used 0.5 KB for the Enron data set and 2 KB for the gmail data set, considering the means and medians of the data sets. Note that previous deduplication studies [5, 6, 10] used the expected average chunk size ranging from 4 to 64 KB.

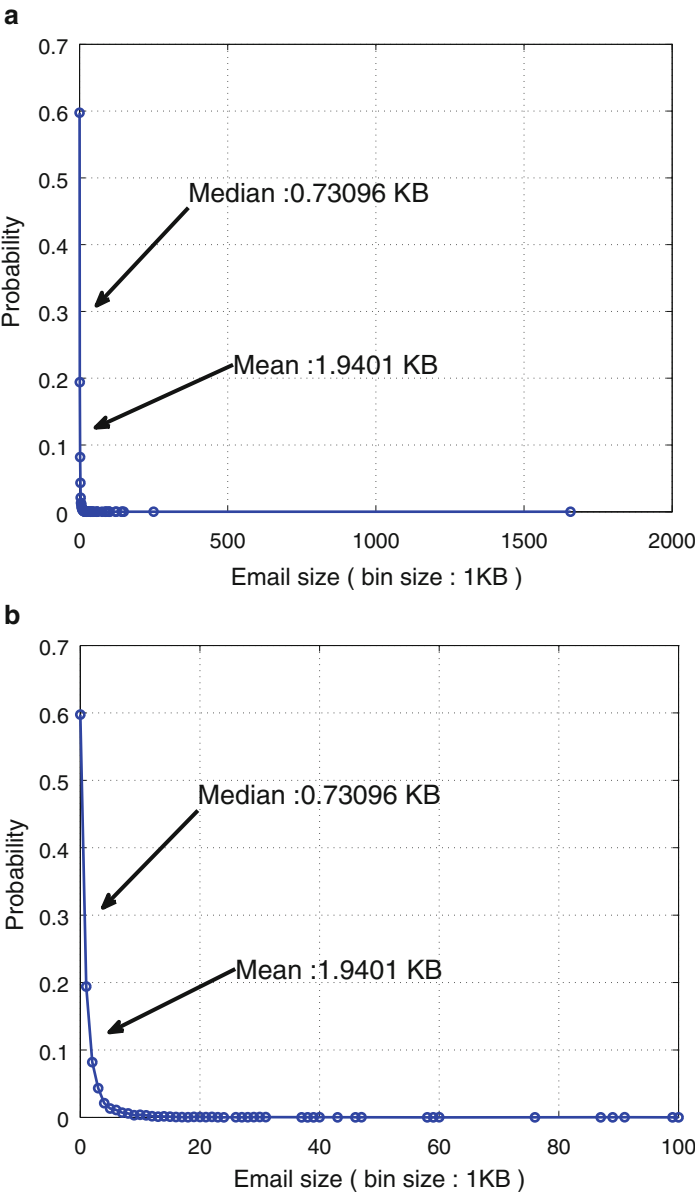


Fig. 3.5 Distribution of Enron corporate email sizes. (a) Complete distribution. (b) Zoomed in to 0–100 KB

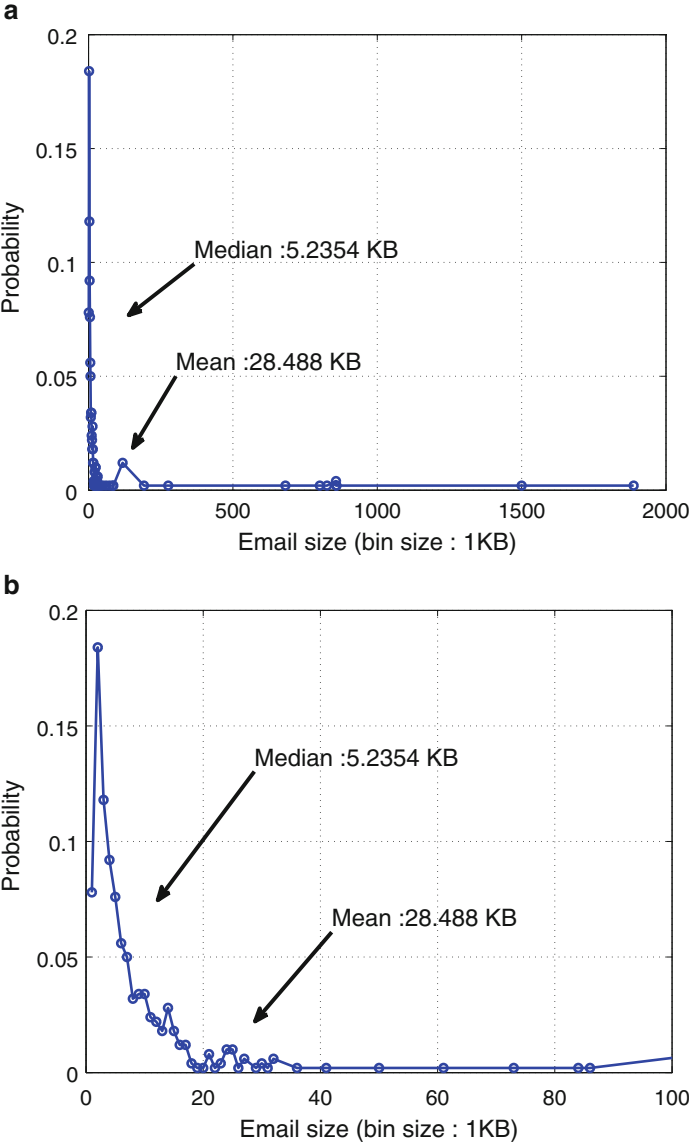


Fig. 3.6 Distribution of gmail personal email sizes. (a) Complete distribution. (b) Zoomed in to 0–100 KB

3.9.3 Deduplication Performance

In this section, we calculate a deduplication ratio that indicates how many redundancies are removed. Overall, we discovered that most redundancies are found in attachments rather than in message bodies. Thus, for corporate (Enron) data sets without attachments, variable-size block deduplication and HEDS showed lower deduplication ratios than file-level deduplication because the low deduplication ratio cannot offset the index overhead. However, for the gmail data set with attachments, variable-size block deduplication and HEDS had a greater deduplication ratio because of the large redundancies in the attachments. In what follows we provide a detailed explanation.

Figure 3.7 shows the deduplication ratio of Enron data sets. ‘App dedup’ means file-level deduplication. ‘Block dedup’ means variable-size block deduplication. ‘Hybrid’ means HEDS with variable thresholds, 1 and 2 KB. All deduplication showed a deduplication ratio over 55 % on average. This means a sending email server sent to two recipients on average. Without the index as shown in Fig. 3.7a, block deduplication and HEDS achieved a 2–3 % higher deduplication ratio than file-level deduplication. However, with the index as shown in Fig. 3.7b, the slight advantage of block deduplication and HEDS is overridden as a result of the chunk index overhead.

For the gmail data set as shown in Fig. 3.8, the deduplication ratio is different compared to the Enron data sets. Figure 3.8a shows deduplication ratios without any chunk index overhead. Since the gmail data set belongs to one person, we saw no benefit in using file-level deduplication. That is, the deduplication ratio of file-level deduplication is 0 %, meaning file-level deduplication cannot reduce the storage size at all. By contrast, block deduplication reduced 15 % more space than file-level deduplication. HEDS with small threshold sizes like 4 KB and 16 KB produces the same space savings as block deduplication, and HEDS with large threshold sizes like 512 KB produces 10 % more space savings than file-level deduplication. Figure 3.8b depicts deduplication ratios including the chunk index overhead. Even after including the overhead, the behavior of the deduplication ratios shown in Fig. 3.8a remains, which indicates that the removed duplicates significantly overcompensate for the overhead.

Next, we investigated the sudden increase in the deduplication ratios observed in Fig. 3.8a with the gmail data set. We found that they were caused by the temporal locality of the attachments. In Table 3.2, we show the changes in a relative deduplication ratio compared to the previous email from the 74th email to the 86th email. The first column shows the email IDs in the data set. The second column displays the received date. The third, fourth, and fifth columns show the size of an entire email, the size of an attached file and the relative deduplication ratio compared to the previous email respectively. Five emails in Table 3.2 are in the same email thread, where each email has the same subject and title of attachment as the other emails. The 74th email shows a 0 deduplication ratio, which shows no deduplication benefit. However, every time the 75th, 80th and 81st emails are received, we acquire

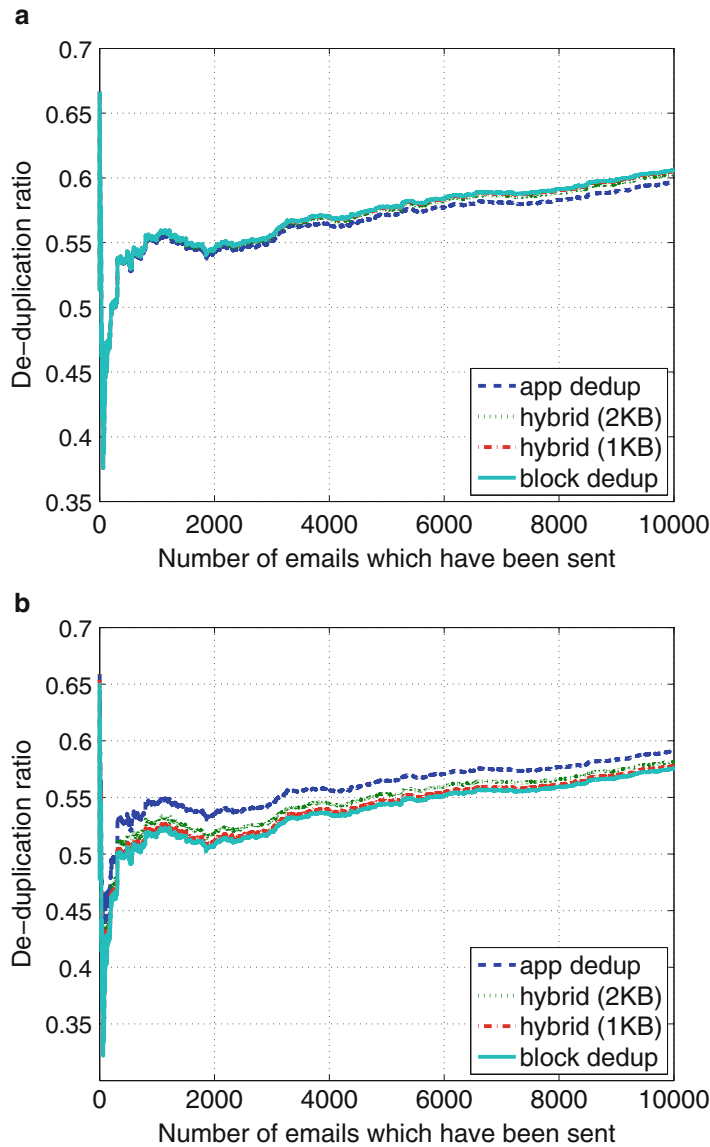


Fig. 3.7 Reduced storage (Enron – corporate data set). (a) Without index. (b) With index

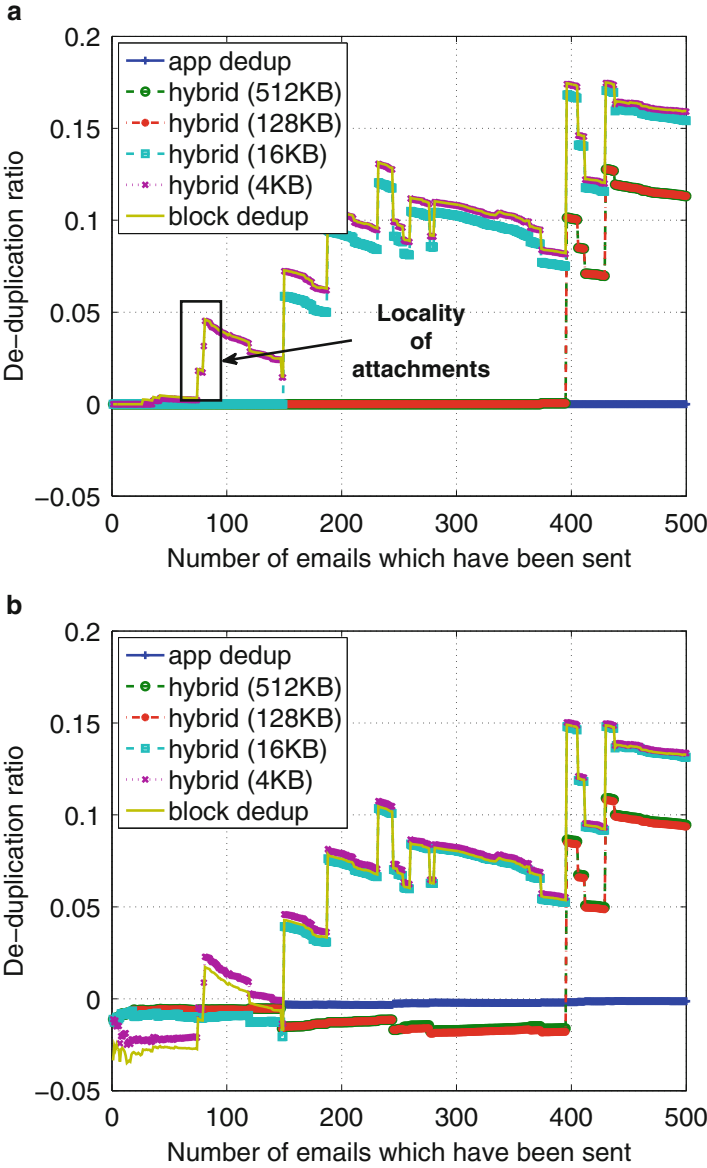


Fig. 3.8 Reduced storage (gmail – single-user data set). (a) Without index. (b) With index

Table 3.2 Locality of attachments

Email ID	Date	Size (bytes)	Attachment size (bytes)	Dedup ratio (%)
74	8 Jan	12,873	9844	0
75	8 Jan	17,805	9844	32.09
80	8 Jan	11,957	9844	33.08
81	8 Jan	12,012	9844	41.07
86	9 Jan	14,896	9593	0

a high deduplication ratio because we do not save the same attachment that was already stored in the 74th email. Interestingly, we see that the 86th email does not show a deduplication ratio, though the title of the attachment is the same as in the other emails. Looking at the attachment, we find that the contents of the attached file in the 86th email have been changed considerably, so even block deduplication cannot find redundancies inside an email. This observation tells us that temporal locality may well be found in emails, and thus, we can exploit the temporal locality with caches.

Table 3.2 illustrates that (1) file-level deduplication does not detect the same attachments in emails with a different message body; (2) block deduplication can detect the same attachments, but it must carry out chunking of the same attachments every time emails are received, resulting in unnecessary CPU and chunk index overhead; furthermore, if the email contents are changed significantly, it does not detect redundant parts in the attachments, as we see at the 86th email; (3) HEDS can detect the same attachments even if a message body is different because it extracts attachments, unlike file-level deduplication. Moreover, an attachment in the next email is not chunked if the hash value of the attachment is found, resulting in less CPU and chunk index overhead over block deduplication.

3.9.4 Memory Overhead

To find a chunk in a new email in the existing chunk index, the chunk index is stored in memory. More chunk indexes means more memory overhead. Here, we evaluate the number of chunk indexes produced with different deduplication approaches. The more chunks an email is separated into, the more chunk indexes are created. For our experiments, our system had large enough memory, and so all the chunk indexes could be stored in memory. In practice, however, memory would contain only a partial chunk index due to the limited cache size and handle continuous incoming emails, which results in a huge demand in the cache. Then a cache management scheme, such as LRU, can be used.

As expected, we find that block-level deduplication shows the largest chunk index overhead, whereas application-level deduplication shows the least overhead. Figure 3.9a, b indicates the accumulated chunk index sizes with the Enron data set

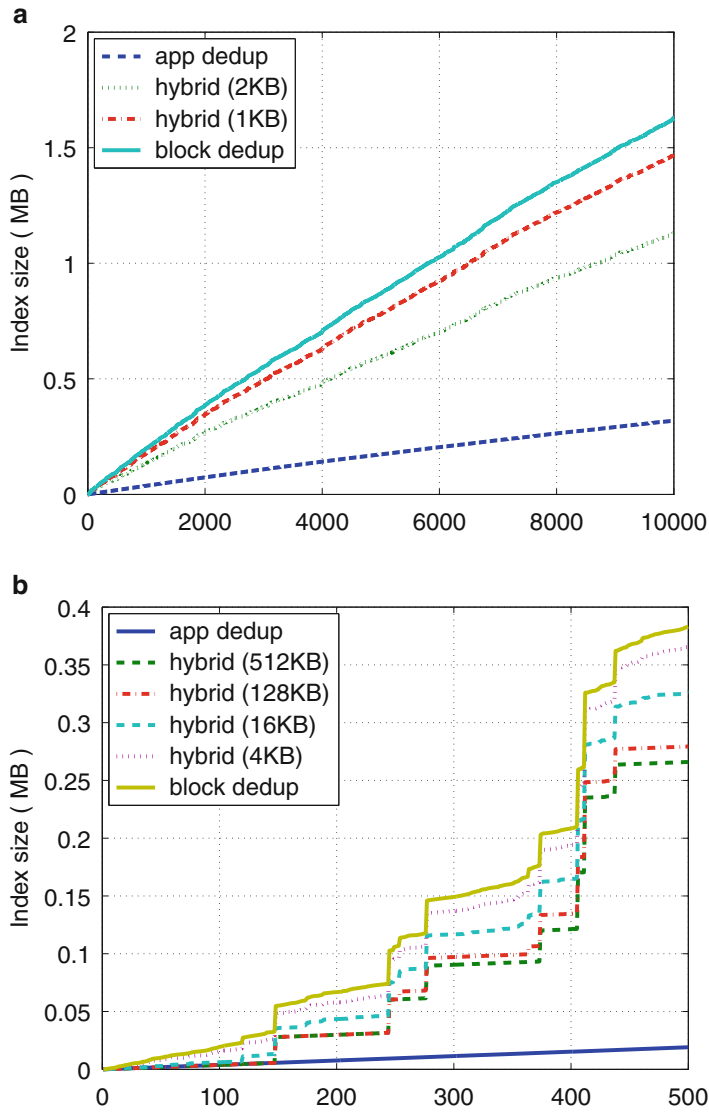


Fig. 3.9 Chunk index overhead. (a) Enron data set. (b) Gmail data set

and the gmail data set respectively. HEDS shows varying chunk index overhead between application-level and block-level deduplication schemes corresponding to different threshold sizes. The chunk index overhead of HEDS with a small size threshold, for example 4 or 16 KB, is almost the same as that of block-level deduplication. It is observed that there are sudden increases in chunk index overhead over time. This is because many chunks are created from a large email that does

not have many redundancies compared to previously saved chunks, resulting in the creation of excessive chunk indexes in memory.

3.9.5 CPU Overhead

Finally, we observe the extra CPU overhead of an application-level, HEDS and a block-level deduplication scheme. Figure 3.10 shows the CPU usage measured with the data sets. As for the Enron data set, application-level deduplication takes 1.6 times as much CPU as sendmail without a deduplication scheme. Block-level deduplication shows the highest CPU usage, and HEDS's CPU usage stays between the two schemes. The gmail data set shows similar relative behavior among the three kinds of schemes but generally shows higher CPU usage because it includes attachments, and thus a greater number of chunks is generated. In short, HEDS achieves a good trade-off between an application-level and a block-level deduplication scheme.

3.10 Summary

We developed a server-based and hybrid deduplication approach (HEDS) for email servers to minimize data storage size while minimizing the overhead of CPU and memory. It performs hybrid data deduplication adaptively, at the granularity of the file level or chunk level, based on the size of emails and the existence of attachments. We implemented and evaluated the hybrid approach on email servers with real email data sets and showed that it achieved a significantly better reduction ratio of storage consumption than file-level deduplication and lower CPU and memory overheads than variable-size block deduplication.

References

1. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* **13**, 422–426 (1970)
2. Bolosky, W., Corbin, S., Goebel, D., Douceur, J.: Single instance storage in Windows 2000. In: *Proceeding of the 4th USENIX Windows Systems Symposium* (2000)
3. FUSE: File in UserSpace. <http://fuse.sourceforge.net/> (2016)
4. Klimt, B., Yang, Y.: The enron corpus: a new dataset for email classification research, pp. 217–226. <http://nyc.lti.cs.cmu.edu/yiming/Publications/klimt-ecml04.pdf> (2004)
5. Lillibridge, M., Eshghi, K., Bhagwat, D., Deolalikar, V., Trezise, G., Camble, P.: Sparse indexing: large scale, inline deduplication using sampling and locality. In: *Proceeding of the USENIX Conference on File and Storage Technologies (FAST)* (2009)
6. Meyer, D.T., Bolosky, W.J.: A study of practical deduplication. In: *Proceeding of the USENIX Conference on File and Storage Technologies (FAST)* (2011)

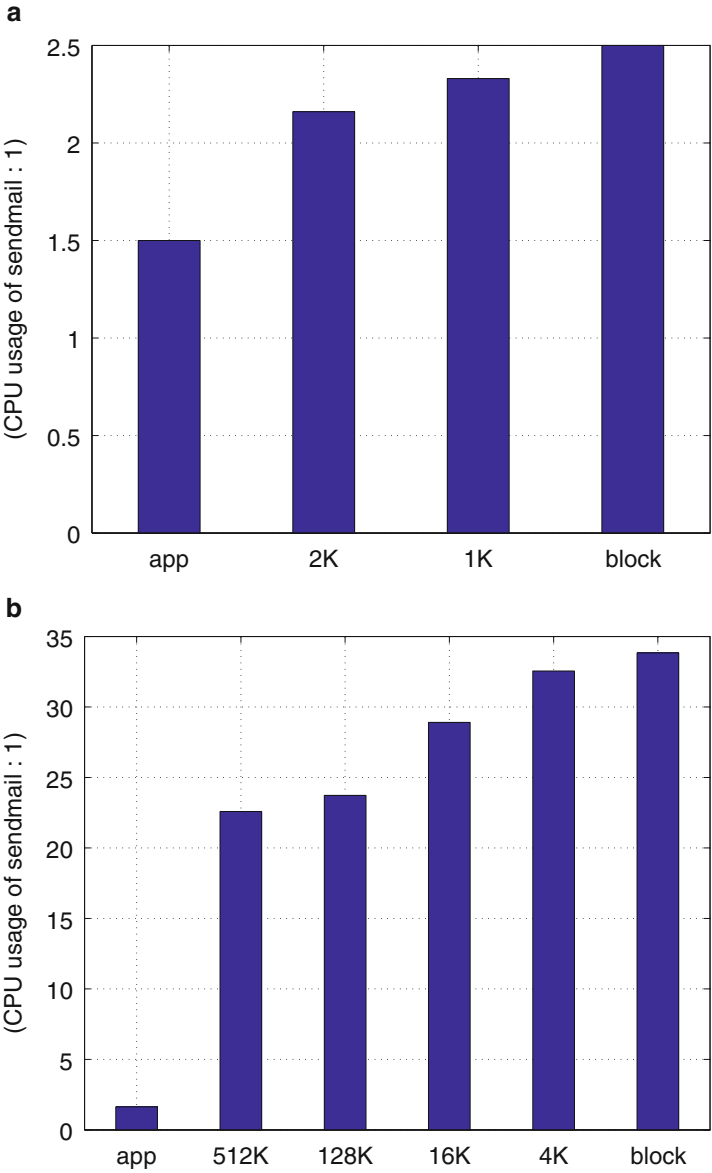


Fig. 3.10 Relative CPU overhead. **(a)** Enron data set. **(b)** Gmail data set

7. Milter.org: Sendmail mail filters. http://www.sendmail.com/sm/partners/milter_partners/open_source_milter_partners/ (2015)
8. National Institute of Standards and Technology (NIST): Secure Hash Standard 1 (SHA1). <http://csrc.nist.gov/publications/fips/fips180-4/fips-180-4.pdf> (2015)
9. Rabin, M.O.: Fingerprinting by random polynomials. Tech. Rep. Report TR-15-81, Harvard University (1981)
10. Zhu, B., Li, K., Patterson, H.: Avoiding the disk bottleneck in the data domain deduplication file system. In: Proceeding of the USENIX Conference on File and Storage Technologies (FAST) (2008)

Chapter 4

SAFE: Structure-Aware File and Email Deduplication for Cloud-Based Storage Systems

Abstract In this chapter, we introduce Structure-Aware File and Email Deduplication for Cloud-based Storage Systems (SAFE), a client-based deduplication technique that is fast and provides the same space savings as variable-size block deduplication using structure-based granularity rather than physical chunk granularity for cloud-based storage. Cloud-based storage, including Dropbox (<http://www.dropbox.com>), JustCloud (<http://www.justcloud.com/>), and Mozy (<http://mozy.com/>), has become popular because people can access data at any time, anywhere and using various types of devices such as laptops, tablets and smartphones. Cloud-based storage services use deduplication techniques to avoid sending and storing duplicate files (or blocks), reducing network bandwidth and storage space, which gives the subsequent benefit of data upload speed. Existing deduplication techniques (file-level and fixed-size block deduplication) that cloud-based storage uses are fast and have low index overhead but find fewer redundancies than variable-size block deduplication. However, owing to excessive CPU and memory overhead from chunking, indexing and fragmentation, variable-size block deduplication cannot be used for cloud-based storage. Thus, we developed SAFE, Structure-Aware File and Email Deduplication, that achieves both fast speeds and shows good space savings in clients by using structure-based granularity for cloud-based storage systems. Evaluation results show that SAFE has as good storage space savings as existing variable-size block deduplication while being as fast as file-level or a large fixed-size block deduplication technique.

4.1 Large Redundancies in Cloud Storage Systems

A structured file is a file that consists of metadata and objects like text and image objects. Typical examples of structured files are compressed files (zip, rar), document files [Microsoft Word document, PowerPoint documents and Portable Document Format (PDF)] and emails. Every day people create large numbers of structured file, and cloud-based storage of document suites contains large amounts of structured files. For example, for one of the data sets that we used, 89 % were structured files and 11 % were unstructured files.

A structured file can be decomposed into various objects with offsets whose positions dynamically change based on the location of the objects. For example, an email is decomposed into multiple objects such as metadata, a message body consisting of text, and attachments. Among attachments, a structured file is further divided into objects like metadata, text, and image objects. Thus, we show that following structures of a file, we can remove redundant objects without expensive chunking.

Based on our observations, we developed SAFE, a fast client-based deduplication technique that runs on the client side for cloud-based storage systems. SAFE is as fast in processing time as file-level or fixed-size block deduplication and provides the same storage space savings as block deduplication using structure-based granularity.

4.2 SAFE Modules

SAFE consists of cooperative modules through which a file is saved to storage. We begin by outlining an architecture of SAFE with modules and elaborating on each module. We explain structures based on which objects are extracted and deduplicated. Finally, we describe how to embed SAFE into a popular cloud storage service, Dropbox.

SAFE incorporates our structure-aware deduplication with existing file-level deduplication. As shown in Fig. 4.1, the SAFE system consists of email parser, file parser, object-level dedup, object manager and storage manager modules. SAFE also exploits file-level deduplication to identify any redundancy of unstructured files for low CPU and memory overhead. SAFE has two parsers, an email parser and a file parser. The email parser module extracts email attachments based on email policy and saves indexes (hash values) of the body and attachments for the reconstruction of emails. The file-level dedup module receives input data (i.e. files) from the email parser or file system. If a file is an unstructured file, such as an image or a video file, SAFE saves the file directly to storage after compression on the store manager module. Otherwise, a structured file is sent to the file parser, where a file is decomposed into objects based on a file policy. The file parser sends parsed objects to the object manager. The object-level dedup module computes hash value of each object and checks whether an object is a duplicate based on the object index table. Finally, the store manager saves unique objects whose indexes are sent from the object-level dedup to storage after compression. We elaborate on each module in what follows.

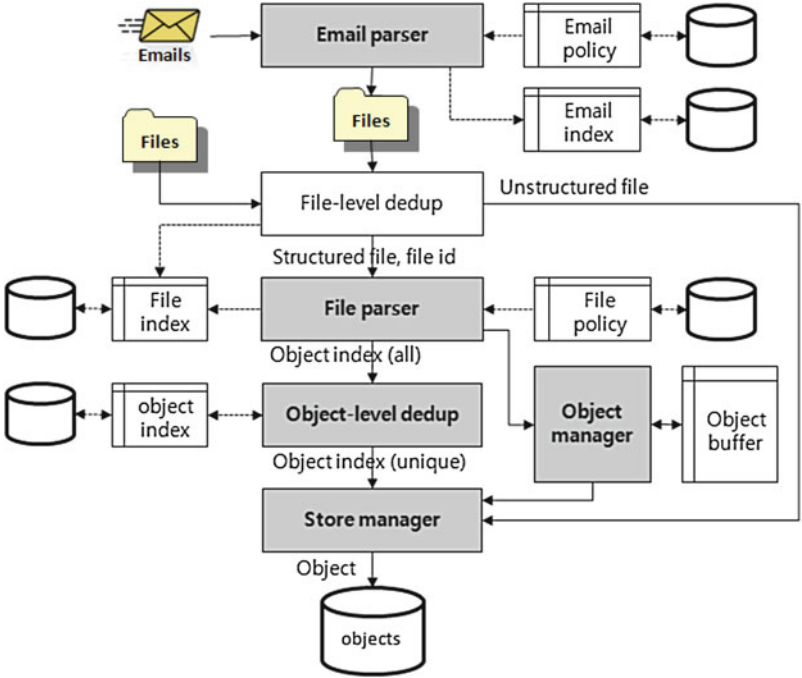


Fig. 4.1 SAFE deduplication architecture

4.3 Email Parser

The email parser runs as a lightweight mail filter on a sendmail server [15]. It intercepts emails using Milter [10] APIs when a Mail Transfer Agent (MTA) of a sendmail server receives an email. A Milter API is a part of the Sendmail Content Management API that can look up, add and modify email messages. Figure 4.2 illustrates how the email parser works. When an email arrives at the MTA on an email server, Milter intercepts and sends the email to the parser, which decomposes the email into metadata, body and attachments based on the email policy. The email policy contains the structure information of an email (Fig. 4.3), and the structure is based on the format of Multipurpose Internet Mail Extensions (MIME) [7]. The email parser decomposes email based on a boundary string given at ‘Content-Type:Boundary=’ in the metadata, which is ‘<boundary>’ in the figure. There can be several attachments that are split by the same boundary string, ‘<boundary>’. Each attachment may be encoded using different encoding types like ‘base64’ that are designated by ‘Content-Transfer-Encoding’. Thus, the email parser runs decoding before processing (sending to other modules) a decomposed attachment.

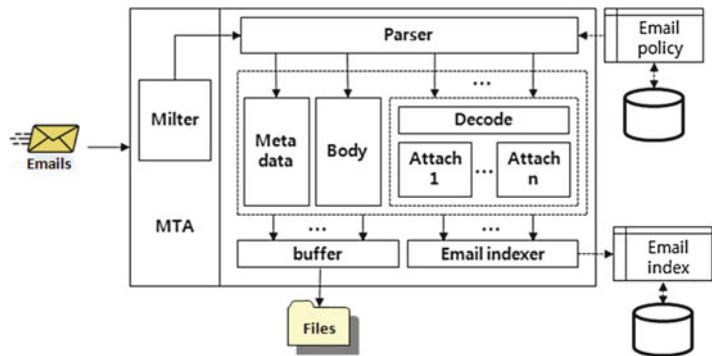
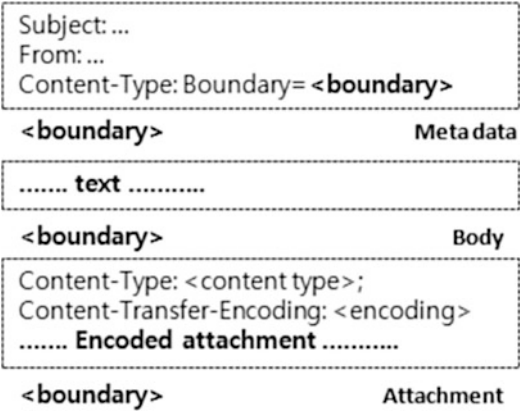


Fig. 4.2 Email parser

Fig. 4.3 Structure of an email



In Fig. 4.2, the email indexer computes SHA-1 [68] hash values of metadata, body, and decomposed attachments; saves all indexes into an email indexer table using a unique email ID, which is a 14 byte string. The buffer has an array data structure where it holds data decomposed from an email and sends the array data (i.e. files) to the file-level dedup where each file is identified as being unique or redundant.

4.4 File Parser

The file parser decomposes three types of structured document files, such as Microsoft Word (docx), PowerPoint (pptx) and Adobe Portable Document Format (PDF) files. SAFE deduplicates a file based on two key aspects: (1) how objects are extracted from a file and (2) what granularity is efficient for deduplication. SAFE uses an object or a combination of objects for granularity. We explain how the file parser works in detail.

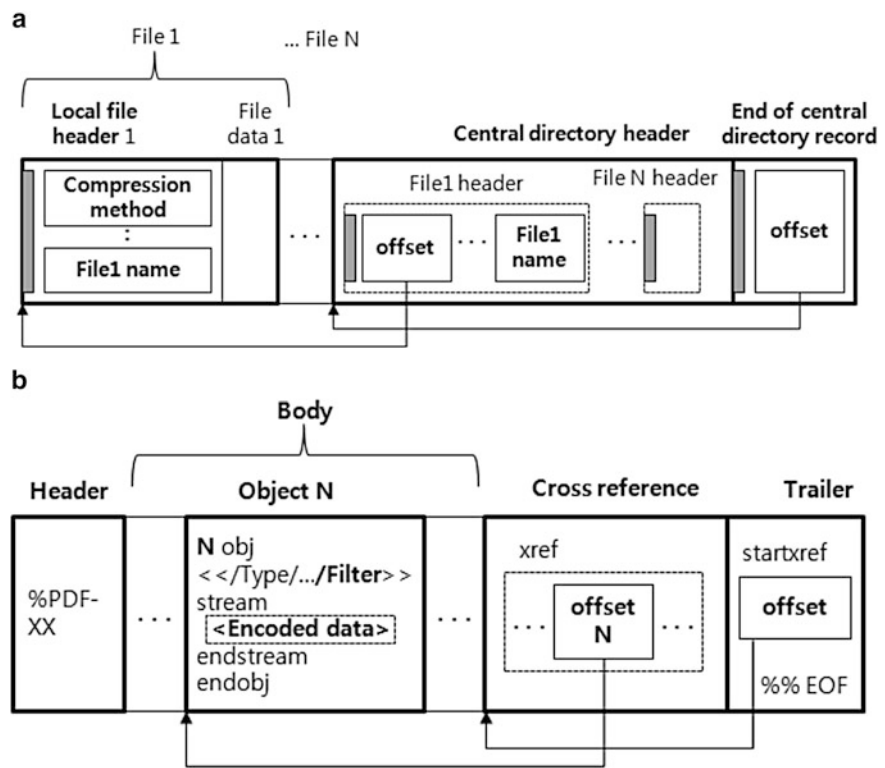


Fig. 4.4 Physical file format. (a) Structure of MS Office Open XML file. (b) Structure of PDF file

MS Word (docx) and PowerPoint (pptx) are based on the structure of MS Office Open XML format, which is standardized at ECMA-376 [6] and ISO/IEC 29500 [8], as shown in Fig. 4.4a. An Open XML file is a ZIP [14] file that contains multiple files such as text and images. An Open XML file format contains different sections (with metadata and data) that are chained with offsets from the end of a file, and data are tracked down along the offsets upwards. In Open XML format, data are located in sections with local file headers at the beginning of a file. The track to a data starts from the ‘end of central directory record’ section with the offset of the ‘central directory header’ section, which describes a directory. The directory has offsets of files in it, and through the offsets, a section of a file is accessed. Each file section consists of a local file header and file data. The local file header contains metadata, such as the compression method and file name. Likewise, other files are accessed through offsets in the ‘central directory header’ section. In Fig. 4.4a, grey bars are signatures. The signatures of ‘end of central directory record’, ‘file header’ in the central directory header, and the local file headers are 0x06064b50, 0x02014b50 and 0x04034b50 respectively. The encryption that comes between local file headers and file data is not shown.

A PDF physical format also contains multiple sections, such as a header, a body, cross references and a trailer. A PDF structure is defined at ISO 32000 [1], and data are accessed through chains of offsets. The header section is at the beginning of a file, which shows the version of the PDF file. The body section contains objects with text or images. The cross reference section has offsets that point to objects in the body section. The trailer section at the end of the file has offsets that point to cross-reference sections. Thus, data in objects are accessed through offsets from the end of a file upwards. The body section contains objects surrounded by ‘obj’ and ‘endobj’, each of which may have a text or an image. Two keywords, including ‘stream’ and ‘endstream’, surround the data. A stream is encoded by a compression algorithm and can be decoded by the corresponding decompression algorithm shown in the metadata of the object, called ‘dictionary’ (i.e. <</Type../Filter/<decompression algorithm>/..>>). According to ISO 32000, there are ten different decompression algorithms, among which FlateDecode and DCTDecode are used to decode a text stream and a JPEG image stream respectively.

Figure 4.5 shows how the file parser works. Dotted lines denote control flows and solid lines are data flows. The output of the file parser is indexes of all objects including individual objects and combined objects of a file. The file parser receives and parses a structured file into objects based on file policy. Encoded objects are decoded based on a decoding algorithm that is specified in the structure. The combiner concatenates small objects into a larger object to reduce the number of indexes of objects. The combined objects are mainly small metadata objects whose contents are always changed for each file, in which case we cannot find any

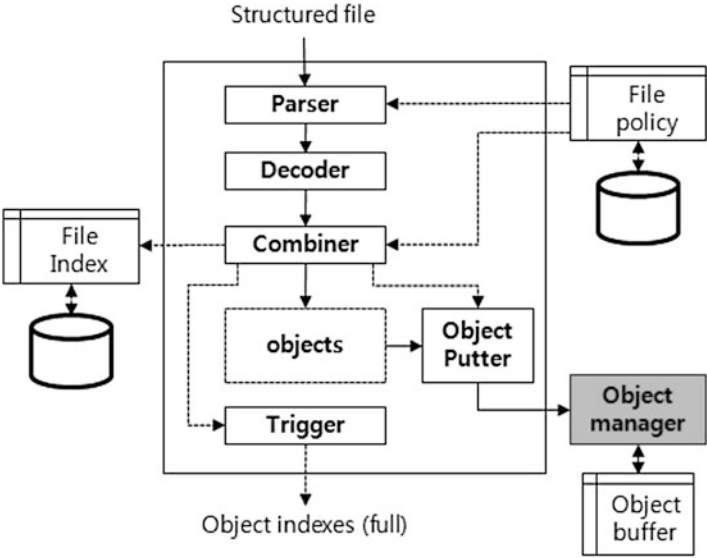


Fig. 4.5 File parser

redundancy of the objects. A parsed object that is not combined comprises a 5-tuple, including the hash value of an object, length of an object, ID of the container that contains an object (file ID for Open XML format and obj ID for PDF), decoding scheme (if specified) and the object itself. A combined object is a concatenation of 5-tuples. The object putter sends an individual object or a combined object to the object manager, which subsequently holds the objects in an object buffer until deduplication of the objects is finished. The trigger combines all object indexes and sends them to the object-level dedup, where object redundancies are identified.

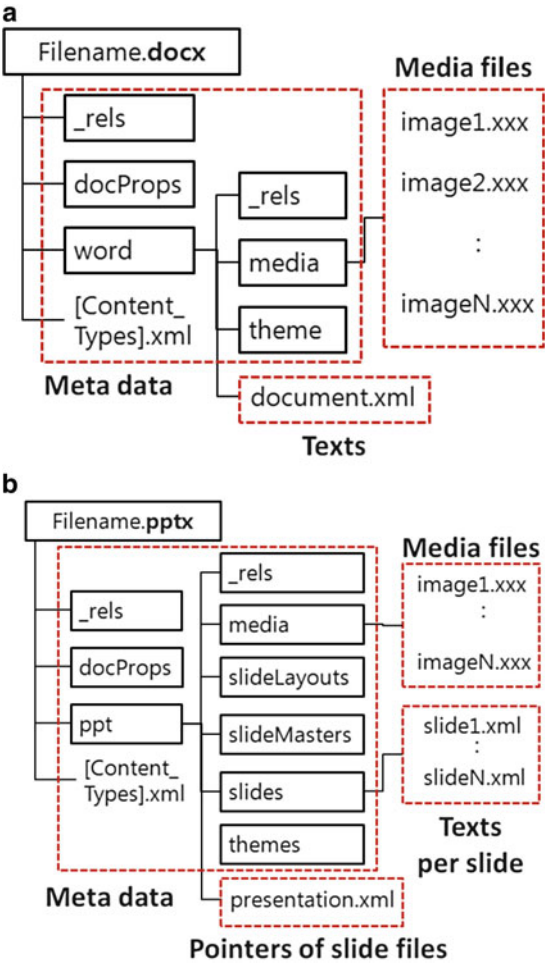
SAFE runs parsing and combining based on a different file policy depending on the file type. To do this, SAFE creates a dynamic instance for each file. SAFE has an abstract base class, `FilePolicy`, that defines functions to be implemented in derived classes such as `DOCXFilePolicy`, `PPTXFilePolicy` and `PDFFilePolicy`. The file parser creates a derived class object corresponding to a file type and executes functions of the class object. Thus, a structured file with a new format can be implemented as a derived class whose basic functions are already defined.

For combining, SAFE puts together metadata objects that are small but uses an image and text(content) objects without combination based on logical structures in accordance with the file type. Figure 4.6 illustrates the logical structure of docx and pptx files. As shown in Fig. 4.6a, the text in a Word file is contained in a document.xml object, while image objects are under a media directory; other directories shown in the figure contain metadata objects. Likewise, a PowerPoint file (Fig. 4.6b) has a media directory but different metadata objects. In addition, texts in the slide are structured into each individual slide<number>.xml. A presentation.xml holds the pointers of slide objects.

4.5 Object-Level Deduplication and Store Manager

The object-level deduplication module receives indexes of objects from the file parser and checks whether each index exists in the object index table. If an index does not exist, the index is unique. Unique indexes are saved to an object index table and sent to the store manager module that fetches objects corresponding to the unique indexes from the object manager module. If an index does exist in the object index table, the index is redundant. Redundant indexes are excluded for storage. The object manager module retrieves an object that the store manager requests from the object buffer. The store manager stores a pair of <object index, object> to object storage after compression. Unstructured files are stored through the store manager without deduplication.

Fig. 4.6 Logical structure of MS Office document file.
(a) Word (docx), (b) PowerPoint (pptx)



4.6 SAFE in Dropbox

In this section, we describe how SAFE can be embedded into a cloud storage service like Dropbox [4]. Dropbox removes redundancy in network and storage using large (4 MB) fixed-size block deduplication. Thus, we address how SAFE can improve data reduction of the current Dropbox with minimal additional overhead of processing and memory.

A recent study [3] revealed the internal mechanisms of Dropbox by measuring and analysing packet traces between clients and Dropbox servers. Dropbox is accessed by Web UI (<http://www.dropbox.com>) or a Dropbox client. We leverage SAFE into a Dropbox client to deduplicate structured files on the client side. Dropbox consists of two types of server, a control server and a storage server.

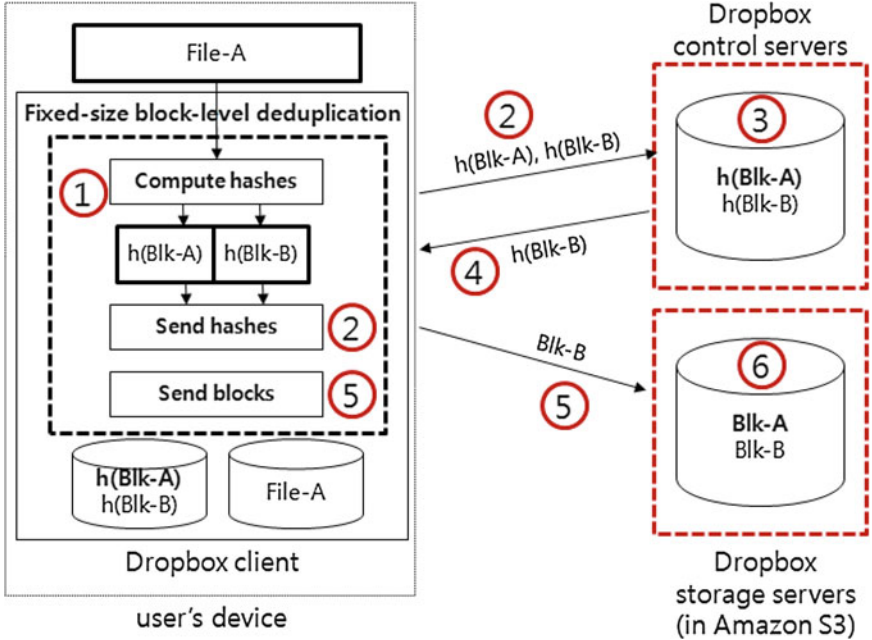


Fig. 4.7 Dropbox internal mechanism

Control servers hold metadata of files such as the hash value of a block and mapping between a file and blocks. Storage servers contain unique blocks in Amazon S3 [2]. Dropbox client synchronizes its own data and indexes with Dropbox servers.

Figure 4.7 shows how Dropbox works. Circles with numbers show the order in which a file is saved. *File-A* is a file and *Blk-X* is a block separated from a file. $h(\text{Blk-X})$ means a hash value of a block. Thick $h(\text{Blk-X})$ and *Blk-X* are considered hash values and blocks that had previously existed before a file is saved. A user device is a mobile phone, tablet, laptop or desktop. Dropbox completes the following steps to save a file. (1) As soon as a user saves *File-A* to a shared folder in a Dropbox client, fixed-size block deduplication of Dropbox splits the file into blocks based on 4 MB granularity and computes object hashes. If the file is larger than 4 MB, then it is the same as an object, and hash values of the file are computed. Dropbox uses SHA256 [13] to compute hash values. (2–4) Dropbox client sends the computed file hash values to a control server that returns only unique hash values not found in a check of previously saved hash values. In this example, a hash of *Blk-B* is returned to a client because the hash of *Blk-A* is found to be a duplicate. (5–6) A Dropbox client sends to the storage server blocks of the returned indexes. Ultimately, storage servers have unique blocks across all Dropbox clients. Note that storage saving occurs in the server (because *Blk-A* is not saved again), and the incurred network cost is reduced because only *Blk-B* is sent.

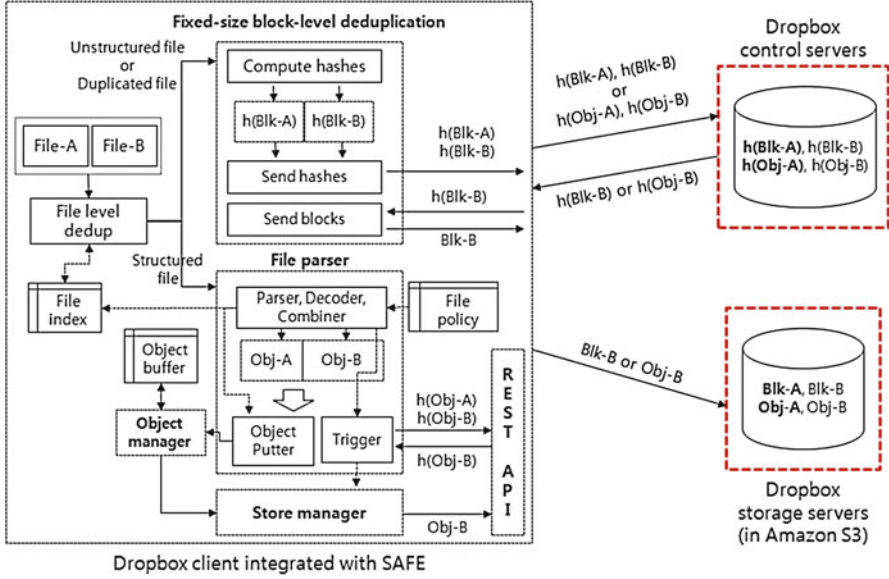


Fig. 4.8 SAFE integration with Dropbox

SAFE can complement the fixed-size block deduplication in a Dropbox client as shown in Fig. 4.8. Suppose that an unstructured file (File-A) and a structured file (File-B) are added to a Dropbox folder. The file-level deduplication module checks duplicate files using the file index table whose entry has a pair of $\langle \text{hash value of file contents, file path of the first unique file} \rangle$. For duplicate files, the entry is added to a file index table without saving the file in local storage. An unstructured file follows the fixed-size block deduplication. A structured file is fed to the file parser, and objects from the file are extracted. The trigger module calls the REST API [5] of Dropbox to send the hash values of objects. The control servers act as an object-level dedup module. We used the SHA256 hash function in SAFE for compatibility with Dropbox. The store manager sends objects corresponding to returned hashes from a control server to a storage server through the REST API. Thus, in the integration of SAFE with Dropbox, control servers function as an object-level dedup module. In Fig. 4.8, thick fonts, such as $h(\text{Blk-X})$, $h(\text{Obj-X})$, Blk-X and Obj-X , already exist before *file-A* and *file-B* are saved.

4.7 Evaluation

We discuss the performance evaluation criteria and data sets used in this section. We then show the evaluation results of the performance and overhead of the proposed SAFE approach, compared with a file-level deduplication that JustCloud [9] and Mozy [11] use, a fixed-size block deduplication that Dropbox [4] uses, and variable-size block deduplication schemes.

4.7.1 Metrics

The major performance metrics are the deduplication ratio and incurred data traffic amount. The deduplication ratio indicates how much storage space can be saved by removing redundancies and is computed using Eq. (4.1):

$$\left(\frac{\text{InputDataSize} - \text{ConsumedStorageSize}}{\text{InputDataSize}} \right) \times 100. \tag{4.1}$$

Data traffic incurred designates how much data are transferred to a storage; it is the amount of unique data of the input data.

As overhead metrics, we measure the processing time and index size. Since overhead is proportional to data size, we compare the processing time and index size overhead relative to the file-level deduplication that has the least overhead.

4.7.2 Data Sets

We collected real data sets of structured files including docx, pptx and pdf from the file systems and emails of five graduate students in the same department. Table 4.1 summarizes the information of data sets collected from file systems and emails. An individual user’s data are labeled ‘P-’#, and ‘Group’ is the sum of all personal data sets, and ‘No.’ is the number of structured files in each data set. For the experiments with email data sets, we deployed two sendmail servers; structured files are attached to emails from a sending sendmail server, and the attached structured files are extracted by the email parser at a receiving sendmail server. Structured files in the file system data sets are fed into the file parser directly.

Figure 4.9 shows the ranges of the file sizes in the email group data set, whose mean value (673 KB) is relatively small compared to the maximum block size, 4 MB, of Dropbox. On the *x*-axis, 10 and 20 indicate 5 and 10 MB respectively. Meanwhile, we measured the percentages of the structured files among all attached files of five people’s emails. As shown in Fig. 4.10, the structured files occupy 89 % of all attached files. PDF occupies 44 %, and the percentage of docx and

Table 4.1 Used data sets

Data set	File system Size (MB)	No.	Emails Size (MB)	No.
P-1	1721	4384	637	955
P-2	509	590	554	720
P-3	266	523	249	480
P-4	869	1499	358	859
P-5	864	1430	744	823
Group	4229	8426	2542	3837

Fig. 4.9 Distribution of file sizes in email data set

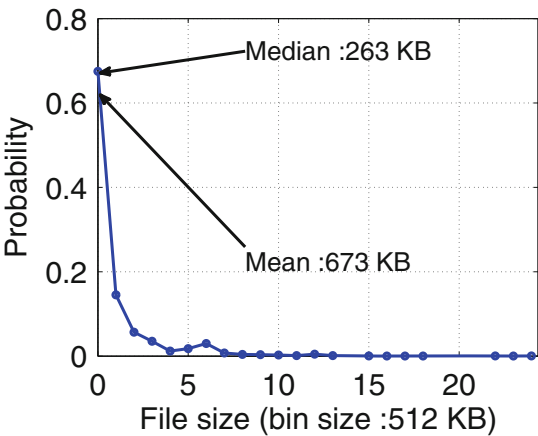
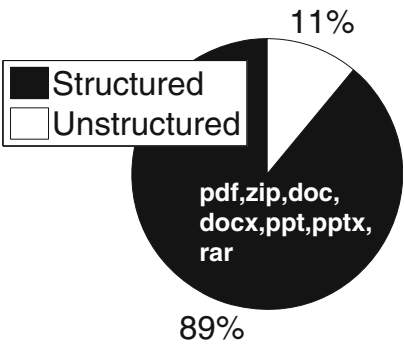


Fig. 4.10 Percentage of structured files in email data sets



pptx is 11 %. Image files, such as jpg, bmp and png, belong to unstructured file types. Despite the small size of data sets, the high percentage of structured files (89 % for all types of structured files and 55 % for docx, pptx and pdf structured files) validates the popularity of the structured file types on which SAFE is based.

The data sets used may be considered relatively small. However, we note that the results obtained in this evaluation will only be stronger if larger data sets of an organization are used since the redundancy levels would become greater. For variable-size block deduplication, we use 2, 8 and 64 KB as the minimum, average and maximum chunk sizes respectively. For fixed-size block deduplication, we use 4 MB as the fixed block size, as does Dropbox. Fixed-size block deduplication is thus the same as file-level deduplication for files smaller than 4 MB. We carried out the evaluations on Fedora 16 Linux operating systems of kernel 2.6.35.9 SMP on an Intel Core 2 Duo 3 GHz.

4.7.3 *Storage Data Reduction Performance*

We first evaluate the deduplication ratio for each data set. The deduplication ratio of a group is larger than that of each personal data set. For the file systems, the high deduplication ratio of a group is due to the same or similar content files shared among people in the same department. For emails, the high deduplication ratio of a group is due to duplicates of multiple-recipient emails as well as the same or similar attachments delivered and updated through email threads.

Figure 4.11 presents the deduplication ratio of six data sets, including personal data sets and a group data set. File, Block-F and Block-V mean file-level deduplication, fixed-size block deduplication and variable-size block deduplication respectively. Deduplication ratios with the email data sets are higher than those with the file system data sets because of the frequent email threads in addition to shared attached files among people in the same department. Compared to the file-level deduplication in Fig. 4.11 on an average based on group data sets, SAFE can further reduce redundancies by 15 % and achieves about 40 % better performance than file-level deduplication. For the email data sets, SAFE shows almost 99 % of the performance level of variable-size block deduplication. Furthermore, SAFE's deduplication ratio is better than variable-size block deduplication in the file system data sets. This is because SAFE can find the boundaries of objects in complicated structured files more efficiently than variable-size block deduplication, especially for PDF, which uses compressions for more individual objects than other structured files such as docx and pptx. Note that file system data sets have twice as many PDF files as email data sets.

4.7.4 *Data Traffic Reduction Performance*

We next evaluate the incurred data traffic for group data sets, as shown in Fig. 4.12. For file system data sets, SAFE shows the lowest data traffic among all deduplication types: concretely, SAFE has the lowest data traffic with the file system data sets and the second to lowest (just behind variable-size block dedup) with the email data sets. This supports the assertion that SAFE can be used as a deduplication technique for personal cloud storage services like Dropbox thanks to the expected decrease in network bandwidth consumption. In addition, for email data sets SAFE reduces data traffic by 56 % in the email group data set (1.4 GB out of 2.5 GB). Compared to file-level and fixed-size block deduplications, SAFE has 30 % less data traffic for the email data sets (and 15 % for the file system data sets), which indicates that SAFE efficiently reduces the network bandwidth requirement storing emails to cloud storage.

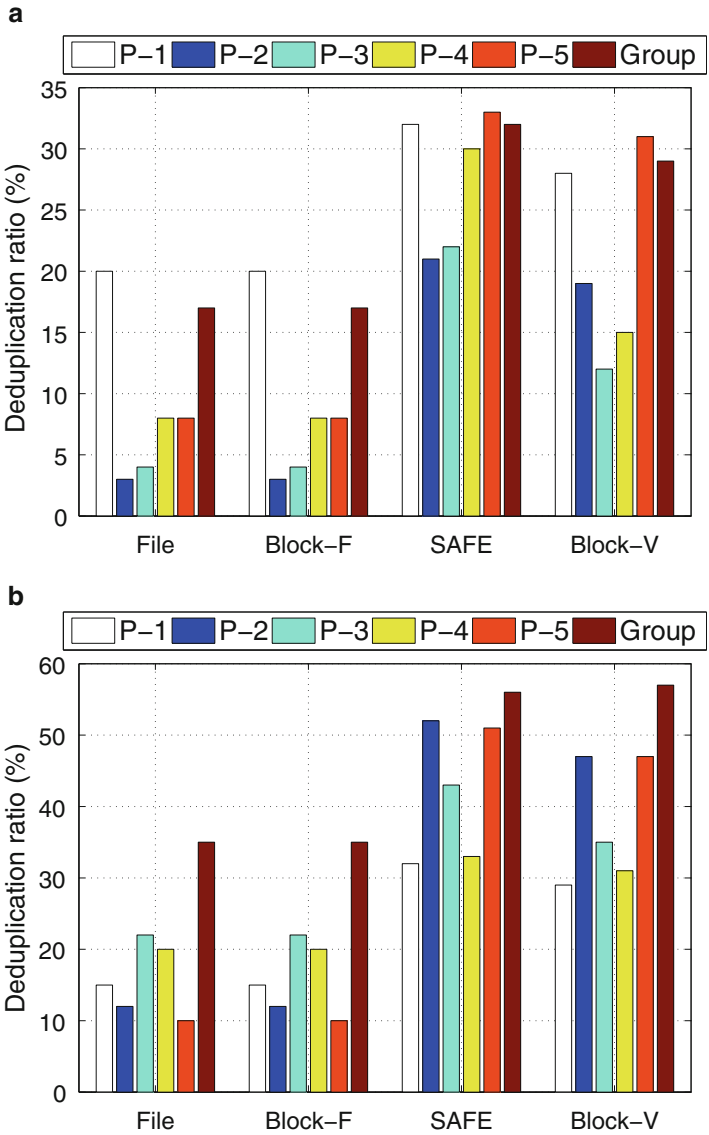


Fig. 4.11 Deduplication ratio. (a) File system data sets. (b) Email data sets

4.7.5 CPU Overhead

Here we show the assessments of the processing time. As shown in Fig. 4.13, the file-level deduplication runs fastest for both data sets types because there is no overhead associated with separating a file. We present the relative processing time

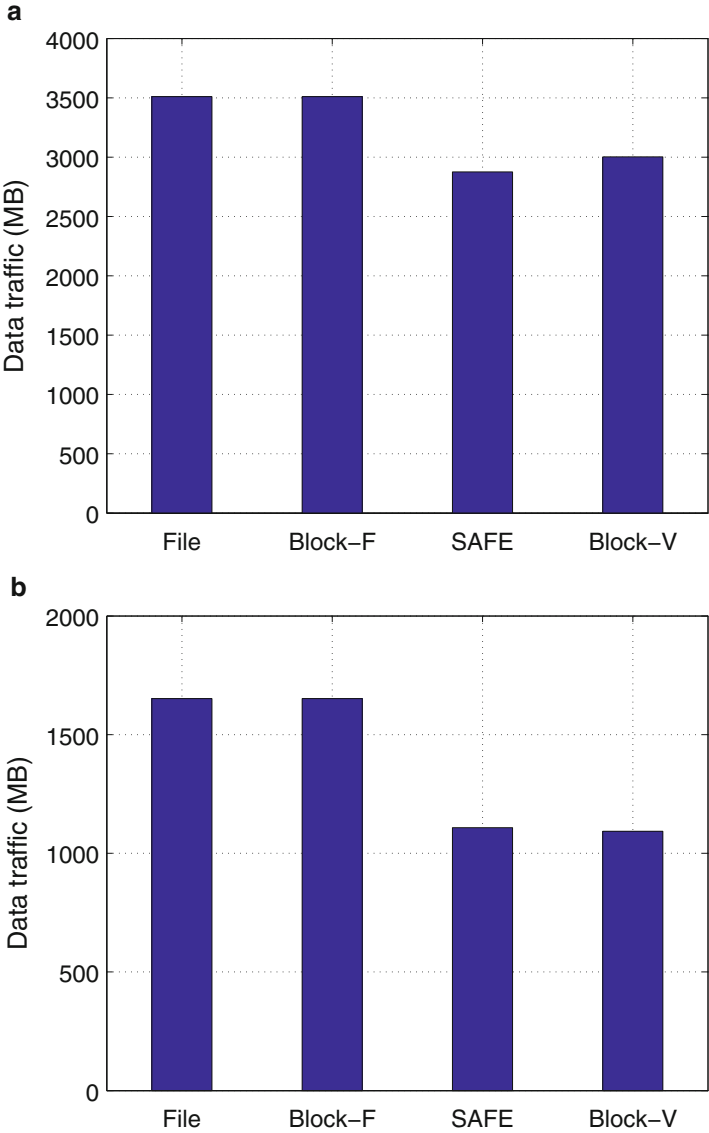


Fig. 4.12 Data traffic incurred (MB). (a) File system data sets. (b) Email data sets

based on file-level deduplication (whose value is 1) that is shown as 0 because the y-axis is set in log scale. The fixed-size block deduplication shows a processing time overhead that is similar to that of file-level deduplication. Even if it is slower than file-level deduplication, SAFE processing is relatively fast on average for the data

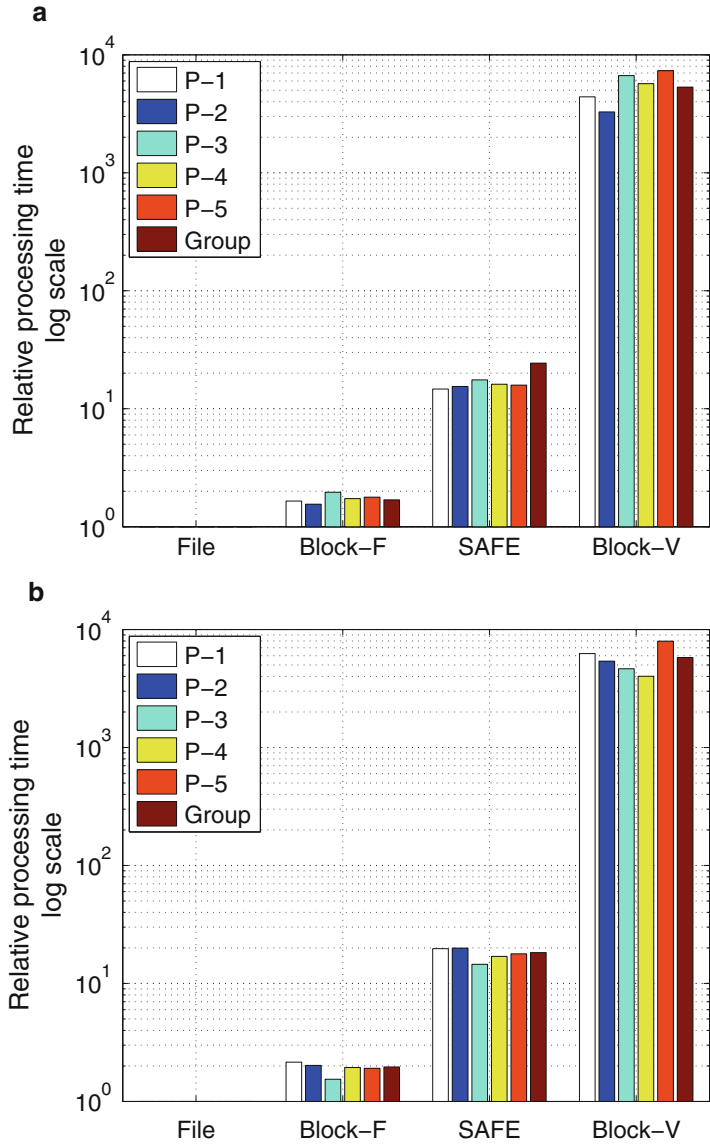


Fig. 4.13 Relative processing time overhead compared to file-level deduplication. (a) File system data sets. (b) Email data sets

sets despite the fact that we do not use enhanced cache management schemes in our implementation. In addition, SAFE is faster by two orders of magnitude than variable-size block deduplication.

4.7.6 *Memory Overhead*

Here we show the assessments of memory overhead. We compare the relative index overhead in Fig. 4.14. Like processing time, we present relative index overhead compared to file-level deduplication. SAFE shows two to three times less index overhead than that seen in variable-size block deduplication. We use a 40 byte hexadecimal string of the SHA1 hash value for a chunk index in all testing deduplication schemes. Though a smaller chunk index could reduce the overhead of variable-size block deduplication, the relative ratios shown in Fig. 4.14 would be maintained. The index overhead increases proportionally to the number of unique chunks. For the email data sets, the numbers of unique chunks for file-level deduplication, fixed-size block deduplication, SAFE and variable-size block deduplication were 2.4 K, 2.5 K, 33 K and 92 K respectively. For the file system data sets, the numbers for each deduplication scheme were 5 K, 5.5 K, 155 K and 248 K respectively. SAFE with the file system data sets shows slightly more chunk index overhead than with the email data sets. This is because the file system data sets had higher percentages of PDF files than the email data sets. PDF files have a relatively complex structure, where files are divided into many small objects, and the current file policy we implemented for PDF saves each object individually without combining. By combining multiple small objects into a large object, as in the file policy for docx and pptx, SAFE would reduce more chunk index overhead for PDF files.

4.8 Summary

We developed a fast client-based deduplication technique, SAFE, that removes redundant objects based on a structure-based granularity instead of using physical chunk granularity. Unlike traditional deduplication, which involves a trade-off between the deduplication ratio and processing overhead, SAFE has the benefits of both a high deduplication ratio and low processing overhead. Our experiments with real data sets and implementation on a cloud storage client show that SAFE achieves more storage space savings, 10–40 %, and 20 % less data traffic on average than file-level and fixed-size block deduplications, which are used in existing cloud-based storage services. In addition, SAFE shows an allowable processing time on average in clients for cloud-base storage systems and is faster by two orders of magnitude than variable-size block deduplication. Thus, SAFE can be used for fast deduplication in clients, with low overhead.

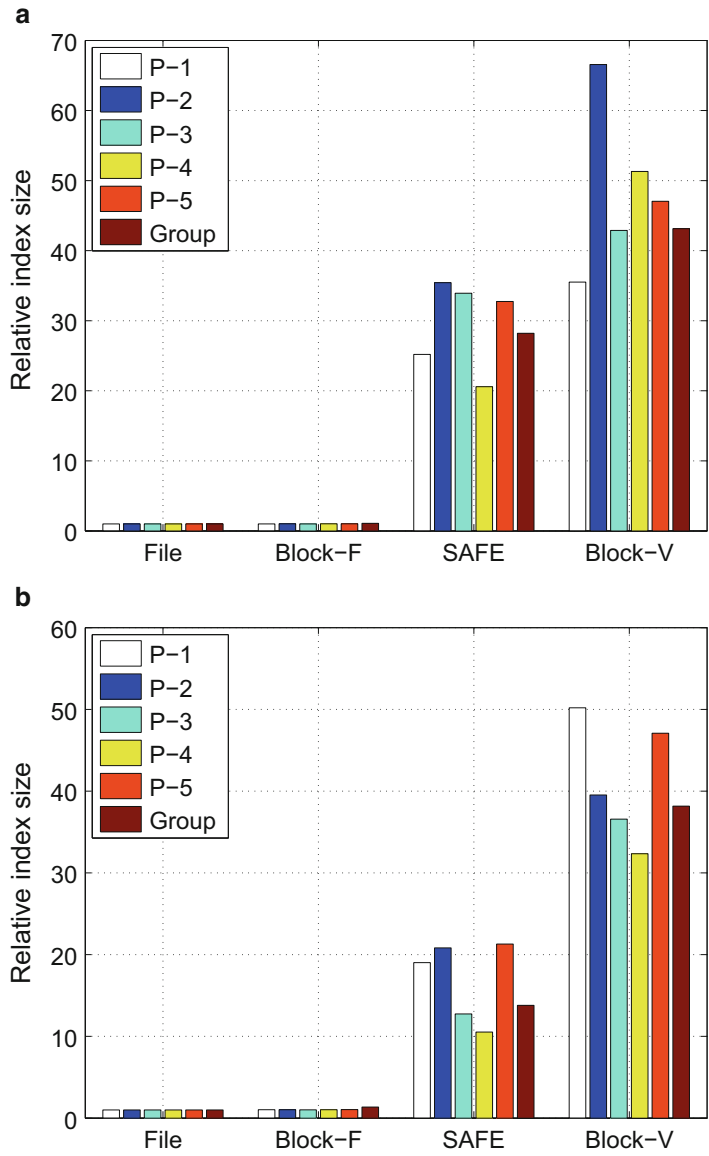


Fig. 4.14 Relative index overhead compared to file-level deduplication. (a) File system data sets. (b) Email data sets

References

1. Adobe: ISO32000: Document management:Portable document format. http://www.adobe.com/content/dam/Adobe/en/devnet/acrobat/pdfs/PDF32000_2008.pdf (2008)
2. Amazon: Amazon simple storage service. <http://aws.amazon.com/s3/> (2016)
3. Drago, I., Mellia, M., Munafo, M.M., Sperotto, A., Sadre, R., Pras, A.: Inside dropbox: understanding personal cloud storage services. In: Proceeding of the 2012 ACM Conference on Internet Measurement Conference (IMC), pp. 481–494 (2012)
4. Dropbox: <http://www.dropbox.com> (2016)
5. Dropbox: REST API. <https://www.dropbox.com/developers/core/docs> (2016)
6. ECMA: Standard ECMA-376: office open XML file formats. <http://www.ecma-international.org/publications/standards/Ecma-376.htm> (2012)
7. Freed, N., Borenstein, N.S.: Multipurpose Internet Mail Extensions (MIME) part one: format of internet message bodies. <http://tools.ietf.org/html/rfc2045> (1996)
8. ISO, IEC: ISO/IEC 29500-1:2008. http://www.iso.org/iso/iso_catalogue/catalogue_tc/catalogue_detail.htm?csnumber=51463 (2008)
9. JustCloud: <http://www.justcloud.com/> (2016)
10. Milter.org: Sendmail mail filters. http://www.sendmail.com/sm/partners/milter_partners/open_source_milter_partners/ (2015)
11. Mozy: <http://mozy.com/> (2016)
12. National Institute of Standards and Technology (NIST): Secure Hash Standard 1 (SHA1). <http://csrc.nist.gov/publications/fips/fips180-4/fips-180-4.pdf> (2015)
13. National Institute of Standards and Technology (NIST): Secure Hash Standard 256 (SHA256). http://csrc.nist.gov/publications/fips/fips180-3/fips180-3_final.pdf (2008)
14. PKWARE: ZIP file format specification. <http://www.pkware.com/documents/casestudies/APPNOTE.TXT> (2014)
15. Sendmail.com: Sendmail. http://www.sendmail.com/sm/open_source/ (2016)

Part III

Network Deduplication

In this part, we focus on removing redundancies generated through networks and propose Software-defined Deduplication as a Network and Storage Service (SoftDance) in Chap. 5. Because nodes are massively connected through networks, redundancy occurs in various domains (storage and network) and in diverse ways, including in the copying and modifying of files, redundant transfers through networks, and backup and replication in servers. Simply leveraging data reduction techniques developed in each domain provides no benefits and even incurs significant redundant processing overhead. SoftDance chains and virtualizes storage deduplication and network redundancy elimination using a software-defined network (SDN) to achieve both storage space and network bandwidth savings while reducing expensive overhead related to processing time and memory size. We show efficient encoding and indexing algorithms for a SoftDance middlebox (SDMB) and an effective control mechanism for an SDN controller. We also explain a prototype of testbed experiments and Mininet-based emulations to evaluate SoftDance on real system environments and typical data centre network topologies.

Chapter 5

SoftDance: Software-Defined Deduplication as a Network and Storage Service

Abstract In this chapter, we focus on removing redundancy in a chain between end-systems through a network. As nodes are massively connected through networks, redundancy occurs in various domains (storage and network) and in diverse ways, including copying and modifying files, redundant transfers through networks, backup and replication on servers. Simply leveraging data reduction techniques developed in each domain does not give benefits and even incurs significant redundant processing overhead. In this chapter, we present SoftDance, software-defined deduplication as a network and storage service. SoftDance chains and virtualizes storage deduplication and network redundancy elimination using a software-defined network (SDN) to achieve both storage space and network bandwidth savings while reducing expensive overhead of processing time and memory size. SoftDance uses encoding and indexing schemes for SoftDance middlebox (SDMB) and control mechanisms for a SDN controller. Evaluation results show SoftDance reduces two to four times more bandwidth than a network-wide redundancy elimination technique and achieves equal/close storage space saving to existing the best storage saving techniques.

5.1 Large Redundancies in Network

Redundancies that occur in various domains (storage and network) consume storage spaces and reduce available network bandwidth as nodes are massively connected through networks. In a storage domain for data reduction, *data deduplication* (*Dedup*) has been proposed [4, 9–12, 18, 23]. Dedup computes indexes of chunks (split from files) and does not store redundant chunks by comparing current indexes with indexes of chunks saved previously. Each index points to a unique chunk. In a network domain, *network redundancy elimination* (*NRE*) has been investigated [1, 2, 21] for data reduction. NRE computes indexes [19] for the incoming packet payload and removes redundant byte strings in packets by checking packets saved previously. Though Dedup and NRE share the same goal of identifying and removing redundant data, the functionalities of the two are orthogonal. Thus, they provide no benefits for each other, and redundant processing overhead is even incurred on both end-systems and networks.

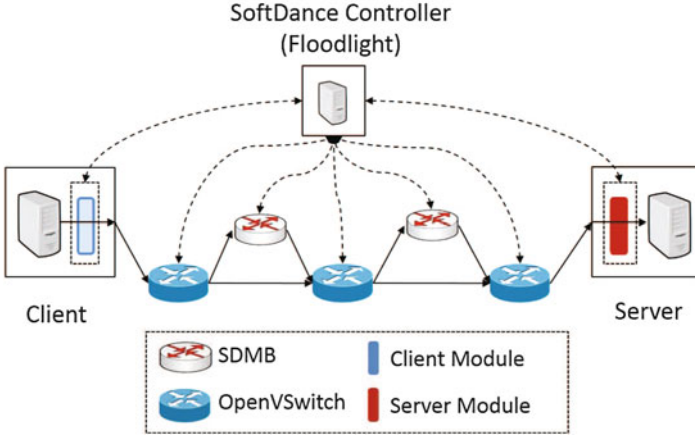


Fig. 5.1 SoftDance architecture

We propose an efficient framework for *software-defined deduplication as a network and storage service* (SoftDance) to save storage space and network bandwidth while reducing processing time and memory overhead. As presented in Fig. 5.1, SoftDance consists of a SoftDance middlebox (SDMB), OpenVSwitches, a SoftDance controller, and lightweight modules on end-systems. SDMB mainly performs encoding and indexing algorithms. SDMB identifies a packet payload for encoding, stores an index of a unique packet payload and replaces the redundant packet payload with an index (called encoding). SDMB also maintains appropriate indexes by communicating with a SoftDance controller. A SoftDance controller provides deduplication function virtualization and control mechanisms by coordinating end-systems and network elements. SoftDance uses a packet payload as a unit of comparison, which enables a zero chunking mechanism through deduplication function virtualization. Chunking is a culprit in expensive processing time, and SoftDance reduces processing time significantly. In addition, SoftDance distributes indexes to reduce memory overhead on SDMBs based on hash-based sampling [20]. Various index distribution algorithms have been designed and implemented in SoftDance controllers.

To validate our approach, we implement the proposed framework and algorithms on both a testbed system and Mininet-based emulation using software-defined network (SDN) technologies. We built a testbed system using OpenVSwitches, a floodlight SDN controller, and Linux-based SDMBs that intercept packets using a userspace netfilter library. Mininet-based emulation compares SoftDance with Dedup and NRE techniques based on typical data centre network (DCN) topologies, including tree, multi-rooted tree and fat tree. Our evaluation results from both testbed and mininet-based emulation show that SoftDance reduces two to four times as much bandwidth as network-wide redundancy elimination (SmartRE) and has storage space savings equal or close to existing storage domain techniques.

Furthermore, in scenarios of both end-systems and networks performing deduplication redundantly, SoftDance attains much more efficient processing and memory overhead.

Our solution considers a network and storage service of backup systems to which application servers (or clients) store redundant data in a DCN. Backup servers are located in centralized places. The software architecture shown in Fig. 5.1 can be deployed on a large scale using a popular cloud solution such as OpenStack [17]. In this case, a SDMB is deployed as a virtual machine where the deduplication function is run. SDMBs can be distributed in multiple clouds in a path between a client and a server.

The rest of the chapter is organized as follows. We begin by explaining SDNs as background information in Sect. 5.2. We describe the design and implementation issues on packet encoding and indexing algorithms of SDMBs and a system coordination scheme of a SoftDance controller in Sect. 5.3. We evaluate our approach in Sect. 5.8, and 5.9 concludes the chapter.

5.2 Software-Defined Network

SoftDance is based on a SDN to set up efficient paths for removing more redundancies and reducing indexes in networks. SDN is a new paradigm that separates the control plane that computes forwarding rules and the data plane that forwards data packets in a network element. As shown in Fig. 5.2, SDN moves the control plane from a switch to a centralized SDN controller that has a global network view and decides paths based on application requirements or policies. When a data packet arrives at a switch without a corresponding forwarding rule, the switch asks a controller the path to be forwarded. Then, the controller sets forwarding rules to the switch, and data packets are forwarded based on the rules.

5.3 Control and Data Flow

To effectively reduce redundancies in chains from clients to servers through a network, SoftDance coordinates clients, servers and SDMBs using a SDN controller. A SDN controller with a global network view controls data transfer service requests of clients and provides an efficient path from a client to a targeted server, leading to low processing and memory overhead. Next, we explain the design and implementation of SoftDance. We start by presenting control and data flows. We then elaborate on the encoding scheme processed at SDMBs. Finally, we describe four distributed hash indexing algorithms.

SoftDance uses control flows to set up a service request from a client and data packets flow based on the set-up through switches and SDMBs. A SoftDance

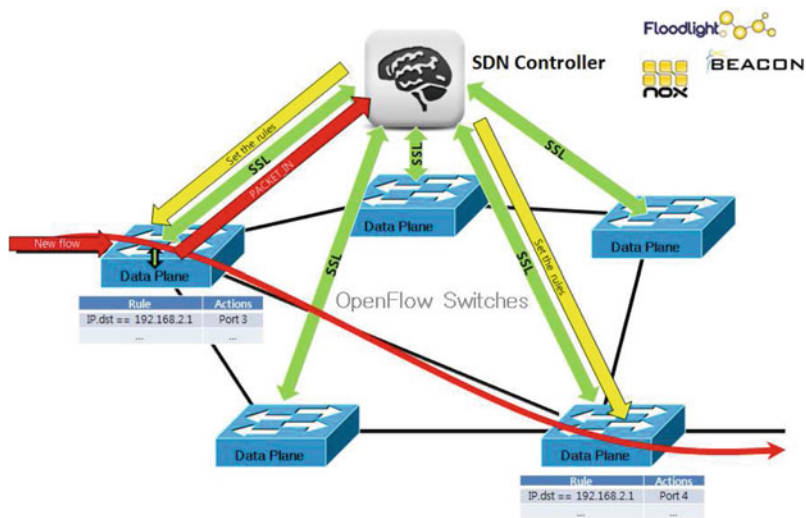


Fig. 5.2 Software defined network

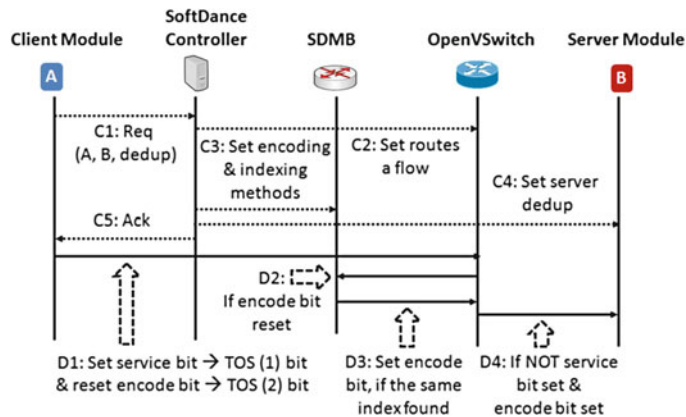


Fig. 5.3 SoftDance control and data flows

controller coordinates control flows through communication with clients, servers, SDMBs and OpenVSwitches.

As shown in Fig. 5.3, the SoftDance process starts with a client’s deduplication service request (C1). The client sends the request along with the client’s and server’s IP addresses to a SoftDance controller. When the SoftDance controller receives the service request from the client, the controller performs Algorithm 4. The controller computes and selects a path between a requested client and a targeted server, retrieves SDMBs and switches on the selected path, and computes hash ranges of retrieved SDMBs. Then the controller pushes flow table entries into switches on

Algorithm 4 SoftDance controller

Input: inPacket(ServiceRequest)

Output: outPacket

```
1: senderIP = getSenderIP(inPacket)
2: // set up service
3: srcIP = getSrcIP(inPacket)
4: dstIP = getDstIP(inPacket)
5: (SDMBList, switchList) ← setupPath(srcIP, dstIP)
6: computeHashRange(SDMBList)
7: pushFlowEntry(switchList, SDMBList)
8: assignHashRange(SDMBList)
9: registerToService(srcIP, dstIP)
10: outPacket ← “confirm”
11: forward outPacket(senderIP)
```

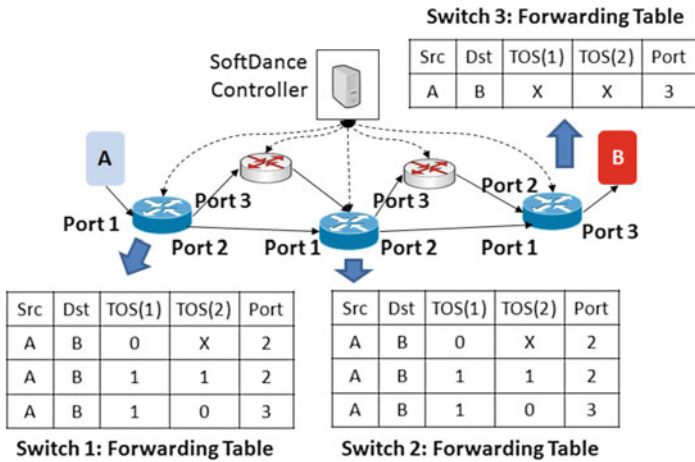


Fig. 5.4 SoftDance forwarding table example

the path (C2). Figure 5.4 illustrates forwarding tables with entries in each switch. The controller sends the computed hash ranges to SDMBs on the path, and the SDMBs set up the hash range for each path (C3). Then the controller sends a configuration message to a destination node for preparing deduplication in the storage system (C4). Finally, the controller registers a service with a pair (A,B) and sends an acknowledgement to the requesting client.

When a client’s SoftDance request has been approved, the client starts sending data packets. To forward a data packet, we use the most significant two bits in type of service (TOS) fields. The first bit in the TOS field is called a service bit, and it indicates whether a data packet uses our deduplication service. The second bit in the TOS field is called an encoding bit, and it indicates whether a data packet has been encoded by the previous SDMB. The client sends a data packet after setting a service bit and resetting an encoded bit (D1). When a switch receives a data packet,

if the service bit is on and the encoded bit off, the switch forwards the data packet to a SDMB (D2). Otherwise, the data packet is forwarded to the next switch. For example, as shown in Fig. 5.4, if the service bit is on and the encoded bit is off (third row), switch 1 forwards the data packet to a SDMB through port 3. Otherwise, data packets are sent to the next switch (switch 2) through port 3. A SDMB checks the redundancy of the data packet while comparing an index of its payload with previously saved indexes. If the same index exists, the data packet is redundant. In this case, a payload is replaced with an index and an encoded bit is set (D3). When a server receives a data packet, if the encoded bit is not set, an index and the data themselves are saved. Otherwise, only the index is stored for future data reconstruction (D4).

5.4 Encoding Algorithms in Middlebox (SDMB)

A SDMB receives SoftDance service packets that are forwarded by a switch and encodes redundant packets among the received packets. In this section, we explain how packets are received and how redundant packets are encoded.

Algorithm 5 explains packet processing in a SDMB. The SDMB computes the path ID of a packet based on a source IP address and a destination IP address from the packet header and retrieves the payload from the packet. Then the SDMB computes a hash range key of the packet using the SHA1 hash key [14]. Though

Algorithm 5 Packet processing in SDMB

Input: inPacket

Output: outPacket

```

1: // pathID is <srcIP> _ <dstIP>
2: pathID = getPathID(inPacket)
3: payload = getPayload(inPacket)
4: hashKey = computeHash(payload)
5: hashRangeKey = computeHashRangeKey(hashKey)
6: if hashRangeKey ∈ hashRange(pathID) then
7:   if hashKey ∈ indexTable then
8:     // redundant packet - encode
9:     replacePayload(hashKey, inPacket, outPacket)
10:    recomputeChecksum(outPacket)
11:   else
12:     // unique packet
13:     saveToIndexTable(hashKey)
14:     outPacket ← inPacket
15:   end if
16: else
17:   outPacket ← inPacket
18: end if
19: forward(outPacket)

```

it is implementation-specific, we use the SHA1 hash key for a uniform distribution of the data set. To compute the hash range key, we take the 18 most significant bits from a SHA1 hash, use a modulo operation with 100 and divide the remainder by 100 to obtain a range of the floating point from 0 to 1. If the computed hash range key is in the hash range that is set by a controller during the set-up phase (discussion of how a controller computes the hash range for a SDMB is deferred to the next section), the SDMB compares the hash key of a packet with hash keys (indexes) saved previously. When the current hash key exists in the index table, the current packet is redundant. The payload of a packet is replaced by a hash key and checksum is recomputed for continuous forwarding. If a packet is unique, a hash key is saved to the index table for future comparison, and the packet is sent to the next hop without encoding the index.

We implement a SDMB as a userspace program that is a callback function based on the `libnetfilter_queue` userspace library [16]. The SDMB runs on a Linux bridge that connects the incoming and outgoing network interfaces. To intercept an incoming packet, we set up the iptables rules in a filter table using OUTPUT for the client module, FORWARD for the SDMB and INPUT for the server module, along with iptables-extension NFQUEUE [15]. Whenever packets come in, they are passed on to a userspace program through the netfilter queue. The userspace program handles incoming packets and a processed packet is forwarded back to either a network element such as a switch and SDMB or a server.

5.5 Index Distribution Algorithms

A SDMB stores indexes of unique packets to compare the redundancy of future packets. As the large amount of indexes causes significant processing and memory overhead, we propose a distributed indexing mechanism. Using hash-based sampling [20], a SoftDance controller distributes hash ranges to SDMBs on a route of a flow, and each SDMB handles only those data packets whose hash range keys belong to a hash range assigned by a controller. In this manner, SoftDance can reduce processing time and memory size by handling a given data packet only once in a flow. In this section, we describe four different index distribution algorithms.

5.5.1 *SoftDANCE-Full (SD-Full)*

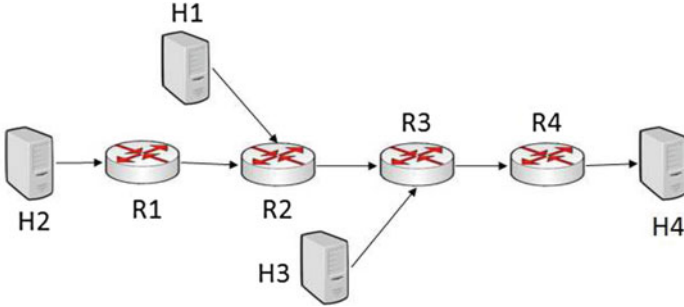
SD-full is an approach with a full hash range (0,1). Thus, using SD-full, a SDMB processes all incoming data packets and holds indexes of the unique packet among the incoming packets. The index size complexity per route is $O(n*m)$, where n and m are the number of unique packets and the number of SDMBs on a path respectively.

Algorithm 6 Compute hash range (uniform, merge)**Input:** sdMBList, pathList**Output:** nodes with hash range

```

1: for all path  $\in$  pathList do ▷ retrieve each path
2:   if approach == "uniform" then
3:     fraction = 1 / numSDMBs(path)
4:   else if approach == "merge" then
5:     totalDegree = getTotalDegree(path)
6:   end if
7:   sdMBs = getSDMBs(path)
8:   range = 0
9:   for all sdMB  $\in$  sdMBs do ▷ a sdMB in a path
10:    if approach == "merge" then
11:      fraction = sdMB.getDegree() / totalDegree
12:    end if
13:    sdMB.lowerBound = range
14:    sdMB.upperBound = range + fraction
15:    range = range + fraction
16:  end for
17: end for

```

**Fig. 5.5** A network topology with three routes**5.5.2 SoftDance-Uniform (SD-Uniform)**

ND-uniform distributes hash ranges uniformly over all SDMBs in a flow path. Each SDMB handles only packets whose hash range key is in its hash range. This scheme reduces index sizes compared to ND-full, with the trade-off of reducing bandwidth saving. As presented in Algorithm 6, a SoftDance controller retrieves a path between a client and a server and computes uniform fractions of SDMBs on a path. Then the controller assigns a disjoint hash range of each SDMB sequentially from the closest SDMB (to client) to the farthest SDMB based on accumulated hash ranges sequentially from the closest SDMB (to client) to the farthest SDMB. For example, in Fig. 5.5, a path $H1-H4$ has three SDMBs. Thus, each SDMB has a fraction of $\frac{1}{3}$ or 0.33. Hash ranges are computed from the first SDMB ($R2$) to the last SDMB ($R4$)

Table 5.1 SD-uniform hash ranges and generated index size

	Path	$R1$	$R2$	$R3$	$R4$
Hash range	$H1-H4$	–	$[0,0.33)$	$[0.33,0.66)$	$[0.66,1)$
	$H2-H4$	$[0,0.25)$	$[0.25,0.5)$	$[0.5,0.75)$	$[0.75,1)$
	$H3-H4$	–	–	$[0,0.5)$	$[0.5,1)$
Index	$H1-H4$	–	1 (A)	0	1 (B)
	$H2-H4$	0	0 (dup A)	1 (B)	0
	$H3-H4$	–	–	1 (A)	0 (dup B)
	Total	0	1	2	1

in a path, starting from 0, by accumulating the hash ranges: $R2$ is assigned $[0,0.33)$, where 0 is inclusive and 0.33 is exclusive. In this manner, $R3$ and $R4$ are assigned $[0.33, 0.66)$ and $[0.66, 1)$ respectively.

Table 5.1 presents an example of how many memory indexes SD-uniform produces on a topology of Fig. 5.5. ‘–’ means a SDMB is not on a path. ‘0 (dup A)’ means a packet A is redundant and none of the redundant index is stored. Suppose all clients send two packets A and B, and the hash range keys of packets A and B are 0.3 and 0.7 respectively. Assuming clients $H1$, $H2$ and $H3$ send sequentially, the total memory size to be stored is 4. Concretely, hash ranges are computed uniformly among SDMBs on a path. When a client $H1$ sends packets A and B, SDMBs $R2$ and $R4$ store indexes of A and B. A SDMB $R3$ just forwards data packets without storing indexes because the hash range keys of A (0.3) and B (0.7) are not in the hash range $[0.33,0.66)$ of $R3$. When a client $H2$ sends packets A and B, SDMB $R2$ finds that data packet A is redundant, and SDMB $R3$ stores an index of data packet B whose hash range key (0.7) is within the hash range $[0.5,0.75)$ of SDMB $R3$. Likewise, a data packet B from a client $H3$ is found to be redundant at SDMB $R4$, so again no index is stored. In this manner, SD-uniform reduces index sizes from 8 (in case each SDMB stores indexes of packet A and B) to 4. The complexity of the index size per path is $O(n)$, where n is the number of unique packets in a flow path.

5.5.3 SoftDANCE-Merge (SD-Merge)

SD-merge assigns disjoint hash ranges only for the SDMBs that have more than one incoming flows of the same destination (merge). As presented in Algorithm 6, SD-merge counts the total incoming degree of merge SDMBs on a path. Then a fraction of a SDMB is computed by Eq. (5.1):

$$\frac{\text{incoming degree of a SDMB}}{\text{total incoming degree of merge SDMBs on a path}}. \quad (5.1)$$

Table 5.2 SD-merge hash ranges and generated index size

	Path	$R1$	$R2$ (merge)	$R3$ (merge)	$R4$
Hash range	$H1-H4$	–	$[0,0.5)$	$[0.5,1)$	–
	$H2-H4$	–	$[0,0.5)$	$[0.5,1)$	–
	$H3-H4$	–	–	$[0,1)$	–
Index	$H1-H4$	–	1 (A)	1 (B)	–
	$H2-H4$	–	0 (dup A)	0 (dup B)	–
	$H3-H4$	–	–	1 (A), 0 (dup B)	–
	Total	0	1	2	0

The `sdMB.getDegree()` function returns 0 if a SDMB is not a merge node, leading the fraction to 0. Hash ranges are computed by accumulating fractions, starting from 0 like SD-uniform. In Fig. 5.5, for a path $H1-H4$, there are two merge SDMBs, $R2$ and $R3$. Thus, $R2$ and $R3$ are assigned $[0,0.5)$ and $[0.5,1)$ respectively, but $R4$ is not assigned hash ranges; that is, incoming packets to $R2$ are just forwarded to the next hop (in this case, $H4$) without encoding.

Table 5.2 shows how many indexes SD-merge produces. Paths $H1-H4$ and $H2-H4$ have two merge SDMBs ($R2$ and $R3$), and $H3-H4$ has only one merge SDMB ($R3$). When a client $H1$ sends data packets A and B, $R2$ stores the index of packet A, while $R3$ stores the index of packet B. Data packets sent from $H2$ are found to be redundant at $R2$ and $R3$. When a client $H3$ sends data packets, an index of data packet A is stored at $R3$, but an index of data packet B is found to be redundant. $R1$ and $R4$ just forward packets because their hash ranges are outside of $[0,1)$. The total index size of SD-merge is now 3, which is lower than that of SD-uniform. This shows that assigning hash ranges to only merge nodes can find more redundant packets.

5.5.4 SoftDANCE-Optimize (SD-opt)

Because both SD-uniform and SD-merge assign the hash range based on the flow path information, they may not be able to consider dynamic conditions, such as network traffic, packet redundancy and resource constraints. To obtain better hash ranges, we use a linear programming (LP) formation of SmartRE [2] for the SD-opt scheme. Algorithm 7 shows how to compute hash ranges based on LP. To run LP, SD-opt needs input constants: (1) redundancy profile that indicates how many packets across paths are redundant (denoted by $\text{match}_{p,q}$) and how many bytes across paths are redundant (denoted by $\text{matchSize}_{p,q}$); (2) the number of packets that passed SDMBs on a path p (denoted by packet_p); (3) the number of unique packets that passed SDMBs on a path p (denoted by $\text{packet}_{p,\text{unique}}$). SDMBs maintain these input constants, and a SoftDance controller uses the input constants that are collected from SDMBs. The `solveLP()` function runs LP, computes fractions of SDMBs that are

Algorithm 7 runOptHashRange: compute hash range (optimize)**Input:** sdMBList, pathList**Output:** nodes with hash range

```

1: rProfileList  $\leftarrow$  importRProfile()  $\triangleright$  matchp,q, matchSizep,q
2: pathList(packetp)  $\leftarrow$  importPacketCounts()
3: pathList(packetp,unique)  $\leftarrow$  importUniqueCounts()
4: // hash range is set to lowerbound, upperbound in a node
5: solveLP(sdMBList, pathList, rProfileList)  $\triangleright$  by LP
6: range = 0
7: for all path  $\in$  pathList do
8:   for all sdMB  $\in$  path do  $\triangleright$  a sdMB in a path
9:     fraction = sdMB.fraction()  $\triangleright$  set by solveLP()
10:    setHashRange(sdMB, range, range+fraction)
11:    range = range + fraction
12:   end for
13: end for

```

the results of LP and sets the fractions into SDMBs. The setHashRange() function computes the hash range with fractions of SDMBs on a path; specifically, lower and upper bounds are computed for each hash range.

The adopted formulation of SD-opt is different from the formulation of SmartRE [2]. SoftDance stores only indexes, while SmartRE stores packets as well as indexes, which changes the memory constraints in our formulation. Also, SoftDance performs indexing and encoding on SDMBs, but SmartRE runs storing packets and decodes encoded packets, which changes the processing constraints in our formulation. The formulation of SD-opt has three constraints: memory constraints, processing constraints and fraction constraints. For memory constraints in Eq. (5.2), each SDMB stores all indexes of unique packets that are within the assigned hash ranges, and the index sizes in the SDMB should be less than the size of available memory. $d_{p,r}$ is a fraction of a packet that a SDMB r can hold on a path p . indexSize is 40 bytes of a hash key string. packet_{p,unique} is the number of unique packets on path p . M_r is the maximum available memory of a SDMB r :

$$\forall r, \sum_{p:r \in p} d_{p,r} \times \text{packet}_{p,\text{unique}} \times \text{indexSize} \leq M_r. \quad (5.2)$$

For the processing constraint in Eq. (5.3), each SDMB checks the hash range and index table to verify redundancy and encodes redundant packets. The total processing should be less than the maximum available processing capability, L_r . packet_p is the number of packets passing a SDMB on a path p . match_{p,q} is the number of packets matched across paths p and q :

$$\forall r, \sum_{p:r \in p} d_{p,r} \text{packet}_p + \sum_{p,q:r \in p,q} d_{q,r} \text{match}_{p,q} \leq L_r; \quad (5.3)$$

$$\forall p, \sum_{r:r \in p} d_{p,r} \leq 1. \quad (5.4)$$

The third constraint is shown in Eq.(5.4), where the maximum sum of the fraction on a path is 1. The objective shown in Eq.(5.5) is to find the largest amount of redundant packets considering the storage space and bandwidth savings. $\text{matchSize}_{p,q}$ is the total size of matched packets across paths p and q . Our objective is different from that of SmartRE, which is to achieve only bandwidth savings:

$$\max \left(\sum_p \sum_r \sum_{q:r \in q} d_{q,r} \times \text{matchSize}_{p,q} \right). \quad (5.5)$$

5.6 Implementation

In this section, we show how a SoftDance controller is deployed with Floodlight, how to create REST URI so that a controller communicates with middleboxes. SoftDance controller exploits CPLEX optimizer to compute the optimal index allocation to middleboxes on paths. We explain how to install CPLEX optimizer on a Linux server and how to run CPLEX optimizer in Java step by step.

5.6.1 Floodlight, REST, JSON

We use Floodlight [6] to implement a SoftDance controller. We implement a Floodlight module [5] that computes hash ranges. A client module, SDMBs and a server module communicate with the SoftDance controller through the REST API using cURL [22]. We add REST API URIs to the Floodlight module for communication. SDMBs use the C++ JSON parser [8] to parse JSON data (with hash ranges) that are delivered from the SoftDance controller. A few important URIs are shown in Table 5.3. For example, `/wm/re/set/<op>` is used for a SDMB to send node information, including IP and MAC addresses, to a controller using the POST method. `<op>` is one of ‘uniform’, ‘merge’, and ‘opt’. `/wm/re/cmd/<op>/<ip>` is used for a SDMB to receive hash range from the controller.

Figure 5.6 describes the JSON format of an example responded to by the ‘get hash range’ URI. SD-opt requires input constants including $\text{packet}_{p,\text{unique}}$, packet_p , $\text{match}_{p,q}$, $\text{matchSize}_{p,q}$, M_r and L_r to run LP. In our prototype, SDMBs maintain the input constants during each SoftDance service.

5.6.2 CPLEX Optimizer: Installation

Network optimization is the goal of Internet Service Providers (ISPs). One example of the goals of optimization is the maximization of available bandwidth. LP is one

Table 5.3 REST API URIs

URI	Method	Description
/wm/re/cmd/<op>/<ip>	GET	Get hash range (ip: ip address of sdMB)
/wm/re/cmd/hashRange/<op>	GET	Signal to compute hash range
/wm/re/set/<op>	POST	Send node information(ip, mac addr) to controller
/wm/re/get/<op>	GET	Retrieve paths

Fig. 5.6 JSON format
example: response of hash
range URI

```
"hashRanges": [
  {
    "hashRangeLB": "0.5",
    "hashRangeUB": "1.0",
    "path": "192.168.2.12_192.168.2.5"
  },
  {
    "hashRangeLB": "0.5",
    "hashRangeUB": "1.0",
    "path": "192.168.2.11_192.168.2.5"
  }
]
```

way to obtain optimized values, and CPLEX is a tool to run LP. When an optimized value needs to be computed on a server module, we run a server program using the CPLEX API.

This section and the next two sections show how to run LP in a server program written in Java. The program runs a linear program using IBM ILOG CPLEX Java API. You will learn how to install CPLEX in Linux and how to run CPLEX using Interactive Optimizer. Finally, you will learn how to run CPLEX using a Java application with the CPLEX API.

This document is based on Ubuntu 12.04 and IBM ILOG CPLEX Optimization Studio V12.6. You can download it for free by joining the IBM Academic Initiative. Eclipse is used for the Integrated Development Environment (IDE).

First, we download IBM CPLEX Studio 12.6 (cplex_studio126.linux-x86-64.bin) and copy the file to the '/opt' directory. Then we change permission to run the installation file.

```
root@server:/opt# chmod 755 cplex_studio126.linux-x86-64.bin
root@server:/opt# ls -l
-rwxr-xr-x 1 root root cplex_studio126.linux-x86-64.bin
:
```

We install CPLEX by typing the file name. During installation, simply hit 'enter' key six times

```

root@server:/opt# cplex_studio126.linux-x86-64.bin
:
Choose Locale ...

    1- Deutsch
-> 2- English
    3- Español
    4- Français
    5- Português (Brasil)

CHOOSE LOCALE BY NUMBER:          // <— Enter
=====
IBM ILOG CPLEX Optimization Studio 12.6
=====

Preparing CONSOLE Mode Installation ...

=====
:
PRESS <ENTER> TO CONTINUE:          // <— Enter

    Default Install Folder: /opt/ibm/ILOG/CPLEX_Studio126

ENTER AN ABSOLUTE PATH, OR PRESS <ENTER> ... // <— Enter
:

=====
Ready To Install
=====

install IBM ILOG CPLEX Optimization Studio 12.6
onto your system at the following location:

    /opt/ibm/ILOG/CPLEX_Studio126

PRESS <ENTER> TO INSTALL:          // <— Enter

=====
Pre-Installation Summary
=====

Please Review the Following Before Continuing:

Product Name:
    IBM ILOG CPLEX Optimization Studio 12.6

Install Folder:
    /opt/ibm/ILOG/CPLEX_Studio126

Product Version
    12.6.0.0

```

```
Disk Space Information (for Installation Target):
    Required: 1,310,151,998 Bytes
    Available: 646,420,684,800 Bytes

PRESS <ENTER> TO CONTINUE:          // <— Enter

:
IBM ILOG CPLEX Optimization Studio 12.6
has been successfully installed to:

    /opt/ibm/ILOG/CPLEX_Studio126

PRESS <ENTER> TO EXIT THE INSTALLER: // <— Enter
```

We are ready to use CPLEX in Eclipse (with Java CPLEX API) and add cplex.jar. We open Eclipse in Linux (assuming Eclipse was installed). The first step is to create a new Java project, as shown in Fig. 5.7.

We move to the project-properties-java build path-libraries and click ‘Add External JARs’ (Fig. 5.8). We select ‘cplex.jar’ in the /opt/ibm/ILOG/CPLEX_Studio126/cplex directory. As a result, cplex.jar is added to the library of a project as in Fig. 5.9. Then we click the ‘OK’ button.

We create a skeleton CPLEX program by creating a new class, ‘Hello’, with ‘cplextest’ as the package name (Fig. 5.10) and insert codes. We need to import IloCplex and IloException classes.

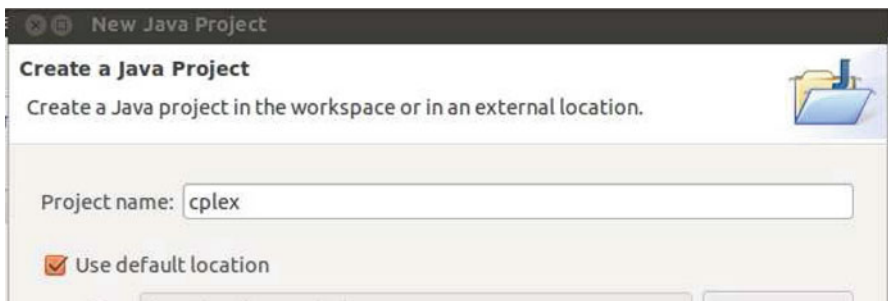


Fig. 5.7 Create new Java project

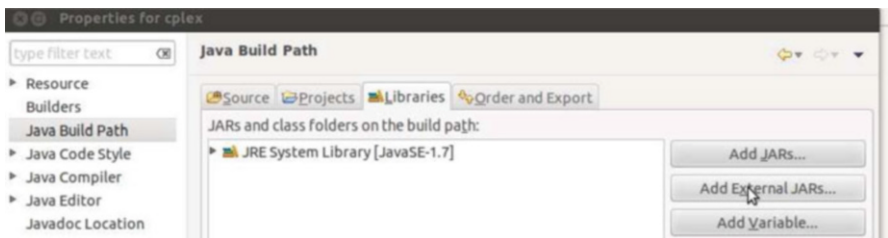


Fig. 5.8 Add external JARs in a project

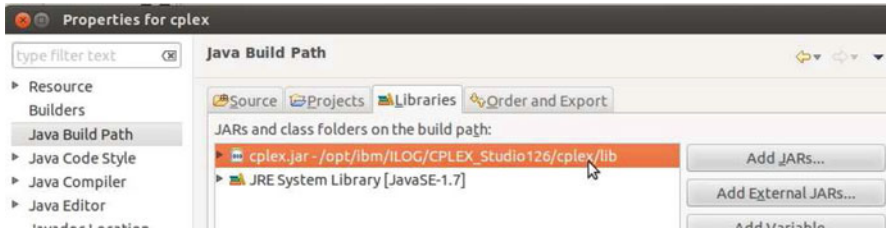


Fig. 5.9 cplex.jar is added to a project

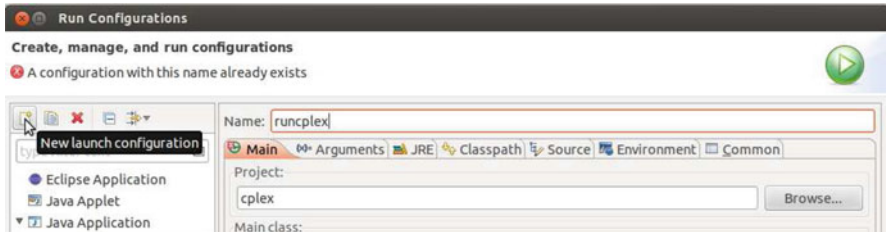


Fig. 5.10 Create a new class, 'Hello'

```
package cplextest;

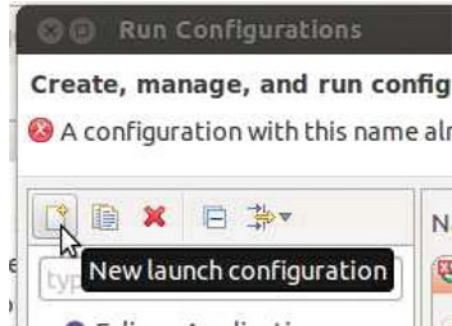
import ilog.concert.IloException;
import ilog.cplex.IloCplex;

public class Hello {
    public static void main(String[] args) {
        try {
            IloCplex cplex = new IloCplex();
            System.out.println("Hello_Cplex");
        } catch (IloException e) {
            System.err.println("cplex_exception:_" + e);
        }
    }
}
```

We create a run configuration with a library path by typing 'Run' and 'Run Configuration'. We click 'new launch configuration' button and create a new configuration (Fig. 5.11). We fill out the name ('cplex') and Main class (cplextest.hello). Then we go to 'Arguments'. In 'VM arguments', we add a path (-Djava.library.path=/opt/ibm/ILOG/CPLEX_Studio126/cplex/bin/x86-64_linux) to the CPLEX library and click 'apply'. The path is a directory where cplexXX.so is located.

All we see as output in the console is 'Hello Cplex'. Even though we still do not run the CPLEX program, we correctly instantiate a CPLEX object using 'new()'. This is a starting point to run the CPLEX program.

Fig. 5.11 Create a run configuration



5.6.3 CPLEX Optimizer: Run Simple CPLEX Using Interactive Optimizer

After we install IBM ILOG CPLEX, LP can be solved by the command line interactive optimizer. We compare the result from the interactive optimizer with one acquired by the Java CPLEX program. For simplicity, let us assume that we need to solve the following LP problem:

```
// LP problem

maximize
  x1 + 2x2 + 3x3
subject to
  -x1 + x2 + x3 <= 20
  x1 - 3x2 + x3 <= 30
bounds
  x1 >= 0
  x1 <= 40
  x2 >= 0
  x3 >= 0
ends
```

To run Interactive Optimizer, we need to set up a PATH environment variable. We go to the home directory, open `.bashrc`, and add a path of `cplex` to the PATH environment variable as follows. We make the environment variable effective by typing `source .bashrc`. We see that the ILOG CPLEX path is in the PATH environment variable. We can run `cplex` in any directory.

```
root@server: /opt/ibm/ILOG/CPLEX_Studio126/cplex/bin/
x86-64_linux# cd
root@server:~# vi .bashrc
:
PATH=$PATH:/opt/ibm/ILOG/CPLEX_Studio126/cplex/bin/x86-64_linux:.
export PATH
:

root@server:~# source .bashrc
root@server:~# env | grep PATH
```

```
:
PATH=/opt/ibm/ILOG/CPLEX_Studio126/cplex/bin/x86-64_linux:.
:
```

We create an example.lp file with the previously assumed LP problem using any editor (here you can use VI editor or nano editor in Linux).

```
root@server:~# vi example.lp
maximize
  x1 + 2 x2 + 3 x3
subject to
-x1 + x2 + x3 <= 20
x1 - 3 x2 + x3 <= 30
bounds
x1 >= 0
x1 <= 40
x2 >= 0
x3 >= 0
end
```

We now run cplex and solve the problem. We solve a LP problem using Interactive Optimizer. Objective is 202.5, and we obtain three solution values, 40, 17.5 and 42.5 for x1, x2 and x3 respectively.

```
root@server:~# cplex

Welcome to IBM(R) ILOG(R) CPLEX(R) Interactive Optimizer 12.6.0.0
  with Simplex, Mixed Integer & Barrier Optimizers
5725-A06 5725-A29 5724-Y48 5724-Y49 5724-Y54 5724-Y55 5655-Y21
Copyright IBM Corp. 1988, 2013. All Rights Reserved.

CPLEX> read example.lp
Problem 'example.lp' read.
Read time = 0.00 sec. (0.00 ticks)
CPLEX> opt
Tried aggregator 1 time.
No LP presolve or aggregator reductions.
Presolve time = 0.00 sec. (0.00 ticks)

Iteration log . . .
Iteration:    1    Dual infeasibility =      0.000000
Iteration:    2    Dual objective    =    202.500000

Dual simplex - Optimal: Objective = 2.0250000000e+02
Solution time = 0.00 sec. Iterations = 2 (1)
Deterministic time = 0.00 ticks (4.50 ticks/sec)

CPLEX> disp sol var -
Variable Name      Solution Value
x1                  40.000000
x2                  17.500000
x3                  42.500000
CPLEX> quit
root@server:~#
```

5.6.4 CPLEX Optimizer: Run Simple CPLEX Using Java Application (with CPLEX API)

SoftDance-optimize runs CPLEX using a Java application on the middlebox. In this section, we explain how to implement and deploy a Java application to run CPLEX. Our goal is to compute the same solution using Interactive Optimizer. We use the same source code as a reference site [3]. Interested readers can find the CPLEX Java API [7]. The following are the CPLEX Java codes:

```
package cplextest;

import ilog.concert.IloException;
import ilog.concert.IloNumVar;
import ilog.cplex.IloCplex;

public class Hello {
    public static void main(String[] args) {
        try {
            IloCplex cplex = new IloCplex();

            //-----
            // 0. Define constants
            //-----
            // objective
            double[] obj_co = {1.0, 2.0, 3.0}; // coefficients in object,
                                              // 1 x1 + 2 x2 + 3 x3

            // bounds of variables
            double[] lb = {0.0, 0.0, 0.0};           // lower bound
            double[] ub = {40.0, Double.MAX_VALUE, Double.MAX_VALUE};
                                              // upper bound

            // coefficients of constraints
            double[] const1_co = {-1, 1, 1};
            double[] const2_co = { 1, -3, 1};

            //-----
            // 0. Define variables
            //-----
            // variables
            IloNumVar[] x = cplex.numVarArray(3, lb, ub);
                // x1,x2,x3 with lower and upper bound
                // 0 <= x1 <= 40
                // 0 <= x2 <= infinite
                // 0 <= x3 <= infinite

            //-----
            // 1. create a LP problem
            //-----
            // add objective
            // x1 + 2x2 + 3x3
            cplex.addMaximize(cplex.scalProd(x, obj_co));
```

```

// add constraints
// -x1 + x2 + x3 <= 20
cplex.addLe(cplex.scalProd(x, const1_co), 20);
// x1 - 3x2 + x3 <= 30
cplex.addLe(cplex.scalProd(x, const2_co), 30);

//-----
// 2. solve the problem
//-----
if (cplex.solve()) {

    //-----
    // 3. get solution (objective, variables)
    //-----
    // objective
    cplex.output().println("Solution_value=" +
                           cplex.getObjValue());

    // variables
    double[] vars = cplex.getValues(x);
    int numVars = cplex.getNcols();
    for (int i=0; i < numVars; i++) {
        int index = i+1;
        cplex.output().println("x" + index + "=" + vars[i]);
    }

    //-----
    // 4. use the solution
    //-----
    // <— codes to use the solution value and
    //      value of variables (vars[i])

} else {
    cplex.output().println("No_solution_exists");
}
cplex.end();

} catch (IloException e) {
    System.err.println("cplex_exception: " + e);
}
}
}

```

When we run cplex Java application in middlebox, we see the objective and values of variables that are the same as those in Interactive Optimizer. The variables, including x_1 , x_2 and x_3 , can be used for other Java programs. Thus, for SoftDance, a controller contains the CPLEX programs and runs the CPLEX programs to compute the index hash range by SoftDance-optimize.

```

Tried aggregator 1 time.
No LP presolve or aggregator reductions.
Presolve time = 0.00 sec. (0.00 ticks)

```

```
Iteration log . . .
Iteration:      1      Dual infeasibility =    0.000000
Iteration:      2      Dual objective   =   202.500000
Solution value = 202.5                      // <— objective
x1 = 40.0                                // <— variable
x2 = 17.5                                // <— variable
x3 = 42.5                                // <— variable
```

5.7 Setup

We begin by describing our setup of the testbed experiment and emulation along with the topology and metrics.

5.7.1 Experiment

We deployed a testbed experiment to verify that SoftDANCE works practically in a physical testbed system. As shown in Fig. 5.12, the experiment consists of three clients (*H1*, *H2* and *H3*), a server (*H4*), four SDMBs (*R1* to *R4*), OpenVSwitches and a controller. The controller is connected to all nodes through an out-of-band network (not shown in figure). There are three paths, and each client sends the same data set as other clients. Thus, the redundancy of all data sets is $\frac{2}{3}$.

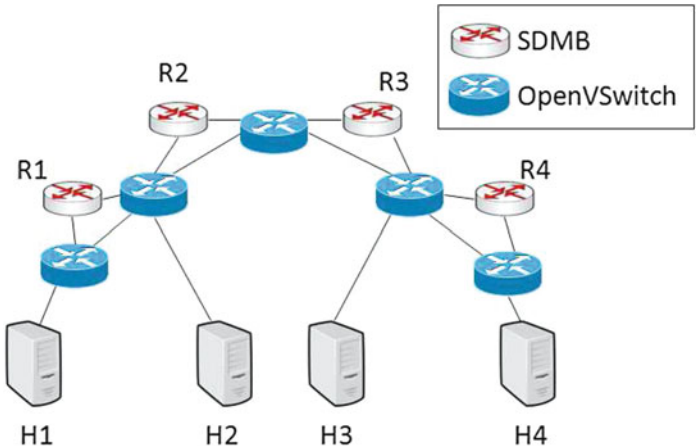


Fig. 5.12 Experiment topology

5.7.2 Emulation

We also set up several topologies, such as a tree, a multi-rooted tree and a fat tree based on mininet [13] (Fig. 5.13). The purpose of choosing the topologies is to validate SoftDANCE on typical topologies in the data centre network (DCN). In all topologies, we choose a server that is on the far right side: $H8$ in the tree, $H16$ in multi-rooted tree and fat tree. Other hosts act as clients. $R\{x\}$ is a SDBM, and $S\{x\}$ is an OpenVSwitch. A controller communicates with all nodes through an in-band (not shown in figure). We do not use a multi-path because our main purpose is to measure performance and overhead on the same path for all compared techniques: a switch is chosen by a spanning tree in both the multi-root tree and fat tree. Thus, the number of nodes selected for the multi-root tree and fat tree are the same with a single core switch. As in the experiment, all clients send the same data set as other clients, so the redundancy is $\frac{6}{7}$ for the tree and $\frac{14}{15}$ for multi-root tree/fat tree.

5.8 Evaluation

We measure the performance and overhead of SoftDance compared to other existing storage and network data reduction techniques. We then compare overall and per-topology performance and overhead of SoftDance with those of others. Finally, we contrast SoftDance with combined existing techniques. SoftDance is compared with client Dedup, server Dedup and network-wide RE (SmartRE). We implemented existing techniques for comparison. Client Dedup is denoted by ClientD. Server Dedup is divided into file-granularity Dedup (File Dedup), fixed-size block granularity Dedup (Fix Dedup) and variable-size chunk granularity Dedup (Var Dedup) based on granularity. SmartRE distributes hash ranges based on its optimization LP [2]. SmartRE is denoted by SRE. We also compare our approaches, including SD-full, SD-uniform, SD-merge and SD-opt.

5.8.1 Metrics

We use two metrics for measuring performance: storage space saving and network bandwidth saving, and two other metrics for measuring overhead: processing time and memory size. To demonstrate the storage space saving, we use the deduplication ratio. The deduplication ratio is a typical means of showing how much storage space is saved and is computed using Eq. (5.6). Network bandwidth saving is computed using Eq. (5.7). For overhead metrics, we measure the processing time at clients, a server and RE boxes. We also measure the size of memory of clients, a server and RE boxes:

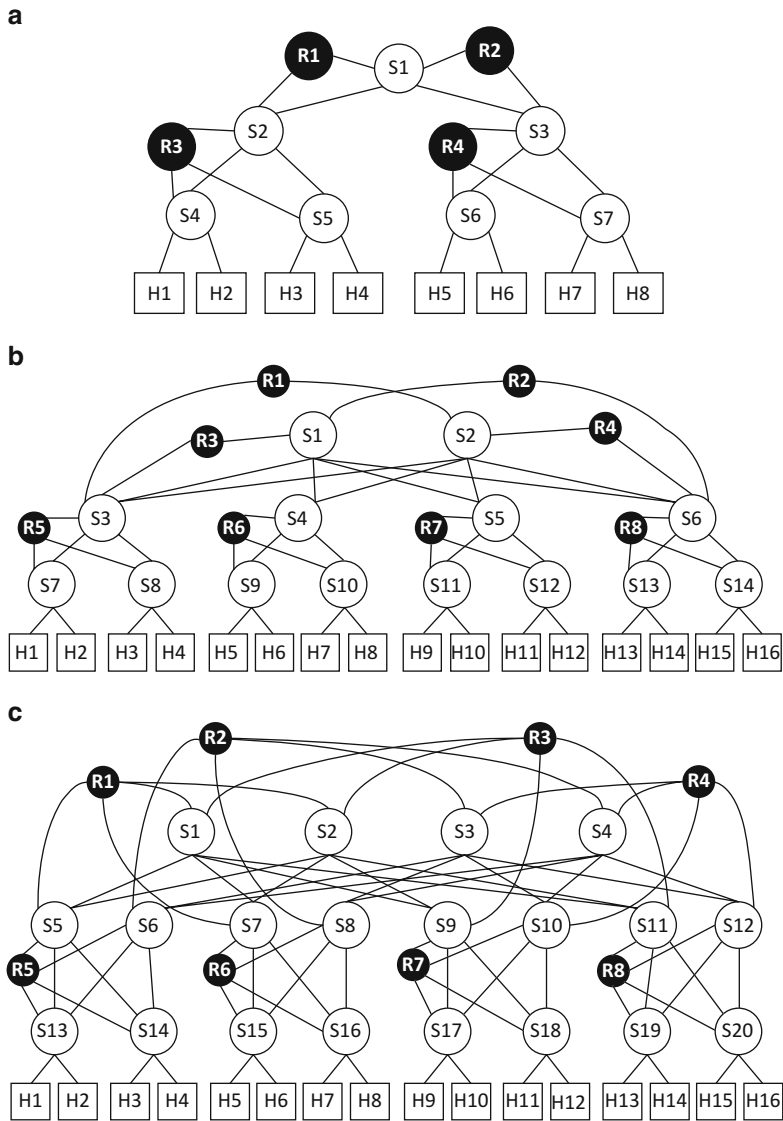


Fig. 5.13 Emulation topology. (a) Tree. (b) Multi-root tree. (c) Fat tree

$$\frac{\text{volume of redundant data eliminated}}{\text{total volume sent by all clients}} * 100, \quad (5.6)$$

$$\frac{\text{Reduced traffic size}}{\text{Total traffic size without redundancy elimination}} * 100. \quad (5.7)$$

5.8.2 Data Sets

For the data set, we use campus log data that were captured at a university data centre. The log data are backed up to storage servers every week. The data set that was used has rare redundancies under 2 %. Thus, for all techniques used, the intra-redundancy ratio found when a single client sends a data set is under 2 % at most.

5.8.3 Storage Space and Network Bandwidth Saving

We present the overall performance of SoftDance compared to existing techniques across all evaluation topologies: experiment and emulation (tree and multitree/fat-tree). For this purpose, we compare the relative value normalized for all metrics as in Figs. 5.14 and 5.15. We use a log scale for Fig. 5.15a, b, because the gap between the largest and smallest one is huge, and multiply 10^4 and 10^3 respectively to read the figures easily.

For storage space saving, as shown in Fig. 5.14a, SoftDance shows the closest performance to server Dedup (ServerD), which is the best for saving storage space using existing techniques. SD-full shows exactly the same space saving as the server Dedup. This indicates that SD-full does not miss any redundancy throughout the network. SD-uniform, SD-merge and SD-opt have space savings similar to that of the server Dedup. Meanwhile, client Dedup has the worst performance in terms of space saving, with only 1.6 % saving by eliminating the redundancy inside a client. This shows that client Dedup does not deal with redundancy across clients. SmartRE does not contribute storage space saving because it runs only on a network in an application agonistic fashion. In SoftDance approaches, SD-merge shows better performance than SD-uniform because it finds more redundancies at merge SDMBs.

For bandwidth saving, in Fig. 5.14b, SoftDance shows two to four times more bandwidth saving than SmartRE. The reason why SmartRE shows lower bandwidth saving than SoftDance is that SmartRE fails to find inter-path redundancies passing different ingress routers that are encoders. For example, in Fig. 5.12, two duplicate packets that arrive as encoders ($R1$ and $R2$) from different hosts ($H1$ and $H2$) are determined to be unique, and thereafter they cross to a server without eliminating redundancy as if they were unique packets. To investigate our argument, we choose a multi-root tree (Fig. 5.13) and build a test case where an edge SDMB is connected only to a client (e.g. $R5$ is connected only to $H1$ but not $H2$, $H3$ or $H4$). We choose four clients ($H1$, $H5$, $H9$, $H13$) and one server ($H16$). The bandwidth saving on the test case shows only approximately 4%, which is the sum of the intra-path redundancies from each client: 70 % inter-path redundancy is not detected ($\frac{3}{4}(75\%) - 4\%$).

In SoftDance, SD-opt (optimize) shows a storage space saving similar to that of SD-full, considering the resource constraints, but has lower bandwidth saving than SD-merge. Note that SD-opt optimizes based on matches between packets and seeks an overall benefit of space and bandwidth savings rather than only a single benefit. Thus, a single benefit can be lower than under heuristic approaches.

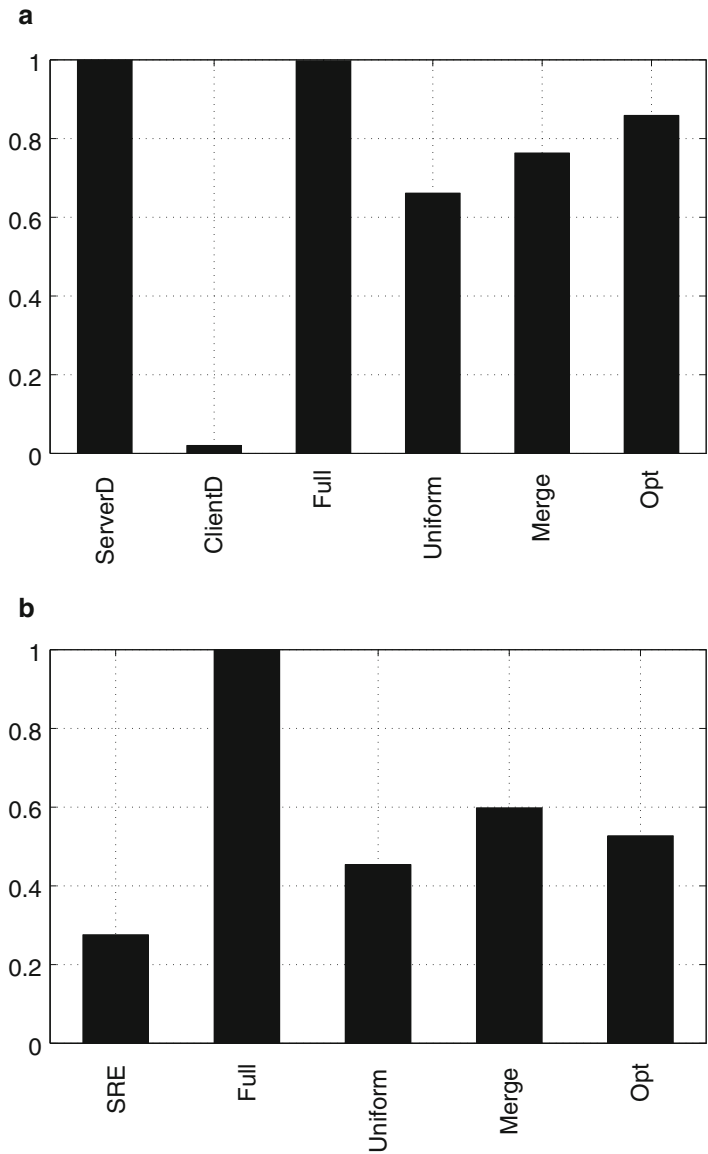


Fig. 5.14 Comparison of performance. (a) Storage space saving. (b) Bandwidth saving

5.8.4 CPU and Memory Overhead

For processing time in Fig. 5.15a, SoftDance (denoted by Full) is the lowest among all techniques. Client Dedup (denoted by ClientD) and Var Dedup (denoted

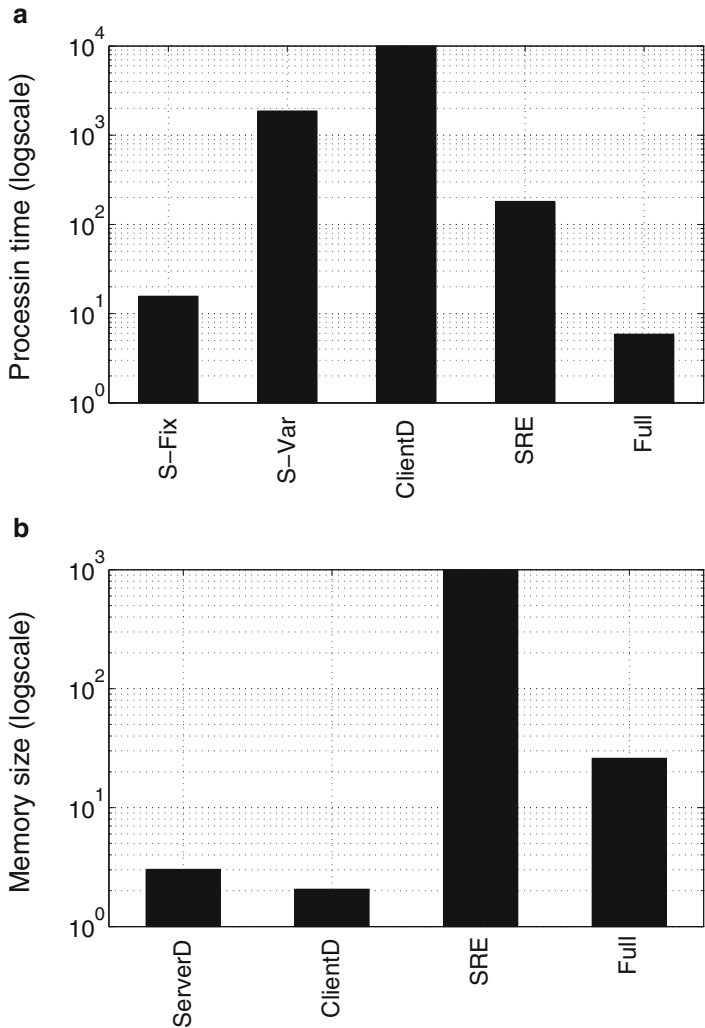


Fig. 5.15 Comparison of overhead. (a) Processing time: per node. (b) Memory size: per node

by S-Var) have 100 times and 10 times higher processing times than SmartRE owing to expensive chunking. Even SmartRE (denoted by SRE) shows a larger processing time than SoftDance because of sliding fingerprinting. For memory size in Fig. 5.15b, SoftDance (denoted by Full) has 40 times less memory than SmartRE. This is because SoftDance only stores indexes but SmartRE saves packets as well as indexes in caches for encoders and decoders. Evicting indexes and packets from caches can reduce memory size but may lead to consuming more bandwidth because the same incoming packets as previously evicted packets are not encoded (that is, they are found to be unique). However, SoftDance (Full) still consumes more

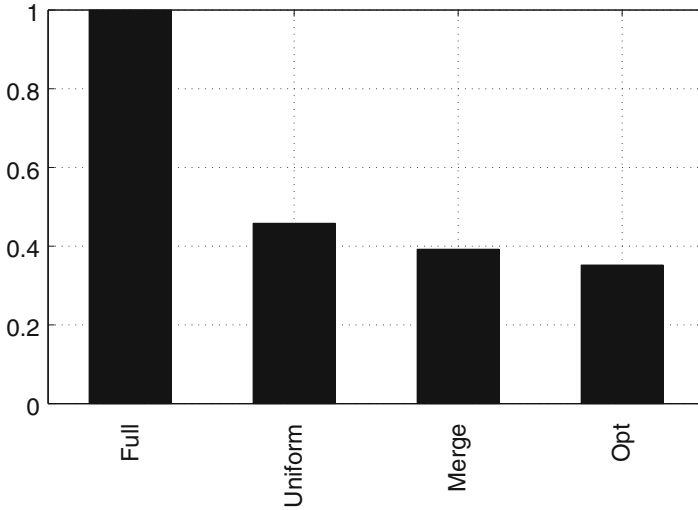


Fig. 5.16 Memory size among SoftDance approaches: per node

memory than server Dedup and client Dedup owing to the larger number of indexes on fine-grained granularity (SoftDance uses a 1.5 KB packet payload, but server Dedup and client Dedup use 8 KB chunk (or block) granularity). To reduce memory size, SoftDance distributes indexes based on hash-based sampling [20]. Figure 5.16 demonstrates that the memory size (concretely index size) required by SD-full can be reduced by up to three times by SD-opt.

5.8.5 Performance and Overhead per Topology

We show the performance per metric and changes in performance depending on different topologies. The differences among topologies are twofold: number of clients and location of clients. First, the number of clients in tree topology is more than that in experiments, resulting in more redundancy because each client sends the same data set as other clients in our evaluation. Second, in experiments, clients are attached to both edge and interior switches, whereas in emulation, clients are attached to only edge switches. Our focus is the performance each technique achieves with respect to the differences.

Storage space saving increases as the number of clients increases for all techniques (except for SD-opt) as shown in Fig. 5.17a. This is because redundancies increase proportionally to the number of clients that send the same data set. SD-full shows exactly the same space saving as server Dedup, which indicates that SD-full does not miss any redundancies throughout the network. In experiments, SD-merge achieves greater space saving than SD-uniform, which indicates that a merge SDMB

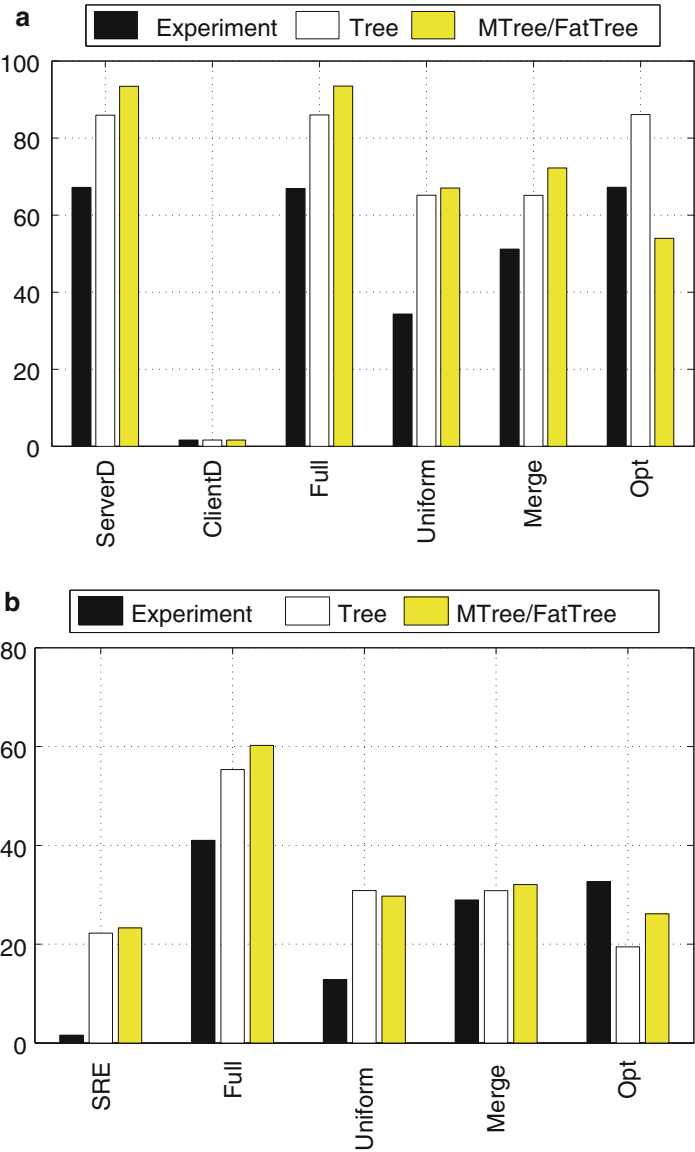


Fig. 5.17 Performance per topology. (a) Storage space saving (%). (b) Bandwidth saving (%)

has more redundant packets (originating from different paths) than a forwarding node with one incoming degree. SD-opt achieves the most space saving among distributed indexing approaches. However, we find that SD-opt in multi-root tree/fat tree shows a somewhat anomalous result that was expected to be higher than in the tree topology. We are investigating the anomaly.

In Fig. 5.17b, SoftDance shows more bandwidth saving than SmartRE. In experiments, SmartRE produced much – 10 to 40 times – less bandwidth saving than SoftDance, while in tree (or mtree/fattree), SmartRE had 2 to 3 times less bandwidth saving than SoftDance. The significant difference between performance in experiments and in a tree (or mtree/fatree) for SmartRE is not caused by an increase in the number of clients but by the fact that SmartRE fails to find inter-path redundancies passing different ingress routers.

As the number of clients increases, processing time increases (Fig. 5.18a) for all techniques (except SmartRE). However, the velocity of change is different; SoftDANCE (Full) increases more slowly in processing time than others. Other SoftDance approaches, such as SD-uniform, SD-merge and SD-opt (not shown here), have almost the same processing time as SD-full. Client Dedup and variable-size server Dedup are not shown as readable figures owing to excessive processing time. Interestingly, in experiments, SmartRE showed greater processing time than in tree topologies. We find that computers used for REboxes in experimentation are much slower than a computer used for emulation (tree and multi-root tree/fat tree), which amplifies processing time slowed by fingerprinting. Memory size increases in proportion to increases in the number of clients (Fig. 5.18b). For SmartRE (not shown in figure), memory size in SDMBs is X, 1.1X and 2.5X in experiments, trees and multi-root tree/fat tree respectively, where X requires 40 times more memory than SD-full. SD-full has larger indexes than server Dedup and client Dedup, but SoftDance can reduce the indexes by utilizing an indexing scheme like SD-merge to reduce the size of memory of SD-full.

5.8.6 *SoftDance vs. Combined Existing Deduplication Techniques*

We tested some scenarios in which client data are transferred across network links to be stored on a server, while each Dedup and NRE could be performed for the benefit of its own domain. The data may go through various forms of deduplication processes redundantly, thereby incurring significant processing and memory overhead. For comparison with SoftDance, we envision two combined approaches that can be used as a network and storage service using existing techniques: client Dedup (storage service) + SmartRE (network service) and server Dedup (storage service) + SmartRE (network service).

For storage space saving as shown in Fig. 5.19a, SoftDance shows the best space saving, equal to ServerD+SRE. Both combined approaches rely on storage services, including client Dedup and server Dedup, because SmartRE is not applicable for storage space saving. For bandwidth saving as shown in Fig. 5.19b, SoftDance saves the most bandwidth compared to the two combined approaches. For the two combined approaches, bandwidth saving is determined by the performance of SmartRE. For processing time, SoftDance outperforms the two combined

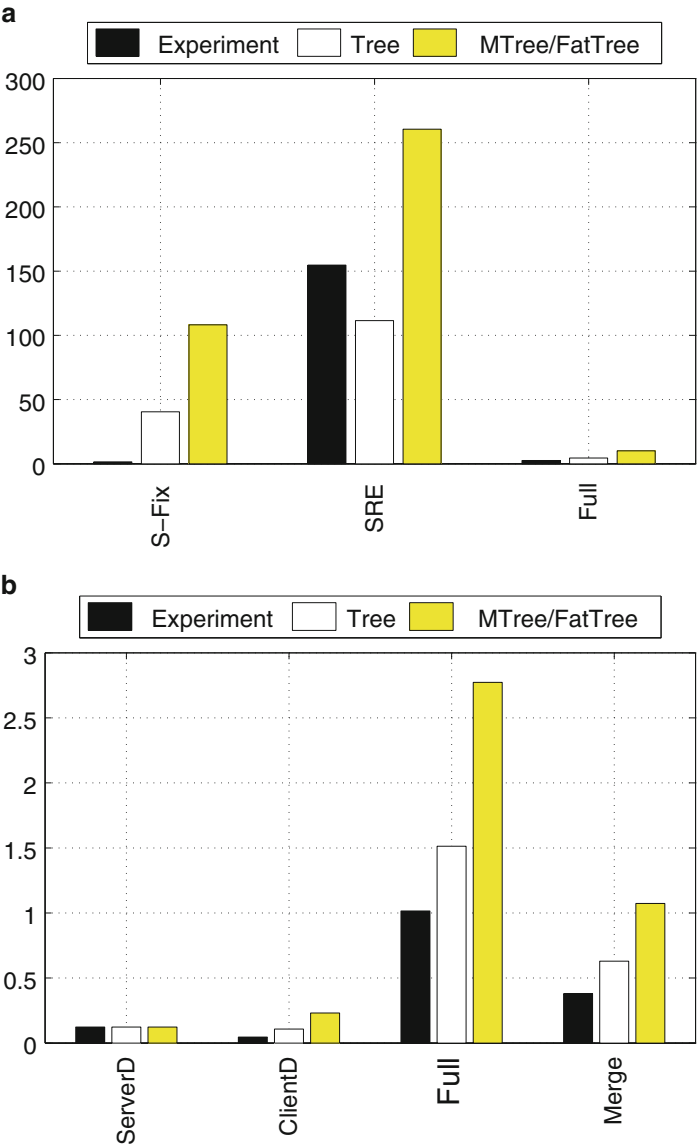


Fig. 5.18 Overhead per topology. (a) Processing time: per node (s). (b) Memory size: per node (MB)

approaches (Fig. 5.20a). The slow processing time of the two combined approaches is due to expensive chunking and fingerprinting. For memory size (Fig. 5.20b), SoftDance requires less memory than the two combined approaches. This is attributable to the fact that SmartRE itself stores packets as well as indexes. The slight reduction

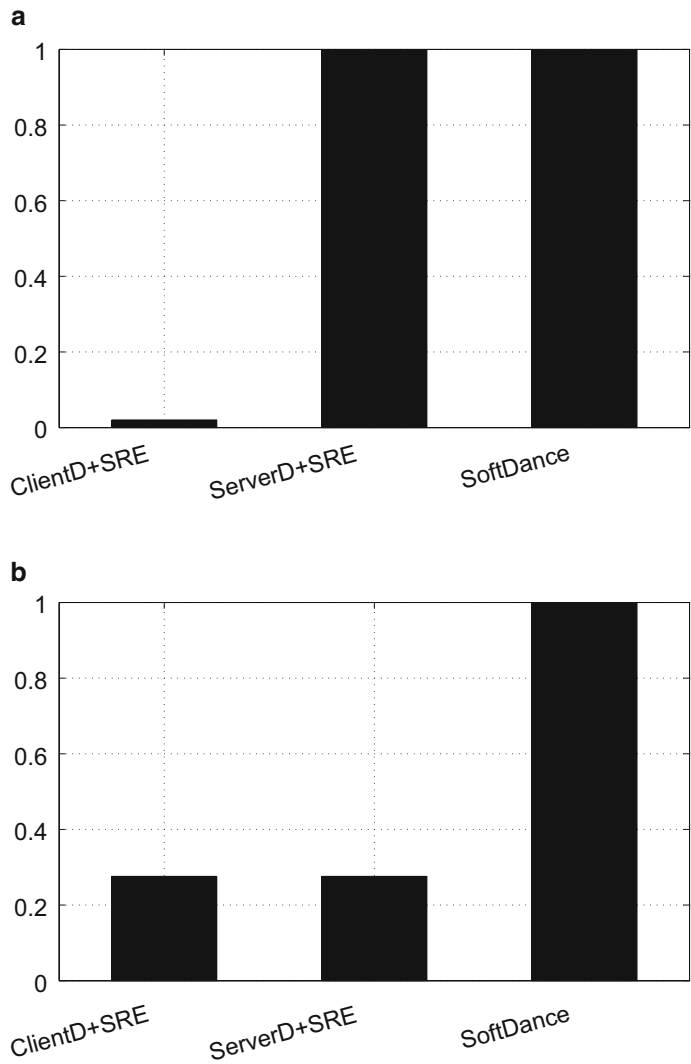


Fig. 5.19 Performance of combined approaches. **(a)** Storage space saving. **(b)** Bandwidth saving

in memory size inside a client becomes invalid owing to the excessive memory required by SmartRE. Overall, the evaluation results show that in the scenarios of both end-systems and networks performing deduplication redundantly, SoftDance achieves a very high level of efficiency in terms of processing and memory overhead.

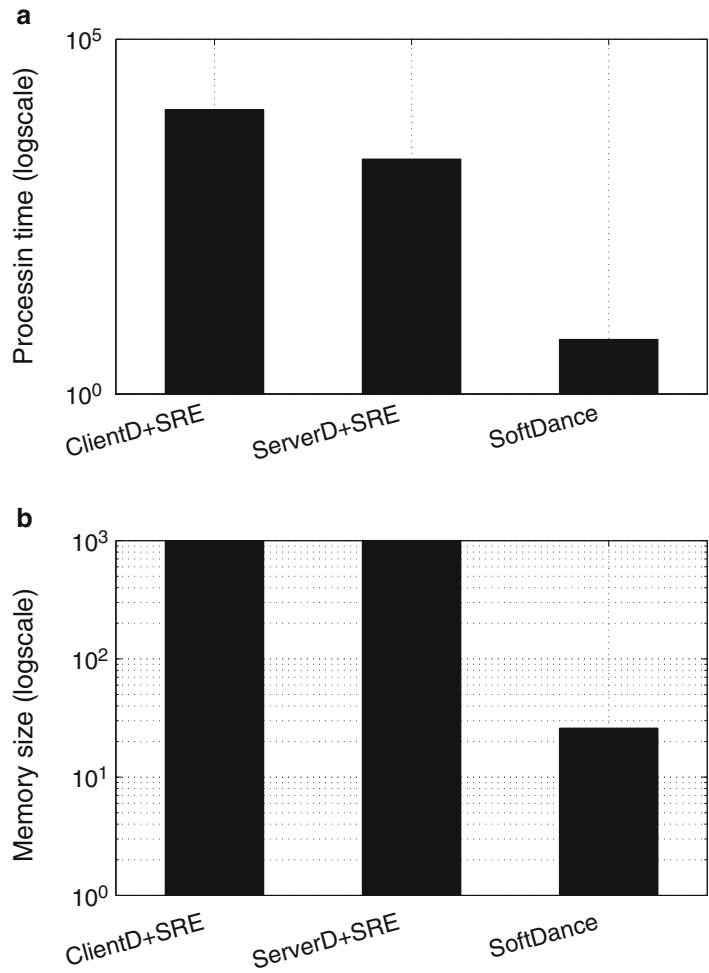


Fig. 5.20 Overhead of combined approaches. (a) Processing time: per node. (b) Memory size: per node

5.9 Summary

In this chapter, we presented SoftDance, an efficient software-defined deduplication system as a network and a storage service that produces savings in both storage space and network bandwidth while significantly reducing processing time and memory size. We developed efficient encoding and indexing algorithms for a SoftDance middlebox (SDMB) and an effective control mechanism for a SDN controller. We also built a prototype of testbed experiments and Mininet-based emulations to evaluate SoftDance on real system environments and typical DCN

topologies. Our evaluation results show that SoftDance saves two to four times more bandwidth than a RE technique (SmartRE) and as much or almost as much storage space as the Dedup technique with low overhead while achieving very high levels of efficiency in terms of processing and memory overhead.

References

1. Anand, A., Gupta, A., Akella, A., Seshan, S., Shenker, S.: Packet caches on routers: the implications of universal redundant traffic elimination. In: Proceeding of the ACM SIGCOMM 2009 Conference on Data Communication (2008)
2. Anand, A., Sekar, V., Akella, A.: SmartRE: an architecture for coordinated network-wide redundancy elimination. In: Proceeding of the ACM SIGCOMM 2009 Conference on Data Communication (2009)
3. Bard: CPLEX Tutorial. <http://www.me.utexas.edu/~bard/LP/LP%20Handouts/CPLEX%20Tutorial%20Handout.pdf> (2006)
4. Bolosky, W., Corbin, S., Goebel, D., Douceur, J.: Single instance storage in Windows 2000. In: Proceeding of the 4th USENIX Windows Systems Symposium (2000)
5. Floodlight: Floodlight module. <https://floodlight.atlassian.net/wiki/display/floodlightcontroller/Module+Applications> (2016)
6. Floodlight: Floodlight sdn controller. <http://www.projectfloodlight.org/floodlight/> (2016)
7. IBM: ILOG CPLEX Optimization Studio Java API. http://www-01.ibm.com/support/knowledgecenter/SSSA5P_12.2.0/ilog.odms.ide.help/html/refjavaopl/html/overview-summary.html (2016)
8. Json.org: C++ json parser. <http://sourceforge.net/projects/jsoncpp/> (2015)
9. Kim, D., Choi, B.Y.: HEDS: hybrid deduplication approach for email servers. In: 2012 Fourth International Conference on Ubiquitous and Future Networks (ICUFN) (2012)
10. Kim, D., Song, S., Choi, B.Y.: SAFE: structure-aware file and email deduplication for cloud-based storage systems. In: Proceeding of the 2nd IEEE International Conference on Cloud Networking (2013)
11. Lillibridge, M., Eshghi, K., Bhagwat, D., Deolalikar, V., Trezise, G., Camble, P.: Sparse indexing: large scale, inline deduplication using sampling and locality. In: Proceeding of the USENIX Conference on File and Storage Technologies (FAST) (2009)
12. Meyer, D.T., Bolosky, W.J.: A study of practical deduplication. In: Proceeding of the USENIX Conference on File and Storage Technologies (FAST) (2011)
13. Mininet.org: Mininet. <http://mininet.org/> (2016)
14. National Institute of Standards and Technology (NIST): Secure Hash Standard 1 (SHA1). <http://csrc.nist.gov/publications/fips/fips180-4/fips-180-4.pdf> (2015)
15. Netfilter.org: Iptables extensions. <http://ipset.netfilter.org/iptables-extensions.man.html> (2015)
16. Netfilter.org: Libnetfilter_queue. http://www.netfilter.org/projects/libnetfilter_queue/ (2016)
17. openstack.org: OpenStack. <https://github.com/okws/sfslite/blob/master/crypt/rabinpoly.h> (2006)
18. Quinlan, S., Dorward, S.: Venti: a new approach to archival storage. In: Proceeding of the USENIX Conference on File and Storage Technologies (FAST) (2002)
19. Rabin, M.O.: Fingerprinting by random polynomials. Tech. Rep. TR-15-81, Harvard University (1981)
20. Sekar, V., Reiter, M.K., Willinger, W., Zhang, H., Kompella, R.R., Andersen, D.G.: CSAMP: a system for network-wide flow monitoring. In: Proceeding of Networked Systems Design and Implementation (2008)

21. Spring, N.T., Wetherall, D.: A protocol-independent technique for eliminating redundant network traffic. In: Proceeding of the ACM SIGCOMM 2000 Conference on Data Communication (2000)
22. Stenberg, D.: Curl. <http://curl.haxx.se/> (2016)
23. Zhu, B., Li, K., Patterson, H.: Avoiding the disk bottleneck in the data domain deduplication file system. In: Proceeding of the USENIX Conference on File and Storage Technologies (FAST) (2008)

Part IV

Future Directions

In this part, we present ongoing and future work and conclude. In Chap. 6, we present mobile deduplication, an enhancement of client-based deduplication for popular files in mobile devices, where each file is deduplicated based on the file's structure with respect to a low-capacity mobile device. This approach considers the security of files and the performance of encryption algorithms based on systems with different CPU types such as an Android mobile device (ARM CPU) and desktop Linux server (Intel CPU). Mobile deduplication uses structure-aware deduplication for files to improve processing time. We demonstrate how the performance of encryption changes depending on the level of security and systems with different CPU types. In Chap. 7, we conclude by discussing open problems, research and implementation directions.

Chapter 6

Mobile De-Duplication

Abstract In this chapter, we show our ongoing and future work, which represents an enhancement of client-based deduplication for popular files in mobile devices, where each file is deduplicated based on the file's structure considering a low-capacity mobile device. We also consider file security and observed performance of encryption algorithms based on systems with different CPU types, such as an Android mobile device (ARM-CPU) and a desktop Linux server (Intel-CPU).

6.1 Large Redundancies in Mobile Devices

Currently, massive files are often created and used in mobile devices. Large amounts of documents and image files are generated and used in mobile devices. In addition, the watching of streaming video is one of the major uses of mobile devices.

We address two issues. The first issue is that large redundancies exist in files of mobile devices. For example, nowadays there is a very popular application for taking pictures of moving objects, which is known as burst shooting mode. In this mode, it is possible to take 30 pictures/s and save good pictures and delete bad ones, but this application may experience large redundancies between similar pictures. Also, video files consist of I-frames that have images and P-frames that hold delta information between I-frames. In scenes where actors talk in the same background, large portions of background become redundant: that is, the I-frames have large redundancies that can be deleted. The second issue is that security becomes critically important in deduplication in mobile devices, and the encryption function should be fast and consume low amounts of energy because of the low energy capacity of mobile devices.

Thus, our approaches are to use structure-aware deduplication for files based on files' structured formats with strong and fast encryption. We choose many different types of files, including documents, emails and image files, in mobile devices. For structure-aware deduplication, we decompose a file into objects and deduplicate objects based on a structure library where the structure formats are defined. For security, the encryption algorithm has different performances and strengths. Generally, stronger encryption is slower due to more computation. Thus, we develop

an idea that for more security-sensitive objects, strong encryption should be used, but for less-security-sensitive objects, weaker encryption can be used for fast performance.

For varying systems with different CPU types, we measured the performance of strong encryption algorithms, like Advanced Encryption Standard (AES) [1], and weaker encryption algorithms, like Blowfish [12], Data Encryption Standard (DES) and 3DES [10], and RC2 [11]. The results revealed that the performance of encryption can be affected by encryption strength as well as the CPU type (Intel or ARM).

6.2 Approaches and Observations

We propose structured-based deduplication using encryption functions based on different levels of security. Thus, in our approaches, we mainly focus on two purposes: efficiently decomposing and reconstructing files in mobile devices, and using fast and strong encryption in the security of decomposed objects. For the first purpose, we apply structure-aware deduplication that has been used for SAFE. We investigate how efficiently it deduplicates image and video files in mobile devices, with a greater focus on the second purpose, security. The issues of security and privacy as they related to deduplication are discussed in many studies [3–5, 9].

6.3 JPEG and MPEG4

JPEG [6] is a popular compression technique for digital photography, and current mobile devices use JPEG as a default image format owing to its small footprint compared to other image files. JPEG uses an efficient compression algorithm such as discrete cosine transform (DCT). We argue that JPEG efficiently reduces redundancies in a single image, and our approach reduces redundancies among similar images. MPEG4 [7] is a popular compression method for audio and video files. For example, streaming files on YouTube are MPEG4 files.

6.4 Evaluation

We show how encryption algorithms are applied depending on the file, system and CPU type. Overall, AES outperforms other compared encryption algorithms on an Intel system in terms of performance. However, for ARM systems like smartphones, Blowfish shows the best performance among the algorithms compared.

6.4.1 Setup

For a Linux system, we install ImageMagick [8] to convert a JPEG file to different types of image files, such as GIF, BMP, TIFF or PNG, which allows us to compare the performance of an encryption algorithm based on different file types. The following codes, called ‘convertImages.sh’, shows the shell script codes to convert image types. ‘convert <source image file> <destination image file>’ is the command to convert file types. Initially we create all types of image files and then measure the running time for the encryption of all files. To run encryption on both Linux and Nexus 7, openssl is used with the command ‘openssl enc <enc> -in imgFile -out outFile -pass pass:password’. <enc> is a list of encryption algorithms including AES256, Blowfish, DES, DES3 and RC2. We use the same image files for Linux and Nexus 7.

```
# convertImages.sh
#!/bin/bash

#####
## constants
#####
IMG_TYPES=( 'jpg' 'gif' 'bmp' 'png' 'tiff' )
IMG_DIR="images"
INPUT_DIR="input"
TMP_FILES="temp.tmp"

NUM_IMG_TYPES=${#IMG_TYPES[@]}

#####
## remove previous files
#####
rm -rf $TMP_FILES

#####
## read jpg files
#####

## get file names
ls -l $INPUT_DIR | awk -f getFileNames.awk >> $TMP_FILES;

## get file numbers
fileNums='ls -l $INPUT_DIR | wc -l '
fileNums='expr $fileNums - 1'      ## exclude a line with list

## read file names in a input directory
count=1
cat $TMP_FILES | while read line
do
    ## exclude empty filename
    if [ -n "$line" ]
    then
```

```

fileName=$line
srcFile=$INPUT_DIR/"$fileName
echo "["$count"/"$fileNums"]_"$srcFile".._converting"

for (( i=0; i<$NUM_IMG_TYPES; i++ ))
do
    destFile=$IMG_DIR/"${IMG_TYPES[ $i ] }"/"${count}".$$
        {IMG_TYPES[ $i ]}

    if [ $i -eq 0 ] ## jpg file
    then
        ## copy jpg file
        cmd="cp_"$srcFile_"_"$destFile
    else
        ## convert other files
        cmd="convert_"$srcFile_"_"$destFile
    fi
    echo $cmd
    $cmd

done
count='expr $count + 1'
fi
done

echo
echo "convert_...._done"

```

6.4.2 Throughput and Running Time per File Type

As shown in Figs. 6.1 and 6.2, we observed that the encryption algorithms showed different performances on each system with different CPU types. As shown in Fig. 6.1, we measured the throughput of the encryption algorithms on a Linux machine with Intel I5 and on a Nexus 7 with ARM for different file types, including audio, document, image, text and email files. We found that in Linux, AES outperformed other encryption algorithms, but in Nexus 7, Blowfish had the highest performance. We found this result interesting because a previous study [2] asserted that Blowfish always had the best performance. The reason for AES's superior performance on the Linux system was that the Intel CPU had hardware-support instruction sets for AES, that is, a call AES New Instruction (AES_NI). In the same vein, Fig. 6.2 shows that AES is the fastest on a Linux system, but Blowfish is the fastest on Nexus 7.

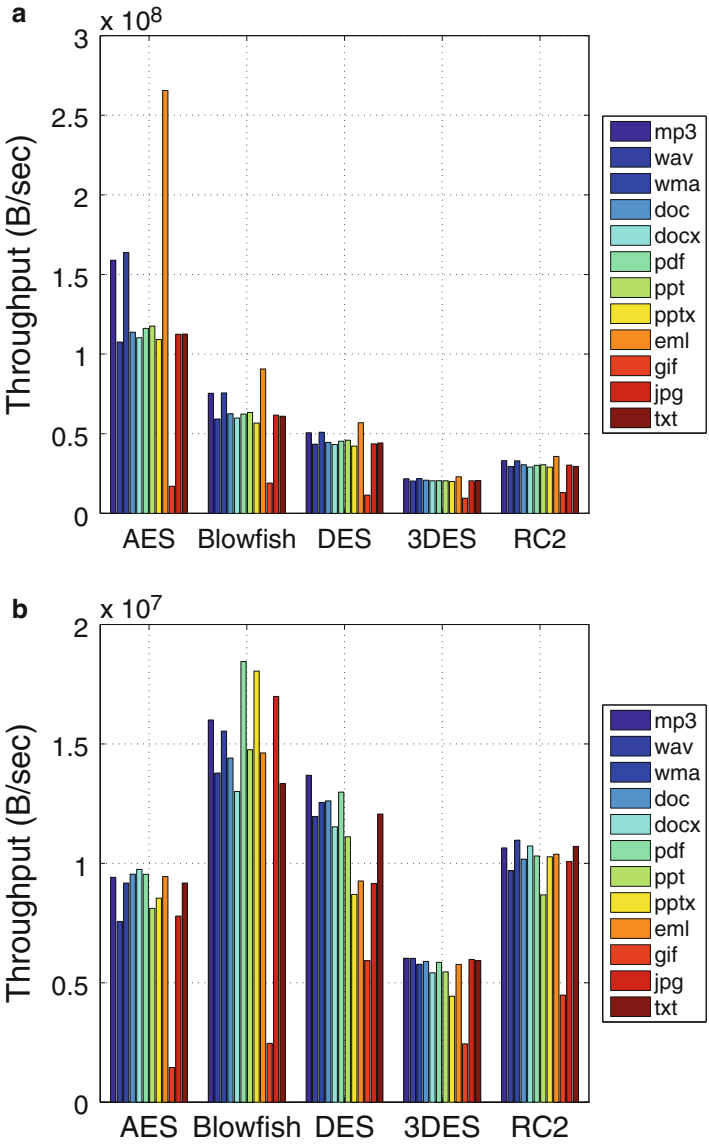


Fig. 6.1 Throughput of encryption algorithms per file type. (a) Linux (Intel I5). (b) Nexus 7 (ARM)

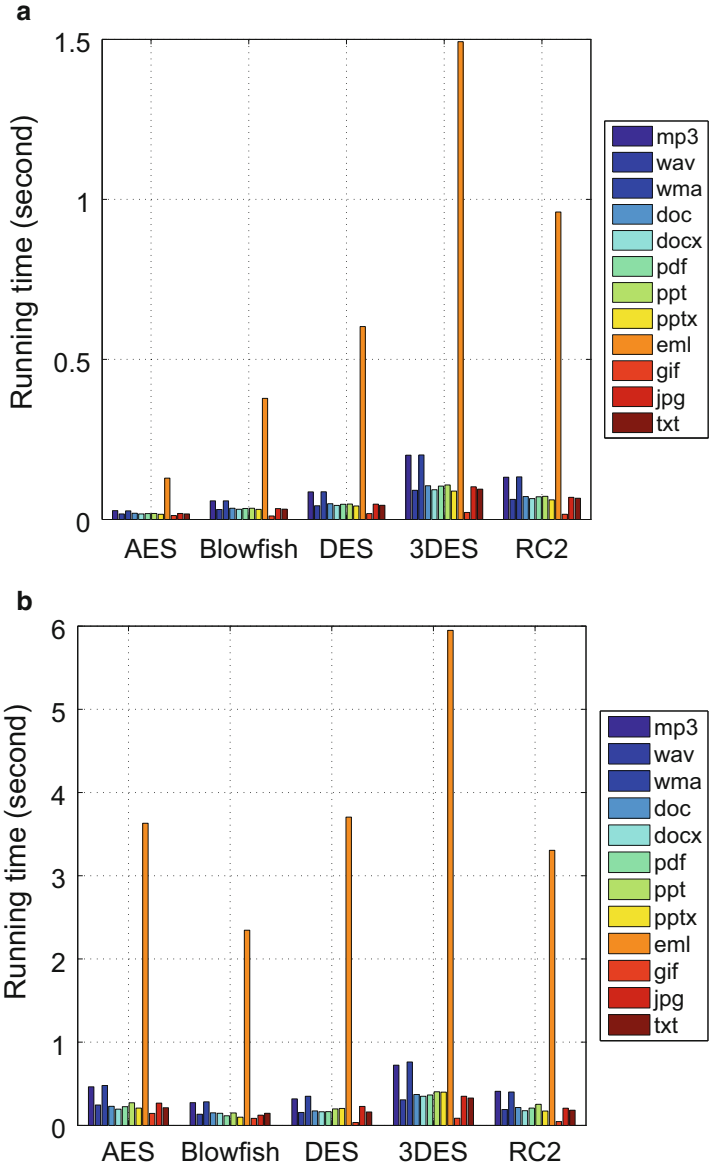


Fig. 6.2 Running time of encryption algorithms per file type. (a) Linux (Intel I5). (b) Nexus 7 (ARM)

6.4.3 *Throughput and Running Time per File Size*

We measured the throughput and processing time of encryption algorithms on Linux and Nexus 7, varying the data size from 4 KB to 1 GB. Overall, on both systems, throughput and processing time increase with the growth in the data size. However, for throughput as shown in Fig. 6.3, there is a threshold size (here, 10 MB) after which the throughput decreases. For processing time as shown in Fig. 6.4, a very small data size (e.g. 4 KB) takes longer than relatively larger data (e.g. 128 KB). However, the processing time from 4 KB to 128 KB decreases for both AES and Blowfish on both systems. These results show the importance of granularity in deduplication, considering the performance of encryption in deduplication. Because 128 KB is equal to 32×4 KB, encrypting a 128 KB object is 32 times faster than encrypting 32 4 KB objects. However, using 128 KB granularity leads to finding fewer redundancies using 4 KB granularity. As a result, we need to select a granularity taking into account the balance between removing redundancies and encryption processing time.

6.5 Summary

In this chapter, we discussed a method of client-based deduplication for popular files on mobile devices, mobile deduplication that removes redundancies from files on mobile devices. Considering the low capacity of mobile devices, we propose to use structure-aware deduplication on files to improve processing time. Also, we observed that the performance of encryption changed depending on the strength level of security as well as on the system and its CPU type. In future work, we will investigate efficient structure-aware deduplication for JPEG [6] and MPEG4 [7] files in terms of storage space savings and processing time overhead.

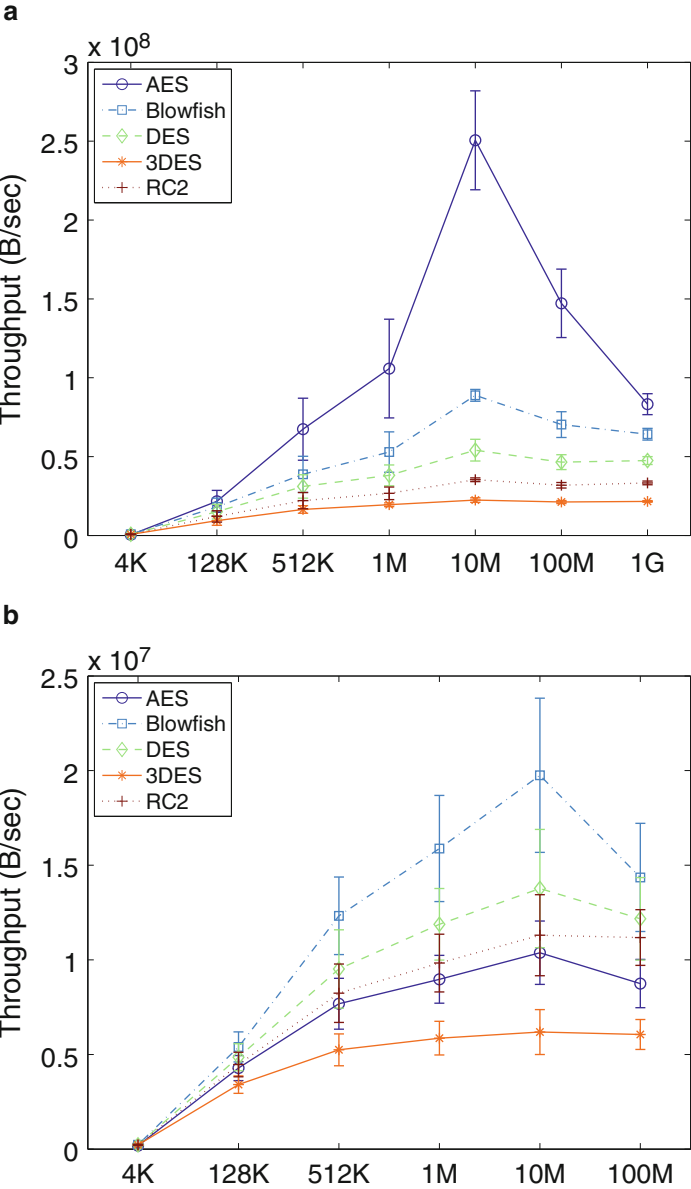


Fig. 6.3 Throughput of encryption algorithms per file size. **(a)** Linux (Intel I5). **(b)** Nexus 7 (ARM)

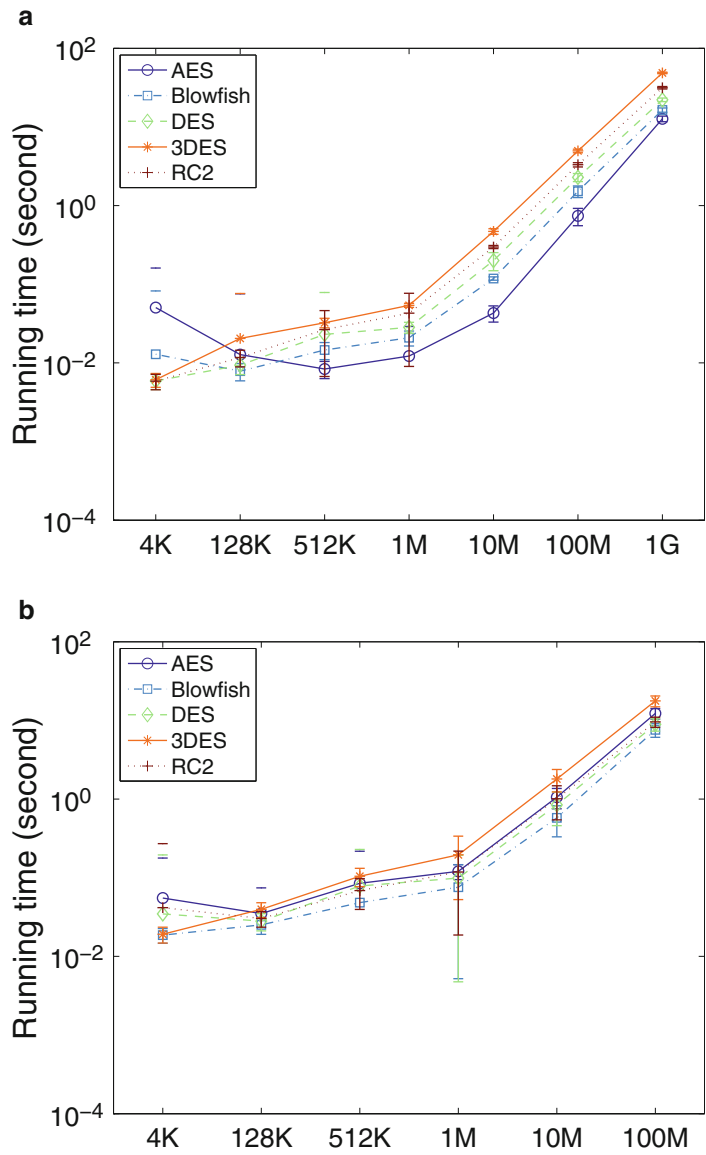


Fig. 6.4 Running time of encryption algorithms per file size. (a) Linux (Intel I5). (b) Nexus 7 (ARM)

References

1. Dworkin, M.J., Barker, E.B., Nechvatal, J.R., Foti, J., Bassham, L.E., Roback, E., Dray, J.F. Jr.: Advanced encryption standard. http://www.nist.gov/manuscript-publication-search.cfm?pub_id=901427 (2001)
2. Elminaam, D.S.A., Kader, H.M.A., Hadhoud, M.M.: Performance evaluation of symmetric encryption algorithms. *Int. J. Comput. Sci. Netw. Secur. (IJCSNS)* **8**(12), 280–286 (2008)
3. Halevi, S., Harnik, D., Pinkas, B., Shulman-Peleg, A.: Proofs of ownership in remote storage systems. In: *Proceedings of the 18th ACM Conference on Computer and Communications Security, CCS '11*, pp. 491–500. ACM, New York (2011)
4. Harnik, D., Pinkas, B., Shulman-Peleg, A.: Side channels in cloud services: deduplication in cloud storage. *IEEE Secur. Priv.* **8**(6), 40–47 (2010)
5. Hu, W., Yang, T., Matthews, J.N.: The good, the bad and the ugly of consumer cloud storage. *ACM SIGOPS Oper. Syst. Rev.* **44**(3), 110–115 (2010)
6. ISO: JPEG, digital compression and coding of continuous-tone still images. http://www.iso.org/iso/catalogue_detail.htm?csnumber=18902 (2011)
7. ISO: MPEG4, Coding of audio-visual objects: Part 12: ISO base media file format. http://www.iso.org/iso/iso_catalogue/catalogue_ics/catalogue_detail_ics.htm?csnumber=38539 (2004)
8. License, A.: ImageMagick. <http://www.imagemagick.org/script/index.php> (2016)
9. Mulazzani, M., Schrittwieser, S., Leithner, M., Huber, M., Weippl, E.: Dark clouds on the horizon: using cloud storage as attack vector and online slack space. In: *Proceedings of the 20th USENIX conference on Security, SEC'11* (2011)
10. National Institute of Standards and Technology (NIST): Data Encryption Standard (DES). <http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf> (1999)
11. Rivest, R.: A description of the RC2(r) encryption algorithm, RFC2268. <https://www.ietf.org/rfc/rfc2268.txt> (1994)
12. Schneier, B.: Description of a new variable-length key, 64-bit block cipher (blowfish). In: *Fast Software Encryption, Cambridge Security Workshop*, pp. 191–204. Springer, London (1994). <http://dl.acm.org/citation.cfm?id=647930.740558>

Chapter 7

Conclusions

Abstract In this chapter, we summarize design and implementation components for efficient deduplication framework on data chains from clients to servers through networks. We point out key points and strength of three components on server side (Hybird Email Deduplication System - HEDS), client side (Structure-Aware File and Email Deduplication for Cloud-based Storage System - SAFE), and network side (Software-Defined Deduplication as as Nework and Storage Service - SoftDance). We end this chapter by describing promising future directions.

In this era of data explosion, huge redundancies exist on storage devices and networks. Existing deduplication solutions, such as storage data deduplication and network redundancy elimination, are not as efficient as they could be at optimizing data movement from clients through networks to servers.

We have designed and proposed an efficient deduplication framework to optimize data chains from clients to servers through networks and to make components for the framework. We developed components such as Hybrid Email Deduplication System (HEDS) on the server side, Structure-Aware File and Email Deduplication for Cloud-based Storage Systems (SAFE) on the client side, and Software-Defined Deduplication as a Network and Storage Service (SoftDance) on the network side for the deduplication framework. HEDS efficiently achieves a trade-off of file-level and block deduplication for email systems. SAFE exploits structure-based granularity rather than using physical chunk granularity, which enables it to perform very fast file-level deduplication and provide the same space savings as block deduplication with low overhead. SoftDance, as an in-network deduplication, chains storage data deduplication and network redundancy elimination functions using a software-defined network (SDN) and achieves storage space savings and network bandwidth savings with low processing time and memory overhead on storage devices and networks. We also presented a mobile deduplication method focusing on popular files such as image and video files on mobile devices.

Further investigations and studies on deduplication are in order, especially in connection with network and system reliability, storage workload and scalability, and efficient accessibility in multi-user cloud environments.

Part V

Appendixes

Appendices

In this part, we show all implementation codes used in the book. In Appendix A, we show codes to compute SHA1 hash keys that are used for indexes to check data redundancies. Codes have two parts; one is the code to compute SHA1 hash keys, the other is the wrapper codes for the former code. Using the latter (wrapper) code gives more flexibility than using the former code. In Appendix B, we show the codes of an index table implementation using an unordered map. An unordered map is a STL library of C++ and a container with a key and value pair. In an index table, the key is an SHA1 key. In Appendix C, we display the codes of a Bloom filter using four hash functions. In Appendix D, we show codes used to acquire fingerprints based on Rabin fingerprinting [1]. This kind of fingerprint is used to determine chunk boundaries to extract variable-size chunks in a file. The fingerprint is 64 bit, which is smaller than SHA1 hash keys, and the computation needed to obtain a fingerprint is faster than that involved in SHA1 hash keys. This is why fingerprinting is used to find chunks.

Appendices E and F display snippet codes to extract variable-size chunks from a file. Two chapters explain chunking core and wrapper classes to find chunk boundaries and retrieve chunks based on those boundaries. In Appendix G, we show sample codes using a libnetfilter_queue library. The codes are used to capture an incoming packet, show the IP and TCP header and change lowercase letters in a payload into capital letters. Changing the payload requires recomputing the packet checksum and putting the checksum into the packet header so that the packet is received by the receiver or the next forwarder. If the checksum is not recomputed, the packet is simply rejected at the receiver or the next forwarder because of the incorrect checksum with a changed payload.

Appendix A

Index Creation with SHA1

This chapter presents codes to compute SHA1 hash keys. The codes are written in C++. Codes have two parts: one part is the code to compute SHA1 hash keys, the other is the wrapper codes for the former code. The latter (wrapper) codes give more flexibility than the former code. `sha1Wrapper.cc` includes codes for testing. `sha1.h` and `sha1.cc` [2] were created by Paul Bakker.

A.1 `sha1Wrapper.h`

```
#ifndef SHA1_WRAPPER_H
#define SHA1_WRAPPER_H

#include <iostream>
#include <string>
#include "sha1.h"
using namespace std;

#define HASH_SIZE 20

class Sha1Wrapper {
public:

    //
    // get SHA1 hash key of a file
    //
    // filePath : a file path
    // return
    // - 20 bytes hash string
    //
    string getHashKeyOfFile( string filePath );

    //

```

```

// get SHA1 hash key of a byte stream
//
// data : a byte stream
// return
// - 20 bytes hash string
//
string getHashKey(string data);

```

```
private:
```

```
};
```

```
#endif
```

A.2 sha1Wrapper.cc

```
#include "sha1Wrapper.h"
```

```
string
Sha1Wrapper::getHashKeyOfFile(string filePath) {
```

```

    string ret;
    unsigned char output[HASH_SIZE];
    char buf[10];
    int i;

    sha1_file((char*)filePath.c_str(), output);

```

```

    for (i=0; i < HASH_SIZE; i++)
    {
        sprintf(buf, "%02x", output[i]);
        ret += buf;
    }
    //cout << ret << endl;
    return ret;
}

```

```
string
Sha1Wrapper::getHashKey(string data) {
```

```

    string ret;
    unsigned char output[HASH_SIZE];
    char buf[10];
    int i;

    sha1((unsigned char*)data.c_str(), data.length(), output);

```

```

    for (i=0; i < HASH_SIZE; i++)
    {

```

```

        sprintf(buf, "%02x", output[i]);
        ret += buf;
    }
    //cout << ret << endl;
    return ret;
}

#ifdef SHAIWRAPPER_TEST

int main() {

    Sha1Wrapper obj;
    string filePath = "hello.dat";
    //string filePath = "Demo_NAl_spring2012 (2).docx";
    string data = "hello_danny_how_are_you_??_";

    string hashKey;

    // get hash key of a file
    hashKey = obj.getHashKeyOfFile(filePath);
    cout << "hashkey_of_" << filePath << "_:" << hashKey << endl;
    cout << endl;

    // get hash key of data
    cout << data << endl;
    hashKey = obj.getHashKey(data);
    cout << "hashkey_of_data_:" << hashKey << endl;

    return 0;
}

#endif

```

A.3 sha1.h

```

/**
 * \file sha1.h
 *
 * Based on XySSL: Copyright (C) 2006–2008 Christophe Devine
 *
 * Copyright (C) 2009 Paul Bakker <polarssl_maintainer
 * at polarssl dot org>
 *
 * This program is free software; you can redistribute it and/or
 * modify
 * it under the terms of the GNU General Public License as
 * published by
 * the Free Software Foundation; either version 2 of the
 * License, or
 * (at your option) any later version.
 *

```

```

* This program is distributed in the hope that it will be
* useful,
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
* GNU General Public License for more details.
*
* You should have received a copy of the GNU General Public
* License along
* with this program; if not, write to the Free Software
* Foundation, Inc.,
* 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.
*/
#ifndef SHA1_H
#define SHA1_H

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// added by Daehee Kim — used at sha1.c —————
// #define DEBUG 0 // for debugging

#define SHA1_SIZE 20 // SHA1 hash full size : bytes
#define LSB 1 // to cut hash value from LSB
#define MSB 2 // to cut hash value from MSB
#define BITS_PER_BYTE 8 // 8 bits per 1 byte

#define YES 1 // for comparing two hashes
#define NO 0 // for comparing two hashes

void show_hash(unsigned char *hash, int hash_size,
               int partial_size, int flag);
int same_hash(unsigned char *hash1, unsigned char *hash2,
              int hash_size, int partial_size, int flag);

// added by Daehee Kim — used at sha1.c —————

/**
 * \brief SHA-1 context structure
 */
typedef struct
{
    unsigned long total[2]; /*!< number of bytes processed */
    unsigned long state[5]; /*!< intermediate digest state */
    unsigned char buffer[64]; /*!< data block being processed */

    unsigned char ipad[64]; /*!< HMAC: inner padding */
    unsigned char opad[64]; /*!< HMAC: outer padding */
}
sha1_context;

#ifdef __cplusplus
extern "C" {
#endif

```

```

/**
 * \brief          SHA-1 context setup
 *
 * \param ctx      context to be initialized
 */
void sha1_starts( sha1_context *ctx );

/**
 * \brief          SHA-1 process buffer
 *
 * \param ctx      SHA-1 context
 * \param input    buffer holding the data
 * \param ilen     length of the input data
 */
void sha1_update( sha1_context *ctx, unsigned char *input,
                  int ilen );

/**
 * \brief          SHA-1 final digest
 *
 * \param ctx      SHA-1 context
 * \param output    SHA-1 checksum result
 */
void sha1_finish( sha1_context *ctx, unsigned char output[20] );

/**
 * \brief          Output = SHA-1( input buffer )
 *
 * \param input    buffer holding the data
 * \param ilen     length of the input data
 * \param output    SHA-1 checksum result
 */
void sha1( unsigned char *input, int ilen, unsigned char
           output[20] );

/**
 * \brief          Output = SHA-1( file contents )
 *
 * \param path     input file name
 * \param output    SHA-1 checksum result
 *
 * \return         0 if successful, 1 if fopen failed,
 *                or 2 if fread failed
 */
int sha1_file( char *path, unsigned char output[20] );

/**
 * \brief          SHA-1 HMAC context setup
 *
 * \param ctx      HMAC context to be initialized
 * \param key      HMAC secret key
 * \param keylen   length of the HMAC key

```

```

*/
void shal_hmac_starts( shal_context *ctx , unsigned char *key ,
                      int keylen );
/**
 * \brief          SHA-1 HMAC process buffer
 *
 * \param ctx      HMAC context
 * \param input    buffer holding the data
 * \param ilen     length of the input data
 */
void shal_hmac_update( shal_context *ctx , unsigned char *input ,
                      int ilen );

/**
 * \brief          SHA-1 HMAC final digest
 *
 * \param ctx      HMAC context
 * \param output   SHA-1 HMAC checksum result
 */
void shal_hmac_finish( shal_context *ctx , unsigned char
                      output[20] );

/**
 * \brief          Output = HMAC-SHA-1( hmac key, input buffer )
 *
 * \param key      HMAC secret key
 * \param keylen   length of the HMAC key
 * \param input    buffer holding the data
 * \param ilen     length of the input data
 * \param output   HMAC-SHA-1 result
 */
void shal_hmac( unsigned char *key, int keylen ,
                unsigned char *input, int ilen ,
                unsigned char output[20] );

/**
 * \brief          Checkup routine
 *
 * \return         0 if successful , or 1 if the test failed
 */

int shal_self_test( int verbose );

#ifdef __cplusplus
}
#endif
#endif /* sha1.h */

```

A.4 sha1.cc

```

/*
 * FIPS-180-1 compliant SHA-1 implementation
 *
 * Based on XySSL: Copyright (C) 2006-2008 Christophe Devine
 *
 * Copyright (C) 2009 Paul Bakker <polarssl_maintainer
 * at polarssl dot org>
 *
 * This program is free software; you can redistribute it and/or
 * modify
 * it under the terms of the GNU General Public License as
 * published by
 * the Free Software Foundation; either version 2 of the
 * License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be
 * useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public
 * License along
 * with this program; if not, write to the Free Software
 * Foundation, Inc.,
 * 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.
 */
/*
 * The SHA-1 standard was published by NIST in 1993.
 *
 * http://www.itl.nist.gov/fipspubs/fip180-1.htm
 */

#include "sha1.h"

/*
 * 32-bit integer manipulation macros (big endian)
 */
#ifndef GET_ULONG_BE
#define GET_ULONG_BE(n,b,i) \
{ \
    (n) = ( (unsigned long) (b)[(i)    ] << 24 ) \
    | ( (unsigned long) (b)[(i) + 1] << 16 ) \
    | ( (unsigned long) (b)[(i) + 2] <<  8 ) \
    | ( (unsigned long) (b)[(i) + 3]      ); \
}
#endif

#ifndef PUT_ULONG_BE

```

```

#define PUT_ULONG_BE(n,b,i)                                \
{                                                            \
    (b)[(i)      ] = (unsigned char) ( (n) >> 24 );      \
    (b)[(i) + 1] = (unsigned char) ( (n) >> 16 );        \
    (b)[(i) + 2] = (unsigned char) ( (n) >> 8 );         \
    (b)[(i) + 3] = (unsigned char) ( (n)                ); \
}
#endif

/*
 * SHA-1 context setup
 */
void shal_starts( shal_context *ctx )
{
    ctx->total[0] = 0;
    ctx->total[1] = 0;

    ctx->state[0] = 0x67452301;
    ctx->state[1] = 0xEFCDAB89;
    ctx->state[2] = 0x98BADCFE;
    ctx->state[3] = 0x10325476;
    ctx->state[4] = 0xC3D2E1F0;
}

static void shal_process( shal_context *ctx, unsigned char
                          data[64] )
{
    unsigned long temp, W[16], A, B, C, D, E;

    GET_ULONG_BE( W[ 0], data,  0 );
    GET_ULONG_BE( W[ 1], data,  4 );
    GET_ULONG_BE( W[ 2], data,  8 );
    GET_ULONG_BE( W[ 3], data, 12 );
    GET_ULONG_BE( W[ 4], data, 16 );
    GET_ULONG_BE( W[ 5], data, 20 );
    GET_ULONG_BE( W[ 6], data, 24 );
    GET_ULONG_BE( W[ 7], data, 28 );
    GET_ULONG_BE( W[ 8], data, 32 );
    GET_ULONG_BE( W[ 9], data, 36 );
    GET_ULONG_BE( W[10], data, 40 );
    GET_ULONG_BE( W[11], data, 44 );
    GET_ULONG_BE( W[12], data, 48 );
    GET_ULONG_BE( W[13], data, 52 );
    GET_ULONG_BE( W[14], data, 56 );
    GET_ULONG_BE( W[15], data, 60 );

#define S(x,n) ((x << n) | ((x & 0xFFFFFFFF) >> (32 - n)))

#define R(t)                                              \
(                                                         \
    temp = W[(t - 3) & 0x0F] ^ W[(t - 8) & 0x0F] ^      \
          W[(t - 14) & 0x0F] ^ W[ t          & 0x0F],   \
    ( W[t & 0x0F] = S(temp,1) )                          \
)

```

```

#define P(a,b,c,d,e,x)                                     \
{                                                            \
    e += S(a,5) + F(b,c,d) + K + x; b = S(b,30);          \
}

    A = ctx->state[0];
    B = ctx->state[1];
    C = ctx->state[2];
    D = ctx->state[3];
    E = ctx->state[4];

#define F(x,y,z) (z ^ (x & (y ^ z)))
#define K 0x5A827999

    P( A, B, C, D, E, W[0] );
    P( E, A, B, C, D, W[1] );
    P( D, E, A, B, C, W[2] );
    P( C, D, E, A, B, W[3] );
    P( B, C, D, E, A, W[4] );
    P( A, B, C, D, E, W[5] );
    P( E, A, B, C, D, W[6] );
    P( D, E, A, B, C, W[7] );
    P( C, D, E, A, B, W[8] );
    P( B, C, D, E, A, W[9] );
    P( A, B, C, D, E, W[10] );
    P( E, A, B, C, D, W[11] );
    P( D, E, A, B, C, W[12] );
    P( C, D, E, A, B, W[13] );
    P( B, C, D, E, A, W[14] );
    P( A, B, C, D, E, W[15] );
    P( E, A, B, C, D, R(16) );
    P( D, E, A, B, C, R(17) );
    P( C, D, E, A, B, R(18) );
    P( B, C, D, E, A, R(19) );

#undef K
#undef F

#define F(x,y,z) (x ^ y ^ z)
#define K 0x6ED9EBA1

    P( A, B, C, D, E, R(20) );
    P( E, A, B, C, D, R(21) );
    P( D, E, A, B, C, R(22) );
    P( C, D, E, A, B, R(23) );
    P( B, C, D, E, A, R(24) );
    P( A, B, C, D, E, R(25) );
    P( E, A, B, C, D, R(26) );
    P( D, E, A, B, C, R(27) );
    P( C, D, E, A, B, R(28) );
    P( B, C, D, E, A, R(29) );
    P( A, B, C, D, E, R(30) );
    P( E, A, B, C, D, R(31) );

```

```

P( D, E, A, B, C, R(32) );
P( C, D, E, A, B, R(33) );
P( B, C, D, E, A, R(34) );
P( A, B, C, D, E, R(35) );
P( E, A, B, C, D, R(36) );
P( D, E, A, B, C, R(37) );
P( C, D, E, A, B, R(38) );
P( B, C, D, E, A, R(39) );

#undef K
#undef F

#define F(x,y,z) ((x & y) | (z & (x | y)))
#define K 0x8F1BBCDC

P( A, B, C, D, E, R(40) );
P( E, A, B, C, D, R(41) );
P( D, E, A, B, C, R(42) );
P( C, D, E, A, B, R(43) );
P( B, C, D, E, A, R(44) );
P( A, B, C, D, E, R(45) );
P( E, A, B, C, D, R(46) );
P( D, E, A, B, C, R(47) );
P( C, D, E, A, B, R(48) );
P( B, C, D, E, A, R(49) );
P( A, B, C, D, E, R(50) );
P( E, A, B, C, D, R(51) );
P( D, E, A, B, C, R(52) );
P( C, D, E, A, B, R(53) );
P( B, C, D, E, A, R(54) );
P( A, B, C, D, E, R(55) );
P( E, A, B, C, D, R(56) );
P( D, E, A, B, C, R(57) );
P( C, D, E, A, B, R(58) );
P( B, C, D, E, A, R(59) );

#undef K
#undef F

#define F(x,y,z) (x ^ y ^ z)
#define K 0xCA62C1D6

P( A, B, C, D, E, R(60) );
P( E, A, B, C, D, R(61) );
P( D, E, A, B, C, R(62) );
P( C, D, E, A, B, R(63) );
P( B, C, D, E, A, R(64) );
P( A, B, C, D, E, R(65) );
P( E, A, B, C, D, R(66) );
P( D, E, A, B, C, R(67) );
P( C, D, E, A, B, R(68) );
P( B, C, D, E, A, R(69) );
P( A, B, C, D, E, R(70) );
P( E, A, B, C, D, R(71) );

```

```

    P( D, E, A, B, C, R(72) );
    P( C, D, E, A, B, R(73) );
    P( B, C, D, E, A, R(74) );
    P( A, B, C, D, E, R(75) );
    P( E, A, B, C, D, R(76) );
    P( D, E, A, B, C, R(77) );
    P( C, D, E, A, B, R(78) );
    P( B, C, D, E, A, R(79) );

#undef K
#undef F

    ctx->state[0] += A;
    ctx->state[1] += B;
    ctx->state[2] += C;
    ctx->state[3] += D;
    ctx->state[4] += E;
}

/*
 * SHA-1 process buffer
 */
void sha1_update( sha1_context *ctx, unsigned char *input,
                  int ilen )
{
    int fill;
    unsigned long left;

    if( ilen <= 0 )
        return;

    left = ctx->total[0] & 0x3F;
    fill = 64 - left;

    ctx->total[0] += ilen;
    ctx->total[0] &= 0xFFFFFFFF;

    if( ctx->total[0] < (unsigned long) ilen )
        ctx->total[1]++;

    if( left && ilen >= fill )
    {
        memcpy( (void *) (ctx->buffer + left),
                (void *) input, fill );
        sha1_process( ctx, ctx->buffer );
        input += fill;
        ilen -= fill;
        left = 0;
    }

    while( ilen >= 64 )
    {
        sha1_process( ctx, input );
        input += 64;
    }
}

```

```

        ilen -= 64;
    }

    if( ilen > 0 )
    {
        memcpy( (void *) (ctx->buffer + left),
                (void *) input, ilen );
    }
}

static const unsigned char sha1_padding[64] =
{
    0x80, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
};

/*
 * SHA-1 final digest
 */
void sha1_finish( sha1_context *ctx, unsigned char output[20] )
{
    unsigned long last, padn;
    unsigned long high, low;
    unsigned char msglen[8];

    high = ( ctx->total[0] >> 29 )
        | ( ctx->total[1] << 3 );
    low  = ( ctx->total[0] << 3 );

    PUT_ULONG_BE( high, msglen, 0 );
    PUT_ULONG_BE( low,  msglen, 4 );

    last = ctx->total[0] & 0x3F;
    padn = ( last < 56 ) ? ( 56 - last ) : ( 120 - last );

    sha1_update( ctx, (unsigned char *) sha1_padding, padn );
    sha1_update( ctx, msglen, 8 );

    PUT_ULONG_BE( ctx->state[0], output, 0 );
    PUT_ULONG_BE( ctx->state[1], output, 4 );
    PUT_ULONG_BE( ctx->state[2], output, 8 );
    PUT_ULONG_BE( ctx->state[3], output, 12 );
    PUT_ULONG_BE( ctx->state[4], output, 16 );
}

/*
 * output = SHA-1( input buffer )
 */
void sha1( unsigned char *input, int ilen, unsigned char
           output[20] )
{
    sha1_context ctx;

```

```

    sha1_starts( &ctx );
    sha1_update( &ctx, input, ilen );
    sha1_finish( &ctx, output );

    memset( &ctx, 0, sizeof( sha1_context ) );
}

/*
 * output = SHA-1( file contents )
 */
int sha1_file( char *path, unsigned char output[20] )
{
    FILE *f;
    size_t n;
    sha1_context ctx;
    unsigned char buf[1024];

    if( ( f = fopen( path, "rb" ) ) == NULL )
        return( 1 );

    sha1_starts( &ctx );

    while( ( n = fread( buf, 1, sizeof( buf ), f ) ) > 0 )
        sha1_update( &ctx, buf, (int) n );

    sha1_finish( &ctx, output );

    memset( &ctx, 0, sizeof( sha1_context ) );

    if( ferror( f ) != 0 )
    {
        fclose( f );
        return( 2 );
    }

    fclose( f );
    return( 0 );
}

/*
 * SHA-1 HMAC context setup
 */
void sha1_hmac_starts( sha1_context *ctx, unsigned char *key,
                      int keylen )
{
    int i;
    unsigned char sum[20];

    if( keylen > 64 )
    {
        sha1( key, keylen, sum );
        keylen = 20;
        key = sum;
    }
}

```

```

    }

    memset( ctx->ipad, 0x36, 64 );
    memset( ctx->opad, 0x5C, 64 );

    for( i = 0; i < keylen; i++ )
    {
        ctx->ipad[i] = (unsigned char)( ctx->ipad[i] ^ key[i] );
        ctx->opad[i] = (unsigned char)( ctx->opad[i] ^ key[i] );
    }

    sha1_starts( ctx );
    sha1_update( ctx, ctx->ipad, 64 );

    memset( sum, 0, sizeof( sum ) );
}

/*
 * SHA-1 HMAC process buffer
 */
void sha1_hmac_update( sha1_context *ctx, unsigned char *input,
                      int ilen )
{
    sha1_update( ctx, input, ilen );
}

/*
 * SHA-1 HMAC final digest
 */
void sha1_hmac_finish( sha1_context *ctx, unsigned char output
                      [20] )
{
    unsigned char tmpbuf[20];

    sha1_finish( ctx, tmpbuf );
    sha1_starts( ctx );
    sha1_update( ctx, ctx->opad, 64 );
    sha1_update( ctx, tmpbuf, 20 );
    sha1_finish( ctx, output );

    memset( tmpbuf, 0, sizeof( tmpbuf ) );
}

/*
 * output = HMAC-SHA-1( hmac key, input buffer )
 */
void sha1_hmac( unsigned char *key, int keylen,
                unsigned char *input, int ilen,
                unsigned char output[20] )
{
    sha1_context ctx;

    sha1_hmac_starts( &ctx, key, keylen );
    sha1_hmac_update( &ctx, input, ilen );

```

```

    sha1_hmac_finish( &ctx , output );

    memset( &ctx , 0, sizeof( sha1_context ) );
}

//-----
// Input parameter : two hash values
// Output parameter : same or not
// —> if same : return 1
// —> otherwise : return 0
//
//-----
// hash : full hash value
// hash_size : size of full hash value
// partial_size : size to be shown
// flag : LSB or MSB
//-----
int same_hash(unsigned char *hash1, unsigned char *hash2,
              int hash_size, int partial_size, int flag)
{
    int i;

    // STEP1. check size parameter based on "hash full size"
    if (partial_size > hash_size) {
        printf("error: same_hash(): size is larger than hash_size\n");
        exit(1);
    }

    // STEP2.
    // Later, I might use bit-operations
    // for MSB —> shift right, for LSB —> & MASK
    switch (flag) {
        case LSB :
            for (i=(hash_size - partial_size) ; i < hash_size; i++)
                if (hash1[i] != hash2[i])
                    return NO;
            break;
        case MSB :
            for (i=0 ; i < partial_size; i++)
                if (hash1[i] != hash2[i])
                    return NO;
            break;
        default :
            printf("error: show_ neither_LSB_nor_MSB\n");
            exit(1);
    }

    return YES;
}

//-----
// Input parameter : two hash values
// Output parameter : same or not

```

```

// —> if same : return 1
// —> otherwise : return 0
//
// String compare
//_____
/*
int same_hash(unsigned char hash1[20], unsigned char hash2[20])
{
    int i;

    for (i=0; i < 20; i++) {
        if (hash1[i] != hash2[i])
            return 0;
    }

    return 1;
}
*/

//_____
// Show hash value based on size and flag
//
// e.g. suppose full hash value is "hash[0] ..... hash[19]"
//      that is, 20 bytes hash value,
//
//      "show(hash, 20, 3, LSB)" shows
//      <hash[17] hash[18] hash[19]>
//
//      "show(hash, 20, 3, MSB)" shows
//      <hash[0], hash[1], hash[3]>
//
//      if you want to show full hash values,
//      "show(hash, 20, 20, LSB)" or "show(hash, 20, 20, MSB)"
//      can be called.
//
//_____
// hash : full hash value
// hash_size : size of full hash value
// partial_size : size to be shown
// flag : LSB or MSB
void show_hash(unsigned char *hash, int hash_size,
               int partial_size, int flag) {

    int i;

    // STEP1. check size parameter based on "hash full size"
    if (partial_size > hash_size) {
        printf("error: show_size is larger than hash_size\n");
        exit(1);
    }

    // STEP2.
    // Later, I might use bit-operations
    // for MSB —> shift right, for LSB —> & MASK

```

```
switch (flag) {  
    case LSB :  
        for (i=(hash_size - partial_size) ; i < hash_size; i++)  
            printf ("%x_", hash[i]);  
        break;  
    case MSB :  
        for (i=0 ; i < partial_size; i++)  
            printf ("%x_", hash[i]);  
        break;  
    default:  
        printf("error: show neither_LSB_nor_MSB\n");  
        exit(1);  
}  
printf("\n");  
}  
  
/*  
 * FIPS-180-1 test vectors  
 */  
static unsigned char sha1_test_buf[3][57] =  
{  
    { "abc" },  
    { "abcdefghijklmnopqrstuvwxyz0123456789abcdefghijklmnopq"  
      },  
    { "" }  
};  
  
static const int sha1_test_buflen[3] =  
{  
    3, 56, 1000  
};  
  
static const unsigned char sha1_test_sum[3][20] =  
{  
    { 0xA9, 0x99, 0x3E, 0x36, 0x47, 0x06, 0x81, 0x6A, 0xBA, 0x3E,  
      0x25, 0x71, 0x78, 0x50, 0xC2, 0x6C, 0x9C, 0xD0, 0xD8, 0x9D,  
      },  
    { 0x84, 0x98, 0x3E, 0x44, 0x1C, 0x3B, 0xD2, 0x6E, 0xBA, 0xAE,  
      0x4A, 0xA1, 0xF9, 0x51, 0x29, 0xE5, 0xE5, 0x46, 0x70, 0xF1,  
      },  
    { 0x34, 0xAA, 0x97, 0x3C, 0xD4, 0xC4, 0xDA, 0xA4, 0xF6, 0x1E,  
      0xEB, 0x2B, 0xDB, 0xAD, 0x27, 0x31, 0x65, 0x34, 0x01, 0x6F,  
      }  
};  
  
/*  
 * RFC 2202 test vectors  
 */  
static unsigned char sha1_hmac_test_key[7][26] =  
{  
    { "\x0B\x0B\x0B\x0B\x0B\x0B\x0B\x0B\x0B\x0B\x0B\x0B\x0B"  
      "\x0B\x0B"  
      },  
    { "\x0B\x0B\x0B\x0B" },  
    { "Jefe" },
```

```

    { "\xAA\xAA\xAA\xAA\xAA\xAA\xAA\xAA\xAA\xAA\xAA\xAA"
    { "\xAA\xAA\xAA" }
    { "\xAA\xAA\xAA\xAA" },
    { "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0A\x0B\x0C\x0D\x0E"
    { "\x0F\x10" }
    { "\x11\x12\x13\x14\x15\x16\x17\x18\x19" },
    { "\x0C\x0C\x0C\x0C\x0C\x0C\x0C\x0C\x0C\x0C\x0C\x0C"
    { "\x0C\x0C\x0C" }
    { "\x0C\x0C\x0C\x0C" },
    { "" }, /* 0xAA 80 times */
    { "" }
};

static const int sha1_hmac_test_keylen[7] =
{
    20, 4, 20, 25, 20, 80, 80
};

static unsigned char sha1_hmac_test_buf[7][74] =
{
    { "Hi_There" },
    { "what_do_ya_want_for_nothing?" },
    { "\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD"
      "\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD"
      "\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD"
      "\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD"
      "\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD\xDD" },
    { "\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD"
      "\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD"
      "\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD"
      "\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD"
      "\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD\xCD" },
    { "Test_With_Truncation" },
    { "Test_Using_Larger_Than_Block-Size_Key_and_Hash_Key_First" },
    { "Test_Using_Larger_Than_Block-Size_Key_and_Larger"
      "_Than_One_Block-Size_Data" }
};

static const int sha1_hmac_test_buflen[7] =
{
    8, 28, 50, 50, 20, 54, 73
};

static const unsigned char sha1_hmac_test_sum[7][20] =
{
    { 0xB6, 0x17, 0x31, 0x86, 0x55, 0x05, 0x72, 0x64, 0xE2, 0x8B,
      0xC0, 0xB6, 0xFB, 0x37, 0x8C, 0x8E, 0xF1, 0x46, 0xBE, 0x00
    },
    { 0xEF, 0xFC, 0xDF, 0x6A, 0xE5, 0xEB, 0x2F, 0xA2, 0xD2, 0x74,
      0x16, 0xD5, 0xF1, 0x84, 0xDF, 0x9C, 0x25, 0x9A, 0x7C, 0x79
    },
    { 0x12, 0x5D, 0x73, 0x42, 0xB9, 0xAC, 0x11, 0xCD, 0x91, 0xA3,
      0x9A, 0xF4, 0x8A, 0xA1, 0x7B, 0x4F, 0x63, 0xF1, 0x75, 0xD3
    },

```

```

    { 0x4C, 0x90, 0x07, 0xF4, 0x02, 0x62, 0x50, 0xC6, 0xBC, 0x84,
      0x14, 0xF9, 0xBF, 0x50, 0xC8, 0x6C, 0x2D, 0x72, 0x35, 0xDA
    },
    { 0x4C, 0x1A, 0x03, 0x42, 0x4B, 0x55, 0xE0, 0x7F, 0xE7, 0xF2,
      0x7B, 0xE1 },
    { 0xAA, 0x4A, 0xE5, 0xE1, 0x52, 0x72, 0xD0, 0x0E, 0x95, 0x70,
      0x56, 0x37, 0xCE, 0x8A, 0x3B, 0x55, 0xED, 0x40, 0x21, 0x12
    },
    { 0xE8, 0xE9, 0x9D, 0x0F, 0x45, 0x23, 0x7D, 0x78, 0x6D, 0x6B,
      0xBA, 0xA7, 0x96, 0x5C, 0x78, 0x08, 0xBB, 0xFF, 0x1A, 0x91
    }
};

/*
 * Checkup routine
 */
int sha1_self_test( int verbose )
{
    int i, j, buflen;
    unsigned char buf[1024];
    unsigned char shalsum[20];
    sha1_context ctx;

    // #####
    // SHA-1 TEST - BEGIN
    // #####
    for( i = 0; i < 3; i++ )
    {
        if( verbose != 0 )
            printf( "SHA-1 test #%d: ", i + 1 );

        sha1_starts( &ctx );

        if( i == 2 )
        {
            memset( buf, 'a', buflen = 1000 );

            for( j = 0; j < 1000; j++ )
                sha1_update( &ctx, buf, buflen );
        }
        else
            sha1_update( &ctx, sha1_test_buf[i],
                          sha1_test_buflen[i] );

        sha1_finish( &ctx, shalsum );

        // #####
        // print sha1 value (hexa decimal) - begin
        // #####
        int k;
        printf( "%d: ", i );
        for (k=0; k < 20; k++) {
            printf( "%x", shalsum[k] );
            // printf( "(%d)%x ", k, shalsum[k] );

```

```

    }
    printf( "\n: " );
    // printf( "\n" );
    // #####
    // print sha1 value (hexa decimal) - end
    // #####

    if( memcmp( shalsum, shal_test_sum[i], 20 ) != 0 )
    {
        if( verbose != 0 )
            printf( "failed\n" );

        return( 1 );
    }

    if( verbose != 0 )
        printf( "passed\n" );
}

// #####
// SHA-1 TEST - END
// #####

// #####
// HMAC-SHA-1 TEST - BEGIN
// #####

if( verbose != 0 )
    printf( "\n" );

for( i = 0; i < 7; i++ )
{
    if( verbose != 0 )
        printf( "HMAC-SHA-1_test_#%d:", i + 1 );

    if( i == 5 || i == 6 )
    {
        memset( buf, '\xAA', buflen = 80 );
        sha1_hmac_starts( &ctx, buf, buflen );
    }
    else
        sha1_hmac_starts( &ctx, sha1_hmac_test_key[i],
                          sha1_hmac_test_keylen[i] );

    sha1_hmac_update( &ctx, sha1_hmac_test_buf[i],
                      sha1_hmac_test_buflen[i] );

    sha1_hmac_finish( &ctx, shalsum );

    buflen = ( i == 4 ) ? 12 : 20;

    if( memcmp( shalsum, sha1_hmac_test_sum[i],
                buflen ) != 0 )

```

```
{
    if( verbose != 0 )
        printf( "failed\n" );

    return( 1 );
}

if( verbose != 0 )
    printf( "passed\n" );
}

// #####
// HMAC-SHA-1 TEST - END
// #####

if( verbose != 0 )
    printf( "\n" );

return( 0 );
}
```

Appendix B

Index Table Implementation using Unordered Map

In this chapter, we present the codes for an index table implementation using unordered map. Unordered map is a STL library of C++ and a container with a pair consisting of a key and a value. In the index table, the key is a sha1 key. The value can be null or some metadata. cacheInterface.h shows function definitions, and cache.h implements the function definitions. Though the key data type is a string, the data type can vary. Thus, we use a template for key and value. cache.cc contains codes to test the index table.

B.1 cacheInterface.h

```
//-----  
//  
// Cache interface  
//  
// coded by Daehee (Danny) Kim  
// 7/20/2015  
//  
//-----  
#ifndef CACHE_INTERFACE  
#define CACHE_INTERFACE  
  
template <typename KeyT, typename ValueT>  
class CacheInterface {  
public :  
  
    //  
    // get a value for a key  
    //  
    virtual ValueT get(KeyT key) = 0;
```

```
//  
// set a key and a value  
//  
virtual void set(KeyT key, ValueT value) = 0;  
  
//  
// check if there are no entries  
//  
virtual bool empty() = 0;  
  
//  
// get number of entries  
//  
virtual int size() = 0;  
  
//  
// get size of all entries  
//  
virtual int sizeOfAllEntries() = 0;  
  
//  
// get size of all entries (double value)  
//  
virtual double sizeOfAllEntriesDouble() = 0;  
  
//  
// get size of keys of all entries  
//  
virtual int sizeOfKeys() = 0;  
  
//  
// get size of keys of all entries (double value)  
//  
virtual double sizeOfKeysDouble() = 0;  
  
//  
// get size of values of all entries  
//  
virtual int sizeOfValues() = 0;  
  
//  
// check if an entry with key exists  
//  
virtual bool exist(KeyT key) = 0;  
  
//  
// show all entries  
//  
virtual void showAll() = 0;  
  
//  
// remove all entries  
//
```

```

    virtual void removeAll() = 0;

private:
};

#endif

```

B.2 cache.h

```

//-----
//
// Cache implementation
// with unordered_map
//
// coded by Daehee (Danny) Kim
// 7/20/2015
//
//-----
#ifndef UNORDERED_MAP_CACHE_H
#define UNORDERED_MAP_CACHE_H

#include <iostream>
#include <unordered_map>
#include "cacheInterface.h"
using namespace std;

#define HASH_SIZE 20

template <typename KeyT, typename ValueT>
class UMapCache : public CacheInterface<KeyT, ValueT> {
public:
    virtual ValueT get(KeyT key);
    virtual void set(KeyT key, ValueT value);
    virtual bool empty();
    virtual int size(); // number of entries
    virtual int sizeOfAllEntries(); // size of all entries
    virtual double sizeOfAllEntriesDouble();
    // size of all entries (double value)
    virtual int sizeOfKeys(); // size of keys of all entries
    virtual double sizeOfKeysDouble();
    // size of keys of all entries (double value)
    virtual int sizeOfValues(); // size of values of all entries
    virtual bool exist(KeyT key);
    virtual void showAll();
    virtual void removeAll(); // remove all entries

private:

```

```

    unordered_map<KeyT, ValueT> container;
};

template <typename KeyT, typename ValueT>
ValueT
UMapCache<KeyT, ValueT>::get(KeyT key) {
    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    iter = container.find(key);

    if (iter == container.end())
        return "";
    else
        return iter->second;
}

template <typename KeyT, typename ValueT>
void
UMapCache<KeyT, ValueT>::set(KeyT key, ValueT value) {
    pair<KeyT, ValueT> entry(key, value);
    container.insert(entry);
}

template <typename KeyT, typename ValueT>
bool
UMapCache<KeyT, ValueT>::empty() {
    return container.empty();
}

template <typename KeyT, typename ValueT>
int
UMapCache<KeyT, ValueT>::size() {
    return container.size();
}

template <typename KeyT, typename ValueT>
int
UMapCache<KeyT, ValueT>::sizeOfAllEntries() {

    int tmpSize = 0;

    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    for (iter = container.begin(); iter != container.end(); iter++)
    {
        tmpSize += (iter->first).length() + (iter->second).length();
    }
    return tmpSize;
}

```

```

template <typename KeyT, typename ValueT>
double
UMapCache<KeyT, ValueT>::sizeOfAllEntriesDouble() {

    double tmpSize = 0;

    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    for (iter = container.begin(); iter != container.end(); iter++)
    {
        tmpSize += (iter->first).length() + (iter->second).length();
    }
    return tmpSize;
}

template <typename KeyT, typename ValueT>
int
UMapCache<KeyT, ValueT>::sizeOfKeys() {

    int tmpSize = 0;

    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    for (iter = container.begin(); iter != container.end(); iter++)
    {
        tmpSize += (iter->first).length();
    }
    return tmpSize;
}

template <typename KeyT, typename ValueT>
double
UMapCache<KeyT, ValueT>::sizeOfKeysDouble() {

    double tmpSize = 0;

    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    for (iter = container.begin(); iter != container.end(); iter++)
    {
        tmpSize += (iter->first).length();
    }
    return tmpSize;
}

template <typename KeyT, typename ValueT>
int
UMapCache<KeyT, ValueT>::sizeOfValues() {

    int tmpSize = 0;

    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    for (iter = container.begin(); iter != container.end(); iter++)
    {
        tmpSize += (iter->second).length();
    }
}

```

```

    return tmpSize;
}

template <typename KeyT, typename ValueT>
bool
UMapCache<KeyT, ValueT>::exist(KeyT key) {
    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    iter = container.find(key);

    if (iter != container.end())
        return true;
    else
        return false;
}

template <typename KeyT, typename ValueT>
void
UMapCache<KeyT, ValueT>::showAll() {
    typename unordered_map<KeyT, ValueT>::const_iterator iter;
    for (iter = container.begin(); iter != container.end(); iter++)
    {
        cout << iter->first << ",_" << iter->second << endl;
    }
}

template <typename KeyT, typename ValueT>
void
UMapCache<KeyT, ValueT>::removeAll() {
    container.clear();
}

#endif

```

B.3 cache.cc

```

#include "cache.h"

#ifdef CACHE_TEST

int main() {

    UMapCache<string, string> cache;
    string key = "1";
    string value = "Danny";
    string key2 = "2";
    string value2 = "Kim";
    string key3 = "3";

```

```

// check if cache is empty
cout << "****_current_cache_****" << endl;
if (cache.empty())
    cout << "empty" << endl;
else
    cout << "filled" << endl;
cout << endl;

// save an entry
cout << "****_save_entries_****" << endl;
cout << "<" << key << "," << value << ">" << endl;
cout << "<" << key2 << "," << value2 << ">" << endl;
cache.set(key, value);
cache.set(key2, value2);
cout << endl;

// check if cache is empty
cout << "****_current_cache_****" << endl;
if (cache.empty())
    cout << "empty" << endl;
else
    cout << "filled" << endl;
cout << endl;

// get an entry
cout << "****_get_an_entry_****" << endl;
cout << "key=_ " << key << "_ ";
cout << cache.get(key) << endl;
cout << endl;

// get number of entries
cout << "****_get_number_of_entries_****" << endl;
cout << "size:_ " << cache.size() << endl;
cout << endl;

// check if an entry with key exists
cout << "****_existence_of_a_key_****" << endl;
string tmp = key2;
if (cache.exist(tmp))
    cout << tmp << "_exists" << endl;
else
    cout << tmp << "_doesn't_exist" << endl;
cout << endl;

// show all entries
cout << "****_show_all_entries_****" << endl;
cache.showAll();
cout << endl;

// show size of all entries in bytes
cout << "size_of_all_entries_(bytes):_"
    << cache.sizeOfAllEntries() << endl;
cout << "size_of_all_entries(double_value)_(bytes):_"
    << cache.sizeOfAllEntriesDouble() << endl;

```

```

// show size of keys of all entries in bytes
cout << "size_of_keys_{}(bytes):_:"
    << cache.sizeOfKeys() << endl;
cout << "size_of_keys(double_value)(bytes):_"
    << cache.sizeOfKeysDouble() << endl;

// show size of values of all entries in bytes
cout << "size_of_values_{}(bytes):_"
    << cache.sizeOfValues() << endl;
cout << endl;

// remove all entries
cout << "****_remove_all_entries_****" << endl;
cache.removeAll();
cout << "size_of_all_entries:_:" << cache.sizeOfAllEntries()
    << endl;
cout << endl;

return 0;
}
#endif

```

Appendix C

Bloom Filter Implementation

In this chapter, we present the codes of a Bloom filter using four hash functions. The hash functions are not one-way hash functions like SHA1. However, we measure the size of the Bloom filter based on a SHA1 hash key (160 bits) because codes were developed to use SHA1 hash keys. Each hash function computes the index of the Bloom filter bit array. These codes are written in C. `bf.h` defines functions for the Bloom filter, and `bf.c` implements the defined functions. A gcc compiler is needed to compile.

C.1 `bf.h`

```
//-----  
//  
// hash key -> Bloom filter  
// -> if any checks say "not 1"(that is , 0),  
//      then fingerprint is new  
// check bf[hash1(fingerprint)] == 1,  
// check bf[hash2(fingerprint)] == 1,  
//      :  
//  
// m : number of bits in an bloom filter array  
// n : number of bits of a fingerprint  
// k : number of hash functions  
//  
// based on DDFS (Data Domain File System)~\cite{Zhu:FAST08}  
// paper(page 274),  
//  
// To archive 2% false positive , smallest size of bloom filter  
// is  $m = 8 * n$  bits ( $m/n = 8$ ), and the number of  
// hash function is 4 ( $k=4$ )  
//
```

```

// So, we use following configurations for a bloom filter.
//
// n = 160 bits (SHA1 hash key)
// m = 8 * 160 = 1280 bits
// k = 4 (four hash functions)
//
//
// I choose prime number as MAX_SIZE_BUF (here, 1283 over 1280).
// This is because mod() by prime number shows good uniform
//      distribution
// ,reducing primary cluster.
// (Weiss book — Data structure and Algorithm in C++ :
//  third edition~\cite{Weiss:Addison_Wesley05})
//
//_____
#ifndef BF_H
#define BF_H

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// maximum number of bits of bloom filter array
#define MAX_SIZE_BUF      11
#define MAX_SIZE_HF      4 // maximum number of hash functions

#define HASH1_MOD        10000
#define HASH2_MOD        100000
#define HASH3_MOD        1000000
#define HASH4_MOD        10000000

#endif

```

C.2 bf.c

```

//
// we will use simple hash functions with mod()
//
// 1. temp ← fingerprint % hash_mod_value
// 2. index of bloom filter ← temp % max_size
//
// Also, in order to choose hash function dynamically in run
//      time,
// I use function pointer for hash functions
//
#include "bf.h"

typedef unsigned long long int  u_int64_t;

// input : fingerprint
// return: index in bloom filter

```

```

int hash1(u_int64_t fingerprint)
{
    u_int64_t temp;
    int         bf_index; // index in the bloom filter

    temp      = fingerprint / HASH1_MOD;
    bf_index  = temp % MAX_SIZE_BF;

#ifdef DEBUG
    printf("hash1():_fingerprint_=%llu\n", fingerprint);
    printf("hash1():_temp_=%llu\n", temp);
    printf("hash1():_bf_index_=%d\n", bf_index);
    //exit(0);
#endif

    return bf_index;
}

// input : fingerprint
// return: index in bloom filter
int hash2(u_int64_t fingerprint)
{
    u_int64_t temp;
    int         bf_index; // index in the bloom filter

    temp      = fingerprint / HASH2_MOD;
    bf_index  = temp % MAX_SIZE_BF;

#ifdef DEBUG
    printf("hash2():_fingerprint_=%llu\n", fingerprint);
    printf("hash2():_temp_=%llu\n", temp);
    printf("hash2():_bf_index_=%d\n", bf_index);
#endif

    return bf_index;
}

// input : fingerprint
// return: index in bloom filter
int hash3(u_int64_t fingerprint)
{
    u_int64_t temp;
    int         bf_index; // index in the bloom filter

    temp      = fingerprint / HASH3_MOD;
    bf_index  = temp % MAX_SIZE_BF;

#ifdef DEBUG
    printf("hash3():_fingerprint_=%llu\n", fingerprint);
    printf("hash3():_temp_=%llu\n", temp);
    printf("hash3():_bf_index_=%d\n", bf_index);
#endif

    return bf_index;
}

```

```

}

// input : fingerprint
// return: index in bloom filter
int hash4(u_int64_t fingerpt)
{
    u_int64_t temp;
    int bf_index; // index in the bloom filter

    temp = fingerpt / HASH4_MOD;
    bf_index = temp % MAX_SIZE_BF;

#ifdef DEBUG
    printf("hash4():_fingerpt=_%llu\n", fingerpt);
    printf("hash4():_temp_=_%llu\n", temp);
    printf("hash4():_bf_index_=_%d\n", bf_index);
#endif

    return bf_index;
}

// input parameter : the name of function
int hash_func(int func(), u_int64_t fingerpt)
{
    return (*func)(fingerpt);
}

// initialize bloom filter
void
bf_init(int *bf)
{
    int i;

    // set all elements to 0
    for (i=0; i < MAX_SIZE_BF; i++)
        bf[i] = 0;
}

// insert 1s to bloom filter
// for all hash functions
void
bf_insert(int *bf, u_int64_t fingerpt)
{
    bf[hash_func(hash1, fingerpt)] = 1;
    bf[hash_func(hash2, fingerpt)] = 1;
    bf[hash_func(hash3, fingerpt)] = 1;
    bf[hash_func(hash4, fingerpt)] = 1;
}

```

```

//
// see if results of all hash functions
// are 1s.
// if all are 1s, return 1
// otherwise, return 0
//
int
bf_lookup(int *bf, u_int64_t fingerprint)
{
    // check index by hash1 function
    if (bf[hash_func(hash1, fingerprint)] != 1)
        return 0;

    // check index by hash2 function
    if (bf[hash_func(hash2, fingerprint)] != 1)
        return 0;

    // check index by hash3 function
    if (bf[hash_func(hash3, fingerprint)] != 1)
        return 0;

    // check index by hash4 function
    if (bf[hash_func(hash4, fingerprint)] != 1)
        return 0;

    return 1;
}

void
bf_show(int *bf)
{
    int i;

    // printf("#####\n");
    printf(">>>_Bloom_Filter_(%d_bits)_\n", MAX_SIZE_BF);
    // printf("#####\n");

    for (i=0; i < MAX_SIZE_BF; i++)
        printf("%d_", bf[i]);

    printf("\n");
}

#ifdef BF_TEST

int main()
{
    //_____
    // test variables
    //_____
    printf("#####\n");
    printf("#_Test_:_Input_Data_#####\n");
    printf("#####\n");
}

```

```

u_int64_t fingerpt1 = 4543863031426141731;;
u_int64_t fingerpt2 = 4543863041425141743;;

printf("[bloom_filter]_fingerpt1_#####: %llu\n", fingerpt1);
printf("[bloom_filter]_fingerpt2_#####: %llu\n", fingerpt2);
printf("\n");

//-----
// STEP1. create bloom filter
//-----
int bf[MAX_SIZE_BF];

//-----
// STEP2. initialize bloom filter
//-----
#ifdef DEBUG
printf("#####\n");
printf("[bloom_filter]_initialize\n");
printf("#####\n");
#endif

bf_init(bf);

#ifdef DEBUG
bf_show(bf);
printf("\n");
#endif

//-----
// STEP3. insert into bloom filter
//-----
printf("#####\n");
printf("[bloom_filter]_insert_#####: %llu\n", fingerpt1);
printf("#####\n");

bf_insert(bf, fingerpt1);

printf("\n");
bf_show(bf);
printf("\n");

//-----
// STEP3. insert into bloom filter
// bf_lookup returns 1 if fingerpt exists
//-----
printf("#####\n");
printf("[bloom_filter]_lookup_: %llu\n", fingerpt1);
printf("#####\n");
if (bf_lookup(bf, fingerpt1))
    printf("[bloom_filter]_exist_#####: %llu\n", fingerpt1);
else
    printf("[bloom_filter]_doesn't_exist_: %llu\n", fingerpt1);

printf("\n");

```

```
printf("#####\n");
printf("[bloom_filter]_lookup_:_%llu_\n", fingerpt2);
printf("#####\n");
if (bf_lookup(bf, fingerpt2))
    printf("[bloom_filter]_exist_:_%llu_\n", fingerpt2);
else
    printf("[bloom_filter]_doesn't_exist_:_%llu_\n", fingerpt2);

printf("\n");
return 0;
}

#endif
```

Appendix D

Rabin Fingerprinting Implementation

In this chapter, we present codes to acquire a fingerprint based on Rabin fingerprinting [1]. This fingerprint is used to determine chunk boundaries for variable-sized chunks in variable-sized block deduplication. The fingerprint is 64 bit, which is smaller than SHA1 hash keys, and the computation for obtaining the fingerprint is faster than computing a SHA1 hash key. This is why fingerprinting is used to obtain chunks. We show a header file (rabinpoly.h), an implementation file of defined functions (rabinpoly.cc) [3] and a testing program (rabinpoly_main.cc). rabinpoly.h and rabinpoly.cc were developed by David Mazieres in 2000, and we added some codes to it. There are different variables whose types are `u_int64_t`. To check the value of the variables (to understand how Rabin fingerprints are calculated), you can use the `%llu` format. `bzero()` was replaced by `memset()`.

D.1 rabinpoly.h

```
// -*-c++-*-
/* $Id: rabinpoly.h,v 1.4 2002/01/07 21:30:21 athicha Exp $ */

/*
 *
 * Copyright (C) 2000 David Mazieres (dm@uun.org)
 *
 * This program is free software; you can redistribute it and/or
 * modify it under the terms of the GNU General Public License as
 * published by the Free Software Foundation; either version 2,
 * or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```

* See the GNU General Public License for more details.
*
* You should have received a copy of the GNU General Public
  License
* along with this program; if not, write to the Free Software
* Foundation, Inc., 59 Temple Place, Suite 330, Boston,
* MA 02111-1307 USA
*
*/

#ifndef _RABINPOLY_H_
#define _RABINPOLY_H_

// #include "async.h"

// #####
// by Danny — begin
// #####
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <assert.h>

typedef unsigned int      u_int;
typedef unsigned char     u_char;

#define BOUNDARY_SIZE      48 // size of region to be boundary

// #####
// by Danny — end
// #####

u_int64_t polymod (u_int64_t nh, u_int64_t nl, u_int64_t d);
u_int64_t polygcd (u_int64_t x, u_int64_t y);
void polymult (u_int64_t *php, u_int64_t *plp,
               u_int64_t x, u_int64_t y);
u_int64_t polymmult (u_int64_t x, u_int64_t y, u_int64_t d);
int polyirreducible (u_int64_t f); // check if the polynomial
                                   // is irreducible
u_int64_t polygen (u_int degree); // generate polynomial with
                                   degree

// -----
// rabinpoly
// -----
typedef struct rabinpoly {

    int shift;
    u_int64_t T[256]; // Lookup table for mod

    u_int64_t poly; // Actual polynomial
} rabinpoly;

```

```

void rb_calcT (rabinpoly *rb);
void rb_init(rabinpoly *rb, u_int64_t poly);
u_int64_t rb_append8(rabinpoly *rb, u_int64_t p, u_char m);

enum {size = 48};

//-----
// window
//-----
typedef struct window {

    //----- commented by Danny
    // size(48) means overlapping size for boundary region
    //----- commented by Danny

    u_int64_t fingerprint;
    int bufpos;
    u_int64_t U[256];    //
    u_char buf[size];    //

} window;

void w_init(window *w, rabinpoly *rb, u_int64_t poly);
u_int64_t w_slide8 (window *w, rabinpoly *rb, u_char m);
void w_reset(window *w);

#endif /* !_RABINPOLY_H_ */

```

D.2 rabinpoly.cc

```

/* $Id: rabinpoly.C,v 1.1 2001/01/29 22:49:13 benjie Exp $ */

/*
 *
 * Copyright (C) 1999 David Mazieres (dm@uun.org)
 * :
 */

#include "rabinpoly.h"
#define INT64_1      0x0000000000000001LL
#define MSB64        0x8000000000000000LL

//
// returns bit position where a bit is 1 among 64 bits
//
// e.g.    100          ———> return 3
// (1 : a bit in the 3rd place from left)
//
// e.g.    10010010     ———> return 2

```

```

// (1 : a bit in the 2nd place from left)
//
static inline int
fls64(u_int64_t mask)
{
    int bit;

    if (mask == 0)
        return (0);

    // if a bit in a mask is "0", continue until a bit becomes "1"
    for (bit = 1; ((mask & 0x1ULL) == 0); bit++)
    {
        // 100 ← mask          , bit = 1
        // 10  ← mask >> 1      , bit = 2
        // 1   ← mask >> 1      , bit = 3
        mask = mask >> 1;
    }

    return (bit);
}

// polynomial mod
u_int64_t
polymod(u_int64_t nh, u_int64_t nl, u_int64_t d)
{
    int i, k;
    assert (d);

    k = fls64(d) - 1;
    d <= 63 - k;

    if (nh)
    {
        if (nh & MSB64)
        {
            nh ^= d;    // nh XOR (shit-lefted poly)
        }
        for (i = 62; i >= 0; i--)
            // if (nh & INT64_1 << i) {
            if (nh & INT64_1 << i) { // << first, & second
                nh ^= d >> 63 - i; // -, + first, >>, << second, ^third
                nl ^= d << i + 1;
            }
    }

    for (i = 63; i >= k; i--)
    {
        if (nl & INT64_1 << i)
        {
            nl ^= d >> 63 - i;
        }
    }
}

```

```

    return nl;
}

u_int64_t
polygcd (u_int64_t x, u_int64_t y)
{
    for (;;) {
        if (!y)
            return x;
        x = polymod (0, x, y);
        if (!x)
            return y;
        y = polymod (0, y, x);
    }
}

void
polymult (u_int64_t *php, u_int64_t *plp, u_int64_t x, u_int64_t
y)
{
    int i;

    u_int64_t ph = 0, pl = 0;
    if (x & 1)
        pl = y;
    for (i = 1; i < 64; i++)
        // if (x & (INT64_1 << i)) {
        if (x & (INT64_1 << i)) {
            ph ^= y >> (64 - i);
            pl ^= y << i;
        }
    if (php)
        *php = ph;
    if (plp)
        *plp = pl;
}

u_int64_t
polymmult (u_int64_t x, u_int64_t y, u_int64_t d)
{
    u_int64_t h, l;

    polymult (&h, &l, x, y);
    return polymod (h, l, d);
}

int
polyirreducible (u_int64_t f)
{
    int i, m;

    u_int64_t u = 2;
    // int m; // by Danny

```

```

    m = (fls64(f) - 1) >> 1;
    for (i = 0; i < m; i++) {
        u = polymmilt(u, u, f);
        if (polygcd(f, u ^ 2) != 1)
            return 0;
    }
    return 1;
}

#####
// rabinpoly - begin
#####

//
// initialize rabinpoly
//
void
rb_init(rabinpoly *rb, u_int64_t p)
{
    rb->poly = p;
    rb_calcT(rb);
}

u_int64_t
rb_append8(rabinpoly *rb, u_int64_t p, u_char m)
{
    return ((p << 8) | m) ^ rb->T[p >> (rb->shift)];
}

void
rb_calcT(rabinpoly *rb)
{
    int xshift, j;

    // poly should be larger than 0x100
    assert((rb->poly) >= 0x100);

    xshift = fls64(rb->poly) - 1;
    rb->shift = xshift - 8;

    u_int64_t T1 = polymod(0, INT64_1 << xshift, rb->poly);

    for (j = 0; j < 256; j++)
    {
        rb->T[j] = polymmilt(j, T1, rb->poly)
                    | ((u_int64_t) j << xshift);
    }
}

```

```

// #####
// rabinpoly - end
// #####

// #####
// window - begin
// #####

// poly : p(t)
// fingerprint : fingerprint
// bufpos : position of buffer
void
w_init(window *w, rabinpoly *rb, u_int64_t poly)
{
    int i;

    //-----
    // initialize
    //-----
    rb_init(rb, poly);
    w->fingerprint = 0;
    w->bufpos      = -1;

    u_int64_t sizeshift = 1;

    // rb_append8 : append eight 0 bits after 1 bit
    // sizeshift      = 1          <— 1
    // sizeshift      = 256        <— 1 00000000
    // sizeshift      = 65536      <— 1 00000000 00000000
    // sizeshift      = 16777216   :
    // sizeshift      = 4294967296  :
    // sizeshift      = 109951162776 :
    // sizeshift      = 281474976710656 :
    // sizeshift      = 72057594037927936 :
    // sizeshift      = 1          <— 1 (loop)
    // sizeshift      = 256
    // :
    // :
    // sizeshift      = 72057594037927936 <— To 48th

    for (i = 1; i < size; i++)
    {
        sizeshift = rb_append8(rb, sizeshift, 0);
    }

    // see bottom for all results of U[i]
    for (i = 1; i < 256; i++)
    {
        w->U[i] = polymmult(i, sizeshift, poly);
    }
}

```

```

    //bzero (buf, sizeof (buf));          // by Danny
    memset(w->buf, '\0', sizeof(w->buf)); // by Danny
}

u_int64_t
w_slide8(window *w, rabinpoly *rb, u_char m)
{
    if (++(w->bufpos) >= size)
        w->bufpos = 0;
    u_char om = w->buf[w->bufpos];
    w->buf[w->bufpos] = m;
    return w->fingerprint =
        rb_append8(rb, w->fingerprint ^ w->U[om], m);
}

void
w_reset (window *w) {
    w->fingerprint = 0;
    memset(w->buf, '\0', sizeof(w->buf)); // by Danny
}

// #####
// window — end
// #####

```

D.3 rabinpoly_main.cc

```

//
// The purpose
//
// Get fingerprint of string A
//
//  $f(A) = A(t) \bmod P(t)$ 
//
#include "rabinpoly.h"

#define FINGERPRINT_PT 0xbfe6b8a5bf378d83LL

u_int64_t
fingerprint(unsigned char *data, int count)
{
    // define
    window *w;
    rabinpoly *rb;

    // allocate memory
    w = (window*)malloc(sizeof(window));
    rb = (rabinpoly*)malloc(sizeof(rabinpoly));
}

```

```

int i;
u_int64_t poly = FINGERPRINT_PT;

// init & reset window
w_init(w, rb, poly);
w_reset(w);

u_int64_t fp = 0;
for (i = 0; i < count; i++)
    fp = rb_append8 (rb, fp, data[i]);

// deallocate memory
free(w);
free(rb);

return fp;
}

int main()
{
    //-----
    // STEP1. get data
    //-----
    unsigned char data[] = "hello_tom_danny";
    u_int64_t fingerpt = 0;
    int size;

    //-----
    // STEP2. get size of data (=number of chars)
    //-----
    size = sizeof(data) / sizeof(unsigned char);
#ifdef DEBUG
    printf("rabinpoly_main_:data_size=%d\n", size);
#endif

    //-----
    // STEP3. get fingerprint of data
    //-----
    printf("rabinpoly_main_:input_data\n");
    printf("%s\n", data);
    printf("rabinpoly_main_:fingerpt=%lu\n",
           fingerprint(data, size));

    return 0;
}

```

Appendix E

Chunking Core Implementation

In this chapter, we present snippet codes to extract variable-sized chunks from a file (owing to the large size of the codes). Chunking core implementation requires Rabin fingerprint functions in Appendix D and utility functions in util.cc. We show process_chunk() function in chunk_sub.cc.

E.1 chunk.h

```
#ifndef _CHUNK_H_
#define _CHUNK_H_

#include "common.h"
#include "rabinpoly.h"

// #define DEBUG 1

// #####
// Constants
// #####
// predetermined irreducible polynomial
#define FINGERPRINT_PT 0xbfe6b8a5bf378d83LL
#define BREAKMARK_VALUE 0x78 // for boundary region

#define ONE_BYTE 1 // size for sliding window
#define MAX_NUM_CHUNKS 8192 // maximum number of chunks

// directory & file
// the directory where chunks are saved
#define CHUNK_DIR "./chunk_dir"
```

```

#define DOC_DIR           "./doc_dir" // the directory in which a
                           document
                           // has hash keys for it.
                           // a document = sum (chunks of each hash
                           key)
#define HEADER_DIR       "./header_dir"
#define CHUNK_INDEX_DIR  "./chunk_index_dir"
#define BREAKPOINT_DIR   "./breakpoint_dir"

#define DOC_FILENAME      "doc.tmp"

#####
// Functions
#####
int get_filesize(char *filepath);

//-----
// a file —> chunks
//-----
int getChunksIndexes(unsigned char* data, int data_size,
                    int avg_chunk_size,
                    int min_chunk_size, int max_chunk_size,
                    int *begin_indexes, int *end_indexes);
int get_chunk_size(int chunk_b_pos, int cur_pos, int b_size);
void process_chunk(unsigned char *data, int data_size,
                  int *num_of_breakpoints, int b_size, int avg_chunk_size,
                  int min_chunk_size, int max_chunk_size, int *num_of_chunks,
                  int *begin_indexes, int *end_indexes);
void set_breakpoint(int *num_of_breakpoints,
                  int *chunk_b_pos, int *chunk_e_pos, int *cur_pos,
                  char *reason, int b_size, int data_size, int *num_of_chunks,
                  int *begin_indexes, int *end_indexes);
void get_chunk(int fd, unsigned char **chunk, int f_chunk_b_pos,
               int chunk_size, int file_size);
void save_chunk(int fd, int chunk_index_fd, int chunk_b_pos,
               int chunk_e_pos, int file_size);
void
save_last_chunk(char *data,
               int *cur_pos, int *num_of_breakpoints,
               int *chunk_b_pos, int *chunk_e_pos,
               int data_size, int b_size );

void
save_chunk_with_max_chunk_size(char *data,
                              int *cur_pos, int *num_of_breakpoints,
                              int *chunk_b_pos, int *chunk_e_pos,
                              int data_size, int b_size,
                              int *num_of_chunks, int *begin_indexes,
                              int *end_indexes );
u_int64_t fingerprint(unsigned char *data,
                      int count, u_int64_t pt);
void save_chunk_index(int chunk_index_fd,
                     unsigned char *hash_str, int chunk_size);

```

```

//=====
// chunks → a file
//=====
void remove_previous_doc(char *doc_filepath);
void get_filepath(char *ret_filepath, char *dir, char *filename);
void get_doc_filepath(char *doc_filepath, char *doc_dir,
    char *mail_hash_key);
void get_chunk_filepath(char *chunk_filepath, char *chunk_dir,
    char *chunk_index);
void get_header_filepath(char *header_filepath, char *header_dir,
    char *mail_hash_key);
void get_chunk_index_filepath(char *chunk_index_filepath,
    char *chunk_index_dir, char *mail_hash_key);
void get_breakpoint_filepath(char *breakpoint_filepath,
    char *breakpoint_dir, char *mail_hash_key);
void append_header(char *header_filepath, char *doc_filepath);
void append_chunk(char *chunk_filepath, char *doc_filepath);
void add_header_into_reassembled_doc(char *header_filepath,
    char *doc_filepath,
    char *header_dir, char *mail_hash_key);
void add_body_into_reassembled_doc(char *doc_filepath,
    char *chunk_index_filepath, char *chunk_dir);

void reassemble_doc(char *chunk_index_filepath,
    char *reassembled_doc_dir,
    char *header_dir, char *chunk_dir, char *mail_hash_key,
    char *dataset);
int is_same_size(char *doc1_filepath, char *doc2_filepath);
void save_chunks_into_doc(char *doc_filepath, char *chunk_dir,
    char **chunk_index_arr, int chunk_index_arr_size);
void get_chunk_from_byte_stream(unsigned char *buf,
    int begin_index,
    int len, unsigned char *chunk);

// overloaded functions
void strncpy(unsigned char *dest, unsigned char *src, int size);
int strlen(unsigned char *s);

#endif

```

E.2 chunk_main.cc

```

#include "chunk.h"

int main(int argc, char *argv[])
{
    //#####
    // Variables
    //#####

```

```

// common
int num_of_params; // number of command-line parameters

// related to files
char data_filename[CHUNK_BUF_SIZE]; // input file path

int data_filesize = 0; // data file size

// related to chunks
int fd; // file descriptor to check boundary
int size; // size of input data
unsigned char *buf; // input data

int offset = 0; // file offset
int chunk_b_pos = 0 ; // beginning position of a chunk in a file
int chunk_e_pos = -1; // ending position of a chunk in a file
// *** NOT 0 ** due to "set_breakpoint() in chunk_sub.c"

// !!!! Accumulated chunk size !!!!
// This value is compared to minimum and maximum chunk size
int cur_chunk_size = 0;

// file which contains chunk indexes for a document
int chunk_index_fd;
char chunk_index_filepath[CHUNK_BUF_SIZE]; // chunk index file
path

// parameters
int avg_chunk_size; // expected average chunk size
int min_chunk_size; // minimum chunk size
int max_chunk_size; // maximum chunk size

// temporary command
char cmd[CHUNK_BUF_SIZE];

int num_of_breakpoints = 0;

#####
// Check parameters
#####
// get number of command-line parameters
num_of_params = argc - 1;

strcpy(data_filename , argv[1]); // input data filename
avg_chunk_size = atoi(argv[2]);
min_chunk_size = atoi(argv[3]);
max_chunk_size = atoi(argv[4]);

#####
// chunk
#####

```

```

// get filesize
data_filesize = get_filesize(data_filename);

// open file
//
fd = open(data_filename , O_RDONLY);
if (fd < 0) {
    printf("file_open_error_: _fd\n");
    exit(1);
}

int num_of_chunks = 0;
int begin_indexes[MAX_NUM_CHUNKS]; // array of begin index
int end_indexes[MAX_NUM_CHUNKS];   // array of end index
unsigned char *chunk;
int len;
int i;

buf = (unsigned char*)malloc(sizeof(unsigned char)
    * data_filesize);
while ( (size = read(fd, buf, data_filesize)) > 0)
{
    process_chunk(buf, size, &num_of_breakpoints, BOUNDARY_SIZE,
        avg_chunk_size, min_chunk_size, max_chunk_size,
        &num_of_chunks, begin_indexes, end_indexes);
}

// close files
close(fd);
return 0;
}

```

E.3 chunk_sub.cc

```

:
omitted due to limitation of pages
:
//
// _____
//
// save chunks based on rabin fingerprint
// that is, variable sized chunk
//
// input :
// - data,
// size of data(which is the same as input file size),
// b_size : boundary size,
// expected average chunk size
// minimum chunk size

```

```

//    maximum chunk size
//
// output : chunk boundaries
//

// *** COMMENTS : The index in a data starts from 0 including
//                  f_cur_pos ...
//
void
process_chunk(unsigned char *data, int data_size,
              int *num_of_breakpoints,
              int b_size, int avg_chunk_size, int min_chunk_size,
              int max_chunk_size, int *num_of_chunks,
              int *begin_indexes, int *end_indexes)
{
    //-----
    // STEP1. Variables
    //-----
    int cur_pos      = 0;      // current index of a cursor
    int chunk_b_pos  = 0;      // starting index of each chunk
    int chunk_e_pos  = 0;      // ending index of each chunk
    int cur_chunk_size = 0;    // current size of each chunk

    unsigned char *b_region; // boundary region
    int size_of_b_region;    // size of boundary region
    u_int64_t fingerprint;   // fingerprint of a boundary region
    u_int64_t low_order_bits; // low order bits of fingerprint

    int size_remained_from_next_line;
    int file_last_index;

    //-----
    // STEP2. Check boundary based on
    //         Rabin fingerprint
    //
    //         slide window by 1 byte
    //-----
    for (cur_pos=0; cur_pos < data_size; cur_pos += ONE_BYTE)
    {
        //-----
        // STEP2-1. At the end of data,
        //           save the last chunk and exit
        //-----
        size_remained_from_next_line = data_size - (cur_pos + 1);
        if ( size_remained_from_next_line < b_size ) // last part
        {
            // get chunk_b_pos and chunk_e_pos
            set_breakpoint(num_of_breakpoints, &chunk_b_pos,
                          &chunk_e_pos, &cur_pos,
                          (char*)"LAST_CHUNK", b_size, data_size,
                          num_of_chunks, begin_indexes, end_indexes);

```

```

    break;
}

//-----
// STEP2-2. get boundary region
//-----

// allocate boundary region
b_region = (unsigned char *)malloc(sizeof(unsigned char)
                                   *b_size);
memset(b_region, '\0', b_size);

// get boundary region
strncpy(b_region, data + cur_pos, b_size);
b_region[b_size] = '\0';

//-----
// STEP2-3. compute rabin fingerprint
//-----
fingerprint = fingerprint(b_region, strlen(b_region),
                          FINGERPRINT_PT);

//-----
// STEP2-4. compare to breakpoint value
//           to extract chunk
//
//           fingerprint % K(avg_chunk_size) == BREAKMARK_VALUE
//-----
low_order_bits = fingerprint % avg_chunk_size;

//-----
// STEP2-5. chunk size is larger than maximum chunk size
//-----
cur_chunk_size = get_chunk_size(chunk_b_pos, cur_pos,
                                b_size);
if (cur_chunk_size >= max_chunk_size)
{
    //get chunk_b_pos and chunk_e_pos
    set_breakpoint(num_of_breakpoints, &chunk_b_pos,
                  &chunk_e_pos, &cur_pos,
                  (char*)"MAX_CHUNK_SIZE", b_size, data_size,
                  num_of_chunks, begin_indexes, end_indexes);

    // set position for the next chunk
    chunk_b_pos = chunk_e_pos + 1;
    cur_pos     = chunk_b_pos;
}

//-----
// STEP3-5-3 chunk size is less than minimum chunk size or
//           is in between minimum chunk size and maximum chunk
//           size

```

```

//
//      ( $f(A) \bmod K == x$ )
//      (chunk size is less than minimum or
//      is in range of minimum and maximum chunk size)
//
//      :  $f(A) \longrightarrow \text{fingerprint}$ 

//      :  $K \longrightarrow \text{expected average chunk size}$ 
//      :  $x \longrightarrow \text{BREAKMARK\_VALUE}$ 
// _____
if (low_order_bits == BREAKMARK_VALUE)
{
    if (cur_chunk_size < min_chunk_size)
    {
        // do not set breakpoint
        ;
    }
    else if ( (cur_chunk_size >= min_chunk_size)
              && (cur_chunk_size < max_chunk_size) )
    {

        // get chunk_b_pos and chunk_e_pos
        set_breakpoint(num_of_breakpoints, &chunk_b_pos,
                      &chunk_e_pos, &cur_pos,
                      (char*)"BREAKMARK", b_size, data_size,
                      num_of_chunks, begin_indexes, end_indexes
                      );

        // set position for the next chunk
        chunk_b_pos = chunk_e_pos + 1;
        cur_pos      = chunk_b_pos;

    }
}

// _____
// STEP2-2. free boundary region
// _____
free(b_region);

} // end of data
}

:
omitted due to limitation of pages
:
```

E.4 common.h

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h> // for STDOUT_FILENO
#include <fcntl.h>  // for O_RDONLY
```

```

#include <sys/stat.h> // get_filesize

#define TRACE      2
#define DEBUG_TEMP 3

#define CHUNK_BUF_SIZE      500

//—————
// UTIL
//—————
int get_str_array_size(char *str);
void split_to_str_array(char *str, char *str_array[]);

```

E.5 util.cc

```

#include "common.h"

const static char *delimiter = ",";

// input string
// output : array_size
//         if error, return -1
int
get_str_array_size(char *str)
{
    int index = 0;
    char *ptr;
    int temp;

    if (str == NULL) {
        printf("no_input_string");
        return -1;
    }

    // first one
    ptr = strtok(str, delimiter);
    if (ptr == NULL) {
        printf("no_output");
        return 0;
    }
    index++;

    // from second
    while( ptr = strtok( NULL, delimiter)) {
        index++;
    }

    return index;
}

// input string

```

```

// output : string array, and array size
void
split_to_str_array(char *str, char *str_array[])
{
    int index = 0;
    char *ptr;
    int temp;
    int tmp_len;

    if (str == NULL) {
        printf("no_input_string");
        return;
    }

    // first one
    ptr = strtok(str, delimiter);
    if (ptr == NULL) {
        printf("no_output");
        return;
    }

    str_array[index] = (char*)malloc(sizeof(char) * strlen(ptr));
    strcpy(str_array[index++], ptr);

    // from second
    while( ptr = strtok( NULL, delimiter)) {

        // NULL should be inserted
        // Otherwise, several items may have unexpected length
        // (expecially an unexpected character at the end)
        tmp_len = strlen(ptr);
        str_array[index] = (char*)malloc(sizeof(char) * (tmp_len+1)
        );
        strncpy(str_array[index], ptr, tmp_len);
        // str_array[index][tmp_len] = NULL;
        str_array[index][tmp_len] = '\0';
        index++;
    }
}

#ifdef UTIL_DEBUG
// To test split_to_str_array() function

int main()
{
    char init_str[100];    // <<— use array
    char str[100];         // <<— use array
                           // if a pointer points out the array
                           // use strcpy to create temporary array

    char **str_array;
    int str_array_size;    // number of members in an array

```

```
int i;

// print string
strcpy( init_str , "Hello ,Bye ,Succeed ,");
strcpy( str , init_str);
printf(" str_=%s\n", str);

// get array size
str_array_size = get_str_array_size( str);
strcpy( str , init_str); // use initial string

str_array = (char**)malloc(sizeof(char*) * str_array_size);
if ( str_array_size > 0)
    split_to_str_array( str , str_array);

for (i=0; i < str_array_size; i++) {
    printf("%s\n", str_array[i]);
}
#endif
```

Appendix F

Chunking Wrapper Implementation

The Chunking wrapper class defines functions that separate data into variable-sized chunks based on beginning and ending indexes computed in the Chunking core class. The functions used to obtain variable-sized chunks are `getChunks()` functions. Also, the Chunking wrapper class has a function that can be used to obtain fixed-size blocks from data, called `getBlocks()`. `chunkWrapperTest.cc` shows examples of how to obtain variable-sized chunks and fixed-size blocks. The Chunking wrapper class uses file operations that are implemented in `FileOper` class that also require the C++ Boost library. To obtain the `FileOper` class, please contact the authors.

F.1 `chunkInterface.h`

```
#ifndef CHUNK_INTERFACE_H
#define CHUNK_INTERFACE_H

#include <iostream>
#include <string>
using namespace std;

class ChunkInterface {
public:

    /** getter and setter **/

    //
    // get average chunk size
    //
    virtual int getAvgChunkSize() = 0;

    //
    // set average chunk size
```

```

//
virtual void setAvgChunkSize(int avgChunkSize) = 0;

//
// get minimum chunk size
//
virtual int getMinChunkSize() = 0;

//
// set minimum chunk size
//
virtual void setMinChunkSize(int minChunkSize) = 0;

//
// get maximum chunk size
//
virtual int getMaxChunkSize() = 0;

//
// set maximum chunk size
//
virtual void setMaxChunkSize(int maxChunkSize) = 0;

/** methods */

//
// get chunks from a data
//
virtual string *getChunks(string data, int & numOfChunks) = 0;

//
// get chunks from a data
//
// avgChunkSize : average chunk size
// minChunkSize : minimum chunk size
// maxChunkSize : maximum chunk size
//
virtual string *getChunks(string data, int & numOfChunks,
                           int avgChunkSize, int minChunkSize,
                           int maxChunkSize) = 0;

//
// get fixed-size blocks from a data
//
virtual string *getBlocks(string data, int & numOfChunks) = 0;

protected:
    int avgChunkSize_;
    int minChunkSize_;
    int maxChunkSize_;
};

#endif

```

F.2 chunkWrapper.h

```

#ifndef CHUNK_WRAPPER_H
#define CHUNK_WRAPPER_H

#include <iostream>
#include "chunkInterface.h"
#include "common.h"
#include "chunk.h"
#include "rabinpoly.h"
using namespace std;

enum CHUNK_TYPE { VARIABLE, FIXED };

class ChunkWrapper : public ChunkInterface {
public:

    ChunkWrapper();
    ChunkWrapper(int avgChunkSize, int minChunkSize,
                int maxChunkSize);

    virtual int getAvgChunkSize();
    virtual void setAvgChunkSize(int avgChunkSize);
    virtual int getMinChunkSize();
    virtual void setMinChunkSize(int minChunkSize);
    virtual int getMaxChunkSize();
    virtual void setMaxChunkSize(int maxChunkSize);

    virtual string *getChunks(string data, int & numOfChunks);
    virtual string *getChunks(string data, int & numOfChunks,
                            int avgChunkSize, int minChunkSize,
                            int maxChunkSize);

    // fixed blocks
    virtual string *getBlocks(string data, int & numOfChunks);

private:

};

#endif

```

F.3 chunkWrapper.cc

```

#include "chunkWrapper.h"

// default constructor
ChunkWrapper::ChunkWrapper() {
    setAvgChunkSize(8192);
}

```

```

    setMinChunkSize(2048);
    setMaxChunkSize(645536);
}

// constructor
ChunkWrapper::ChunkWrapper(int avgChunkSize,
    int minChunkSize, int maxChunkSize) {

    setAvgChunkSize(avgChunkSize);
    setMinChunkSize(minChunkSize);
    setMaxChunkSize(maxChunkSize);
}

int
ChunkWrapper::getAvgChunkSize() {
    return avgChunkSize_;
}

void
ChunkWrapper::setAvgChunkSize(int avgChunkSize) {
    avgChunkSize_ = avgChunkSize;
}

int
ChunkWrapper::getMinChunkSize() {
    return minChunkSize_;
}

void
ChunkWrapper::setMinChunkSize(int minChunkSize) {
    minChunkSize_ = minChunkSize;
}

int
ChunkWrapper::getMaxChunkSize() {
    return maxChunkSize_;
}

void
ChunkWrapper::setMaxChunkSize(int maxChunkSize) {
    maxChunkSize_ = maxChunkSize;
}

//
// variable sized chunking
//
string *
ChunkWrapper::getChunks(string data, int & numOfChunks) {

    string *chunks;

    int beginIndexes[MAX_NUM_CHUNKS];
    int endIndexes[MAX_NUM_CHUNKS];

```

```

int i, len;

// get indexes of beginning and ending of chunks
numOfChunks = getChunksIndexes((unsigned char*)data.c_str(),
                                data.length(), getAvgChunkSize(), getMinChunkSize(),
                                getMaxChunkSize(), beginIndexes, endIndexes);

// create string array
chunks = new string[numOfChunks];
for (i=0; i < numOfChunks; i++) {
    len = endIndexes[i] - beginIndexes[i] + 1;
#ifdef DEBUG
    cout << "###_chunk[" << i
           << ":_begin_index=_< " << beginIndexes[i]
           << ",_len=_< " << len << endl;
#endif
    chunks[i] = data.substr(beginIndexes[i], len);
}

return chunks;
}

//
// variable sized chunking with parameters
//
string *
ChunkWrapper::getChunks(string data, int & numOfChunks,
                        int avgChunkSize, int minChunkSize, int maxChunkSize) {

    string *chunks;

    int beginIndexes[MAX_NUM_CHUNKS];
    int endIndexes[MAX_NUM_CHUNKS];
    int i, len;

    // get indexes of beginning and ending of chunks
    numOfChunks = getChunksIndexes((unsigned char*)data.c_str(),
                                    data.length(), avgChunkSize, minChunkSize,
                                    maxChunkSize, beginIndexes, endIndexes);

#ifdef DEBUG
    cout << "numOfChunks(after)=< " << numOfChunks << endl;
#endif

    // create string array
    chunks = new string[numOfChunks];
    for (i=0; i < numOfChunks; i++) {
        len = endIndexes[i] - beginIndexes[i] + 1;
#ifdef DEBUG
        cout << "###_chunk[" << i
               << ":_begin_index=_< " << beginIndexes[i]
               << ",_len=_< " << len << endl;
#endif
    }
    chunks[i] = data.substr(beginIndexes[i], len);
}

```

```

    }

    return chunks;
}

//
// fixed size blocks
//
string *
ChunkWrapper::getBlocks(string data, int & numOfChunks) {

    string *chunks;
    int chunkIndex = 0;
    size_t beginOffset = 0, endOffset = 0;

    int chunkSize = getAvgChunkSize();

    // get number of fixed chunks
    numOfChunks = (data.length() / chunkSize) + 1;

#ifdef DEBUG
    cout << "data.length()_=_ " << data.length() << endl;
    cout << "avg_chunk_size_=_ " << chunkSize << endl;
    cout << "numOfChunks_=_ " << numOfChunks << endl;
#endif

    // get fixed chunks
    chunks = new string[numOfChunks];
    endOffset = beginOffset + (chunkSize - 1);
    while (endOffset < data.length()) {
#ifdef DEBUG
        cout << "offset_(" << beginOffset << ",_";
        cout << endOffset << ")" << endl;
#endif
        endif
        chunks[chunkIndex++] = data.substr(beginOffset, chunkSize);

        beginOffset = endOffset + 1;
        endOffset = beginOffset + (chunkSize - 1);
    }

    // get fixed last chunk
#ifdef DEBUG
    cout << "offset_(" << beginOffset << ",_";
    cout << data.length() - 1 << ")" << endl;
#endif
endif

    chunks[chunkIndex] =
        data.substr(beginOffset, data.length() - beginOffset);

    return chunks;
}

```

F.4 chunkWrapperTest

```

#include "chunkWrapper.h"
#include "fileOper.h"
#include "shalWrapper.h"

#ifdef CHUNK_WRAPPER_TEST

int main(int argc, char* argv[]) {

    string *chunks;
    string *fixedChunks;
    int numOfChunks = 0;
    int i;

    int avgChunkSize, minChunkSize, maxChunkSize;
    FileOper fileOper;
    ShalWrapper shalObj;

    // get configuration
    avgChunkSize = 8192;
    minChunkSize = 2048;
    maxChunkSize = 65535;

    // initialize object
    ChunkWrapper obj(avgChunkSize, minChunkSize, maxChunkSize);

    // get chunks from a data
    // string data = fileOper.getData("document.xml.changed");
    string data = fileOper.getData("chunk_lib/body");
    cout << endl;

    //-----
    // Variable sized chunking
    //-----
    cout << "*****_variable_sized_chunking*****" <<
        endl;
    cout << "average_chunk_size_=" << obj.getAvgChunkSize() <<
        endl;
    cout << "minimum_chunk_size_=" << obj.getMinChunkSize() <<
        endl;
    cout << "maximum_chunk_size_=" << obj.getMaxChunkSize() <<
        endl;

    //cout << "data size : " << data.length() << endl;
    //cout << "hash key : " << shalObj.getHashKey(data) << endl;
    chunks = obj.getChunks(data, numOfChunks);
    for (i=0; i < numOfChunks; i++) {
        cout << "chunk_" << i << "]" << chunks[i].length() << endl;
    }
    cout << endl;
    delete[] chunks;
}


```

```

//-----
// Variable sized chunking
//-----
cout << "*****_variable_sized_chunking_***" << endl;
avgChunkSize = 2048;
minChunkSize = 512;
maxChunkSize = 65535;
cout << "average_chunk_size_=" << avgChunkSize << endl;
cout << "minimum_chunk_size_=" << minChunkSize << endl;
cout << "maximum_chunk_size_=" << maxChunkSize << endl;
chunks = obj.getChunks(data, numOfChunks,
                        avgChunkSize, minChunkSize, maxChunkSize
                        );
for (i=0; i < numOfChunks; i++) {
    cout << "chunk_" << i << "]" << chunks[i].length() << endl;
}
cout << endl;
delete[] chunks;

//-----
// Fixed sized chunking
//-----
cout << "*****_fixed_sized_chunking_*****" << endl;
fixedChunks = obj.getBlocks(data, numOfChunks);
for (i=0; i < numOfChunks; i++) {
    cout << "chunk_" << i << "]" << fixedChunks[i].length()
        << endl;
}
cout << endl;
delete[] fixedChunks;

return 0;
}
#endif

```

Appendix G

Sample Programs Using libnetfilter_queue Library

In this chapter, we show the sample codes using the libnetfilter_queue library. The codes are used to capture incoming packets, show IP and TCP headers, and change lowercase letters in a payload into uppercase letters. Please note that changing the payload requires us to recompute the checksum of the packet and put the checksum into a packet header so that the packet is received by a receiver or the next forwarder. If checksum is not recomputed, the packet is simply rejected at the receiver or next forwarder because of an incorrect checksum with a changed payload. For testing, first compile and build by typing ‘make’. Then we need to add an iptables rule. For the receiver side, we type ‘iptables -I INPUT -p tcp -dport 50000 -j NFQUEUE --queue-num 0’ so that a packet with destination port 50000 is intercepted and forwarded to NFQUEUE. The intercepted packet is processed by the cb() function in ndedup_main.cc. Then the processed packet is received by a receiver (or sent on to the next forwarder) depending on the iptables rule. In other words, ‘iptables -I FORWARD -p tcp -dport 50000 -j NFQUEUE --queue-num 0’ is used for forwarding to the next forwarder.

G.1 ndedup.h

```
#ifndef NETWORK_DEDUP
#define NETWORK_DEDUP

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <unistd.h>
#include <netinet/in.h>
```

```

#include <linux/types.h>
#include <linux/netfilter.h>           /* for NF_ACCEPT */

#include <libnetfilter_queue/libnetfilter_queue.h>

#include <linux/ip.h>
// #include <linux/tcp.h>
#include <netinet/tcp.h>
#include <linux/udp.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#include <string>
#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;

#define NUM_BITS 16                    // number of bits in a word
                                       // for computing checksum

/* offset in nf_packet_payload */
#define TCP_CHECKSUM_FIRST_OFFSET 36
#define TCP_CHECKSUM_SECOND_OFFSET 37

class NDedup {
public:

    /* Common */
    void set(unsigned char *nf_packet_payload);

    /* IP */
    unsigned int ip_version();
    unsigned int ip_ihl();
    unsigned int ip_tos();
    unsigned int ip_tot_len();
    unsigned int ip_id();
    unsigned int ip_ttl();
    unsigned int ip_protocol();
    unsigned int ip_checksum(); // checksum of ip header(20 bytes)
    unsigned int ip_saddr();
    unsigned int ip_daddr();
    char * ip_saddr_str();
    char * ip_daddr_str();

    void showIPInfo();
    unsigned int ip_computeChecksum(unsigned char
                                    *nf_packet_payload);

    /* TCP */
    unsigned int tcp_sport();
    unsigned int tcp_dport();

```

```

unsigned long tcp_seq ();
unsigned long tcp_ack_seq ();
unsigned int tcp_hdr_len ();
unsigned short tcp_urg ();           // flag
unsigned short tcp_ack ();           // flag
unsigned short tcp_psh ();           // flag
unsigned short tcp_rst ();           // flag
unsigned short tcp_syn ();           // flag
unsigned short tcp_fin ();           // flag
unsigned int tcp_window ();          // window size
unsigned int tcp_checksum ();        // checksum of pseudo IP header
                                           // and TCP segment
// pseudo IP header + TCP segment (TCP header + TCP body)
//
// pseudo IP header (96 bits)
// = ip_saddr(32 bits),
//   ip_daddr(32 bits),
//   reserved (8 bits),
//   protocol (8 bits),
//   length(TCP Header + TCP body) in bytes (16 bits)
//
// checksum field (16 bits) is cleared when
// checksum is computed.

void showTCPInfo ();
unsigned int tcp_computeChecksum(unsigned char
                                *nf_packet_payload);
void setTCPChecksum(unsigned char *nf_packet_payload ,
                   unsigned int checksum);

/* UDP */
// it is now shown for UDP here.

/* Block */
// string hashKey();
unsigned int block_begin_offset ();
unsigned int block_end_offset ();
unsigned int block_size ();
string block ();

void showBlockInfo ();
void saveBlockToFile ();

private:

/* Common */
unsigned int computeChecksum(unsigned int *words ,
                           int num_inputs);
// unsigned char *nf_packet_payload_;

/* IP */
unsigned int ip_version_;
unsigned int ip_ihl_;           // (32 bits word) header length

```

```

unsigned int ip_tos_;
unsigned int ip_tot_len_;           // (bytes) total length =
                                     header + data
                                     // 20 B ~ 64 KB

unsigned int ip_id_;
unsigned int ip_ttl_;
unsigned int ip_protocol_;
unsigned int ip_checksum_;
unsigned int ip_saddr_;
unsigned int ip_daddr_;
char ip_saddr_str_[50];           // source address
char ip_daddr_str_[50];           // destination address

/* TCP */
unsigned int tcp_sport_;
unsigned int tcp_dport_;
unsigned long tcp_seq_;
unsigned long tcp_ack_seq_;
unsigned int tcp_hdr_len_;
unsigned short tcp_urg_;          // flag
unsigned short tcp_ack_;          // flag
unsigned short tcp_psh_;          // flag
unsigned short tcp_rst_;          // flag
unsigned short tcp_syn_;          // flag
unsigned short tcp_fin_;          // flag
unsigned int tcp_window_;         // window size
unsigned int tcp_checksum_;

/* UDP */
// it is not shown for UDP here.

/* Block (data block in a packet) */
// string hashKey_;
unsigned int block_begin_offset_;
unsigned int block_end_offset_;
unsigned int block_size_;
string block_;

/* SUB-FUNCTIONS */
void printChar(unsigned char c);
void showBits(int num);
int getBit(int num, int pos);
int setBit(int num, int pos, int bitValue);
int onesComplement(int num);
int addNumbers(int num1, int num2);
};

#endif

```

G.2 ndedup.cc

```
#include "ndedup.h"

#####
// Common
#####

// set IP, TCP, UDP header information
// REFERENCES :
// http://jve.linuxwall.info/ressources/code/nfqueue_recorder.c
void
Ndedup::set(unsigned char *nf_packet_payload) {

    int i;

    //-----
    // IP Information
    //-----
    // get a IP packet header (= payload of netfilter)
    struct iphdr *iph = ((struct iphdr *) nf_packet_payload);

    // set information
    ip_version_ = iph->version;
    // ihl is specified in 32 bits words. Thus, to get the size
    // in bytes,
    // multiply the value by 4
    ip_ihl_ = iph->ihl * 4; // (in bytes)
    ip_tos_ = iph->tos;
    ip_tot_len_ = ntohs(iph->tot_len);
    ip_id_ = ntohs(iph->id);
    // ip_frag_off_ = ntohs(iph->frag_off);
    ip_ttl_ = iph->ttl;
    ip_protocol_ = iph->protocol;

    // 10~11 chars —> checksum
    ip_checksum_ = nf_packet_payload[11];
    ip_checksum_ += (nf_packet_payload[10] << 8);

    // 12~15 chars (source address), 16~19 chars (dest address)
    ip_saddr_ = nf_packet_payload[15];
    ip_saddr_ += ((nf_packet_payload[14]) << 8);
    ip_saddr_ += ((nf_packet_payload[13]) << 16);
    ip_saddr_ += ((nf_packet_payload[12]) << 24);
    ip_daddr_ = nf_packet_payload[19];
    ip_daddr_ += ((nf_packet_payload[18]) << 8);
    ip_daddr_ += ((nf_packet_payload[17]) << 16);
    ip_daddr_ += ((nf_packet_payload[16]) << 24);

    sprintf(ip_saddr_str_, "%d.%d.%d.%d", nf_packet_payload[12],
        nf_packet_payload[13], nf_packet_payload[14],
        nf_packet_payload[15]);
```

```

sprintf(ip_daddr_str_, "%d.%d.%d.%d", nf_packet_payload[16],
        nf_packet_payload[17], nf_packet_payload[18],
        nf_packet_payload[19]);

// printf("xxx saddr = 0x%x\n", ip_saddr_);
// printf("xxx daddr = 0x%x\n", ip_daddr_);

//-----
// TCP Information
//-----
if (ip_protocol() == 6) {

    // get TCP header
    struct tcphdr *tcp = ((struct tcphdr *) (nf_packet_payload +
        (iph->ihl << 2)));

    // set information
    tcp_sport_ = ntohs(tcp->source);
    tcp_dport_ = ntohs(tcp->dest);
    tcp_seq_ = ntohl(tcp->seq);
    tcp_ack_seq_ = ntohl(tcp->ack_seq);

    // tcp->doff contains contains the number of 32 bit words
    // that represent the header size. Therefore,
    // to get the number of bytes, multiple this number by 4
    tcp_hdr_len_ = (tcp->doff << 2); // (in bytes)

    tcp_urg_ = tcp->urg;
    tcp_ack_ = tcp->ack;
    tcp_psh_ = tcp->psh;
    tcp_rst_ = tcp->rst;
    tcp_syn_ = tcp->syn;
    tcp_fin_ = tcp->fin;
    tcp_window_ = ntohs(tcp->window);

    tcp_checksum_ = ntohs(tcp->check);
}

//-----
// UDP Information
//-----
// not shown here.

//-----
// Block Information
//-----
// set information
block_ = "";
block_size_ = ip_tot_len() - (ip_ihl() + tcp_hdr_len())
);
if (block_size() > 0) {
    block_begin_offset_ = ip_ihl() + tcp_hdr_len();
    block_end_offset_ = ip_tot_len() - 1;
    // nf_packet_payload_ = nf_packet_payload;

```

```

        for (i=block_begin_offset(); i <= block_end_offset(); i++) {
            block_ += nf_packet_payload[i];
        }

        // cout << "block = " << block_ << endl;
    }
}

// #####
// IP
// #####

unsigned int
NDedup::ip_version() {
    return ip_version_;
}

unsigned int
NDedup::ip_ihl() {
    return ip_ihl_;
}

unsigned int
NDedup::ip_tos() {
    return ip_tos_;
}

unsigned int
NDedup::ip_tot_len() {
    return ip_tot_len_;
}

unsigned int
NDedup::ip_id() {
    return ip_id_;
}

unsigned int
NDedup::ip_ttl() {
    return ip_ttl_;
}

unsigned int
NDedup::ip_protocol() {
    return ip_protocol_;
}

unsigned int
NDedup::ip_checksum() {
    return ip_checksum_;
}

unsigned int

```

```

NDedup::ip_saddr() {
    return ip_saddr_;
}

unsigned int
NDedup::ip_daddr() {
    return ip_daddr_;
}

char *
NDedup::ip_saddr_str() {
    return ip_saddr_str_;
}

char *
NDedup::ip_daddr_str() {
    return ip_daddr_str_;
}

void
NDedup::showIPInfo() {

    // show ip information
    printf("****_IP_header_****\n");
    printf("version_____: %u_____\n", ip_version());
    printf("header_length_: %u_(byte)\n", ip_ihl());
    // printf("tos          : %u          \n", ip_tos());
    printf("total_length_: %u_(byte)\n", ip_tot_len());
    printf("id_____: %u_____\n", ip_id());
    printf("ttl_____: %u_____\n", ip_ttl());
    printf("protocol_____: %u_____\n", ip_protocol());
    printf("checksum_____: 0x%0x_____\n", ip_checksum());
    printf("source_____: %s_____\n", ip_saddr_str());
    printf("destination_____: %s_____\n", ip_daddr_str());
}

// Compute checksum of IP header (20 bytes).
// Checksum field is cleared before checksum is computed.
//
// return value : IP checksum
unsigned int
NDedup::ip_computeChecksum(unsigned char *nf_packet_payload) {
    int i;

    int num_inputs = ip_ihl() / 2;
    unsigned int words[num_inputs]; // 16-bit words

    //-----
    // make inputs
    //-----
    // make 16-bit words
    for (i=0; i < num_inputs; i++) {
        words[i] = nf_packet_payload[i*2 + 1];
        words[i] += ((nf_packet_payload[i*2]) << 8);
    }
}

```

```

    }
    // clear checksum word (6th position)
    words[5] = 0;

    return computeChecksum(words, num_inputs);
}

// #####
// TCP
// #####
unsigned int
NDedup::tcp_sport() {
    return tcp_sport_;
}

unsigned int
NDedup::tcp_dport() {
    return tcp_dport_;
}

unsigned long
NDedup::tcp_seq() {
    return tcp_seq_;
}

unsigned long
NDedup::tcp_ack_seq() {
    return tcp_ack_seq_;
}

unsigned int
NDedup::tcp_hdr_len() {
    return tcp_hdr_len_;
}

unsigned short
NDedup::tcp_urg() {
    return tcp_urg_;
}

unsigned short
NDedup::tcp_ack() {
    return tcp_ack_;
}

unsigned short
NDedup::tcp_psh() {
    return tcp_psh_;
}

unsigned short
NDedup::tcp_rst() {
    return tcp_rst_;
}

```

```

}

unsigned short
NDedup::tcp_syn () {
    return tcp_syn_;
}

unsigned short
NDedup::tcp_fin () {
    return tcp_fin_;
}

unsigned int
NDedup::tcp_window () {
    return tcp_window_;
}

unsigned int
NDedup::tcp_checksum () {
    return tcp_checksum_;
}

void
NDedup::showTCPInfo () {

    // show TCP information
    printf ("****_TCP_header_****_\n");
    printf ("sport: %u\n", tcp_sport ());
    printf ("dport: %u\n", tcp_dport ());
    printf ("seq: %lu\n", tcp_seq ());
    printf ("ack_seq: %lu\n", tcp_ack_seq ());
    printf ("header_length: %u (byte)\n", tcp_hdr_len ());
    printf ("flag_urgent: %u\n", tcp_urg ());
    printf ("flag_ack: %u\n", tcp_ack ());
    printf ("flag_push: %u\n", tcp_psh ());
    printf ("flag_rst: %u\n", tcp_rst ());
    printf ("flag_syn: %u\n", tcp_syn ());
    printf ("flag_fin: %u\n", tcp_fin ());
    printf ("window_size: %u (byte)\n", tcp_window ());
    printf ("checksum: 0x%0x\n", tcp_checksum ());

}

// Compute checksum of TCP
// Checksum field is cleared before checksum is computed.
//
// TCP checksum is computed with pseudo header and TCP segment.
// TCP segment consists of TCP header and TCP data.
//
// Pseudo header consists of ip source address(32 bits),
// ip destination address(32 bits), reserved (8 bits),
// protocol (8bits), and length of combined TCP header and
// TCP data (16 bits)
//

```

```

// *** IMPORTANT
// If TCP data is not aligned to 16-bits, 0 is padded.
// E.g. 0x0A is last bits in a TCP data,
// it is changed to 0x0A00 (8-bit 0s are padded)
//
// For more information, refer to following URLs
// http://www.tcpipguide.com/free/
// t_TCPChecksumCalculationandtheTCPPseudoHeader-2.htm
// http://www.personal.uni-jena.de/~p6lost2/DC/software
//      /tutorials/TCP_IP_checksum.pdf
//
// return value : TCP checksum
unsigned int
Ndedup::tcp_computeChecksum(unsigned char *nf_packet_payload) {
    int i;
    int index = 0;

    // TCP data length in bytes
    int tcp_data_len = ip_tot_len() - (ip_ihl() + tcp_hdr_len());

    int num_inputs = /* Pseudo Header */
        2 + // ip source address (4 bytes -> 2 16-bit words)
        2 + // ip dest address
        1 + // reserved(8 bits) + protocol(8 bits)
        1 + // length of TCP segment in bytes
        /* TCP Header */
        (tcp_hdr_len() / 2) +
        /* TCP Data */
        ((tcp_data_len + 1) / 2);
    // 1 is used to round up size for padding 0.

    unsigned int words[num_inputs]; // 16-bit words

    //-----
    // make inputs
    //-----

    /* Pseudo Header */
    words[index++] = (ip_saddr() >> 16); // left 16 bits
    words[index++] = (ip_saddr() & 65535); // right 16 bits
    words[index++] = (ip_daddr() >> 16);
    words[index++] = (ip_daddr() & 65535);
    words[index++] = ip_protocol(); // "ip reserved" is 0
    words[index++] = tcp_hdr_len() + tcp_data_len;

    /* TCP Header */
    for (i=ip_ihl(); i < (ip_ihl() + tcp_hdr_len()); i=i+2) {
        words[index] = (nf_packet_payload[i] << 8);
        words[index++] += nf_packet_payload[i+1];
    }

    // clear checksum word to 0
    words[14] = 0; // index of checksum is placed onto 14th index
                  // out of (pseudo header + TCP segment)

```

```

    /* TCP Data */
    for (i=(ip_ihl() + tcp_hdr_len()); i < ip_tot_len(); i=i+2) {
        words[index] = (nf_packet_payload[i] << 8);
        if ((i+1) < ip_tot_len()) {
            words[index++] += nf_packet_payload[i+1];
        }
    }

/*
    for (i=0; i < index; i++) {
        printf("word[%d] = 0x%0x, ", i, words[i]);
        showBits(words[i]);
    }
*/

    return computeChecksum(words, num_inputs);
}

// TCP checksum position in nf_packet_payload
// 66th, 67th bytes (position starts from 0)
void
NDedup::setTCPChecksum(unsigned char *nf_packet_payload,
                        unsigned int checksum) {

    printf("checksum_=_0x%0x\n", checksum);

    // to nf_packet_payload
    nf_packet_payload[TCP_CHECKSUM_FIRST_OFFSET]
        = (checksum >> 8) & 65535;
    nf_packet_payload[TCP_CHECKSUM_SECOND_OFFSET] = checksum
        & 65535;

    // to object
    tcp_checksum_ = checksum;
}

#####
// UDP
#####
// it is not shown for UDP

#####
// Block
#####

unsigned int
NDedup::block_begin_offset() {
    return block_begin_offset_;
}

```

```

unsigned int
NDedup::block_end_offset() {
    return block_end_offset_;
}

// in bytes
// IP total length - (IP header size + TCP header size)
unsigned int
NDedup::block_size() {
    return block_size_;
}

string
NDedup::block() {
    return block_;
}

void
NDedup::showBlockInfo() {

    // show Block information
    printf("****_Block_****\n");
    printf("begin_offset_:::%u\n", block_begin_offset());
    printf("end_::offset_:::%u\n", block_end_offset());
    printf("size_:::%u\n", block_size());
    // printf("%s\n", block());
}

void
NDedup::saveBlockToFile() {

    ofstream outF;

    // set filename
    stringstream s;
    string filename;
    s << "storage/" << ip_id();
    filename = s.str();

    // open file
    outF.open(filename.c_str(), ofstream::out | ofstream::trunc);

    // save data
    outF << block();

    // close file
    outF.close();
}

```

```

// #####
// INTERNAL FUNCTIONS
// #####

// compute and return checksum
unsigned int
NDedup::computeChecksum(unsigned int *words, int num_inputs) {
    int sum;
    int carry;
    int checksum;
    int i;

#ifdef DEBUG
    //-----
    // inputs
    //-----
    printf(">>>_Input_numbers_(%d)\n", num_inputs);

    for (i=0; i < num_inputs; i++) {
        printf("word[%d]_=0x%x", i, words[i]);
        showBits(words[i]);
    }
    printf("\n");
#endif

    //-----
    // add numbers
    //-----

#ifdef DEBUG
    printf(">>>_Add_numbers_with_wrapping_around_carry\n");
#endif

    sum = addNumbers(words[0], words[1]);
    for (i=2; i < num_inputs; i++) {
        sum = addNumbers(sum, words[i]);
    }

#ifdef DEBUG
    showBits(sum);

    printf("\n");
#endif

    //-----
    // 1's complement
    //-----

#ifdef DEBUG
    printf(">>>_1's_complement\n");
#endif
    checksum = onesComplement(sum);
#ifdef DEBUG
    printf("checksum_=0x%x", checksum);
    showBits(checksum);
#endif

```

```

    return checksum;
}

// print hexa-decimal value of a character
void
NDedup::printChar(unsigned char c) {
    static const char* const lut = "0123456789ABCDEF";
    char output[2 + 1];

    output[0] = lut[c >> 4];
    output[1] = lut[c & 15];

    output[2] = '\0';
    cout << output;
}

void
NDedup::showBits(int num) {
    int i;

    for (i=(NUM_BITS-1); i >= 0; i--) {
        printf("%d", getBit(num, i));
    }
    printf("\n");
}

// get value of a bit for a position.
//      3210 (bit position)
// e.g. 0100 → bit(2) returns 1
int
NDedup::getBit(int num, int pos) {
    int mask = 1;
    int i;

    for (i=0; i < pos; i++) {
        mask *= 2;
    }

    // printf("num = %d, pos = %d, mask = %d\n", num, pos, mask);

    num &= mask;
    return num >> pos;
}

// set value of a bit for a position
//      3210
// e.g. num(9) = 1001 → setbit (num, 3, 0) → 0001
//
// for error, return -1
// Otherwise, return changed number
int
NDedup::setBit(int num, int pos, int bitValue) {

```

```

int i;

if (bitValue == 1) {
    int mask = bitValue;
    mask = mask << pos;
    num |= mask;
} else if (bitValue == 0) { // bitValue = 0
    int mask = 1;
    for (i=0; i < pos; i++) {
        mask *= 2;
    }
    mask = ~mask;
    num &= mask;

} else {
    return -1;
}

return num;
}

```

// compute 1's complement
// return computed 1's complement

```

int
NDedup::onesComplement(int num) {

    int i;

    for (i=(NUM_BITS-1); i >=0; i--) {
        if (getBit(num,i) == 0)
            num = setBit(num, i, 1);
        else // bit == 1
            num = setBit(num, i, 0);
    }

    return num;
}

```

// add numbers with wrapping around carry
// return an added number

```

int
NDedup::addNumbers(int num1, int num2) {

    int sum;
    int carry;

    // add numbers
    sum = num1 + num2;

    // check carry
    carry = sum >> NUM_BITS;
    // printf("carry = %d\n", carry);

    // wrap around carry

```

```

int mask = (int)(pow(2, NUM_BITS) - 1);
if (carry > 0) {
    sum &= mask;
    sum += carry;
}
return sum;
}

```

G.3 ndedup_main.cc

```

#include "ndedup.h"

#ifdef NDEDUP_TEST

void printBytes(unsigned char *input, int len) {
    static const char* const lut = "0123456789ABCDEF";
    char output[len * 2 + 1];
    for (int i = 0; i < len ; ++i) {
        const unsigned char c = input[i];
        output[i * 2] = lut[c >> 4];
        output[i * 2 + 1] = lut[c & 15];
    }
    output[len * 2] = '\0';
    cout << output << endl;
}

void printChar(unsigned char c, int len) {
    static const char* const lut = "0123456789ABCDEF";
    char output[2 + 1];

    output[0] = lut[c >> 4];
    output[1] = lut[c & 15];

    output[2] = '\0';
    cout << output << endl;
}

// returns packet id
static u_int32_t print_pkt(struct nfq_data *tb,
    int *nf_payload_size,
    unsigned int *block_begin_offset,
    unsigned int *block_end_offset) {

    //int id = 0;
    u_int32_t id = 0;
    struct nfqnl_msg_packet_hdr *ph;
    // struct nfqnl_msg_packet_hw *hwph;
    // u_int32_t mark, ifi;
    //int ret;
    unsigned char *nf_packet;

```

```

// network dedup object
NDedup nd;

ph = nfq_get_msg_packet_hdr(tb);
if (ph) {
    id = ntohl(ph->packet_id);
}

/*
hwph = nfq_get_packet_hw(tb);
if (hwph) {
    int i, hlen = ntohs(hwph->hw_addrlen);

    printf("hw_src_addr=");
    for (i = 0; i < hlen-1; i++)
        printf("%02x:", hwph->hw_addr[i]);
    printf("%02x ", hwph->hw_addr[hlen-1]);
}

mark = nfq_get_nfmark(tb);
if (mark)
    printf("mark=%u ", mark);

ifi = nfq_get_indev(tb);
if (ifi)
    printf("indev=%u ", ifi);

ifi = nfq_get_outdev(tb);
if (ifi)
    printf("outdev=%u ", ifi);

ifi = nfq_get_physindev(tb);
if (ifi)
    printf("physindev=%u ", ifi);

ifi = nfq_get_physoutdev(tb);
if (ifi)
    printf("physoutdev=%u ", ifi);

ret = nfq_get_payload(tb, &nf_packet);
if (ret >= 0) {
    printf("payload_len=%d\n", ret);
}

fputc('\n', stdout);
*/

//-----
// Initialize NDedup object
//-----
nfq_get_payload(tb, &nf_packet);
nd.set(nf_packet);

//-----

```

```

    // IP Information
    //-----
    nd.showIPInfo();

    //-----
    // TCP Information
    //-----
    if (nd.ip_protocol() == 6) {
        nd.showTCPInfo();
    }

    //-----
    // UDP Information
    //-----
    // later .. implemented ...

    //-----
    // Block Information
    //-----
    if (nd.block_size() > 0) {
        nd.showBlockInfo();
        nd.saveBlockToFile();

        *nf_payload_size = nd.ip_tot_len();
        *block_begin_offset = nd.block_begin_offset();
        *block_end_offset = nd.block_end_offset();

    }

    return id;
}

static int cb(struct nfq_q_handle *qh, struct nfgenmsg *nfmsg,
              struct nfq_data *nfa, void *data) {

    int nf_payload_size = 0;
    unsigned char *nf_payload;
    u_int32_t id;
    int i;
    int len;

    NDedup nd;

    unsigned int block_begin_offset, block_end_offset;

    // get nf_payload
    len = nfq_get_payload(nfa, &nf_payload);
    nd.set(nf_payload);

    // get id and nf_payload_size
    id = print_pkt(nfa, &nf_payload_size,
                  &block_begin_offset, &block_end_offset);

```

```

if (nf_payload_size > 0) {
    //printf("payload size      = %d\n", nf_payload_size);
    //printf("block  b_offset = %u\n", block_begin_offset);
    //printf("block  e_offset = %u\n", block_end_offset);

    //-----
    // payload before modification
    //-----
    printf("****_nf_payload(IP_header+_TCP_header+"_
           "TCP_data)****_n");
    printf(">>>_before_modification\n");
    printf("ip_checksum=_0x%0x\n",
           nd.ip_computeChecksum(nf_payload));
    printf("tcp_checksum=_0x%0x\n",
           nd.tcp_computeChecksum(nf_payload));
    if (len) {
        printBytes(nf_payload, len);
    }

    //#####
    // modify nf_payload
    //#####
    //-----
    // modify data
    //-----
    for (i=block_begin_offset; i <= block_end_offset; i++) {
        printf("%c", toupper(nf_payload[i]));
        nf_payload[i] = toupper(nf_payload[i]);
    }
    /*
    for (i=block_begin_offset; i <= block_end_offset; i++) {
        if (i == block_begin_offset) {
            printf("%c", toupper(nf_payload[i]));
            nf_payload[i] = toupper(nf_payload[i]);
        } else
            printf("%c", nf_payload[i]);
    }
    */

    //-----
    // modify header information
    // - ip_tot_len
    // - ip_checksum
    // - tcp_checksum
    //-----
    unsigned int modified_tcp_checksum;
    modified_tcp_checksum = nd.tcp_computeChecksum(nf_payload);
    printf("modified_tcp_checksum=_0x%0x\n",
           modified_tcp_checksum);
    nd.setTCPChecksum(nf_payload, modified_tcp_checksum);

    //-----
    // payload after modification
    //-----

```

```

    printf(">>>_after_modification\n");
    printf("ip_checksum=_0x%0x\n",
           nd.ip_computeChecksum(nf_payload));
    printf("tcp_checksum=_0x%0x\n",
           nd.tcp_computeChecksum(nf_payload));

    if (len) {
        printBytes(nf_payload, len);
    }

}

// put modified payload
printf("entering_callback\n\n");
//return nfq_set_verdict(qh, id, NF_ACCEPT, 0, NULL);
return nfq_set_verdict(qh, id, NF_ACCEPT, nf_payload_size,
                       nf_payload);

}

int main(int argc, char **argv)
{
    struct nfq_handle *h;
    struct nfq_q_handle *qh;
    struct nfnl_handle *nh;
    int fd;
    int rv;
    char buf[4096] __attribute__((aligned));

    printf("opening_library_handle\n");
    h = nfq_open();
    if (!h) {
        fprintf(stderr, "error_during_nfq_open()\n");
        exit(1);
    }

    printf("unbinding_existing_nf_queue_handler_for_AF_INET\n");
    if (nfq_unbind_pf(h, AF_INET) < 0) {
        fprintf(stderr, "error_during_nfq_unbind_pf()\n");
        exit(1);
    }

    printf("binding_nfnetlink_queue_as_nf_queue_handler_for\n");
    if (nfq_bind_pf(h, AF_INET) < 0) {
        fprintf(stderr, "error_during_nfq_bind_pf()\n");
        exit(1);
    }

    printf("binding_this_socket_to_queue_0'\n");
    qh = nfq_create_queue(h, 0, &cb, NULL);
    if (!qh) {

```

```

        fprintf(stderr, "error_during_nfq_create_queue()
        _____\n");
        exit(1);
    }

    printf("setting_copy_packet_mode\n");
    if (nfq_set_mode(qh, NFQNL_COPY_PACKET, 0xffff) < 0) {
        fprintf(stderr, "can't_set_packet_copy_mode\n");
        exit(1);
    }

    fd = nfq_fd(h);

    while ((rv = recv(fd, buf, sizeof(buf), 0)) && rv >= 0) {
        printf("pkt_received\n");
        // printf("%s", buf);
        nfq_handle_packet(h, buf, rv);
    }

    printf("unbinding_from_queue_0\n");
    nfq_destroy_queue(qh);

#ifdef INSANE
    /* normally, applications SHOULD NOT issue this command,
       since
       * it detaches other programs/sockets from AF_INET,
       too ! */
    printf("unbinding_from_AF_INET\n");
    nfq_unbind_pf(h, AF_INET);
#endif

    printf("closing_library_handle\n");
    nfq_close(h);

    exit(0);
}

#endif

```

References

1. Rabin, M.O.: Fingerprinting by random polynomials. Tech. Rep. Report TR-15-81, Harvard University (1981)
2. Bakker, P.: SHA1 codes. http://asf.atmel.com/docs/latest/uc3l/html/sha1_8h.html
3. Mazieres, D.: Rabin Finterprinting Codes. <https://github.com/okws/sfslite/blob/master/crypt/rabinpoly.h>

Glossary

ARM-CPU Microprocessor technology that is a family of reduced instruction set computing (RISC) architectures and used mainly for low-power devices.

Block Fixed-size data accessed with offsets.

Bloom Filter Small bit array used to check whether duplicate data exist in storage.

C++ Boost Library Peer-reviewed portable C++ source libraries.

Chunk Variable-sized data.

CPLEX optimizer Optimization software to compute a linear program.

Eclipse Integrated development environment.

EDMilter Email deduplication mail filter.

Floodlight Java-based controller system in a software-defined network.

Granularity The unit of data to be compared to check duplicates.

Index cache Memory that contains data hash keys.

Iptables Userspace program that configures firewall in Linux kernel.

JPEG Popular compression technique for digital photography. a lossy compression technique for color images.

JSON JavaScript Object Notation, a lightweight data-interchange format.

Libnetfilter_queue Userspace library providing an API to packets that have been queued by the kernel packet filter.

MPEG Standards for audio and video compression and transmission.

Netfilter Series of hooks to capture packets at various points in protocol stack.

NFQUEUE Iptable target.

Offset-shifting problem Fixed offset is shifted when data before the offset are inserted in a file.

OpenVSwitch Virtual switch for software-defined network.

Packet Decoding Reconstructing the encoded packet.

Packet Encoding Replacing the redundant payload within a packet with an index.

Rabin Fingerprint Technique to find chunk boundary.

Redundancy Elimination Solution to remove duplicate data transferred through network.

REST Software architecture where data are accessed through HTTP API.

Storage Data Deduplication Solution to remove duplicate data when data are stored in storage.

Time Complexity Amount of time taken by an algorithm to run.

Unordered_map Data structure with a pair consisting of a key and value; data are saved in no particular order.